

Machine Learning

Course Notes (654AA)

Version 1.2

Fabio Piscitelli

Professor: Alessio Micheli

University of Pisa — Master's Degree in Computer Science, A.Y. 2026/27

September 29, 2026

Contents

1	Introduction to Machine Learning	1
1.1	What is Machine Learning	1
1.1.1	Learning as a universal principle	1
1.1.2	Why make machines learn: luxury or necessity?	1
1.1.3	When ML is needed	1
1.1.4	Application areas	2
1.1.5	Some recent successes	2
1.1.6	The ultimate goal	3
1.2	Why study Machine Learning	3
1.2.1	Modern ML	4
1.2.2	Three views of ML	4
1.3	Course information	4
1.3.1	Prerequisites and goals	4
1.3.2	Course structure	5
1.3.3	Practical organisation	6
1.3.4	Exam	6
1.3.5	Materials and bibliography	8
1.4	Mathematical background	8
1.4.1	Dot product and norms	9
1.4.2	Partial derivatives and gradient	9
1.4.3	Probability density: the Gaussian	10
1.5	Summary and next steps	11
2	ML in the master's curricula	12
2.1	The master's curricula	12
2.2	The role of Machine Learning	12
2.3	Synergy with Computational Mathematics	13
2.4	Notes for specific categories of students	13
3	Fundamental concepts of ML	14
3.1	Machine Learning in the computational setting	14
3.1.1	When to use ML	14
3.2	Learning as function approximation	15
3.2.1	The guiding example: digit recognition	15
3.2.2	Examples of pairs $x \rightarrow f(x)$	15
3.3	The ingredients of an ML system	16
3.3.1	Data	16

3.3.2	Task	17
3.3.3	Model	20
3.3.4	Learning algorithm	22
3.4	Inductive bias	22
3.4.1	Learning Boolean functions: an ill-posed problem	23
3.4.2	Restricting the space: conjunctive rules	23
3.4.3	The unbiased learner	24
3.4.4	Equivalent inductive and deductive systems	24
3.4.5	Language bias or search bias?	24
3.5	Tasks and loss functions	25
3.5.1	Regression: squared error	26
3.5.2	Classification: 0/1 loss	26
3.5.3	Clustering and vector quantization (preview)	27
3.5.4	Density estimation (preview)	27
3.6	Generalisation and validation	27
4	Generalisation and validation (introduction)	29
4.1	ML in a nutshell	29
4.2	The fundamental problems of ML	29
4.2.1	An ill-posed problem	29
4.2.2	Generalisation and overfitting	29
4.3	A case study: polynomial fitting	30
4.3.1	Effect of the polynomial degree	30
4.3.2	Training and test error as M varies	32
4.3.3	Effect of the amount of data	33
4.4	Towards Statistical Learning Theory	33
4.4.1	The formal setting (simplified)	33
4.4.2	VC-dimension and risk bound	34
4.4.3	Why SLT is important	35
4.5	Validation	36
4.5.1	The two goals of validation	36
4.5.2	Hold-out	36
4.5.3	K-fold cross-validation (preview)	36
4.6	Accuracy in classification	37
4.6.1	Confusion matrix	37
4.6.2	ROC curve	38
4.7	The design cycle	38
4.8	Interpretation errors	39
5	Linear models and K-nearest neighbors	41
5.1	Data notation	41
5.2	Linear models	41
5.2.1	Univariate linear regression	41

5.2.2	Notation for multidimensional inputs	43
5.3	Classification with linear models	43
5.3.1	The problem	43
5.3.2	Hyperplanes and decision regions	44
5.3.3	Using the model: two examples	45
5.3.4	Useful properties of the hyperplane	45
5.4	The learning problem	46
5.4.1	Formulation	46
5.4.2	Replacing the loss with a smooth function	46
5.4.3	Direct approach: the normal equations	47
5.4.4	Iterative approach: gradient descent	48
5.4.5	Summary for classification	51
5.4.6	Back to the problem: is it a good solution?	52
5.4.7	Inductive bias of the linear model	52
5.5	Limits of linear models	53
5.5.1	Regression of non-linear problems	53
5.5.2	Classification: linear separability	53
5.6	Extending the linear model	53
5.6.1	“Linear” in the parameters, not in the inputs	54
5.6.2	Linear Basis Expansion (LBE)	54
5.7	Regularisation: complexity control	55
5.7.1	Ridge regression (Tikhonov regularisation)	55
5.7.2	Other regularisation techniques	57
5.7.3	Other improvements (optional)	58
5.7.4	Multi-class classification (optional)	58
5.7.5	A look ahead	58
5.8	K-nearest neighbors	58
5.8.1	Eager and lazy learning	59
5.8.2	1-nearest neighbor	59
5.8.3	K-nearest neighbors	59
5.8.4	Behaviour as k varies	59
5.8.5	Variants	61
5.8.6	An extreme of ML	61
5.8.7	K-NN versus linear model	61
5.8.8	The Bayes optimal classifier	62
5.8.9	Inductive bias of K-NN	63
5.8.10	Limits of K-NN	63
5.8.11	K-NN design choices	65
5.8.12	What next?	65

6	Neural networks (part 1): from the neuron to the MLP	67
6.1	Neural networks as an ML tool	67
6.2	The biological inspiration	67
6.3	The artificial neuron	68
6.4	The Perceptron	69
6.4.1	McCulloch and Pitts networks (1943)	70
6.4.2	The hidden layer as a representation	71
6.5	Learning for a single unit	72
6.5.1	The Perceptron learning algorithm	72
6.5.2	Delta rule: two readings	73
6.5.3	Representing and learning	73
6.5.4	The Perceptron convergence theorem	74
6.6	Perceptron versus LMS	76
6.7	Sigmoidal activation functions	77
6.7.1	Other activation functions	77
6.7.2	Derivatives	78
6.7.3	LMS with a sigmoidal unit	78
6.8	Neural networks: the Multi-Layer Perceptron	79
6.8.1	View A: a network of units	79
6.8.2	View B: a flexible function	80
6.8.3	Components of a neural network	80
6.8.4	Uniform notation for units	80
6.8.5	Architecture and feedforward processing	81
6.9	Why neural networks are flexible	81
6.9.1	Which tasks	81
6.9.2	The network as an adaptive basis expansion	82
6.9.3	Universal approximation	83
6.9.4	Expressive power and VC-dimension	83
6.9.5	How many layers? (preview)	83
6.9.6	The inductive bias of neural networks	84
6.10	Towards the learning algorithm	84
6.10.1	The credit assignment problem	84
6.10.2	The loading problem	84
6.10.3	The key idea: extending gradient descent	84
6.10.4	Practical issues in training	85
7	Notes on backpropagation	86
7.1	Differential calculus tools	86
7.2	The problem	86
7.3	The iterative algorithm	87
7.4	The derivation	88
7.4.1	The gradient for a generic weight	88

7.4.2	Case 1: output unit ($t = k$)	88
7.4.3	Case 2: hidden unit ($t = j$)	89
7.5	Summary of the formulas	89
7.5.1	Generalisation to m hidden layers	90
7.6	The complete training cycle	90
7.7	Properties and interpretation	91
8	Neural networks (part 2): training in practice	92
8.1	Philosophy	92
8.2	Initial weight values	93
8.3	Multiple minima	93
8.3.1	Do we really need the global minimum?	94
8.4	On-line, batch and mini-batch	94
8.4.1	Mini-batch	95
8.5	The learning rate η	95
8.5.1	LMS: dividing by l	96
8.5.2	Improving the descent	96
8.5.3	Beyond basic SGD	98
8.6	Stopping criteria	98
8.7	Overfitting and regularisation	98
8.7.1	Stop at the minimum?	98
8.7.2	How overfitting arises in a network	99
8.7.3	Regularisation (!)	99
8.7.4	Putting everything together: momentum and weight decay	102
8.8	Number of hidden units	102
8.8.1	Cascade Correlation	102
8.9	Input and output	104
8.10	Strengths and weaknesses of neural networks	104
8.11	Towards the project: the MONK benchmark	105
9	Validation (part 1): model selection and model assessment	108
9.1	Preamble: bias and variance	108
9.2	Motivations	109
9.2.1	How to estimate	109
9.3	The two goals of validation	109
9.3.1	Hold-out	109
9.3.2	Test or model selection?	110
9.4	An instructive counterexample	110
9.5	Grid search	111
9.5.1	Alternatives to grid search	111
9.6	K-fold cross-validation	112
9.6.1	How much data is needed?	112
9.6.2	Recap	113

9.6.3	A complete example of selection and assessment	113
9.7	Special cases	113
9.7.1	Lucky or unlucky sampling	113
9.7.2	Very few data	113
9.8	Error measures for evaluation	114
10	Validation (part 2): formal schemes	115
10.1	Notation	115
10.2	Methodology: identifying the goal	116
10.3	Model selection	116
10.3.1	Hold-out	116
10.3.2	K-fold cross-validation	116
10.4	Risk estimation (model assessment)	117
10.4.1	Hold-out	117
10.4.2	K-fold cross-validation	117
10.5	Combining model selection and model assessment	118
10.5.1	Hold-out training-validation-test	118
10.5.2	K-fold CV for selection + hold-out for the test	118
10.5.3	K-fold CV for the test with internal hold-out for selection	119
10.5.4	Double (nested) K-fold CV	119
11	Validation (part 3): common mistakes and FAQ	122
11.1	Stopping training	122
11.1.1	Do not fix an arbitrary number of epochs	122
11.1.2	Early stopping	122
11.2	Random initialisation and model selection	123
11.3	Sequential hyperparameter selection	123
11.4	What to do after selection and assessment?	124
11.5	Cross-validation can overfit	124
11.6	Which cross-validation?	124
11.7	Examples of model selection	125
11.7.1	Example 1: different models	125
11.7.2	Example 2: number of hidden units	125
11.7.3	Example 3: K-NN	126
11.8	Practical conclusions	126
12	Statistical Learning Theory and VC-dimension	127
12.1	The VC-dimension	127
12.1.1	Shattering	127
12.1.2	Definition of VC-dimension	128
12.1.3	VC-dimension of hyperplanes in the plane	129
12.1.4	Other examples	129
12.1.5	VC-dimension and number of parameters	130

12.2 The analytical risk bound	130
12.3 Structural Risk Minimization	131
12.3.1 SRM for model selection	131
12.3.2 What the bound is for	131
12.3.3 Two practical approaches to SRM	132
13 Support Vector Machines	134
13.1 Part I — SVM for binary classification	134
13.1.1 The separating hyperplane	135
13.1.2 The separation margin	135
13.1.3 Canonical representation and support vectors	136
13.1.4 Computing the margin	136
13.1.5 The quadratic optimisation problem (hard margin)	137
13.1.6 How it is used	139
13.1.7 Why does it improve generalisation?	139
13.1.8 Why it is an elegant approach	139
13.2 Soft margin SVM: allowing errors	140
13.2.1 Slack variables	140
13.2.2 The soft margin primal problem	141
13.2.3 The soft margin dual*	141
13.3 Kernels: SVM for non-linear problems	141
13.3.1 Mapping into a high-dimensional space	142
13.3.2 High-dimensional feature spaces	142
13.3.3 The kernel trick	143
13.3.4 The kernel matrix	143
13.3.5 SVM with kernels	144
13.3.6 “Architectural” view of an SVM	144
13.3.7 Examples of kernels	145
13.4 Part II — SVM for non-linear regression	145
13.4.1 The problem	145
13.4.2 The ϵ -insensitive loss	145
13.4.3 Formulation	146
14 SVM and kernels: practical aspects and critical view	148
14.1 Main features	148
14.2 Results	148
14.2.1 The first famous application	149
14.2.2 The Gaussian mixture example	149
14.3 SVMs in practice	149
14.3.1 The critical role of hyperparameters	150
14.3.2 Myths about SVMs	151
14.4 Kernel methods	151
14.4.1 Kernel as a similarity measure	152

14.4.2 Advanced topics	152
15 Bias-variance and ensembles	154
15.1 The idea	154
15.2 Analysis in regression	154
15.2.1 The expected prediction error	154
15.2.2 Statistics recap	155
15.2.3 The decomposition	156
15.2.4 Interpretation of the three components	156
15.3 Bias, variance and regularisation	158
15.4 Ensemble learning	159
15.4.1 Bagging (bootstrap aggregating)	160
15.4.2 Boosting (e.g. AdaBoost)	160
15.5 Feature selection (outline)	160
16 Convolutional neural networks (CNN)	162
16.1 The problem: recognising characters	162
16.2 Convolution	163
16.2.1 1D convolution: a unit on a stream	163
16.2.2 2D convolution	163
16.2.3 Stride greater than 1	164
16.3 A network of “filters”	165
16.4 Pooling	165
16.5 The CNN as a whole	165
16.6 How they are used	167
16.7 Historical results	167
16.7.1 LeCun’s architectures on the ZIP code dataset	167
16.7.2 MNIST	168
16.7.3 Face recognition and ImageNet	169
16.8 Modern CNNs and GPUs	169
17 Deep learning	171
17.1 What is deep learning	171
17.1.1 A bit of history	171
17.1.2 The general framework	171
17.1.3 The conceptual ingredients: hierarchical abstraction	172
17.1.4 Combining to generalise	173
17.2 Are many layers really needed?	174
17.2.1 An example from logic circuits	174
17.2.2 The case of neural networks	174
17.2.3 Why: compositionality	175
17.2.4 The inductive bias	176
17.2.5 Curse of dimensionality and bias	176

17.2.6 Practical issues	177
17.3 Representation learning	177
17.3.1 Basic ideas	177
17.3.2 Pre-training	178
17.3.3 Transfer learning	179
17.4 Distributed representations	180
17.4.1 Counting the difference	181
17.4.2 Shared attributes: disentangling concepts	181
17.4.3 Deep distributed representations	182
17.5 Further insights and recent results	183
17.6 Techniques	184
17.6.1 Gradient problems	184
17.6.2 ReLU	185
17.6.3 Batch normalization	186
17.6.4 Dropout	186
17.6.5 L1 regularisation	187
17.6.6 Adversarial learning	188
17.7 Applications, challenges and conclusions	188
18 Randomized neural networks	189
18.1 Randomness in Machine Learning	189
18.1.1 Intrinsically random models	189
18.2 Random-weight neural networks	190
18.2.1 General structure	190
18.2.2 Randomised feedforward networks	191
18.2.3 Pros and cons	192
18.2.4 Where they are useful	192
19 Unsupervised learning: K-means and SOM	194
19.1 Unsupervised learning	194
19.1.1 Clustering	194
19.2 Vector quantization	195
19.2.1 The quantization error	196
19.3 K-means	196
19.3.1 On-line version	196
19.3.2 Batch version	197
19.3.3 Strengths and weaknesses	198
19.3.4 Soft-max	198
19.4 Self-Organizing Map	198
19.4.1 Goal	198
19.4.2 Competitive learning and biological inspiration	199
19.4.3 The algorithm	199
19.4.4 The emergence of topological order	200

19.4.5 Visualisation	201
19.5 Evaluating a clustering (outline)	202
20 Recurrent neural networks (RNN)	204
20.1 Why sequential data	204
20.2 From flat data to structured data	204
20.2.1 The new domain: sequences and transductions	204
20.3 Memory in neural networks	205
20.3.1 Approach 1: finite memory (IDNN)	205
20.3.2 Approach 2: recurrent units	206
20.3.3 State transition systems	206
20.3.4 A recurrent network (Simple RNN)	207
20.3.5 Properties	208
20.4 Learning: unfolding	208
20.4.1 Advanced models	209
20.5 Related approaches	209
20.5.1 Transformers and attention	209
20.5.2 Randomised recurrent networks: Echo State Networks	210
20.6 Towards structured domains	211
21 Learning on structured data and graphs	213
21.1 Why structured data and graphs	213
21.1.1 The scenario	213
21.1.2 A bit of history	214
21.2 Deep Graph Networks	214
21.2.1 The graphs considered	214
21.2.2 The representation problem	215
21.2.3 Transductions on graphs	215
21.2.4 Message passing	216
21.2.5 The link with CNNs	216
21.2.6 The general formula	217
21.2.7 NN4G and GCN: iterations as layers	217
21.2.8 GNN and GraphESN: recursive approach	218
21.3 Open problems and some proposals	219
21.3.1 Efficiency: beyond end-to-end backpropagation	219
21.3.2 The problems of depth: the “over-*”	220
21.3.3 Ongoing extensions	221
21.4 Other approaches: kernels for structures	221

Chapter 1

Introduction to Machine Learning

The first lecture has a twofold purpose: to motivate the study of **Machine Learning** (ML) by placing it within the landscape of computer science and artificial intelligence, and to provide practical information about the course (prerequisites, structure, exam, materials). At the end, the basic mathematical concepts needed throughout the course are reviewed.

1.1 What is Machine Learning

1.1.1 Learning as a universal principle

Learning is a principle shared by living beings, societies and — today — machines. As Poggio and Shelton observe (*AI Magazine*, 1999), *the problem of learning is arguably at the very core of the problem of intelligence, both biological and artificial*. Understanding how we learn therefore means understanding a fundamental part of what it means to be intelligent.

Definition — Machine Learning

Machine Learning is a theoretical and applied area of computer science that combines two goals: the artificial intelligence goal of building **computers capable of learning**, and that of having **powerful adaptive and statistical tools** with rigorous foundations in the computational sciences. Operationally, it is the automatic learning, by a system, from **experience** (a set of examples), aimed at solving a computational task.

1.1.2 Why make machines learn: luxury or necessity?

ML is not a technological fad but a **necessity**, for two main reasons:

1. **Growing availability of empirical data** and the need to analyse it. Science itself is shifting its paradigm towards a *data-driven* approach, in which knowledge is extracted from data; ML plays a central, methodological role in this change.
2. **It is hard to explicitly program adaptivity and intelligence**. Alan Turing had already realised this: for many tasks writing the rules by hand is impractical, and learning becomes the only way to endow systems with intelligent behaviour.

The combination of **large amounts of data**, **computing power** (HPC, GPUs) and **flexibility of ML models** has opened a new era of artificial intelligence.

1.1.3 When ML is needed

ML is the right choice when there is **little or no prior knowledge** about a problem (we do not know how to write the rules of the solution), but it is relatively **easy to obtain experience**, i.e.

data for which we know the correct result. Classic examples are classifying emails as *spam*, and recognising characters, faces or speech.

Example — Handwritten digit recognition

Each digit is an 8×8 pixel image. Writing by hand the rules that distinguish a “3” from an “8” for every possible handwriting is practically impossible. Instead, many already-labelled examples of digits are collected and the system is left to **learn a classifier** f that maps each image to the correct class (0, 1, ..., 9).

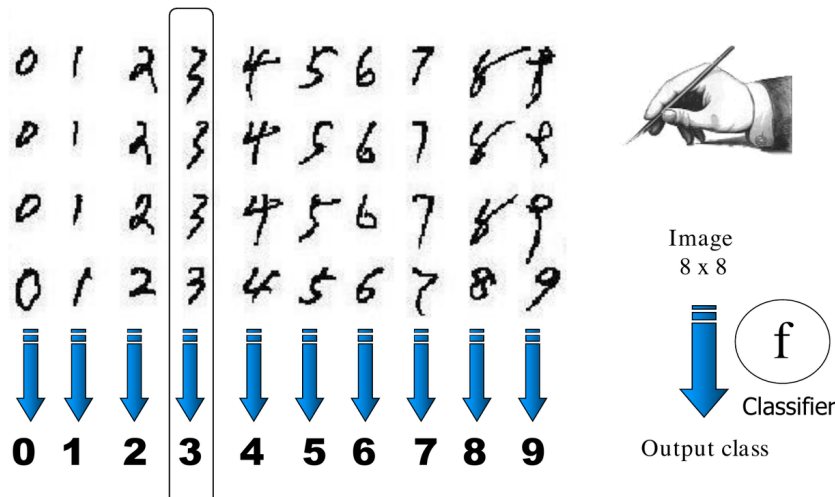


Figure 1.1: Handwritten digit recognition: the classifier f maps an 8×8 image to the output class.

1.1.4 Application areas

ML is now pervasive in real systems (from OCR to human-machine interfaces and search engines) and is the core of an interdisciplinary area that since the 1980s has included:

- **Pattern Recognition** (face and speech recognition), **Computer Vision**, **Natural Language Processing** (from text classification to speech processing up to generative AI);
- **Robotics**, adaptive systems and filters, intelligent sensor networks, personalised components;
- **Knowledge Discovery and Data Mining**, Information Retrieval, analysis of complex data (medicine, biology, chemistry, web, marketing), financial forecasting.

In some fields — NLP, Pattern Recognition, Computer Vision — the impact has been revolutionary, so much so that people speak of a “new AI”.

1.1.5 Some recent successes

The professor shows some emblematic results:

- **Face recognition (DeepFace, CVPR 2014)**: combining deep neural networks and other ML techniques, trained on four million images of more than 4000 people, the system decides whether two photos depict the same person with 97.25% accuracy, just below human accuracy (97.53%).
- **AlphaGo (DeepMind, March 2016)**: defeats the world Go champion.

- **Medicine:** a deep neural network for skin cancer classification, trained on 130,000 cases (more than a doctor sees “in many lifetimes”), reaches the accuracy of board-certified dermatologists (*Nature*, 2017). At CIML in Pisa, the BrAID project applies AI to the diagnosis of Brugada syndrome.
- **Machine translation:** Google Neural Machine Translation (since 2016) and DeepL (since 2017), entirely based on neural networks.
- **Large Language Models and generative AI:** ChatGPT (OpenAI, November 2022), Bard/LaMDA (Google), open-source models such as Dolly, and systems that generate images (Stable Diffusion), video, music.
- Autonomous driving, sensor data analysis, IoT.

ML provides the theoretical and methodological basis of the so-called *AI revolution*, with huge industrial investments. It is considered a **GPT** (*General-Purpose Technology*): a technology capable of deeply transforming pre-existing economic and social structures. The “*ML scientist*” is among the most sought-after profiles on the job market.

Note — Turing Award 2018

On 27 March 2019 the ACM awarded the Turing Award (the “Nobel prize of computing”) to **Yoshua Bengio, Geoffrey Hinton and Yann LeCun** “for conceptual and engineering breakthroughs that have made deep neural networks a critical component of computing”.

Note — Nobel Prizes 2024

In 2024 ML also reached the Nobel prizes: - **Physics:** to **John Hopfield** and **Geoffrey Hinton** for foundational discoveries and inventions that enable machine learning with artificial neural networks (the Hopfield network as associative memory, the Boltzmann machine). - **Chemistry:** to **Demis Hassabis** and **John Jumper** (Google DeepMind) for protein structure prediction with **AlphaFold**, and to **David Baker** for computational protein design. Hassabis: “*I’ve always thought that if we could build AI in the right way, it could be the ultimate tool to help scientists explore the universe around us.*”

1.1.6 The ultimate goal

Anything powerful can be dangerous if misused, but the ultimate goal of AI and ML is to **benefit people** by solving problems big and small, to **accelerate human progress** and to provide intelligence to every other scientific field: augmenting our capabilities while enhancing our humanity (P. Domingos, *Scientific American*, 2018).

1.2 Why study Machine Learning

Within the master’s degree, ML serves to:

- learn the **basic principles of learning processes**, from a computational point of view;
- learn **new computing paradigms**, such as **neural networks**: studied as a computational paradigm since the 1940s with neurobiological inspiration, today they are a set of powerful models for **function approximation** with predictive capabilities, supported by a solid theory (*learning theory*), and they are the basis of **Deep Learning**;
- be able to **apply these models rigorously**;
- have the foundations for all subsequent AI courses.

1.2.1 Modern ML

ML has moved from being a **collection of heuristic methods** (tricks coming from biology, statistics, neural networks, fuzzy logic...) to building **general conceptual frameworks**. A new understanding of the principles governing learning processes has emerged, with three important consequences:

- expressing different models in different mathematical or computer “languages” **does not change the principles**;
- different application fields **do not change the principles**;
- new technological advances **do not change the principles**.

Tip — The common thread of the course

“Machine Learning” proper is the **core of principles and methods** for building flexible and powerful computational models from data. Individual models change, principles remain: this is what is really worth learning.

1.2.2 Three views of ML

1. **As an AI methodology** → building adaptive and intelligent systems.
2. **As statistical learning** (inference of hypotheses according to mathematical principles) → building powerful predictive systems for intelligent data analysis.
3. **As a computing method for innovative application areas** → using models as tools for complex, interdisciplinary problems.

The course considers all of them, with a **critical approach**: for the various methods, their limits and appropriateness of use are discussed, with attention to the underlying principles. There are two practical goals: learning to **develop new ML models** and learning to **apply state-of-the-art methodologies** to problems in other fields.

1.3 Course information

1.3.1 Prerequisites and goals

No course is strictly required: we start from scratch. However, the typical background of a scientific bachelor’s degree is needed: **calculus** (functions, differential calculus), **matrix notation and computation**, **elements of probability and statistics**, **algorithms**.

The course introduces the principles and the critical analysis of the main paradigms (models and algorithms, with a focus on neural networks) for learning from data. Concepts are introduced progressively, from the simplest approaches up to state-of-the-art models, within the conceptual framework of modern ML: **learning functions from examples**. The focus is on:

- critical analysis of the features relevant to the design and use of ML;
- **rigorous experimental evaluation**: there is a big difference between *using* ML tools and using them *correctly*;
- computational aspects of learning systems.

1.3.2 Course structure

The course has a strong structure: it starts from simple models and arrives at the state of the art, and fundamental concepts are introduced through different models. The practical advice is twofold: (1) study the individual methods, (2) look for the **connections and comparisons** among them, so as to abstract the concepts. The common thread is understanding the relationship between **model flexibility** and **complexity control**, a fundamental concept for the ability to **generalise**.

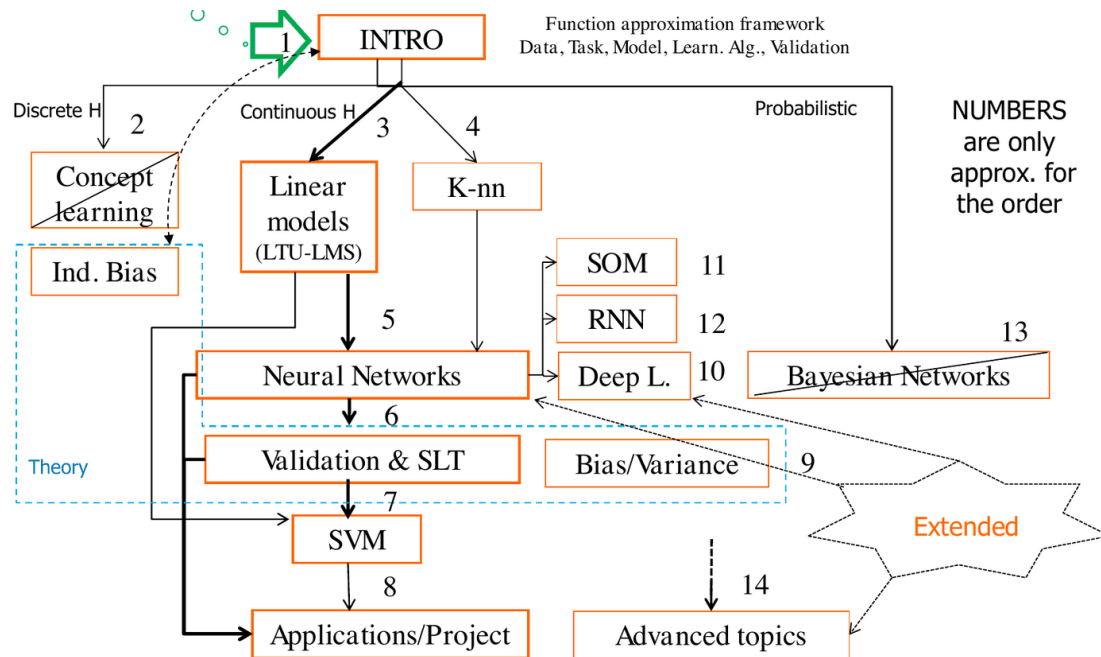


Figure 1.2: Map of the course. Numbers roughly indicate the order of the lectures; the dashed “Theory” box groups the theoretical parts.

The map is not just an ordering of the lectures: it describes the path. “**Building blocks**” (intermediate concepts and models) are introduced and then composed to build ML models — for instance, neural networks are built from linear models, and a complete ML system is built from validation techniques. For this reason the intermediate models should be seen as pieces of a puzzle that comes together at the end.

Note — Order of lectures according to the map

1. Introduction (data, task, model, learning algorithm, validation) — within the framework of **function approximation**.
2. Concept learning and inductive bias (discrete hypothesis space).
3. Linear models (LTU, LMS) (continuous hypothesis space).
4. K-nearest neighbors.
5. Neural networks.
6. Validation and Statistical Learning Theory.
7. Support Vector Machines.
8. Applications and project.
9. Bias/Variance.
10. Deep Learning (and, as an extension, SOM, RNN). 11–14. SOM, RNN, (Bayesian Networks — removed from the syllabus), advanced topics (structured data).

The density of the course is not uniform: the introduction is slow and “soft”, the core (neural networks, SVM, validation) is dense and sequential, the advanced part is fast but less dense.

The (optional) project is announced roughly halfway through the course.

1.3.3 Practical organisation

- **Lecturer:** Prof. Alessio Micheli (Department of Computer Science), full professor, head of the CIML group (*Computational Intelligence & Machine Learning*) with more than 25 years of research in ML/AI, **president of the European Neural Network Society** (annual ICANN conference), co-founder of the IEEE Task Force on Reservoir Computing, associate editor of IEEE TNNLS and *Neural Networks*. Goal of the course: to introduce current ML in 3 months, also retracing its foundations and history.
- **Lectures** (A.Y. 2026/27): Tuesday, Wednesday and Thursday 16–18, Room E. Lectures start on time so that there can be a break.
- **Q&A** with the lecturer: on Tuesday after the lecture (optional time, not part of the lecture) plus some special Q&A lectures. For personal questions, write an email with the tag **[ML-26-Q]** in the subject (mandatory).
- **Announcements:** via Teams and/or Moodle news. Lecture **recordings** are on Teams (*Shared* tab → *Recordings*); official lectures are in person only, streaming is informal and without guaranteed interaction.
- **Make-up lectures:** on Monday afternoons (announced when needed) and in the recovery period from 9 to 18 December 2026 (to be taken into account before planning Christmas holidays). The lectures of 15–17 September 2026 did not take place and will be made up.

Note — For those joining the course late

All information is on the course **Moodle** (elearning.di.unipi.it, with self-enrolment through the Unipi account): slides, bibliography and **FAQ** section. It is also worth using Teams (news and recordings) and classmates' notes. The project and the practice tests are **not mandatory** for the exam. Do not ask the lecturer about things already covered in the FAQ or in the first lecture.

1.3.4 Exam

Warning — Updated rules (A.Y. 2026/27)

The exam rules have changed compared to previous years: the **project is no longer mandatory** and the exam is based on a **mandatory written test**, with an oral exam only if needed.

The exam consists, in chronological order, of four phases.

1. Practice tests during the course (optional). At the end of each cycle of lectures, short and simple practice tests are proposed on Moodle. They serve for **self-assessment** (like homework exercises, with a score that however **does not count for the exam**), help keep up with the lectures and are evidence of active participation. They have a deadline (typically one week), but if you miss some you can continue the series. They prepare for the written test, although format, content and depth are different. They are announced only in class and with a short message on Teams.

2. Project (optional). At the end of the course (December) a project can be carried out:

- group work of **2 students**, no exceptions;
- content: implementation, application and/or analysis of an ML/neural network method or technique with **experimental evaluation** (projects coordinated with the CM course)

are possible);

- format: public presentation of a **poster** in class (one of the last lectures), plus delivery of the developed material; there is a prize for the best poster;
- after the presentation the exam can be taken in any session, but to discuss the project the two group members must take the exam **in the same session**.

The advice is not to apply for the project before having studied the course contents thoroughly. Details will be given in a dedicated lecture.

3. Written test (mandatory). It takes place after the course, in the exam sessions (from January and February):

- it is mandatory to **register** for the exam on esami.unipi.it by the deadline (check it well in advance) and to cancel the registration if you decide not to attend;
- it takes place **in person only**, in the classroom, on the date, time and room indicated on esami.unipi.it; not showing up is equivalent to withdrawing from that session;
- questions must be answered on **pre-printed sheets** with space for free-form answers; bring a pen or pencil;
- answers require **equations, drawings and/or descriptive parts** (quizzes and open questions);
- no course or external material may be used, nor any electronic aid (virtual assistants, chatbots...): the use of any device (laptops, smartphones, smart bands, earphones, smart glasses...) is forbidden, under penalty of exclusion.

4. Oral exam (only if needed) and grade registration. Access to the oral exam requires a sufficient level in the written test (the list of admitted students is published on Moodle). The oral exam is held in the same session as the written test, in person, typically in the lecturer's office: in the days following the grading, the lecturer assigns a slot to each student or group. If needed (depending on the outcome of the written test), the oral exam is an interview on **the whole syllabus** and may include discussion of the written test and of the project. Special needs (priority, inability to attend in person...) are evaluated case by case upon request.

Summary — Summary of the rules

The evaluation takes into account: **result of the written test** + active participation (including tests during the course) + optional extra activities (project, voluntary presentations at the end of the course) + a possible full oral exam, if requested by the lecturer.

- **Sufficient** written test → in a slot a few days later: possible oral exam and **grade proposal** → if accepted, registration.
- **Insufficient** written test or grade not accepted → repeat in a later session, chosen among the many during the year.

Completing 80% of the online tests and a very good project may give a **small bonus** (if the written test is already sufficient or good), but they are **not necessary** either to take the exam or to obtain the maximum grade. The key indicator is the **final knowledge of ML and its depth**.

Tip — The professor's advice

- Follow the lectures using the slides as a guide and study progressively during the course; Q&A lectures are a discussion forum: this is a class, not a set of recordings.
- First **study the course contents**, then work on the project: it increases both the

effectiveness and the efficiency of the work.

- If something is unclear: don't panic, ask the lecturer (Q&A), collaborate with classmates, use books and papers (bibliography and personal research) and the course support tools. ML is a continuously evolving field: it must be studied!

1.3.5 Materials and bibliography

The main reference is the course slides (on Moodle, often published in advance; make sure to use the latest version of the ML-26...-v* files), which however must be complemented with your own notes (the slides alone may not be enough to reconstruct the thread of the lecture) and with books. The lecturer is willing to share students' notes on Moodle. The main textbooks are:

- S. Haykin, *Neural Networks and Learning Machines*, Prentice Hall, 3rd ed., 2008;
- T. M. Mitchell, *Machine Learning*, McGraw-Hill, 1997;
- I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press, 2016 (free online).

Other useful references: Russell–Norvig (*AIMA*), Hastie–Tibshirani–Friedman (*The Elements of Statistical Learning*, free copy online), Cherkassky–Mulier, Bishop (*Pattern Recognition and Machine Learning*, now **free** from the publisher's website; and *Neural Networks for Pattern Recognition*), Duda–Hart–Stork, Shalev-Shwartz–Ben-David (*Understanding Machine Learning*, free online).

For **software libraries**: F. Chollet, *Deep Learning with Python*; A. Géron, *Hands-on Machine Learning with Scikit-Learn, Keras & TensorFlow*; A. Zhang, Z. Lipton, M. Li, A. Smola, *Dive into Deep Learning* (Cambridge University Press, 2023, free online, with examples in PyTorch, NumPy/MXNet, JAX and TensorFlow). The main libraries will be introduced later by the course assistant. Beware: there are a great many technical books and blogs, and one must pay attention to scientific quality.

Note — Course support

- A **course assistant** to help with the development of the project (to be confirmed).
- An experimental **tutor chatbot** (based on ChatGPT) for the ML 2026 course: it helps clarify doubts, deepen concepts and get guidance on slides, definitions and exercises, always based on the official course materials. Details and links will be published in the Moodle news.

In parallel, the course **Computational Mathematics for Learning and Data Analysis** (CM) is very useful, as it provides the deep mathematical foundations of learning methods (it is not, however, mandatory for following ML).

1.4 Mathematical background

To follow the course some basic concepts are needed: multivariable calculus (partial derivatives, gradient), probability (densities, mean, variance, normal distribution, conditional and joint probabilities), matrix calculus (dot product, inverse, norms). Topics such as the linear least-squares problem with the SVD and Lagrange multipliers can be studied in parallel. Recommended references: appendix A of *AIMA*, chapters 2–4 of the *Deep Learning book*, and *Mathematics for Machine Learning* (2020).

The notation used is: x scalar, $\mathbf{x}$ vector, X matrix.

1.4.1 Dot product and norms

Definition — Dot product (inner/dot product)

$$\mathbf{a} \cdot \mathbf{b} = a_1 b_1 + a_2 b_2 + \cdots + a_n b_n = \sum_{i=1}^n a_i b_i = |\mathbf{a}| |\mathbf{b}| \cos \theta$$

Equivalent notations: $\mathbf{a} \cdot \mathbf{b} = \mathbf{a}^T \mathbf{b} = \mathbf{a}^t \mathbf{b} = \langle \mathbf{a}, \mathbf{b} \rangle$, and even just $\mathbf{ab}$ if the context is clear.

The relation with the cosine gives the geometric intuition: the dot product measures **how much two vectors point in the same direction**. It is maximal when they are parallel, zero when they are orthogonal, negative when they point in opposite directions. This reading will come up again and again (for example in linear models, where $\mathbf{w}^T \mathbf{x}$ measures the “alignment” of the input with the weights).

The **Euclidean norm** (modulus, length) of a vector is

$$\|\mathbf{x}\|_2 = \sqrt{\mathbf{x}^T \mathbf{x}} = \sqrt{\sum_i x_i^2} = d(\mathbf{x}, \mathbf{0}),$$

often written simply as $\|\mathbf{x}\|$ or $|\mathbf{x}|$. Other useful norms are the L^1 **norm**, $\|\mathbf{x}\|_1 = \sum_i |x_i|$, and the **max norm**, $\|\mathbf{x}\|_\infty = \max_i |x_i|$.

More generally, an **inner product** $\langle \cdot, \cdot \rangle$ is a symmetric bilinear form that maps a pair of vectors to a scalar:

$$\langle v, w \rangle = \langle w, v \rangle, \quad \langle v + w, u \rangle = \langle v, u \rangle + \langle w, u \rangle, \quad \langle kv, w \rangle = k \langle v, w \rangle.$$

The **Cauchy-Schwarz inequality** holds: $|\langle x, y \rangle| \leq \|x\| \cdot \|y\|$ for all $x, y \in V$.

Note — Tensors

A **tensor**, in the simplified view used in ML, is a multidimensional array of numbers arranged on a regular grid with a variable number of axes (for example $X_{i,j,k}$ has 3 indices). In ML it is used mainly as a notational convenience and to reason about GPU computation; in mathematics tensors have richer properties (related to changes of basis).

1.4.2 Partial derivatives and gradient

The **partial derivative** generalises the derivative to functions of several variables: it is the derivative with respect to a single variable, keeping all the others fixed. For $f(x_1, x_2, x_3)$ one computes $\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \frac{\partial f}{\partial x_3}$.

Definition — Gradient

The **gradient** of f is the vector of its partial derivatives:

$$\nabla f = \text{grad } f = \left(\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n} \right) = \sum_{i=1}^n \frac{\partial f}{\partial x_i} \mathbf{e}_i$$

where the $\mathbf{e}_i$ are the orthogonal unit vectors along the coordinate directions.

Its geometric meaning is fundamental for the whole course: at a point, the gradient is a vector that **points in the direction of steepest ascent**, and its norm indicates **how steep** the slope

is. The gradient therefore indicates where the function grows fastest, while its opposite $-\nabla f$ indicates the direction in which it **decreases** fastest. This is exactly the idea behind **gradient descent**, which we will use to train models by minimising an error function.

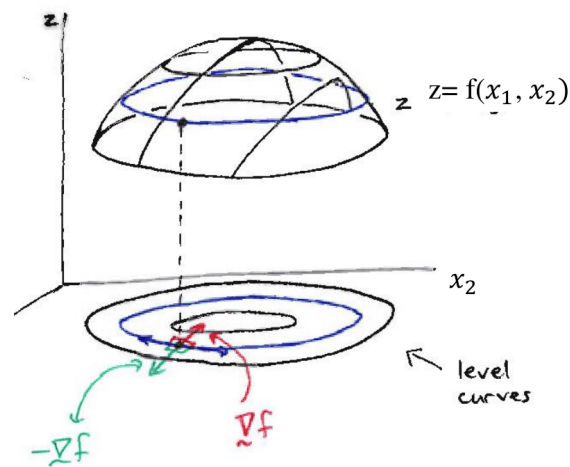


Figure 1.3: Gradient on a surface: ∇f points towards the increase of f , $-\nabla f$ towards its decrease; both are perpendicular to the level curves.

A point where the gradient is zero is called a **stationary point**. It can be a local minimum, a local maximum or a saddle point: a zero gradient alone is not enough to tell which case it is.

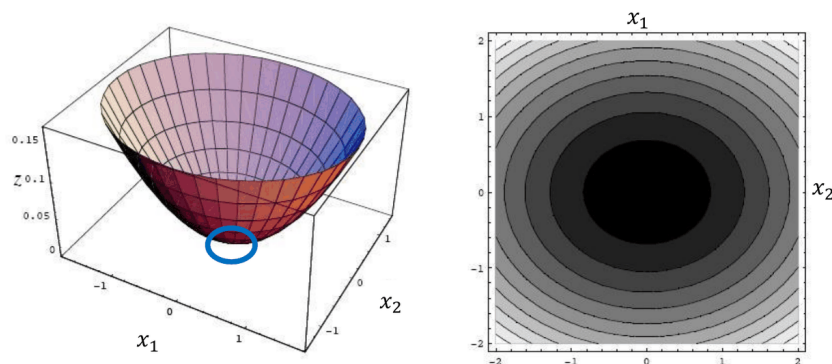


Figure 1.4: A minimum: at the lowest point of the paraboloid the gradient is zero.

1.4.3 Probability density: the Gaussian

The **normal distribution** with mean μ and variance σ^2 has probability density (the “Gaussian function”)

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-(x-\mu)^2/(2\sigma^2)}.$$

With $\mu = 0$ and $\sigma^2 = 1$ we obtain the **standard normal**.

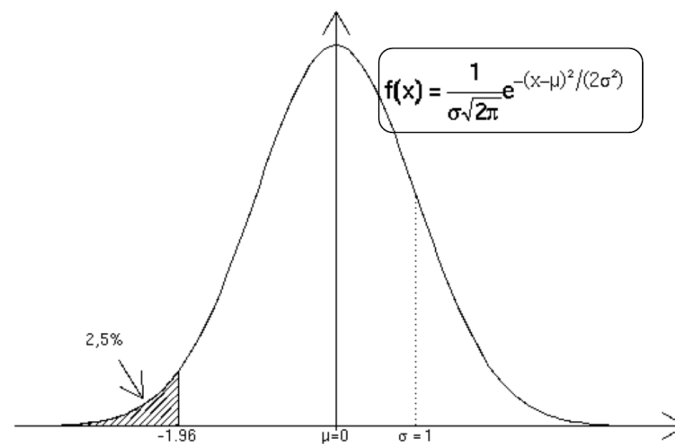


Figure 1.5: Density of the standard normal: the area to the left of -1.96 is 2.5%.

1.5 Summary and next steps

Summary — Summary

ML is the automatic learning of a task from examples: it is necessary when data are abundant but explicit rules are missing. The course approaches it as **function approximation from examples**, building models through successive “building blocks”, paying attention to the principles (which do not change as models change) and to rigorous evaluation. The key concept running through the whole course is the trade-off between **model flexibility** and **complexity control**, on which **generalisation** depends.

The following lectures complete the introduction: lectures 2–3 cover the usefulness of ML, learning as function approximation with a guiding example, and the design components of an ML system (task, hypothesis space, inductive bias, loss function, generalisation); lecture 4 goes deeper into generalisation and validation.

Questions — Possible exam questions

- Why is ML a necessity and not a luxury? Under which conditions is it appropriate to use it?
- What is the gradient and why does $-\nabla f$ indicate the direction of steepest descent?
- What is a stationary point? Is it always a minimum?

Chapter 2

ML in the master's curricula

This slide deck is an “interlude” of the first lecture, which the professor asks students to read on their own. It describes how the Machine Learning course fits into the **Master's Degree in Computer Science** at the University of Pisa and its curricula. It contains no exam material, but it helps to understand the role of the course within the study programme.

2.1 The master's curricula

Since 2017 the master's degree in Computer Science has been organised into four curricula:

- **Artificial Intelligence (AI);**
- **Data and Knowledge**, since 2021 **Big Data Technologies (BD);**
- **ICT Solutions Architect (ICT);**
- **Software: Programming, Principles, and Technologies (SW).**

The organisation into curricula allows students to specialise in an area (with a diploma supplement certifying the curriculum), offers basic methodological courses at the beginning of the programme and aims to show how two more years of study are useful for one's professional future. Regulations, exam instructions and procedures for changing curriculum or study plan are on the department website.

2.2 The role of Machine Learning

ML is connected to **all four** curricula, but it is a **core course of the AI curriculum**, where it is part of the methodological basis for building adaptive and intelligent systems, together with:

- *Artificial Intelligence Fundamentals* (6 ECTS);
- *Computational Mathematics for Learning and Data Analysis* (9 ECTS);
- *Machine Learning* (9 ECTS);
- *Parallel and Distributed Systems: Paradigms and Models* (9 ECTS).

The other core courses of the AI curriculum are *Human Language Technologies*, *Intelligent Systems for Pattern Recognition* and *Smart Applications*, alongside groups of elective exams (Algorithm Engineering, Data Mining, Mobile and Cyber-Physical Systems, Information Retrieval, Computational Neuroscience, Robotics, Semantic Web, etc.).

ML is the pivot of the **intelligent systems area**: many master's courses (Human Language Technologies, Intelligent Systems for Pattern Recognition, Smart Applications, Computational Neuroscience, Robotics, Data Mining, Information Retrieval, Bioinformatics, Big Data Analytics...) directly use its methods.

Note — Study plan of the AI curriculum (since 2021)

60 ECTS of core courses, 27 ECTS from groups of elective exams (one 9-ECTS exam and three 6-ECTS exams) and at least 9 ECTS of free choice, which can also be covered with two 6-ECTS exams.

2.3 Synergy with Computational Mathematics

The professor recommends following **ML** and **CM** (*Computational Mathematics for Learning and Data Analysis*) in parallel: the former provides the methods, the latter the mathematical foundations, and the two courses stimulate each other. CM is however not a prerequisite: many students from other degree programmes or curricula have taken ML without CM.

2.4 Notes for specific categories of students

Warning — For those coming from the Pisa bachelor's degree (IIA course)

The first six lectures will seem “easy”, but they **do not coincide** with the contents of IIA: they contain new parts and different mathematical concepts. The change of perspective is sharp: in IIA simple models were the *tool* for introducing ML concepts, here state-of-the-art models and principles are the *object* of study. From lecture 5–6 onwards everything is new.

For students of the old **AA1** exam (320AA, 6 ECTS), which no longer exists: those who need to switch from AA1 to ML (9 ECTS) take a supplementary exam on a list of topics indicated by the professor; those who need to do the opposite must contact the lecturer.

For any information the reference is Prof. Alessio Micheli (micheli@di.unipi.it), of the *Computational Intelligence & Machine Learning* group of the Department of Computer Science.

Chapter 3

Fundamental concepts of ML

Lectures 2 and 3 provide the **overview and terminology** of Machine Learning before getting into actual models. We start from the general picture — learning as the approximation of an unknown function from examples — and then describe the “ingredients” of an ML system (data, task, model, learning algorithm, validation) and three crucial concepts: **inductive bias**, the **loss function** and **generalisation**. Many concepts only hinted at here will be taken up again and explored in the rest of the course.

3.1 Machine Learning in the computational setting

Restricting ourselves to the computational framework, ML studies **principles, methods and algorithms for learning and prediction**: the system learns from experience (known data) to tackle a computational task, building a **model** (or **hypothesis**) to be used for making predictions.

The most widespread specific framework, and the one adopted by the course, is the following: **infer a model, i.e. a function, from a set of examples, so that it is able to generalise**, that is, to give accurate answers on new data.

3.1.1 When to use ML

ML should be used with **opportunity** (when it is really needed) and **awareness** (of its requirements and limits). Predictive models are useful when:

- there is no theory (or little knowledge) explaining the phenomenon;
- data are **uncertain, noisy or incomplete**, and this hinders the formalisation of an exact solution.

In exchange, ML requires two things: a **source of experience** for training (*representative* data) and a certain **tolerance on the precision** of the results, which will never be guaranteed to be exact.

ML is complementary to traditional techniques (analytical models based on prior knowledge, algorithms and imperative programming, classical AI). It is particularly suitable when:

- **knowledge is too hard to formalise** by hand: humans recognise faces but cannot describe *how* they do it; the same holds for speech recognition;
- **human knowledge is not enough**: for example, predicting the binding strength of molecules to proteins;
- **personalised behaviour** is needed: filtering emails or web pages according to user preferences, individualised human–machine interfaces.

The great challenges of ML are building **intelligent autonomous systems** (robotics, human–robot interaction, search engines), providing **powerful tools for data analysis** (the

tools of the *data scientist*) and opening **new interdisciplinary application areas**, in an era in which scientific progress is increasingly *data-driven*.

3.2 Learning as function approximation

The course adopts a specific but very widespread view: **learning is the approximation of an unknown function from examples**. The advantage is that different tasks (classification, regression, ...) are treated in a uniform framework — mathematically, that of function spaces (Hilbert spaces) — which allows a rigorous formulation.

3.2.1 The guiding example: digit recognition

Let us go back to the example of the first lecture. The input is a collection of images of hand-written digits (8×8 matrices of values); the problem is to build a model that, given an image, **predicts** the digit. This is a **classification problem** with ten classes.

It is a typical ML success case: it is **hard to formalise exactly** the solution (handwriting varies, there is noise and there are ambiguous data), while it is **relatively easy to collect labelled examples**.

Definition — Machine Learning (extended definition)

ML studies and proposes methods for building (**inferring**) dependencies, functions or hypotheses from examples of observed data, that: - **fit** the known examples; - are able to **generalise**, with reasonable accuracy, to new data; according to **verifiable** results, under statistical and computational conditions and criteria, taking into account the **expressiveness** and **algorithmic complexity** of the models and learning algorithms.

3.2.2 Examples of pairs $x \rightarrow f(x)$

In all the following cases we want to infer a general function from known data, i.e. from a **training set** (TR) of pairs $\langle x, f(x) \rangle$:

Problem	Input x	Output $f(x)$
Handwriting recognition	pen movement data	letter of the alphabet
Medical diagnosis	patient properties (symptoms, tests)	disease or recommended therapy
Face recognition	bitmap of the face	name of the person
Spam detection	email message	spam / not spam
<i>Protein folding</i>	amino acid sequence (variable-length string)	3D coordinates of the atoms (sequence of vectors)
<i>Drug design</i>	molecule (graph of atoms and bonds)	binding strength to HIV protease (real number)

The last two examples show that inputs and outputs can be **complex data**: strings, sequences, graphs.

3.3 The ingredients of an ML system

A predictive ML system can be described through its components, which also constitute a **guide to the key design choices**.

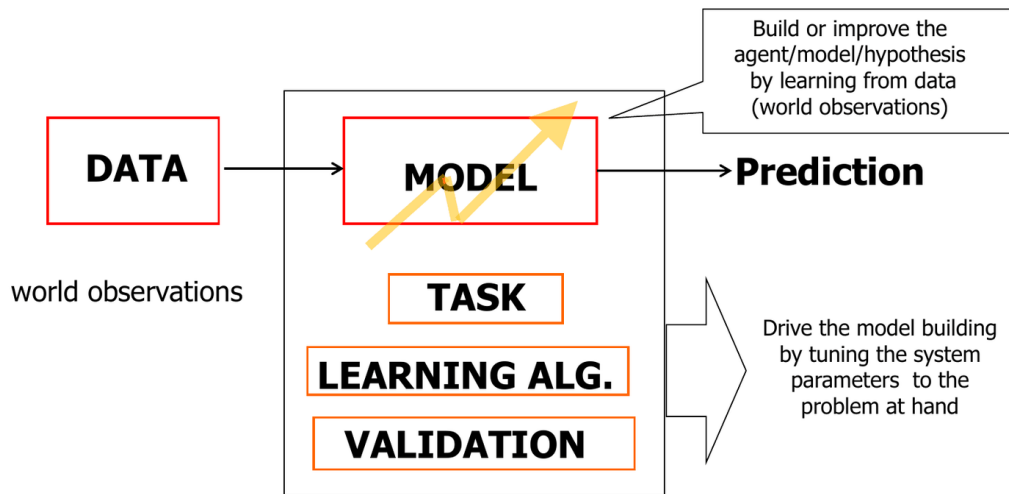


Figure 3.1: Overview of a predictive ML system and its “ingredients”.

- **Data:** the observations of the world, i.e. the available experience.
- **Task:** the purpose of the application, which defines what we want to obtain.
- **Model:** the agent/hypothesis that is built or improved by learning from data.
- **Learning algorithm:** drives the construction of the model by tuning the system’s parameters to the problem.
- **Validation:** evaluates the quality of the obtained model.

3.3.1 Data

Data represent the available facts. The first problem is that of **representation**: how to capture the structure of the objects being analysed.

In the **flat** case (attribute-value language) each object is a **fixed-size vector of properties** (*features*), and the whole dataset is a table of tuples. Attributes can be **categorical/discrete** (colour: yellow, red) or **continuous** (weight, cost), and there may be **missing data** (denoted by “?”).

Note — Data terminology

- Each row $\mathbf{x}$ (vector, in bold) is an **example**, *pattern*, instance, sample.
- The **dataset size** is the number of examples, denoted by l .
- The **input dimension** is the number of features, denoted by n .
- x_i (or x_j) is typically the i -th feature (attribute, component) of $\mathbf{x}$.
- $\mathbf{x}_p$ is typically the p -th pattern (row) of the dataset.
- $x_{p,i}$ is attribute i of pattern p .

Data can be **pre-processed**: variable scaling, encoding, feature selection.

Encoding categorical variables

To use categorical variables in numerical models they must be encoded:

- for **two classes** one uses 0/1 or $-1/+1$;
- for **more classes** one can use 1, 2, 3, ..., but this encoding introduces an artificial **degree of similarity** (class 1 turns out to be “closer” to 2 than to 3). It is suitable only for **ordered categorical** variables (e.g. small, medium, large);
- for symbols without an order one uses the **1-of-k** (or **one-hot**) encoding: each symbol becomes a vector with a single 1, for example $A = (1, 0, 0)$, $B = (0, 1, 0)$, $C = (0, 0, 1)$. All symbols are thus equidistant from each other. It is used for both input and output, and it is useful for the project.

Structured data

Many data are not naturally vectors: **sequences** (lists), **trees**, **graphs**, multi-relational data. Examples: images, time series, strings of a language, DNA and proteins, hierarchical relations, molecules, the link network of web pages. The problem is finding a natural representation for them (this will be discussed in the advanced part of the course).

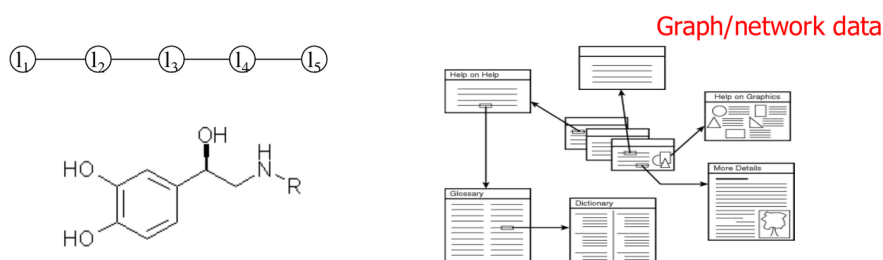


Figure 3.2: Structured data: sequences, molecules and graphs.

Noise, outliers and feature selection

- **Noise** is the addition of external factors to the informative signal: it is due to the randomness of measurements, not to the underlying law (e.g. Gaussian noise).
- **Outliers** are unusual values, inconsistent with most observations (e.g. anomalous measurement errors). They are handled by detecting and removing them in pre-processing or by using **robust** modelling methods.
- **Feature selection** selects a small number of informative features, providing an optimal input representation for the problem.

3.3.2 Task

The task defines the purpose of the application: what knowledge we want to obtain, what the useful nature of the result is, what information is available. We distinguish:

- **predictive tasks** (classification, regression): function approximation — they are the focus of the course;
- **descriptive tasks** (cluster analysis, association rules): finding subsets or groups of unclassified data.

Supervised learning

Definition — Supervised learning

Data: training examples of the form $\langle \text{input}, \text{output} \rangle = \langle \mathbf{x}, d \rangle$ (**labelled** examples) for an unknown function f , known only at the example points. The **target** d (also denoted by t or y) is provided by a “teacher” according to $f(\mathbf{x})$.

Goal: find a good approximation of f , i.e. a **hypothesis** h that can be used to predict on unseen data $\mathbf{x}'$ — a hypothesis that **generalises**.

The target can be:

- **categorical** $\rightarrow$ **classification**: $f(\mathbf{x}) \in \{1, 2, \dots, K\}$ (discrete-valued function);
- **numerical** $\rightarrow$ **regression**: $f(\mathbf{x}) \in \mathbb{R}$ or $\mathbb{R}^K$ (continuous real-valued function).

Thanks to the function approximation formalism, classification and regression have a **unified view**. In statistical terminology, inputs are the **independent variables** and outputs the **dependent variables** or **responses**.

Unsupervised learning

In **unsupervised** learning there is no teacher: the training set contains only **unlabelled** data $\langle \mathbf{x} \rangle$. The goal is for example to find **natural groupings** in the data (**clustering**, i.e. partitioning the data into subsets of “similar” elements), to perform **dimensionality reduction**, visualisation or pre-processing, or to **model the density** of the data.

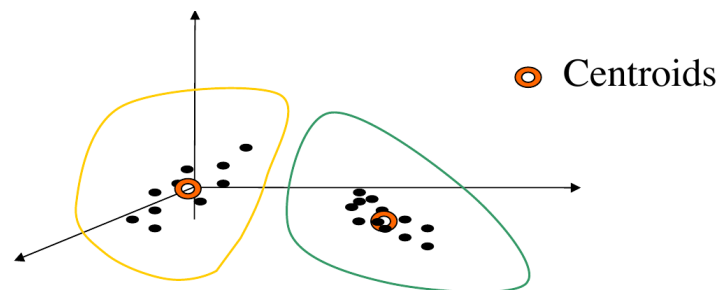


Figure 3.3: Clustering: partition of the data into groups, each represented by its centroid.

Classification

In classification, patterns are seen as members of a class, and the goal is to assign each pattern to the correct class. If there are **two** classes, $f(\mathbf{x})$ is a **Boolean function**: we speak of **binary classification** or **concept learning** (true/false, 0/1, $-1/+1$, negative/positive). With **more than two** classes we have a **multi-class** problem.

Geometrically, classification can be seen as the **partition of the input space into decision regions**. The simplest example is the linear separator in $\mathbb{R}^2$: a line (in general a hyperplane) that separates the points of class 1 from those of class 0.

The separating hyperplane is the set of points $\mathbf{x}$ such that

$$\mathbf{w}^T \mathbf{x} + w_0 = w_1 x_1 + w_2 x_2 + w_0 = 0,$$

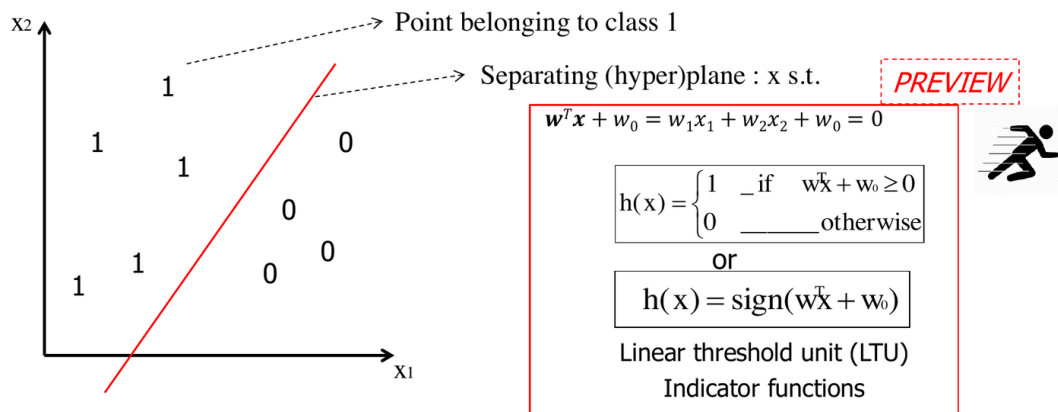


Figure 3.4: A linear separator in the plane: a preview of the Linear Threshold Unit.

and the classifier is

$$h(\mathbf{x}) = \begin{cases} 1 & \text{if } \mathbf{w}^T \mathbf{x} + w_0 \geq 0 \\ 0 & \text{otherwise} \end{cases} \quad \text{or} \quad h(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x} + w_0).$$

This model is called **Linear Threshold Unit (LTU)** and will be studied in detail later. Functions of this kind, with values in $\{0, 1\}$, are called **indicator functions**. The set H of all dichotomies (splits into two classes) induced by hyperplanes is its hypothesis space.

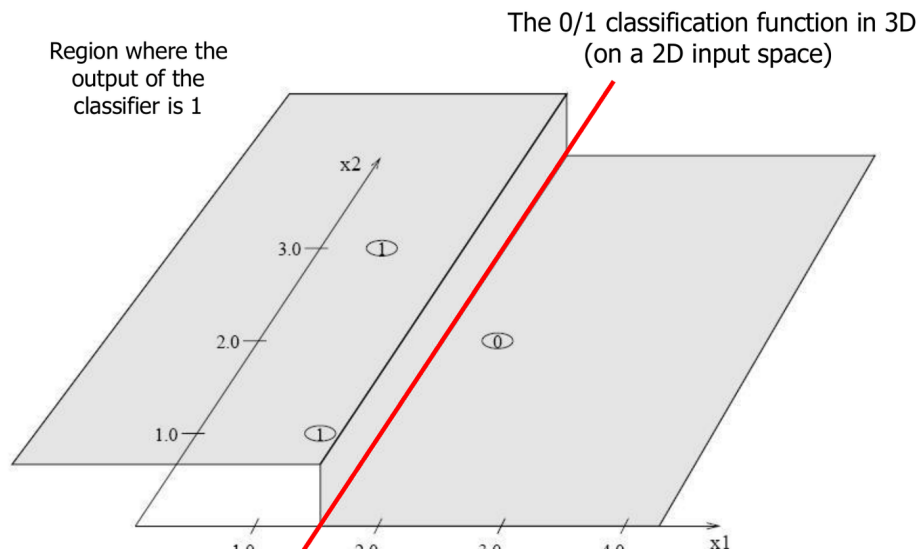


Figure 3.5: The same classification function seen in 3D: it is a “step” function that is 1 in one region and 0 in the other.

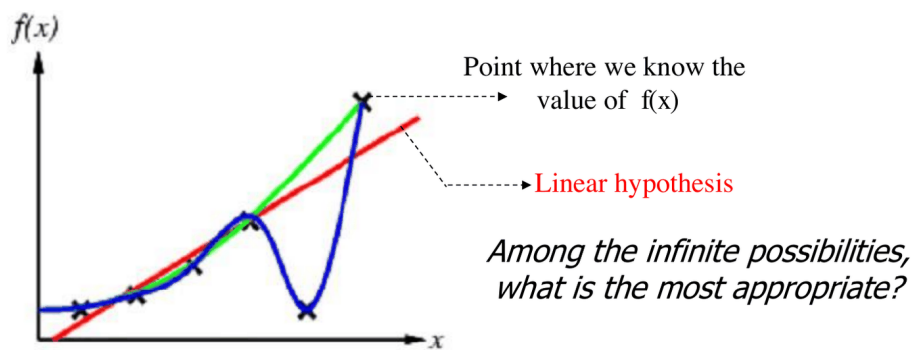
Regression

Regression is the estimation of a real-valued function on the basis of a finite set of **noisy samples**: pairs $(\mathbf{x}, f(\mathbf{x}) + \text{noise})$ are known. It is a form of *curve fitting* (in one dimension in the example, in k dimensions in general).

Example — Regression exercise

Given the points $(1; 2.1)$, $(2; 3.9)$, $(3; 6.1)$, $(4; 8.4)$, $(5; 9.8)$, what is f ? It comes naturally to hypothesise $f(x) = 2x$, which has small errors on all points. But how did we choose it? And how would a machine (for example a neural network) do it? This is exactly what the

learning algorithm must formalise.



An example (**linear hypothesis**): $h_w(x) = w_1x + w_0 = 0.2x - 0.4$

Figure 3.6: Among the infinitely many functions compatible with the data, which is the most appropriate?

The figure poses the key question: among infinitely many possibilities, which function should we choose? The blue curve passes exactly through all points but oscillates unnaturally; the red line ($h_w(x) = w_1x + w_0 = 0.2x - 0.4$) makes small errors but is much simpler. This dilemma is the heart of the problem of **generalisation**.

Other paradigms

- **Semi-supervised learning:** combines labelled and unlabelled examples.
- **Self-supervised learning:** a form of supervised learning in which the labels are **generated automatically from the structure of the data itself**, without manual labelling. The model is **pre-trained** on large amounts of unlabelled data, learning general representations, which are then adapted to specific tasks via **fine-tuning**. Current examples, at the basis of LLMs:
 - **next-word prediction** in a text, i.e. the **autoregressive** self-supervised learning of the GPT family (*Generative Pre-trained Transformer*, OpenAI);
 - reconstruction of **masked words** (BERT, Google);
 - for images, **masked autoencoders** (Meta), which reconstruct hidden portions of the image.
- **Reinforcement learning:** learning with a “critic” that says right/wrong. The algorithm learns a **policy** on how to act given observations of the world; every action has an impact on the environment, which provides feedback. There are no step-by-step examples; the goal is *decision-making*, and it is widely used in modern AI.

3.3.3 Model

The **model** captures the relationships among data (according to the task) through a “language” (numerical, symbolic, ...), tied to the representation used. Above all, **the model defines the class of functions the machine can implement: the hypothesis space**. For example, the set of functions $h(x, \mathbf{w})$ as the parameter $\mathbf{w}$ varies.

Definition — Basic terminology

- **Training example** (supervised): pair $(\mathbf{x}, f(\mathbf{x}) + \text{noise})$, where $\mathbf{x}$ is the feature vector and $d = f(\mathbf{x}) + \text{noise}$ is the **target**.
- **Target function** f : the true (unknown) function.
- **Hypothesis** h : a proposed function, believed to be similar to f ; an expression in a given language describing the relationships among the data.
- **Hypothesis space** H : the set of all hypotheses that the learning algorithm can, in principle, output.

Some examples of different representations:

- **Linear models**: H is a **continuously** parameterised space, and each assignment of $\mathbf{w}$ is a different hypothesis. Examples: the binary classifier $h(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x} + w_0)$ or simple linear regression $h_{\mathbf{w}}(x) = w_1 x + w_0$ (e.g. $2x + 150$).
- **Symbolic rules**: H is **discrete**, for example “if $x_1 = 0$ and $x_2 = 1$ then $h(\mathbf{x}) = 1$, otherwise 0”.
- **Probabilistic models**: estimate $p(\mathbf{x}, y)$.
- **K-nearest neighbors** (regression): predicts the mean of the y values of the nearest neighbours (memory-based model).
- **Neural networks**: beyond the biological inspiration, they are a computational model capable of approximating complex and **non-linear** relationships between input and output. They too, once again, define a **class of functions**.

The main paradigms (languages for H) are:

Paradigm	Examples
Symbolic / rule-based (discrete H)	conjunctions of literals, decision trees, inductive grammars, evolutionary algorithms, Inductive Logic Programming
Sub-symbolic (continuous H)	linear discriminant analysis, multiple linear regression, LTU, neural networks, kernel methods (SVM)
Probabilistic / generative	traditional parametric models, Bayesian networks, Naïve Bayes, Markov models, HMM
Instance-based	nearest neighbor

Some models can be expressed in several different languages.

Theorem — No Free Lunch Theorem

There is no universally “best” learning method (without any knowledge, for all problems): if an algorithm achieves superior results on some problems, it must pay for it with inferior results on others (Devroye 1982; Wolpert and Macready 1997).

This is why the course provides both a **set of models** and the **critical tools to compare them**. However, models are *not* all equivalent: they differ greatly in **flexibility** (the ability, in principle, to approximate arbitrary functions, not only linear ones) and in **complexity control**. Flexible models and principles to control their complexity are the core of ML.

3.3.4 Learning algorithm

The learning algorithm, based on data, task and model, performs a **(heuristic) search in the hypothesis space H** to find the best hypothesis, i.e. the best approximation of the unknown target function. Typically it looks for the h with **minimum error**: the free parameters of the model are fitted to the task (the best $\mathbf{w}$ in linear models, the best rules in symbolic models...). In the regression example, the choice $w_1 = 2$, $w_0 = 0$ is the result of this search.

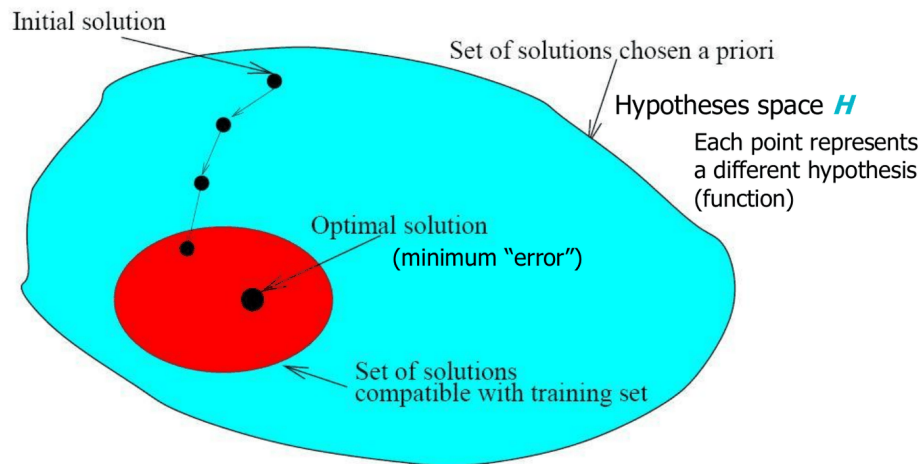


Figure 3.7: Learning as search in the hypothesis space: each point is a different function; one starts from an initial solution and moves, typically with local search, towards the minimum-error one.

H may not coincide with the set of all possible functions and the search cannot be exhaustive: **assumptions** are needed, and this is where **inductive bias** comes into play.

Depending on the context, “learning” takes different names: **inference** (statistics), **abduction/induction** (logic), **adaptation** (biology, systems), **optimisation** (mathematics), **training** (neural networks), **function approximation** (mathematics), regression, *curve fitting*.

After these four ingredients, three concepts remain to be discussed: **inductive bias**, **loss** and **generalisation**.

3.4 Inductive bias

To build a model and a learning algorithm we must make **assumptions** about the nature of the target function. These can concern:

- **constraints on the model**, i.e. on the hypothesis space H (the set of hypotheses we can express): this is the **language bias**;
- **constraints or preferences in the learning algorithm**, i.e. in the search strategy: this is the **search bias**;
- both.

We will see that these assumptions are **indispensable** for obtaining a useful model, i.e. one capable of generalising. We study it with examples in discrete hypothesis spaces, learning a **concept** (a Boolean function), as in Mitchell ch. 2 (e.g. $h_{cat}(\mathbf{x}) = 1$ if the animal $\mathbf{x}$ is a cat, 0 otherwise).

3.4.1 Learning Boolean functions: an ill-posed problem

Suppose we want to learn an unknown Boolean function $y = f(x_1, x_2, x_3, x_4)$ from 7 examples.

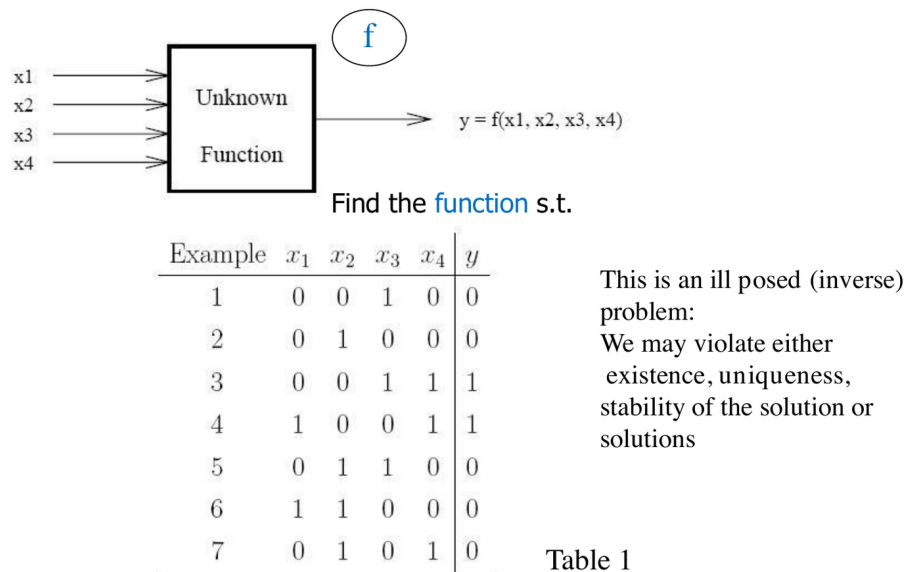


Figure 3.8: Learning a Boolean function of four variables from seven examples.

This is an **ill-posed (inverse) problem**: the solution may violate existence, uniqueness or stability. With 4 binary inputs there are $2^4 = 16$ possible input instances and hence $2^{16} = 65536$ possible Boolean functions. We cannot know which one is correct until we see **all** input-output pairs: after 7 examples there are still $2^9 = 512$ possibilities (the 9 unseen instances can have any output).

In general, for binary inputs and outputs with input dimension n :

$$|H| = 2^{\text{\#input instances}} = 2^{2^n}.$$

A model that uses this whole space is a **lookup table**: a *rote learner* that memorises the examples and classifies $\mathbf{x}$ only if it coincides with an already seen example (otherwise “no answer”). **No inductive bias $\rightarrow$ no generalisation.**

3.4.2 Restricting the space: conjunctive rules

Let us introduce a **language bias**: we only consider **conjunctive rules**, i.e. conjunctions (AND) of literals, such as $h_1 = l_2$, $h_2 = l_1 \wedge l_2$, $h_3 = \text{true}$, $h_4 = \neg l_1 \wedge l_2$. For example “if $x_2 = 1$ then $h(\mathbf{x}) = 1$, otherwise 0”.

The number of semantically distinct hypotheses collapses: for each of the n positions we can have l_i , $\neg l_i$ or “don’t care”, giving 3^n hypotheses, plus 1 because all conjunctions containing $l_i \wedge \neg l_i$ are equivalent to “false”. With $n = 4$ we go from 65536 to $3^4 + 1 = 82$ hypotheses.

Definition — Consistency and Version Space

A hypothesis h is **consistent** with the training set TR if $h(\mathbf{x}) = d(\mathbf{x})$ for every example $\langle \mathbf{x}, d(\mathbf{x}) \rangle \in TR$.

The **version space** $VS_{H,TR}$ is the subset of hypotheses of H consistent with all training examples.

In this reduced space there are efficient algorithms (Mitchell ch. 2) that find the version space without enumerating all hypotheses.

3.4.3 The unbiased learner

The conjunctive assumption, however, is too restrictive: if the target concept is not in H it cannot be represented. For example “if $x_1 = 1$ **or** $x_2 = 1$ then 1” is not a conjunction.

The opposite idea is to choose an H capable of expressing **every teachable concept**: the set of all subsets of the input space X , i.e. the power set $\mathcal{P}(X)$ (disjunctions, conjunctions, negations). With $n = 10$ binary inputs, $|X| = 2^{10} = 1024$ and $|\mathcal{P}(X)| = 2^{1024} \approx 10^{308}$ concepts — more than the atoms in the universe. H surely contains the target concept. But what happens to generalisation?

Theorem — A learner without bias cannot generalise

With an *unbiased learner*, the only instances classified unambiguously by the version space are the training examples themselves: once again we obtain a **lookup table**.

Proof. Let x_i be an unseen (test) instance and let $h \in VS$. Consider h' identical to h everywhere except at x_i , where $h'(x_i) \neq h(x_i)$. Since H contains *all* functions, $h' \in H$; and since h and h' agree on the training set, also $h' \in VS$. Therefore for every hypothesis in the VS that classifies x_i as positive there is another that classifies it as negative: every unseen instance is classified 1 by exactly half of the hypotheses in the VS and 0 by the other half. The VS cannot give any answer.

Example — Trivial example

A single binary input x , $H = \{x, \neg x, 0, 1\}$ (all possible functions). The TR contains only $x = 0 \mapsto d = 0$. The version space is $\{x, 0\}$: both are 0 at $x = 0$. For the test $x = 1$, however, x answers 1 and 0 answers 0. There is no way to decide, unless the whole X is used as the training set.

Tip — Futility of bias-free learning

A learner that makes **no a priori assumptions** about the identity of the target concept **has no rational basis** for classifying unseen instances. Bias (restriction or preference) is not only useful for efficiency: it is **necessary for generalisation**. To learn without bias one would have to show every single instance of X as an example. Bias, however, does not yet tell us *which* solution generalises best.

3.4.4 Equivalent inductive and deductive systems

An elegant way to see the role of bias: an inductive system (a learning algorithm that, using the hypothesis space H , receives examples and a new instance and outputs a classification or “don’t know”) is **equivalent to a deductive system** — a theorem prover — that receives the same examples and the same instance **plus the inductive bias as an additional axiom**. The inductive bias is therefore exactly the set of assumptions that makes the classification produced by the learner “logically deducible”.

3.4.5 Language bias or search bias?

In modern ML, **search bias** is generally preferred. Flexible approaches are used, with very expressive hypothesis spaces and universal approximation capabilities (neural networks, decision trees), avoiding language bias and thus **without excluding a priori** the target function.

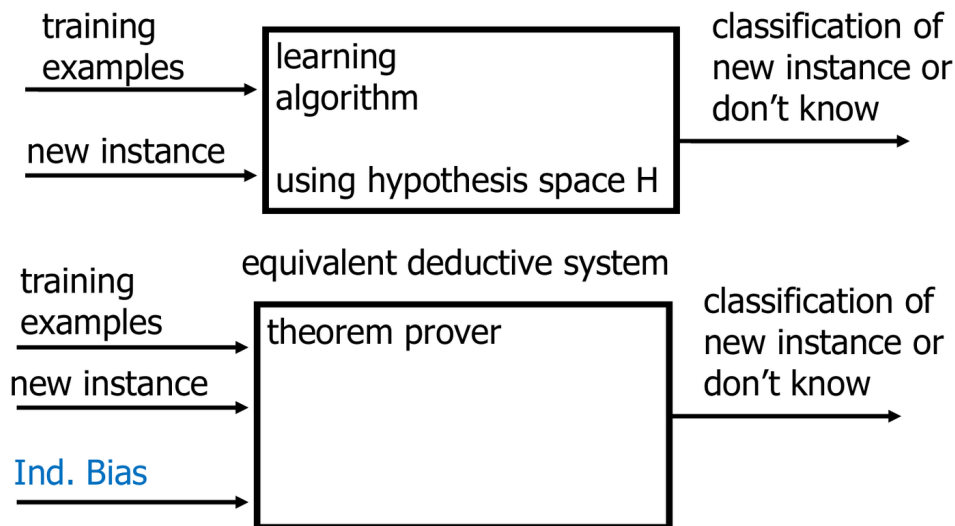


Figure 3.9: Inductive bias makes an inductive system equivalent to a deductive system.

Inductive bias remains, but it is entrusted to the **search strategy** of the learning algorithm, in practice by using an **incomplete** search (which favours certain solutions over others).

Summary — Conclusions on inductive bias

- Learning without bias does not allow extracting regularities from data: we get a lookup table, without generalisation.
- **Every state-of-the-art ML approach has an inductive bias.**
- The problem is characterising the bias of the different models and algorithms.

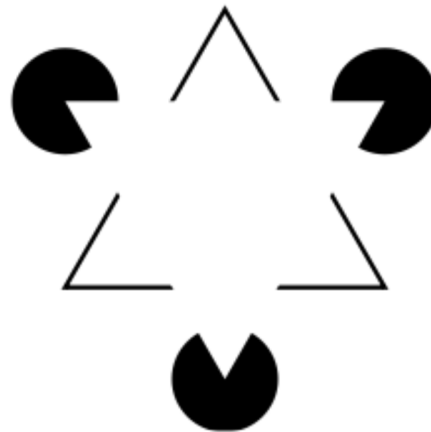


Figure 3.10: The Kanizsa triangle: an example of perceptual bias of our visual system, which “completes” shapes that are not there.

3.5 Tasks and loss functions

We said we want to find a “good” approximation of f . But how is the quality of the approximation measured? The model outputs $h(\mathbf{x})$ for input $\mathbf{x}$, and we want to measure the “distance” between $h(\mathbf{x})$ and the target d . An (“inner”) **loss function** $L(h_{\mathbf{w}}(\mathbf{x}), d)$ computed on the single pattern is used: a high value indicates a poor approximation.

Definition — Error (Risk, Loss)

The **error** is an expected value of the loss L , for example its mean over the l samples:

$$\text{Loss}(h_{\mathbf{w}}) = E(\mathbf{w}) = \frac{1}{l} \sum_{p=1}^l L(h_{\mathbf{w}}(\mathbf{x}_p), d_p).$$

It serves as the **objective function** to be minimised during training and as a measure of error control in the test phase. For now Error, Risk and Loss are considered synonyms; the differences will be clarified later.

The function L changes depending on the task. Let us look at the main cases (useful as a future reference).

3.5.1 Regression: squared error

- **Output:** $d_p = f(\mathbf{x}_p) + e$ (real function plus random error).
- H : a set of real-valued functions.
- **Loss:** the **squared error** $L(h_{\mathbf{w}}(\mathbf{x}_p), d_p) = (d_p - h_{\mathbf{w}}(\mathbf{x}_p))^2$.
- The mean over the dataset is the **Mean Squared Error** (MSE).

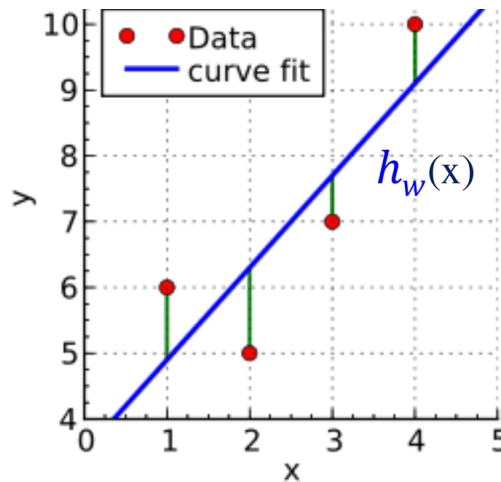


Figure 3.11: The MSE is the mean of the squares of the green segments, i.e. of the vertical distances between the data (in red) and the line $h_w(x) = w_1x + w_0$ (in blue).

$$E(\mathbf{w}) = \frac{1}{l} \sum_{p=1}^l (y_p - h_{\mathbf{w}}(\mathbf{x}_p))^2$$

where $\mathbf{w}$ are the free parameters of the linear model (in the figure the targets are denoted by y).

3.5.2 Classification: 0/1 loss

- **Output:** for example $\{0, 1\}$.
- H : a set of indicator functions.
- **0/1 loss:**

$$L(h_{\mathbf{w}}(\mathbf{x}_p), d_p) = \begin{cases} 0 & \text{if } h_{\mathbf{w}}(\mathbf{x}_p) = d_p \\ 1 & \text{otherwise} \end{cases}$$

- The mean over the dataset is the **percentage of misclassified patterns**: if 20 patterns out of 100 are wrong we have 20% error, i.e. 80% **accuracy**.

3.5.3 Clustering and vector quantization (preview)

- **Goal**: optimally partition an unknown distribution in the input space into regions (clusters), each approximated by a **centre** or **prototype**.
- H : a set of *vector quantizers* $\mathbf{x} \mapsto c(\mathbf{x})$, which map the continuous space into a discrete one.
- **Loss**: the **squared distortion**, which measures how close the pattern is to the centroid of its cluster:

$$L(h(\mathbf{x}_p)) = (\mathbf{x}_p - h(\mathbf{x}_p)) \cdot (\mathbf{x}_p - h(\mathbf{x}_p)) = \|\mathbf{x}_p - h(\mathbf{x}_p)\|^2.$$

3.5.4 Density estimation (preview)

- **Goal**: estimate a density from an assumed class (generative, parametric methods).
- **Output**: a density, for example a normal $p(\mathbf{x} \mid \mu, \sigma^2)$.
- H : a set of densities (here μ and σ^2 are the two unknown parameters).
- **Loss**: $L(h(\mathbf{x}_p)) = -\ln h(\mathbf{x}_p)$, related to **maximising the (log-)likelihood**: minimising the sum of $-\ln$ is equivalent to maximising the product of the probabilities of the observed data.

3.6 Generalisation and validation

This is the **fundamental concept of the course**.

- **Learning** is the search for a good function in a function space starting from known data, typically by minimising an error/loss.
- “Good” with respect to the **generalisation error**: how accurately the model predicts on **new data** (loss measured on new data).

Warning — Generalisation is the crucial point

It is easy to *use* ML tools; it is much harder to use them *well*. The difference lies entirely in generalisation.

Two phases are distinguished:

1. **Learning phase** (*training, fitting*): the model is built from known data (the training data, plus the bias).
2. **Predictive or test phase** (*deployment, use of the model in inference*): the model is applied to new examples. New inputs $\mathbf{x}'$ are taken, the model’s response is computed and compared with the target d' that the model has **never seen**: this is how the generalisation capability is evaluated.

Note — Performance in ML

In ML, “performance” means **generalisation accuracy** (or predictive accuracy), estimated by the error computed on a **test set** kept aside (*hold out*).

Statistical Learning Theory (Vapnik) studies the mathematical conditions under which a model is able to generalise; its basics will be seen in the next lecture.

Evaluating the performance of an ML system coincides with evaluating its predictive accuracy: in a word, **validation! validation! validation!** Validation techniques serve both to **evaluate** generalisation (*model assessment*) and to **manage** it (*model selection*), and will be the subject of the next lecture.

Note — The cost of inference

Even the inference phase alone (the input $\rightarrow$ output *mapping*) can be costly when there are millions of requests, as in Google's services: this is why dedicated chips such as **Tensor Processing Units** (TPUs) were designed, originally conceived for TensorFlow.

Summary — Summary

ML builds a hypothesis h that approximates an unknown function f from examples. Its ingredients are **data** (representation, encoding, noise), **task** (supervised: classification and regression; unsupervised: clustering, density), **model** (which defines the hypothesis space H), **learning algorithm** (search in H for the minimum-error hypothesis) and **validation**. Without **inductive bias** there is no generalisation; in modern ML *search bias* with very expressive models is preferred. Quality is measured with a **loss** that depends on the task (squared, 0/1, distortion, $-\ln$), but what really matters is the error on **new data**.

Questions — Possible exam questions

- What is inductive bias? What is the difference between language bias and search bias?
- Prove that an *unbiased learner* cannot generalise.
- How many hypotheses are there with n binary inputs without constraints? And with conjunctive rules only?
- What is the version space?
- Which losses are used for regression, classification, clustering and density estimation?
- What is meant by generalisation and how is it estimated?

Chapter 4

Generalisation and validation (introduction)

The fourth lecture concludes the introduction to ML by addressing the central theme of the course: **generalisation**. Through a concrete example (polynomial fitting) we see the phenomena of **underfitting** and **overfitting**, give a first formalisation with **Statistical Learning Theory** and the **VC-dimension**, and introduce the basic **validation** techniques, accuracy measures for classification and the design cycle of an ML system.

4.1 ML in a nutshell

Let us recap the ingredients seen in the previous lectures:

- **Data**: the available experience, represented as vectors, structures, etc.
- **Task**: supervised (classification, regression), unsupervised, ... For example, given labelled examples, find a good approximation of the unknown f .
- **Model**: describes the relationships among data and defines the class of functions the machine can implement (the hypothesis space).
- **Learning algorithm**: given task and model, performs a (heuristic) search in the space of hypotheses valid on the data, fitting the free parameters of the model.
- **Validation**: evaluates the generalisation capability of the hypothesis found.

The underlying message remains: *easy use of ML tools versus correct and aware use of ML*.

4.2 The fundamental problems of ML

4.2.1 An ill-posed problem

Inferring general functions from known data is an **ill-posed problem**: in principle the solution is not unique, and with finite data we cannot expect to find the exact solution. This is why we work with a **restricted hypothesis space** (recall inductive bias). Two questions guide the study of models:

- **What can we represent?** (if a function is not representable, it cannot be learned either);
- **What can we learn?** (a secondary question with respect to the first).

4.2.2 Generalisation and overfitting

We distinguish the **learning phase**, in which the model is built (including training), from the **prediction phase**, in which the learned function is evaluated on new samples (generalisation capability).

Definition — Inductive learning hypothesis

Any hypothesis h that approximates f well on the training examples will also approximate f well on new, unseen instances.

The question mark the professor puts next to this hypothesis is intentional: **it is not always true**, and this is exactly where overfitting arises.

Definition — Overfitting

A learner **overfits** the data if it outputs a hypothesis $h \in H$ with true (generalisation, *risk*) error R and empirical (training) error E , when there exists another $h' \in H$ with

$$E' > E \quad \text{but} \quad R' < R.$$

In other words, h' fits the training data *worse* but is *better* on new data.

The critical aspect thus becomes **accuracy estimation**, which can be tackled **theoretically** (Statistical Learning Theory) or **empirically** (training and test error, cross-validation techniques).

4.3 A case study: polynomial fitting

To see concretely the role of model complexity we use a regression example with a parametric model: the hypothesis space is the set of **polynomials of degree M** ,

$$h_{\mathbf{w}}(x) = w_0 + w_1x + w_2x^2 + \dots + w_Mx^M = \sum_{j=0}^M w_jx^j.$$

The complexity of the hypothesis grows with the degree M ; l is the number of examples.

Warning — Artificial example

It is a simplified and unrealistic task: a single input variable, and we know the target function in advance. It only serves to visualise the concepts.

The target function is $\sin(2\pi x)$ with added **Gaussian noise**: this is why the samples do not lie exactly on the “true” green curve.

To find the best parameters, the **sum of squared errors** is minimised:

$$E(\mathbf{w}) = \sum_{p=1}^l (y_p - h_{\mathbf{w}}(x_p))^2,$$

where p indexes the example, y_p is its target, $h_{\mathbf{w}}(x_p)$ is the output of the model at the point x_p (here x is a single variable, $n = 1$).

4.3.1 Effect of the polynomial degree

- $M = 0$ (constant) and $M = 1$ (line): the model is **too simple** with respect to the target function and fails to capture its trend. This is **underfitting**.
- $M = 3$: more flexibility is useful, and the curve approximates the sine well.

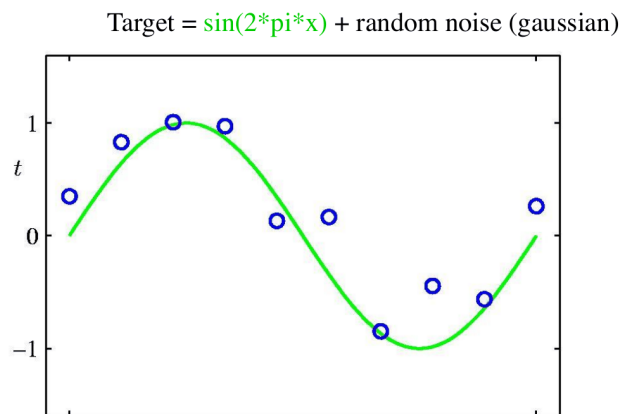


Figure 4.1: The target function $\sin(2\pi x)$ (in green) and the noisy training samples (in blue).

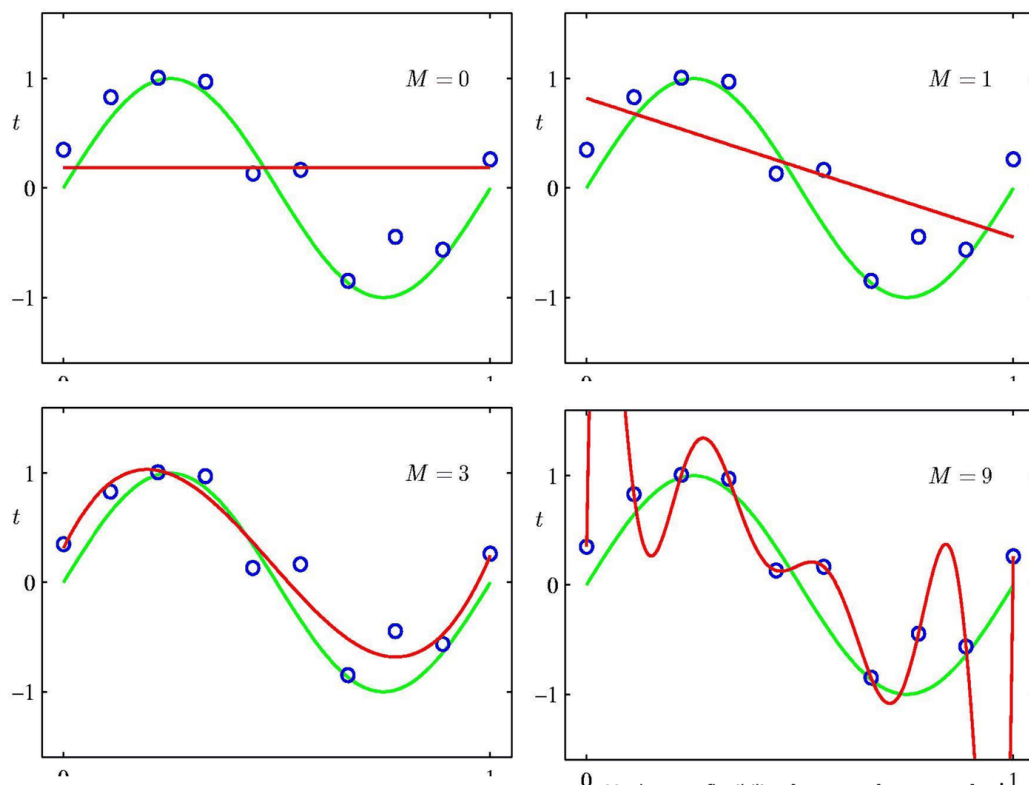


Figure 4.2: Fitting with polynomials of degree $M = 0, 1, 3, 9$ (in red) compared to the true function (in green).

- $M = 9$: with 10 points and 10 parameters the polynomial passes **exactly** through all data: $E(\mathbf{w}) = 0$ on the training set! But the curve oscillates wildly between the points and represents the true function very poorly: the model is too complex and **also learns the noise**. This is **overfitting**. And what about the error on the test set?

4.3.2 Training and test error as M varies

To compare models the **Root-Mean-Square error** is used, $E_{RMS} = \sqrt{2E(\mathbf{w}^*)/l}$, where $E(\mathbf{w}^*)$ is the error of the trained model. Dividing by l allows comparing datasets of different sizes, and the square root brings the error back to the same scale as the target.

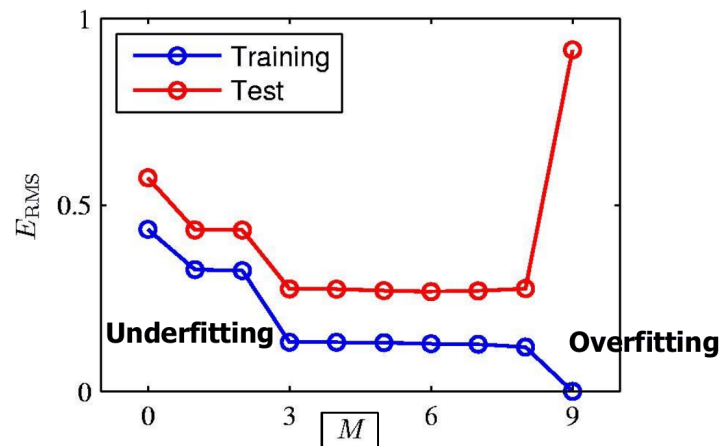


Figure 4.3: Training and test RMS error as the degree M varies: underfitting zone on the left, overfitting zone on the right.

The training error **always decreases** as M increases, because a more flexible model can fit the data better. The test error instead first decreases, then stays stable and finally **explodes** for $M = 9$: this is the signature of overfitting.

The optimal **coefficients** $\mathbf{w}^*$ are also revealing:

	$M = 0$	$M = 1$	$M = 3$	$M = 9$
w_0^*	0.19	0.82	0.31	0.35
w_1^*		-1.27	7.99	232.37
w_2^*			-25.43	-5321.83
w_3^*			17.37	48568.31
w_4^*				-231639.30
w_5^*				640042.26
w_6^*				-1061800.52
w_7^*				1042400.18
w_8^*				-557682.99
w_9^*				125201.43

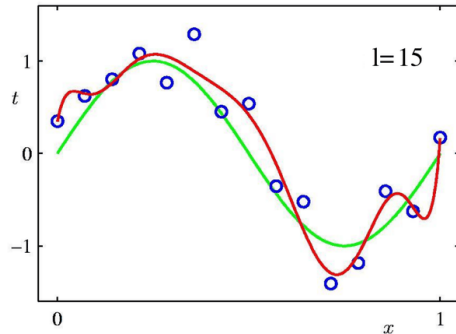
Tip — Huge weights = overfitting

In the degree-9 polynomial the coefficients become **huge** and of alternating sign: they compensate each other to make the curve pass exactly through every point, producing very strong oscillations. This observation is the basis of **regularisation** (which we will see with linear models and neural networks): penalising large weights is a way to control complexity.

4.3.3 Effect of the amount of data

What happens if, with the same model ($M = 9$), we increase the data?

Order Polynomial



Order Polynomial (even more data)

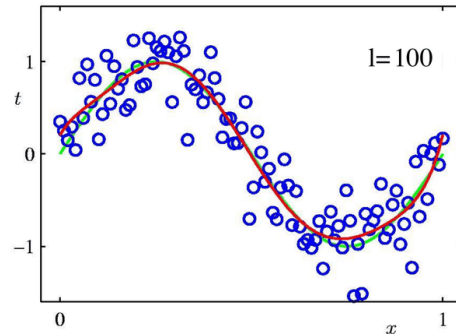


Figure 4.4: Degree-9 polynomial with $l = 15$ and $l = 100$ examples: with more data overfitting is reduced.

With 15 examples the degree-9 polynomial still oscillates, but much less; with 100 examples it follows the sine almost perfectly. **With more data, more complex models can be used without falling into overfitting.**

Summary — What we have learned

Generalisation capability (measured as risk or test error) must be studied in relation to: - the **training error** and the underfitting and overfitting zones; - the **model complexity**; - the **amount of data**.
Statistical Learning Theory is the general theory linking these three elements.

4.4 Towards Statistical Learning Theory

4.4.1 The formal setting (simplified)

We want to approximate an unknown function $f(\mathbf{x})$, where the target is $d = f(\mathbf{x}) + \text{noise}$.

Ideally we would like to minimise the **risk function** (true error), computed over **all** possible data:

$$R = \int L(d, h(\mathbf{x})) dP(\mathbf{x}, d),$$

where $P(\mathbf{x}, d)$ is the (unknown) probability distribution of the data and L is a loss, for example $L(h(\mathbf{x}), d) = (d - h(\mathbf{x}))^2$. We therefore look for $h \in H$ minimising R .

But we only have a finite dataset $TR = \{(\mathbf{x}_p, d_p)\}_{p=1}^l$. So we minimise the **empirical risk** (training error), finding the best values of the free parameters:

$$R_{emp}(h, TR) = \frac{1}{l} \sum_{p=1}^l (d_p - h(\mathbf{x}_p))^2.$$

This is the inductive principle of **Empirical Risk Minimization** (ERM). The crucial question is: **can we use R_{emp} to approximate R ?**

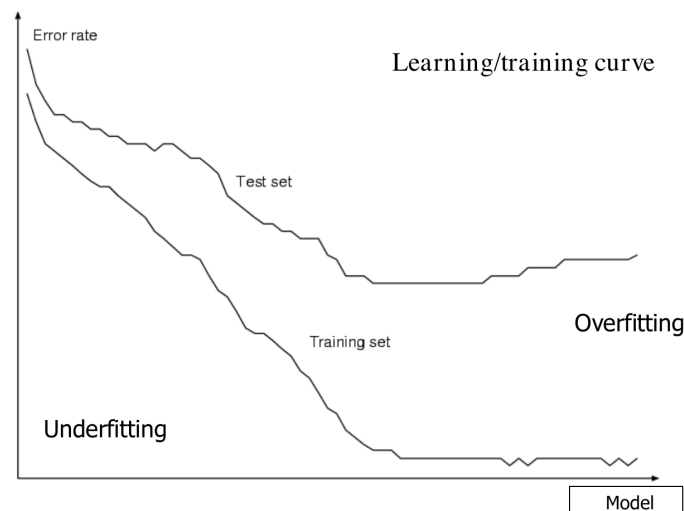


Figure 4.5: Typical learning behaviour as model complexity grows.

4.4.2 VC-dimension and risk bound

The **VC-dimension** (Vapnik-Chervonenkis) is a measure of the **complexity** of H , i.e. of its flexibility in fitting the data (for linear models or polynomials it is related to the number of parameters; it will be formally defined later).

Theorem — VC-bound

With probability $1 - \delta$ it holds:

$$\underbrace{R}_{\text{guaranteed risk}} \leq R_{\text{emp}} + \underbrace{\varepsilon\left(\frac{1}{l}, VC, \frac{1}{\delta}\right)}_{\text{VC-confidence}}$$

where the **VC-confidence** ε is a function that **grows with the VC-dimension** and **decreases with the number of data l** (and with δ).

The parameter δ is the **confidence**: it controls the probability that the bound holds (for example $\delta = 0.01$ means that the bound holds with probability 0.99).

This bound **explains** underfitting and overfitting:

- **more data** (high l) $\rightarrow$ lower VC-confidence, and the bound gets closer to R : the empirical risk becomes a good estimate of the true one;
- **model too simple** (low VC-dim) $\rightarrow$ small ε , but high R_{emp} : **underfitting**;
- **model too complex** (high VC-dim, fixed l) $\rightarrow$ low R_{emp} , but ε grows, and with it the bound on R : **overfitting**.

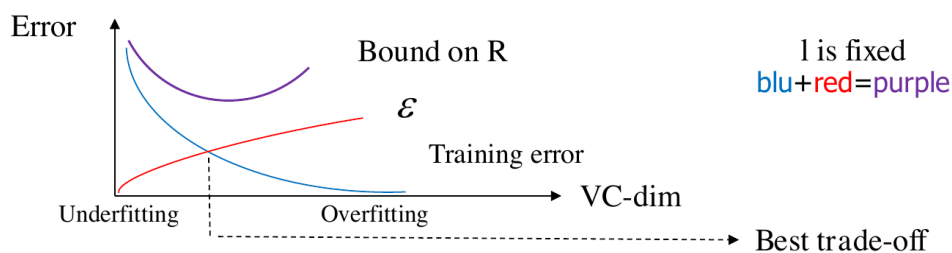


Figure 4.6: Structural Risk Minimization: the bound on R (purple) is the sum of the training error (blue) and the VC-confidence (red); its minimum is the best trade-off.

Definition — Structural Risk Minimization (SRM)

Instead of minimising only R_{emp} , we **minimise the bound**, i.e. the sum of training error and VC-confidence. This formalises the concept of **model complexity control**: a trade-off between complexity (VC-dim) and accuracy on the training set (fitting).

A concrete example of a bound is

$$R \leq R_{emp} + \varepsilon \left(\frac{VC}{l}, -\frac{\ln \delta}{l} \right),$$

and there are various formulations for different function classes and tasks. In simple words: **f can be approximated well from examples provided that we have a good number of data and the model complexity is suited to the problem.** One must fit the data as much as possible to avoid underfitting (high R_{emp}), but not too much, to avoid overfitting (due to the growth of the VC-confidence).

Note — Beyond the classical trade-off

More recent phenomena such as **double descent**, over-parameterisation and *benign overfitting* (typical of deep networks) seem to challenge this view and will be discussed later in the course.

4.4.3 Why SLT is important

SLT makes it possible to frame formally the problem of generalisation and overfitting, providing **analytical upper bounds** on the risk R regardless of the type of algorithm or the details of the model. It shows that ML is **well founded**: the risk can be bounded analytically, and few concepts are fundamental. It has also led to new models (**SVMs**) that directly take complexity control into account, and it explains the main difference between ML and pure optimisation methods (which provide the techniques for *fitting*): in ML the goal is not to minimise the error on the data, but on *future* data.

Practical questions remain open: how to measure complexity? How to find the best trade-off between fitting and complexity?

Questions — Exercises proposed by the professor

- Why does a zero training error not necessarily imply a good solution?
- Relate underfitting and overfitting to the interpretation of the SLT inequality and to the polynomial example.
- Looking at the definition of overfitting, identify h and h' on the SLT bound plot. (*Hint: h is to the right of the minimum, with lower R_{emp} but higher bound; h' is near the minimum.*)
- Is noise the cause of overfitting, or is it the complexity trade-off? Can overfitting occur even with perfectly clean data? (*Yes: even without noise, a very high-degree polynomial interpolating a few points of a smooth function can oscillate a lot between one point and the next; the problem is excessive complexity relative to the available data.*)

4.5 Validation

Evaluating the performance of an ML system means evaluating its **predictive accuracy**. Beware: **performance on the training data gives an overly optimistic evaluation**. Hence the mantra “validation, validation, validation”. Here we give an introduction; validation will be taken up again in dedicated lectures.

4.5.1 The two goals of validation

Definition — Model selection and model assessment

- **Model selection:** estimate the performance (generalisation error) of **different models** in order to choose the best one. It includes searching for the best **hyperparameters** (e.g. the degree of the polynomial). *Returns a model.*
- **Model assessment:** having chosen the final model, estimate its prediction error (risk) on **new test data**, as a measure of the quality of the chosen model. *Returns an estimate.*

Warning — Golden rule

Keep the **goals separate** and use **separate data sets** for each. If the test set is used to choose the model, the final estimate is no longer reliable (it is optimistic).

In an ideal world we would have a large training set (to find the best hypothesis), a large validation set for model selection and a very large external test set. With finite, often small, datasets, we can only **estimate** generalisation.

4.5.2 Hold-out

In the basic technique, **hold-out**, the dataset D is partitioned into three **disjoint** sets:

- **Training set (TR):** used to run the learning algorithm;
- **Validation set (VL)** or *selection set*: used to choose the best model (e.g. hyperparameter tuning);
- **Test set (TS):** used **only** for model assessment, never to choose or tune h .

TR and VL together form the **development/design set**; the test set (for example 25–30% of the data) is kept aside.

4.5.3 K-fold cross-validation (preview)

Hold-out can make poor use of the data, especially when they are few. In **K-fold cross-validation**:

1. D is split into k mutually exclusive subsets $D_1, \dots, D_k$;
2. for each i , train on $D \setminus D_i$ and evaluate on D_i ;
3. the k results are combined.

In this way all data are used both for training and for validation (or testing). It can be applied both to the validation split and to the test split. Open questions are how many folds to

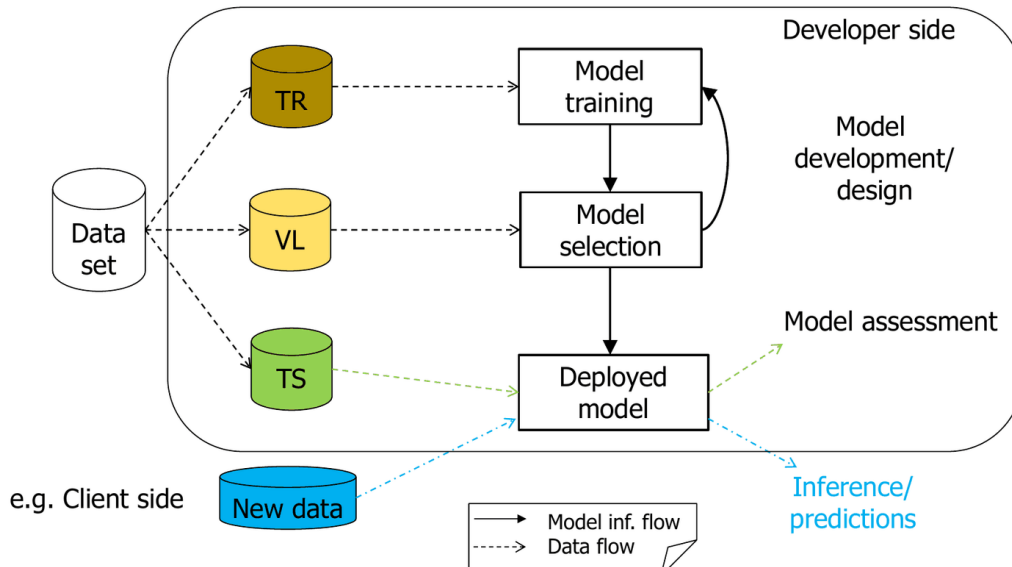


Figure 4.7: Role of TR, VL and TS: model development on the developer side, use in inference on the client side.

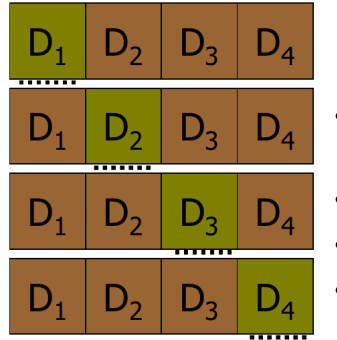


Figure 4.8: 4-fold cross-validation: each fold in turn acts as the validation set.

use (3, 5, 10, up to *leave-one-out*), the often high computational cost and the combination with other schemes (validation set, double K-fold). It will be discussed in detail later.

4.6 Accuracy in classification

4.6.1 Confusion matrix

For a binary classifier, the actual class and the predicted class are compared:

Actual \ Predicted	Positive	Negative
Positive	TP (true positives)	FN (false negatives)
Negative	FP (false positives)	TN (true negatives)

From which:

- **Accuracy** = $\frac{TP + TN}{\text{total}}$: percentage of correctly classified patterns;
- **Sensitivity** (*True Positive rate, Recall*) = $\frac{TP}{TP + FN}$: how many actual positives are recognised;

- **Specificity** (*True Negative rate*) = $\frac{TN}{FP + TN} = 1 - FPR$: how many actual negatives are recognised;
- **Precision** = $\frac{TP}{TP + FP}$: how many of the predicted positives are actually positive.

A false positive is equivalent to a **false alarm**.

Warning — Beware of accuracy

- For binary classification, 50% accuracy is that of a **random** predictor (coin toss).
- With **unbalanced data** (e.g. 99% positives) there is a trivial classifier that always answers “positive” and has 99% accuracy without having learned anything. In these cases accuracy alone is misleading.

4.6.2 ROC curve

The **ROC curve** shows the *True Positive rate* (sensitivity) as a function of the *False Positive rate* (1 – specificity) as the decision threshold of the classifier varies.

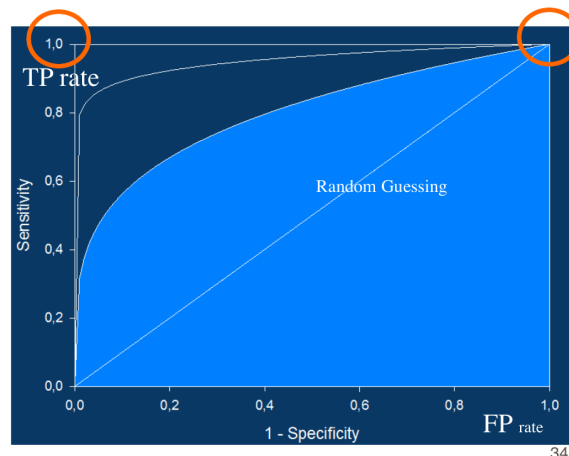


Figure 4.9: ROC curve: the diagonal corresponds to the random classifier; better curves have a larger area under the curve (AUC).

The **diagonal** corresponds to the worst (random) classifier. Better curves rise quickly towards the top-left corner (high TPR with low FPR) and have a higher **AUC** (*Area Under the Curve*): the ideal classifier has AUC = 1, the random one 0.5.

4.7 The design cycle

1. **Data collection:** selection, integration, cleaning. A **large and representative** enough set of examples is needed for training and testing.
2. **Data representation:** depends on the domain and exploits the expert’s prior knowledge; includes feature selection, outlier detection, variable scaling, handling of missing data. **It is often the most critical phase** for overall success.
3. **Model choice:** formulation of the problem and of the hypotheses. One must know the model’s **limits of applicability** and control its complexity.
4. **Model building** (the heart of ML): through the learning algorithm on the training data.
5. **Evaluation:** performance is **predictive accuracy**; interpretation of results, explanation of data and knowledge extraction are added.

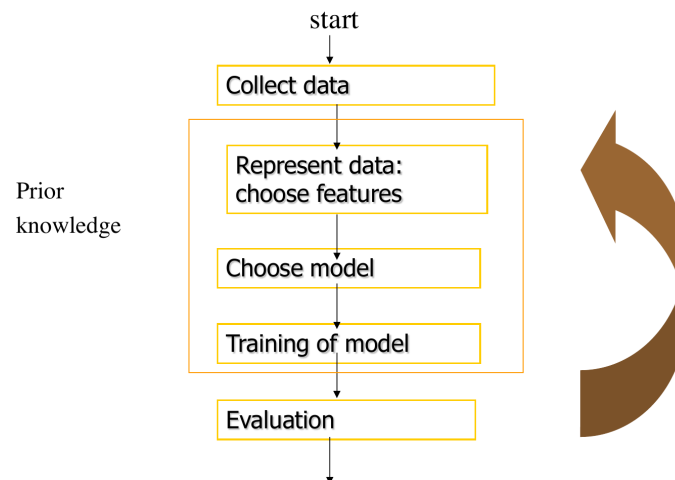


Figure 4.10: The design cycle of an ML system: an iterative process guided by prior knowledge.

6. Deployment.

The cycle is iterative: the results of the evaluation may require going back to previous phases.

4.8 Interpretation errors

For every statistical model (including Data Mining and ML):

- **causality** is (often) assumed and a data set **representative** of the phenomenon is needed. ML does not work with uncorrelated variables or random phenomena (lotteries); uninformative inputs lead to poor models and results;
- **causality cannot be inferred from data analysis alone**: in Florida people are on average older than in the other US states, but it is not the climate that makes them live longer (people move there when retired). Statistical dependence can have reasons external to the data.

Specific to ML: **powerful models increase the risk**, because they also fit “garbage” data; and results that are **not properly validated** may lead to misleading predictions and interpretations.

Summary — Summary

Generalisation depends on the trade-off between **model complexity** and **amount of data**. Models that are too simple **underfit**, models that are too complex **overfit** (low training error, high test error). SLT formalises everything with the **VC-bound** $R \leq R_{emp} + \varepsilon(1/l, VC, 1/\delta)$ and suggests **Structural Risk Minimization**. In practice generalisation is estimated through **validation**: *model selection* (on VL) and *model assessment* (on TS), always on separate data.

Questions — Possible exam questions

- Formally define overfitting.
- Describe the polynomial fitting example: what happens as M and l vary?
- Write the VC-bound and explain how it justifies underfitting and overfitting.
- Difference between risk R and empirical risk R_{emp} ; what is the ERM principle?

- Difference between model selection and model assessment; why are separate sets needed?
- Define accuracy, sensitivity, specificity, precision; what is the ROC curve?

Chapter 5

Linear models and K-nearest neighbors

After the introduction we move on to the first actual models, which occupy two opposite ends of the ML landscape. On one side, **linear models**: rigid, with few parameters, grounded in classical mathematics but already rich in modern concepts (loss function, gradient descent, regularisation). On the other, **K-nearest neighbors**: extremely flexible, local, with no real model to train. Comparing the two shows concretely the trade-off between flexibility and complexity control.

Note — Two distinct lectures

In the 2026/27 slides the two topics are in two separate decks (*linear* and *knn*); the content is the same and here they remain in a single note, with linear models in the first part and K-NN in the second.

With linear models we move from a discrete hypothesis space (concept learning) to a **continuous** one, still restricted but parameterised by real numbers.

5.1 Data notation

Data are organised in a matrix X of size $l \times n$: l rows (the patterns, $p = 1, \dots, l$) and n columns (the features, $i = 1, \dots, n$).

- $\mathbf{x}$ (in bold) is a generic pattern, i.e. a row of the table: example, instance, sample, input vector;
- x_i or x_j (scalar) is component i or j of a pattern $\mathbf{x}$ (omitting the pattern index);
- $\mathbf{x}_p$ is the p -th pattern (row);
- $x_{p,i}$, also written $(\mathbf{x}_p)_i$, is component i of pattern p ;
- for the target we use y_p (or d_p, t_p), with $p = 1, \dots, l$.

When the context is clear some indices are omitted.

5.2 Linear models

The linear model has been the mainstay of statistics. As Hastie writes: “*despite the great advances of modern non-parametric regression techniques, linear models remain important, and we need to understand them well*”. We start from the simplest form, linear in the input variables. The linear model is also a natural **baseline**: the first question when facing a problem is “is it a linear problem?”.

5.2.1 Univariate linear regression

Let us go back to the exercise of the introduction: given the points (1; 2.1), (2; 3.9), (3; 6.1), (4; 8.4), (5; 9.8) we had “guessed” $h(x) = 2x$. Now we want to find the parameters **systematically**.

In the univariate case (one input variable x , one output y) we assume the model

$$h_{\mathbf{w}}(x) = w_1 x + w_0,$$

where w_0, w_1 are real coefficients called **weights** or free parameters. The task is to fit a line to the data. The hypothesis space is infinite (the $\mathbf{w}$ are continuous), but classical mathematics provides an elegant solution dating back to Gauss and Legendre (around 1795). Surprisingly, with this basic tool one can already “learn”, and it contains many relevant concepts of modern ML.

Learning via Least Mean Squares

Training means finding $\mathbf{w}$ that **minimises the empirical error** on the training data: for now we focus only on R_{emp} (complexity control will come later). Given a set of l examples (x_p, y_p) , we look for $h_{\mathbf{w}}(x) = w_1 x + w_0$ that minimises the **sum of squared residuals**:

$$\text{Loss}(h_{\mathbf{w}}) = E(\mathbf{w}) = \sum_{p=1}^l (y_p - h_{\mathbf{w}}(x_p))^2 = \sum_{p=1}^l (y_p - (w_1 x_p + w_0))^2.$$

To get the mean, divide by l . This is the **least squares** method (*Least (Mean) Squares*, LMS), i.e. $\arg \min_{\mathbf{w}} E(\mathbf{w})$ in the L^2 norm.

Tip — Why least squares?

Each candidate line produces **residuals**, i.e. the vertical distances between the points (x_p, y_p) and the line. Different lines have different residuals: minimising the sum of their squares is a natural way to find the line that best approximates the data. In general, least squares is the standard approach to approximately solve **overdetermined systems**, i.e. with more equations (the examples) than unknowns (the parameters).

How it is solved

A local minimum is a **stationary point**, where the gradient is zero:

$$\frac{\partial E(\mathbf{w})}{\partial w_i} = 0, \quad i = 1, \dots, n+1.$$

In the simple case with two parameters we look for w_0, w_1 such that $\frac{\partial E}{\partial w_0} = 0$ and $\frac{\partial E}{\partial w_1} = 0$. Since the loss is **convex** (a parabola in two dimensions), there are no spurious local minima and the solution exists in closed form:

$$w_1 = \frac{\sum_p x_p y_p - \frac{1}{l} \sum_p x_p \sum_p y_p}{\sum_p x_p^2 - \frac{1}{l} \left(\sum_p x_p \right)^2} = \frac{\text{Cov}[x, y]}{\text{Var}[x]}, \quad w_0 = \bar{y} - w_1 \bar{x},$$

where $\bar{x} = \frac{1}{l} \sum_p x_p$ and $\bar{y} = \frac{1}{l} \sum_p y_p$ are the means.

Tip — To be understood, not memorised

The professor points out that this “direct” solution is only meant to show that it **exists** (thanks to the convexity of the loss): it should not be memorised. What matters is being

able to **derive** it by setting the gradient to zero, as in the exercise that follows.

Computing the gradient for a pattern

For a single pattern (x, y) , using the differentiation rules ($\frac{\partial k}{\partial w} = 0$, $\frac{\partial w}{\partial w} = 1$, $\frac{\partial f(w)^2}{\partial w} = 2f(w)\frac{\partial f(w)}{\partial w}$, and “the derivative of the sum is the sum of the derivatives”):

$$\frac{\partial E(\mathbf{w})}{\partial w_i} = \frac{\partial (y - h_{\mathbf{w}}(x))^2}{\partial w_i} = 2(y - h_{\mathbf{w}}(x)) \frac{\partial (y - (w_1 x + w_0))}{\partial w_i},$$

hence

$$\frac{\partial E(\mathbf{w})}{\partial w_0} = -2(y - h_{\mathbf{w}}(x)), \quad \frac{\partial E(\mathbf{w})}{\partial w_1} = -2(y - h_{\mathbf{w}}(x))x.$$

The quantity $\delta = y - h(x)$, the error on the pattern, will be called **delta**. For l patterns we just sum; then we extend to multidimensional inputs.

Example — Worked exercise

With the data $(1; 2.1), (2; 3.9), (3; 6.1), (4; 8.4), (5; 9.8)$: $\bar{x} = 3$, $\bar{y} = 6.06$; $\sum x_p y_p = 2.1 + 7.8 + 18.3 + 33.6 + 49 = 110.8$; $\sum x_p^2 = 55$. Hence $w_1 = \frac{110.8 - 5 \cdot 3 \cdot 6.06}{55 - 45} = \frac{110.8 - 90.9}{10} = 1.99$ and $w_0 = 6.06 - 1.99 \cdot 3 = 0.09$. The line found, $h(x) \approx 1.99x + 0.09$, confirms the intuition $h(x) = 2x$.

5.2.2 Notation for multidimensional inputs

With $\mathbf{x}$ and $\mathbf{w}$ column vectors, the linear model is

$$\mathbf{w}^T \mathbf{x} + w_0 = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_n x_n = w_0 + \sum_{i=1}^n w_i x_i.$$

w_0 is called **intercept**, threshold, **bias** or offset. It is convenient to include a constant component $x_0 = 1$ in the input, so that

$$\mathbf{x}^T = [1, x_1, \dots, x_n], \quad \mathbf{w}^T = [w_0, w_1, \dots, w_n], \quad h(\mathbf{x}_p) = \mathbf{x}_p^T \mathbf{w} = \sum_{i=0}^n x_{p,i} w_i.$$

The linear model is therefore simply a **dot product** between input and weights. (In neural networks the transpose is often omitted.)

5.3 Classification with linear models

5.3.1 The problem

Consider 200 points in $\mathbb{R}^2$, generated by an unknown distribution, 100 for each of two classes. Can we build a rule that predicts the colour of future points? The data might be generated by one Gaussian per class (with different means), or by a mixture of many low-variance Gaussians.

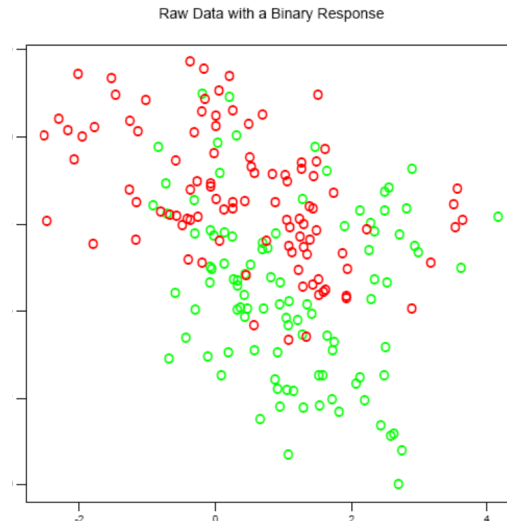


Figure 5.1: The example problem (Hastie-Tibshirani-Friedman): 200 points of two classes.

5.3.2 Hyperplanes and decision regions

The same models used for regression can be used for **classification**, with categorical targets (0/1 or $-1/+1$). The expression $\mathbf{w}^T \mathbf{x}$ defines a **hyperplane**; we exploit the fact that $\mathbf{w}^T \mathbf{x}$ takes positive values on one side and negative values on the other to decide which class a point belongs to. The goal is to find, through learning, a $\mathbf{w}$ that gives good accuracy.

In two dimensions, $\mathbf{w}^T \mathbf{x} = w_0 + w_1 x_1 + w_2 x_2 = 0$ is a line: the **decision boundary**.

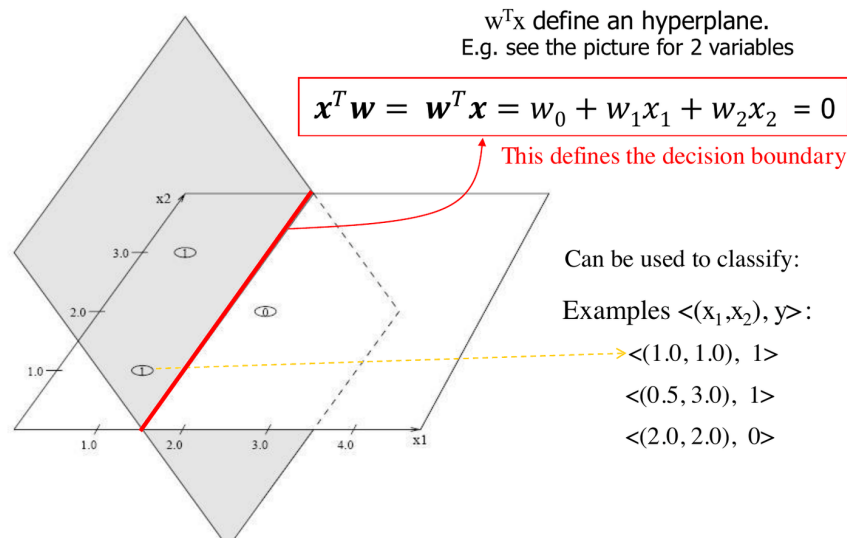


Figure 5.2: The hyperplane $\mathbf{w}^T \mathbf{x}$ and its intersection with the input plane: the decision boundary.

To obtain a classifier a **threshold function** is applied:

$$h(\mathbf{x}) = \begin{cases} 1 & \text{if } \mathbf{w}^T \mathbf{x} + w_0 \geq 0 \\ 0 & \text{otherwise} \end{cases} \quad (\text{output in } [0, 1]), \quad h(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x} + w_0) \quad (\text{output in } [-1, +1]).$$

Including w_0 in $\mathbf{w}$: $h(\mathbf{x}_p) = \text{sign}(\mathbf{x}_p^T \mathbf{w}) = \text{sign}\left(\sum_{i=0}^n x_{p,i} w_i\right)$. This is the **Linear Threshold Unit (LTU)**.

Note — The bias as a threshold

Saying $\mathbf{w}^T \mathbf{x} + w_0 \geq 0$ is equivalent to saying $\mathbf{w}^T \mathbf{x} \geq -w_0$. The two forms identify the same positive region, but the second highlights the role of the bias: $-w_0$ is the **threshold value** that the weighted combination of the inputs must exceed to “fire” the +1 output.

5.3.3 Using the model: two examples**Example — Earthquakes or nuclear explosions? (AIMA)**

Seismic data (1982–1990, Asia): x_1 is the body wave magnitude, x_2 the surface wave magnitude. A learning algorithm finds the boundary $-4.9 + 1.7x_1 - x_2 = 0$. To classify a new event (6, 3):

$$h(6, 3) = \text{sign}(-4.9 + 1.7 \cdot 6 - 1 \cdot 3) = \text{sign}(2.3) = +1 \Rightarrow \text{nuclear explosion.}$$

(Obtaining new training examples, in this case, is decidedly expensive.)

Find h s.t. given (x_1, x_2) return 0/-1 for Earthquakes and 1 for Nuclear Explosion

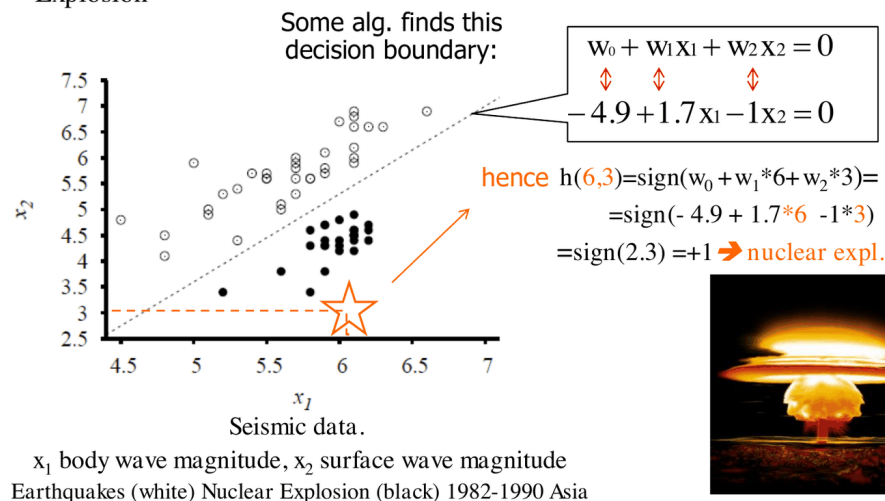


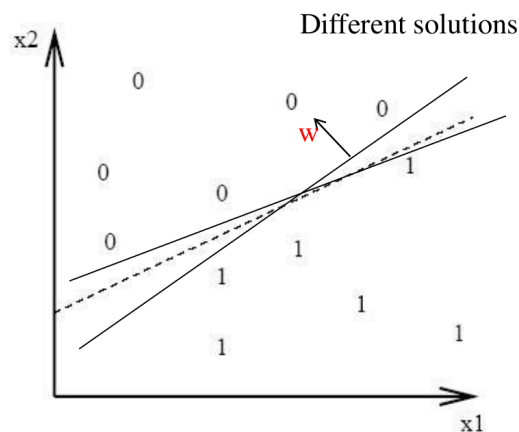
Figure 5.3: Classification of seismic data with a linear decision boundary.

Example — Spam filter

We want $h(\text{mail}) = +1$ for spam and -1 otherwise. The features $\phi_k(\text{mail})$ are for example the presence (0/1) of words or phrases (“free money”) or the length (integer): this is the *bag of words* representation. Each weight w_k expresses the contribution of the feature to the prediction: positive for “free money”, negative for “.edu” or “.unipi”. The classification is $h_{\mathbf{w}}(\mathbf{x}) = \text{sign}(\sum_k w_k \phi_k(\mathbf{x}))$: if the weighted combination is positive, it is spam.

5.3.4 Useful properties of the hyperplane

- If $w_0 = 0$ the line passes through the **origin**.
- If $n > 2$ the boundary is a **hyperplane**.
- **Scale freedom:** multiplying $\mathbf{w}$ by a constant K does not change the boundary.



A linearly separable problem

Figure 5.4: A linearly separable problem admits many separating hyperplanes; $\mathbf{w}$ is orthogonal to the hyperplane.

- **$\mathbf{w}$ is orthogonal to the hyperplane:** taking two points $\mathbf{x}_a, \mathbf{x}_b$ on the hyperplane, $\mathbf{w}^T \mathbf{x}_a + w_0 = 0$ and $\mathbf{w}^T \mathbf{x}_b + w_0 = 0$; subtracting gives $\mathbf{w}^T (\mathbf{x}_a - \mathbf{x}_b) = 0$, i.e. $\mathbf{w}$ is orthogonal to every vector lying on the hyperplane.
- If a separating hyperplane exists, **many exist**, and hence also many algorithms to find them.

The equation of the line can be rewritten as $x_2 = -x_1 \frac{w_1}{w_2} - \frac{w_0}{w_2}$: trying to draw it with different values of $\mathbf{w}$ is a good exercise.

5.4 The learning problem

5.4.1 Formulation

Given l examples $(\mathbf{x}_p, y_p)$ with $y_p \in \{0, 1\}$ or $\{-1, +1\}$ and a loss L , we look for $\mathbf{w}$ minimising the empirical risk

$$R_{emp} = \frac{1}{l} \sum_{p=1}^l L(h(\mathbf{x}_p), y_p).$$

For classification, directly using the 0/1 loss on $\text{sign}(\mathbf{w}^T \mathbf{x})$ makes the problem hard: the function is **piecewise constant**, so the gradient is zero almost everywhere and gives no indication of how to move, and one ends up with combinatorial problems.

5.4.2 Replacing the loss with a smooth function

The idea is to replace the 0/1 loss with a **smooth and differentiable** function, such as the squared error, applied directly to the linear output $\mathbf{w}^T \mathbf{x}$ (not to $h(\mathbf{x})$, which contains the sign):

$$E(\mathbf{w}) = \sum_{p=1}^l (y_p - \mathbf{x}_p^T \mathbf{w})^2 = \|\mathbf{y} - X\mathbf{w}\|^2.$$

Why does it work? With target $y_p = 1$, minimising $(1 - \mathbf{x}_p^T \mathbf{w})^2$ pushes $\mathbf{x}_p^T \mathbf{w}$ towards 1, hence positive: no classification error. With $y_p = -1$ it pushes it towards -1 . Both losses are minimised

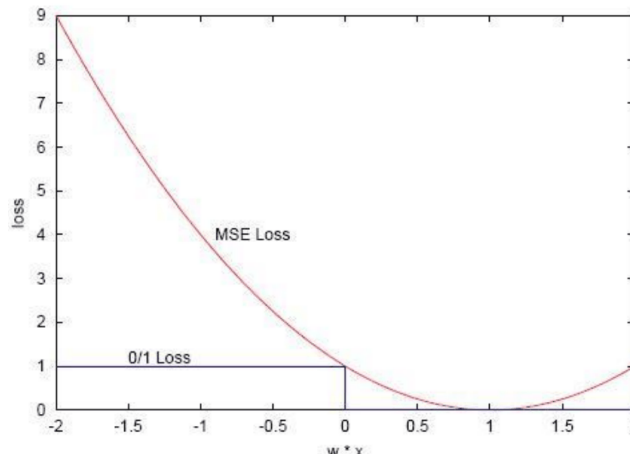


Figure 5.5: 0/1 loss and squared loss for a pattern with target $y = 1$: both are minimised in the region $\mathbf{w}^T \mathbf{x} > 0$.

in the same region (on the right in the plot). The function is **quadratic**, so a minimum always exists (but it is not necessarily unique). The same approach applies identically to regression.

We now see two learning algorithms, both based on LMS and valid for both regression and classification:

1. a **direct approach** based on the **normal equations**;
2. an **iterative approach** based on **gradient descent**.

5.4.3 Direct approach: the normal equations

Differentiating $E(\mathbf{w})$ with respect to the component w_j :

$$\frac{\partial E(\mathbf{w})}{\partial w_j} = \sum_{p=1}^l 2(y_p - \mathbf{x}_p^T \mathbf{w}) \frac{\partial (y_p - \mathbf{x}_p^T \mathbf{w})}{\partial w_j}.$$

In the dot product $\mathbf{x}_p^T \mathbf{w} = x_{p,1}w_1 + x_{p,2}w_2 + \dots$ the only term that depends on w_j is $x_{p,j}w_j$, whose derivative is $x_{p,j}$. Therefore:

$$\frac{\partial E(\mathbf{w})}{\partial w_j} = -2 \sum_{p=1}^l (y_p - \mathbf{x}_p^T \mathbf{w}) x_{p,j} = -2 \sum_{p=1}^l \delta_p x_{p,j},$$

where $\delta_p = y_p - \mathbf{x}_p^T \mathbf{w}$ is the error on pattern p . This form will come back in neural networks.

Setting the gradient to zero and switching to matrix notation we obtain the **normal equations**:

$$(X^T X) \mathbf{w} = X^T \mathbf{y}.$$

Theorem — Least squares solution

If $X^T X$ is non-singular, the solution is unique:

$$\mathbf{w} = (X^T X)^{-1} X^T \mathbf{y} = X^+ \mathbf{y},$$

where X^+ is the **Moore-Penrose pseudoinverse** (defined even if X is not invertible). Otherwise there are infinitely many solutions, and the one with **minimum norm** of $\mathbf{w}$ can be chosen.

In practice the **singular value decomposition** (SVD) is used: $X = U\Sigma V^T$ implies $X^+ = V\Sigma^+U^T$, where Σ^+ is obtained by replacing each non-zero diagonal element with its reciprocal. Applying the SVD one computes directly $\mathbf{w} = X^+\mathbf{y}$, obtaining the minimum-norm solution. **This is the learning algorithm** of the direct approach; it is implemented in all numerical libraries (Numerical Recipes, Armadillo, NumPy, R, Matlab...), with many variants for efficiency and stability.

5.4.4 Iterative approach: gradient descent

Why look for other approaches if the direct solution exists? An iterative approach makes it possible to obtain:

- **more efficient** solutions (the direct one is cubic in the size of the matrix);
- **regularisation** (to reduce model complexity);
- better approximation with noisy data, by stopping the search **before the minimum**;
- above all, a method that also applies to **non-linear models**. It is the basis of the fundamental approaches we will see (neural networks).

The idea

The gradient $\frac{\partial E}{\partial w_j} = -2 \sum_p (y_p - \mathbf{x}_p^T \mathbf{w}) x_{p,j}$ indicates the direction of **ascent** of the error. Moving in the opposite direction ($\Delta \mathbf{w} = -\nabla E(\mathbf{w})$) we go down towards the minimum. It is a **local search**: we start from an initial weight vector and iteratively modify it to decrease the error (*steepest descent*).

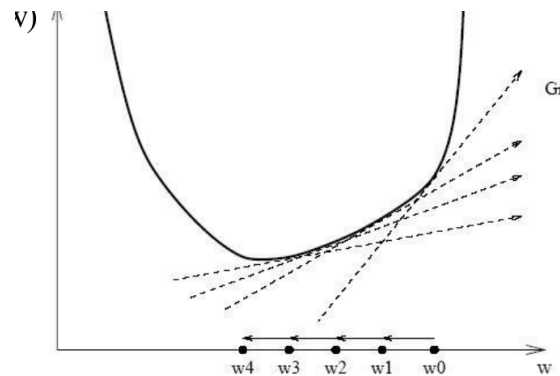


Figure 5.6: Gradient descent in one dimension: at each step the weight moves in the direction opposite to the slope.

With two weights, the error surface $E([w_0, w_1]^T)$ is a **paraboloid** (convex quadratic function), and the negative gradient is our “compass” to find the minimum.

In general the vector $\Delta \mathbf{w}$ has one component for each weight:

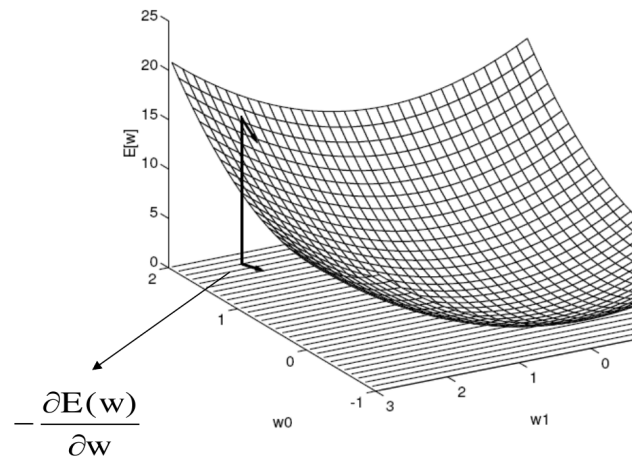
$$\Delta \mathbf{w} = -\frac{\partial E(\mathbf{w})}{\partial \mathbf{w}} = \left(-\frac{\partial E}{\partial w_0}, -\frac{\partial E}{\partial w_1}, \dots, -\frac{\partial E}{\partial w_n} \right)^T,$$

and we can work in multidimensional spaces without needing to visualise them.

The learning rule (delta rule)

We move iteratively with a **learning rule**:

$$\mathbf{w}_{new} = \mathbf{w} + \eta \Delta \mathbf{w},$$



Our "compass" to find the minimum

Figure 5.7: The error surface of a linear model with two weights: each point of the (w_0, w_1) plane is a hypothesis.

where η (eta) is the **learning rate** (*step size*), which controls the speed of descent.

Summary — Gradient descent algorithm

1. Initialise $\mathbf{w}$ with small values; fix $0 < \eta < 1$.
2. Compute $\Delta \mathbf{w} = -\frac{\partial E(\mathbf{w})}{\partial \mathbf{w}}$.
3. Update $\mathbf{w}_{new} = \mathbf{w} + \eta \Delta \mathbf{w}$.
4. Repeat from step 2 until convergence or until $E(\mathbf{w})$ is "small enough".

Notes:

- using $\Delta \mathbf{w}/l$ corresponds to *least mean squares* (dividing by l will be the standard case);
- η controls a **speed/stability** trade-off; it can be gradually reduced towards zero to guarantee convergence while avoiding oscillations around the minimum (many variants will be seen later).

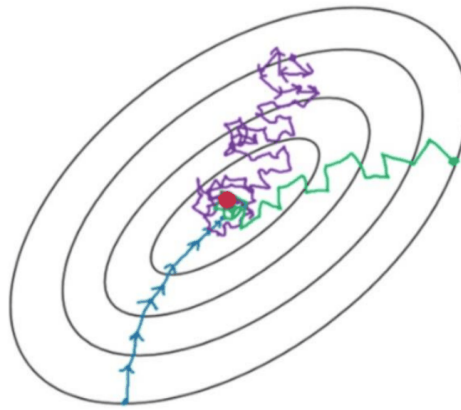
Batch and on-line

- **Batch version:** the gradient is the sum over **all** l patterns, and the weights are updated after seeing the whole training set (an **epoch**). The gradient estimate is more "precise".
- **On-line** (or **stochastic**, SGD) **version:** the weights are updated after **each** pattern, with the gradient of the single pattern:

$$\frac{\partial E_p(\mathbf{w})}{\partial w_j} = -2(y_p - \mathbf{x}_p^T \mathbf{w}) x_{p,j} = -\Delta_p w_j.$$

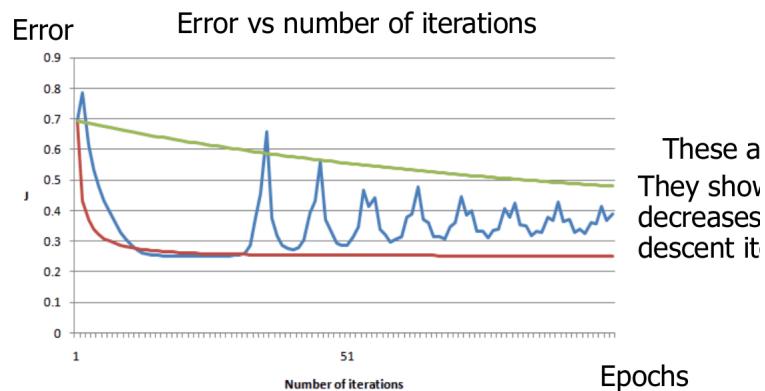
The output of the second pattern is thus computed with weights already updated by the first, and so on. It makes progress at every example and can be faster, but it requires a smaller η .

There are intermediate cases (**mini-batch**), which we will see later.



Paths over the error surface by Batch or On-line version

Figure 5.8: Paths on the error surface: batch (blue, smooth) and on-line (purple and green, more irregular).



Exercise: 1 is slow, 1 is unstable, 1 is good: which one?

Figure 5.9: Learning curves: the error as a function of the iterations (epochs).

Questions — Exercise: which curve is slow, which unstable, which good?

The **green** curve decreases very slowly (learning rate too small); the **blue** one shows spikes and oscillations (learning rate too large, unstable); the **red** one decreases quickly and stabilises (good).

Interpretation: an error-correction rule

Summing up,

$$\Delta w_j = 2 \sum_{p=1}^l \underbrace{(y_p - \mathbf{x}_p^T \mathbf{w})}_{\delta_p} x_{p,j}, \quad \mathbf{w}_{new} = \mathbf{w} + \eta \Delta \mathbf{w}$$

(the constant 2 can be omitted, absorbing it into η). It is an **error-correction** rule, called the **Widrow-Hoff rule** or **delta rule**: each w_j changes **in proportion to the error** (target – output).

- If the error is 0, there is no correction.

- With input $x_j > 0$: if the error is positive (output too low), positive delta $\rightarrow w_j$ increases $\rightarrow$ the output increases $\rightarrow$ the error decreases.
- With input $x_j > 0$: if the error is negative (output too high), negative delta $\rightarrow w_j$ decreases $\rightarrow$ the output decreases $\rightarrow$ the error decreases.
- With a negative input the signs are reversed, with a zero input the weight does not change: the input “decides” how much and in which direction each weight is responsible for the error.

We learn from previous mistakes: “by trying and failing one learns” (Goethe).

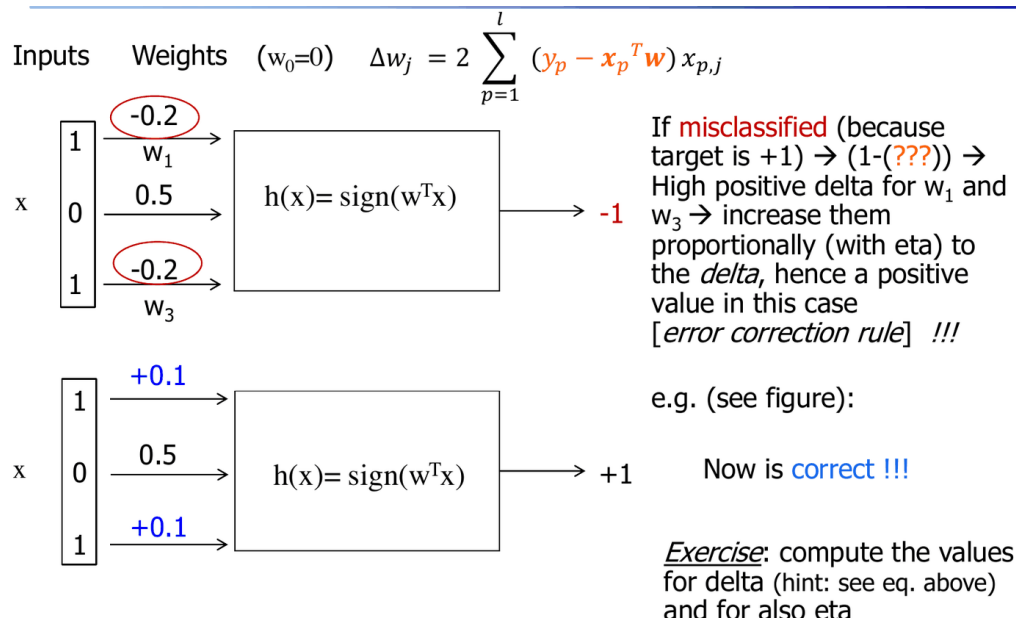


Figure 5.10: The delta rule at work: the weights of the active inputs are increased until the pattern is correctly classified.

In the example, the input is $(1, 0, 1)$ with weights $(-0.2; 0.5; -0.2)$ and $w_0 = 0$: $\mathbf{w}^T \mathbf{x} = -0.4$, so the output is -1 but the target is $+1$. The delta is positive: w_1 and w_3 (whose inputs are 1) are increased, while w_2 (input 0) stays unchanged. After the correction ($w_1 = w_3 = +0.1$) we have $\mathbf{w}^T \mathbf{x} = 0.2 > 0$ and the output is correct.

Tip — Why gradient descent is so important

It is a simple and effective local search method that makes it possible to explore an **infinite and continuous** hypothesis space. It always applies when H is continuous and the loss is differentiable, **not only to linear models**: it will be the basis for training neural networks and deep learning. There are many improvements (Newton and quasi-Newton methods, conjugate gradient...).

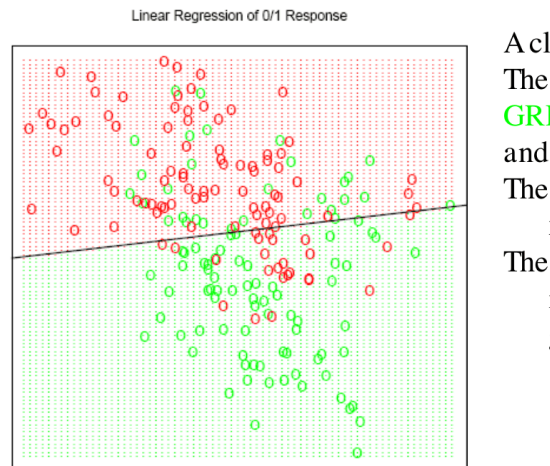
5.4.5 Summary for classification

- The model is trained on TR with LMS applied to $\mathbf{w}^T \mathbf{x}$, through the same gradient descent used for regression.
- The model is **used** to classify by applying the threshold: $h(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x})$.
- The error can be evaluated as **classification error** (0/1 loss, number of wrong patterns),

not only as MSE:

$$\text{mean_err} = \frac{1}{l} \sum_{p=1}^l L(h(\mathbf{x}_p), d_p) = \frac{\text{num_err}}{l}, \quad \text{accuracy} = \frac{l - \text{num_err}}{l}.$$

5.4.6 Back to the problem: is it a good solution?



The *decision boundary* is $\{\mathbf{x} \mid \mathbf{x}^T \mathbf{w} = 0.5\}$.
seems to make many errors on the training

Figure 5.11: Linear solution to the example problem: classes encoded as GREEN = 0, RED = 1, boundary $\mathbf{x}^T \mathbf{w} = 0.5$.

The boundary $\{\mathbf{x} \mid \mathbf{x}^T \mathbf{w} = 0.5\}$ is linear and seems to make many errors on the training set. Is it a bad solution? It depends on how the data were generated:

- **Scenario 1:** each class is generated by a Gaussian with uncorrelated components, same variance and different means. In this case the linear rule is **almost optimal**: the overlap between the classes is unavoidable (due to noise in the data).
- **Scenario 2:** each class is a mixture of 10 Gaussians. In this case the linear model is **too rigid**, and the next models are needed.

Note — Least squares and maximum likelihood

Least squares corresponds to the **maximum likelihood** criterion if the experimental errors are normally distributed.

5.4.7 Inductive bias of the linear model

- **Language bias:** H is the set of linear functions, which can be very restrictive and rigid.
- **Search bias:** ordered search guided by least squares minimisation. One might prefer a different method, which for example restricts the parameter values, obtaining solutions with different properties (in particular with respect to generalisation).

Even for a “simple” model there are many possibilities: a principled approach (ML theory) is needed.

5.5 Limits of linear models

5.5.1 Regression of non-linear problems

If the target function is not linear (like the sine of the polynomial example), a line is **too poor** a solution: it is the $M = 1$ case already seen, well within the underfitting zone.

5.5.2 Classification: linear separability

Definition — Linear separability

Two sets of points in the plane are **linearly separable** if they can be completely separated by a single line. In general, two groups are linearly separable in an n -dimensional space if they can be separated by a hyperplane of dimension $n - 1$.

The linear decision boundary provides exact solutions **only** for linearly separable sets.

The linear model can represent, for example, **conjunctions**. The conjunction $x_1 \wedge x_2 \wedge x_4$ is realised with $1 \cdot x_1 + 1 \cdot x_2 + 0 \cdot x_3 + 1 \cdot x_4 \geq 2.5$ (true only if all three are 1); the two-variable AND with $x_1 + x_2 \geq 1.5$. The weights $\mathbf{w}$ of these solutions can be learned.

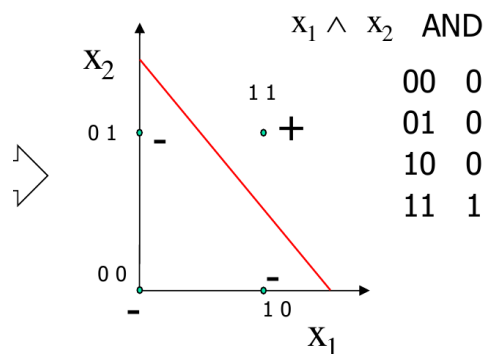


Figure 5.12: AND is linearly separable.

But not everything is separable:

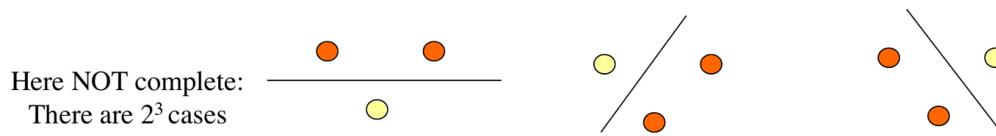
- **3 points:** can we always find a hyperplane for every assignment of labels? Yes, **if they are not collinear**. If they are collinear, the labelling “1–0–1” (0 in the middle) is not separable.
- **4 points:** no. There is a labelling for which no linear classifier is perfect: **XOR** ($00 \mapsto 0$, $01 \mapsto 1$, $10 \mapsto 1$, $11 \mapsto 0$), in which the positives lie on one diagonal and the negatives on the other.

Note — Preview

This “counting how many labellings can be separated” is exactly the idea of the **VC-dimension**: a hyperplane in the plane separates all labellings of 3 (non-collinear) points, but not of 4. The VC-dimension of lines in the plane is therefore 3.

5.6 Extending the linear model

- Given **3 points**, can we always find a separation plane for every assignment of $f(\mathbf{x})$?
No, 3 aligned points with 0 in the middle and others 1; yes if they are not aligned (existence!).



- Given **4 points**, can we always find a separation plane for every assignment of $f(\mathbf{x})$?
No (XOR)
 we can find a labeling such that the linear classifier fails to be perfect

		XOR
		00 0
		01 1
		10 1
		11 0

Exercise: try to redo it by yourself!

Figure 5.13: Limits of linear separation: three non-collinear points are always separable, XOR on four points is not.

5.6.1 “Linear” in the parameters, not in the inputs

In parametric statistical models, “**linear**” **does not refer to the line**, but to the way the **coefficients $\mathbf{w}$** appear in the equation. Transformed inputs such as $x, x^2, x^3, \dots$ can therefore be used to model non-linear relationships between input and output, keeping **the same learning machinery** (least squares). This is exactly the polynomial regression of the introduction:

$$h_{\mathbf{w}}(x) = w_0 + w_1x + w_2x^2 + \dots + w_Mx^M = \sum_{j=0}^M w_jx^j.$$

5.6.2 Linear Basis Expansion (LBE)

Definition — Linear basis expansion

$$h_{\mathbf{w}}(\mathbf{x}) = \sum_{k=0}^K w_k \phi_k(\mathbf{x})$$

where the $\phi_k : \mathbb{R}^n \rightarrow \mathbb{R}$ are fixed transformations of the input. The input vector is augmented with new variables derived from $\mathbf{x}$.

Examples of ϕ :

- polynomial** representation: $\phi(\mathbf{x}) = x_j^2, \phi(\mathbf{x}) = x_jx_i, \dots$;
- non-linear transformations of single inputs: $\log(x_j), \sqrt{x_j}, \dots$;
- transformations of multiple inputs: $\phi(\mathbf{x}) = \|\mathbf{x}\|$;
- splines, ...

For example: $h(\mathbf{x}) = w_1x_1 + w_2x_2 + w_3 \log(x_2) + w_4 \log(x_3) + w_5(x_2x_3) + w_0$.

Typically the number of parameters becomes $K > n$. The model is **linear in the parameters** (and in the ϕ), not in $\mathbf{x}$: the same learning algorithm is used. This holds for regression and also for classification (it is enough to apply the threshold to $\sum_k w_k \phi_k(\mathbf{x})$).

Warning — Pros and cons of basis expansion

Pros: it models more complex relationships, it is more expressive.

Cons: - with many basis functions one easily risks **overfitting**, so complexity control methods are needed (complexity meant as flexibility, i.e. VC-dim, not as computational cost); - the **curse of dimensionality**: the volume of the space grows so fast that the available data become sparse and are no longer enough to support the complexity of the model; - the ϕ are **fixed before** seeing the data. The alternative is adaptive models, non-linear in the parameters, such as **neural networks** (which learn the ϕ) and **SVMs**. The question then remains open: which ϕ should be chosen? (“dictionary” approaches).

5.7 Regularisation: complexity control

How can model complexity be controlled? Among the many approaches there is **coefficient shrinkage**.

5.7.1 Ridge regression (Tikhonov regularisation)

A term that **penalises large weights** is added to the loss:

$$\text{Loss}(\mathbf{w}) = \underbrace{\sum_{p=1}^l (y_p - \mathbf{x}_p^T \mathbf{w})^2}_{\text{error term } (R_{emp})} + \underbrace{\lambda \|\mathbf{w}\|^2}_{\text{penalty term}}, \quad \|\mathbf{w}\|^2 = \sum_j w_j^2.$$

λ is the **regularisation hyperparameter**, a small positive value chosen during **model selection**. The resulting model is *smoother*: models with small or zero weights are favoured, i.e. models that use fewer terms, hence less complex.

Note — Loss and Error are no longer synonyms

From here on **Loss** denotes the objective function used for training (error + penalty), while **Error** E denotes the measure of the model’s error (the data term). So far we had treated them as equivalent.

Tip — The learning goal changes

With regularisation the goal is redefined: we no longer want **only** the model that best fits the data, but **the simplest model that fits the data well**. The loss is a balance between two pans: - **fit to the data** (training error, $\sum_p (y_p - \mathbf{w}^T \mathbf{x}_p)^2$): pushes the model to follow the data and minimise the residuals; - **model simplicity** (Tikhonov penalty, $\|\mathbf{w}\|^2$): punishes large weights, making the model smoother, simpler and better able to generalise; and the **regularisation parameter** λ decides how much weight to give to the penalty.

Solution

- **Direct approach:** $\mathbf{w} = (X^T X + \lambda I)^{-1} X^T \mathbf{y}$. The matrix $X^T X + \lambda I$ is **always invertible** (for $\lambda > 0$): a numerical advantage too.

- **Gradient approach:** we still follow the negative gradient of the loss. The derivative of the penalty term with respect to w_j is $2\lambda w_j$, so the new rule is

$$\mathbf{w}_{new} = \mathbf{w} + \eta \Delta \mathbf{w} - 2\lambda \mathbf{w},$$

where $\Delta \mathbf{w}$ is the negative gradient of the error term alone (multiplied by η), while the penalty is multiplied directly by λ . This is the **weight decay** technique: even with zero error gradient, at each step every weight is reduced by a fraction of its value.

The trade-off governed by λ

- **Small λ :** when minimising the loss the error term dominates; we obtain a model that is too complex (high-norm weights), with a risk of **overfitting**.
- **Large λ :** the penalty dominates; the error on the data may grow too much, and we move towards **underfitting**.

The great advantage is that we have a **concrete realisation of complexity control**, easy to implement and of general applicability.

Tip — Tikhonov and SLT

The penalty pushes all weights towards small values (some even to zero), achieving complexity control: we obtain a model with a lower (or more suitable) VC-dimension, and the trade-off is controlled by a **single parameter**, λ . In the SLT bound plot, increasing λ amounts to moving to the left (less complexity): a well-chosen λ brings us close to the minimum of the bound on R .

Effect on the degree-9 polynomial

Let us go back to the degree-9 polynomial with 10 points, which without regularisation ($\lambda = 0$) had zero training error and very strong overfitting.

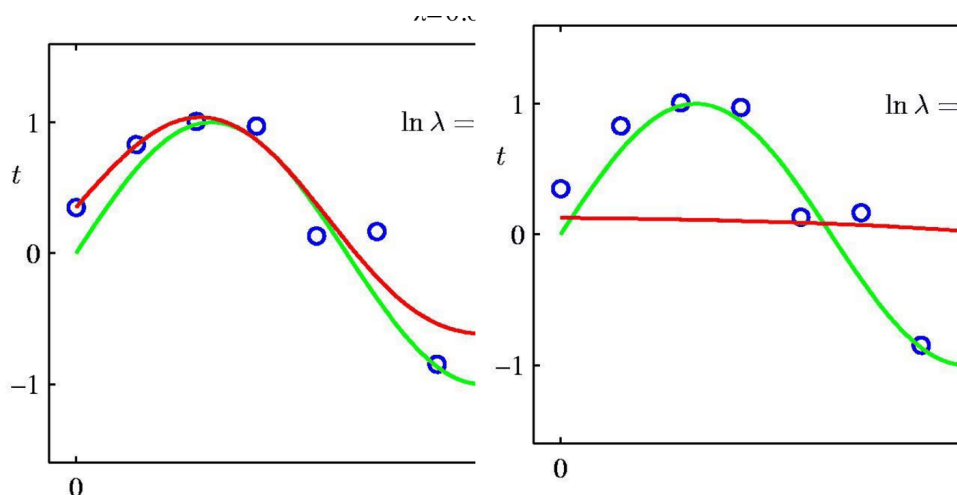


Figure 5.14: Regularised degree-9 polynomial: with $\ln \lambda = -18$ (left) the curve is smooth and follows the true function well; with $\ln \lambda = 0$ (right) it is too flat.

With a suitable λ ($\ln \lambda = -18$, i.e. $\lambda \approx 1.5 \cdot 10^{-8}$) the model works well: it is **worse on the training set** but reduces overfitting. With λ too high ($\ln \lambda = 0$, i.e. $\lambda = 1$) we end up in underfitting.

The coefficients also show the effect of regularisation:

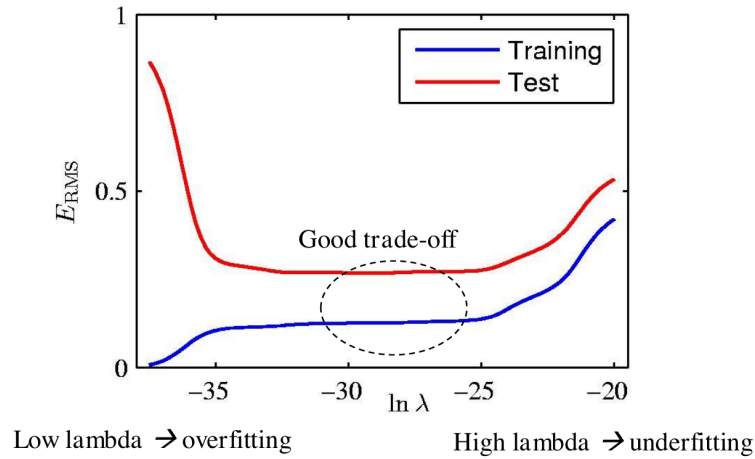


Figure 5.15: Training and test E_{RMS} as $\ln \lambda$ varies: overfitting on the left, underfitting on the right, the good trade-off in the middle.

	$\ln \lambda = -\infty$ ($\lambda = 0$)	$\ln \lambda = -18$	$\ln \lambda = 0$
w_0^*	0.35	0.35	0.13
w_1^*	232.37	4.74	-0.05
w_2^*	-5321.83	-0.77	-0.06
w_3^*	48568.31	-31.97	-0.05
w_4^*	-231639.30	-3.89	-0.03
w_5^*	640042.26	55.28	-0.02
w_6^*	-1061800.52	41.32	-0.01
w_7^*	1042400.18	-45.95	-0.00
w_8^*	-557682.99	-91.53	0.00
w_9^*	125201.43	72.68	0.01

The huge weights of the unregularised case are drastically reduced: regularisation “tames” the oscillations.

5.7.2 Other regularisation techniques

- **Ridge regression** ($\|\cdot\|_2$ norm): penalises the square of the weights and tends to make **all** weights smaller.
- **Lasso** ($\|\cdot\|_1$ norm): penalises the absolute value and tends to bring **some weights exactly to zero** (leaving others large) $\rightarrow$ a form of **feature selection**. Unfortunately the absolute value makes the loss non-differentiable (at zero), and other approaches are needed.
- **Elastic net**: uses both norms.

Tip — Why does L1 zero out weights and L2 does not?

The gradient of λw^2 is $2\lambda w$: it becomes tiny when w is close to zero, so the push towards zero fades and the weight stays small but non-zero. The (sub-)gradient of $\lambda|w|$ is instead $\lambda \text{sign}(w)$: a **constant** push towards zero, even for weights that are already small, which brings them exactly to zero if the error term does not “defend” them.

5.7.3 Other improvements (optional)

- **Data augmentation:** adding noise to the inputs teaches the model to ignore irrelevant variations (robustness and regularisation); oversampling helps with unbalanced datasets.
- **Derived inputs:** a small number of new variables are used, linear combinations of the inputs (Principal Component Regression, Partial Least Squares).

5.7.4 Multi-class classification (optional)

Two simple approaches:

- **1-of-K** encoding of the classes (e.g. red, green, blue $\rightarrow (0, 0, 1), (0, 1, 0), (1, 0, 0)$) and one linear model for each component;
- **OVA** (*one-vs-all*): K binary classifiers are trained, each distinguishing one class from all the others; to classify, the one with the highest (most positive) output is chosen;
- **AVA** (*all-vs-all, one-vs-one*): one classifier for each pair of classes ($K(K - 1)/2$ distinct classifiers); the class with the maximum sum of outputs or with the most votes wins. Each classifier is trained on less data.

With many classes **masking** phenomena may occur (a class “hidden” by the others). There are more sophisticated approaches and models with native multiple outputs. Other well-known linear classifiers are **Linear Discriminant Analysis** and **logistic regression** (which directly models $P(y | \mathbf{x})$).

5.7.5 A look ahead

- The **Perceptron** (Rosenblatt, 1958, biologically inspired) minimises only the misclassifications and is the basis of **neural networks**: several units organised in layers, which implement an **adaptive basis expansion** (the ϕ are learned), i.e. *representation learning*, trained with gradient descent.
- **SVMs** (Vapnik, 1996) regularise through **margin maximisation** between the classes and enlarge the feature space with basis expansion (e.g. polynomials).

Both implement a flexible non-linear approximation for classification and regression.

Summary — Lessons learned from linear models

- It is possible to **learn by tuning free parameters $\mathbf{w}$** , searching in a continuous space guided by a loss function.
- The problem is formulated as **LMS** and two algorithms are derived: **direct** (normal equations, SVD) and **iterative** (gradient descent, delta rule).
- Linear models have a strong **language bias** (only linearly separable problems).
- Two key concepts of the common thread of the course: **basis expansion** (more **flexibility**) and **regularisation** (**complexity control**). We will find them again, in different forms, in neural networks and SVMs.

5.8 K-nearest neighbors

Now we move to the opposite extreme: from a rigid approach to a **very flexible** (and local) one.

5.8.1 Eager and lazy learning

With respect to the *timing* of learning we distinguish:

- **eager**: the training data are analysed and an **explicit hypothesis** is built (like the linear model);
- **lazy**: the data are stored and we wait for a test point; only then is an ad hoc hypothesis built to classify that point.

5.8.2 1-nearest neighbor

Definition — 1-NN algorithm

1. Store the training data $\langle \mathbf{x}_p, y_p \rangle$, $p = 1, \dots, l$.
2. Given an input $\mathbf{x}$ (of dimension n), find the nearest training example:

$$i(\mathbf{x}) = \arg \min_p d(\mathbf{x}, \mathbf{x}_p),$$

- for example with the Euclidean distance $d(\mathbf{x}, \mathbf{x}_p) = \sqrt{\sum_{t=1}^n (x_t - x_{p,t})^2} = \|\mathbf{x} - \mathbf{x}_p\|$.
3. Return y_i .

1-NN is **very flexible**: on the training set it makes **zero errors** (each point is its own nearest neighbour). The decision boundary is not linear and is very irregular, perhaps needlessly noisy (for example in scenario 1, where we know that the optimal solution is almost linear). And on the test data?

5.8.3 K-nearest neighbors

A natural way to classify a new point is to look at its **neighbours** and take their average:

$$\text{avg}_k(\mathbf{x}) = \frac{1}{k} \sum_{\mathbf{x}_i \in N_k(\mathbf{x})} y_i,$$

where $N_k(\mathbf{x})$ is the set of the k patterns closest to $\mathbf{x}$ according to the distance d . If in the neighbourhood of $\mathbf{x}$ one class clearly dominates, $\mathbf{x}$ is likely to belong to it too: the classification rule is the **majority vote** among the members of $N_k(\mathbf{x})$. For targets in $\{0, 1\}$:

$$h(\mathbf{x}) = \begin{cases} 1 & \text{if } \text{avg}_k(\mathbf{x}) > 0.5 \\ 0 & \text{otherwise.} \end{cases}$$

For **regression** the average of the targets of the k neighbours is used directly.

With $k = 15$ the model is still very flexible, but makes some errors on the training set; the boundary is still irregular but less so, and it **adapts to the local densities** of the classes.

K-NN implicitly uses the **Voronoi diagram**: each cell contains all points closer to a certain pattern than to any other, and the edges of the cells are the points equidistant from two patterns. 1-NN assigns to each cell the label of its pattern.

5.8.4 Behaviour as k varies

In K-NN too we find the **trade-off between underfitting and overfitting**, governed by the value of k . The test error has the classic U shape moving between two extremes:

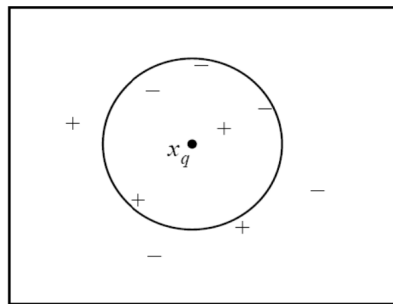


Figure 5.16: 1-NN returns “+” for x_q , 5-NN returns “-”: increasing k “smooths” the decision, useful with noisy data.

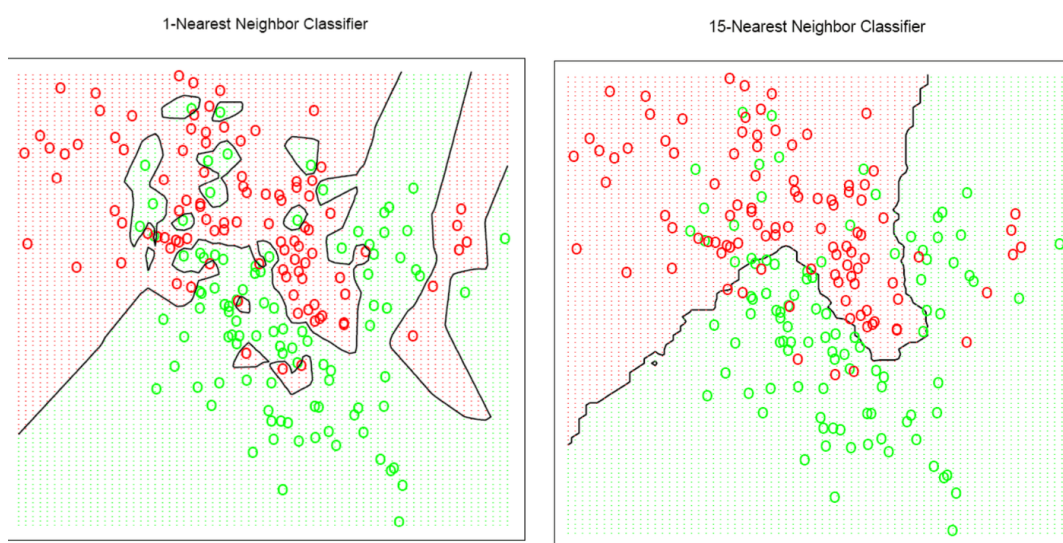
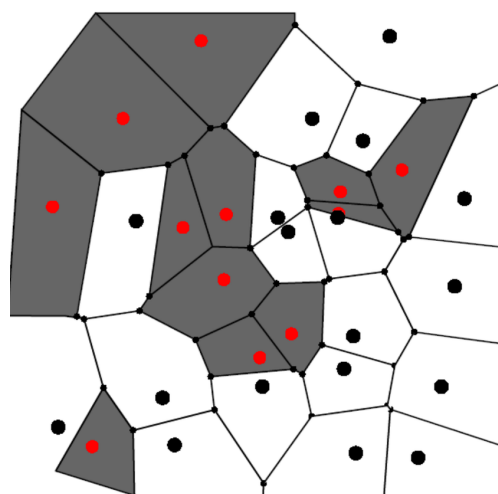


Figure 5.17: Decision boundaries of 1-NN (left) and 15-NN (right) on the same problem.



Each cell consisting of all points closer to x than to any other patterns.

The segments of the Voronoi diagram are all the points in the plane that are equidistant to two patterns.

Figure 5.18: Voronoi diagram: the 1-NN boundary follows the edges of the cells between points of different classes.

- $k = 1$: the **extremely flexible** case (zero training error, jagged boundary, risk of overfitting);
- $k = l$ (all data): a **very rigid** model, which answers with **a single average for all data** (for classification, the most frequent class in the training set), whatever the input.

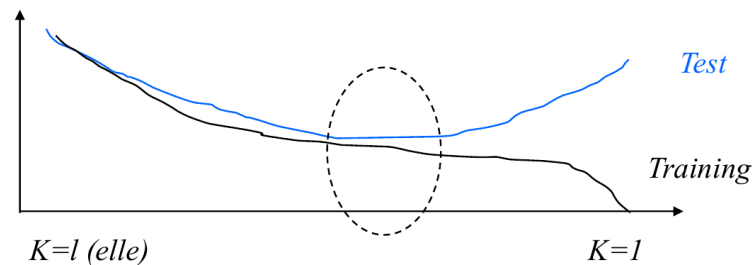


Figure 5.19: K-NN behaviour: from $k = l$ (rigid) to $k = 1$ (flexible) the training error always decreases, the test error is U-shaped.

Where have we seen this before? It is the same trend as in **polynomial fitting** as the degree M grows and in the plot of the SLT **VC-bound** (04 - Generalisation and validation (introduction)): here it is k (read from right to left, i.e. l/k) that acts as complexity control. Intermediate values of k give the best trade-off.

5.8.5 Variants

Multi-class: the most frequent class among the k neighbours is returned:

$$h(\mathbf{x}) = \arg \max_v \sum_{\mathbf{x}_i \in N_k(\mathbf{x})} \mathbb{1}_{v, y_i}, \quad \mathbb{1}_{v, y_i} = \begin{cases} 1 & \text{if } v = y_i \\ 0 & \text{otherwise.} \end{cases}$$

Distance-weighted: closer neighbours count more:

$$h(\mathbf{x}) = \arg \max_v \sum_{\mathbf{x}_i \in N_k(\mathbf{x})} \frac{\mathbb{1}_{v, y_i}}{d(\mathbf{x}, \mathbf{x}_i)^2},$$

and if $d = 0$ for some $\mathbf{x}_i$, y_i is returned directly.

5.8.6 An extreme of ML

K-NN does not build a global hypothesis valid for all instances: **there is no model to train**, but the examples must be stored. It makes **local** estimates (with locally constant functions) instead of a global linear approximation. It is a *lazy* method, memory-based, instance-based, distance-based.

5.8.7 K-NN versus linear model

	Linear	K-NN
Flexibility	rigid (low variance)	flexible (high variance)
Timing	eager	lazy
Type	parametric	instance-based

	Linear	K-NN
Parameters	$n + 1$ (3 in the example)	about l/k “effective parameters”

With small k a few points are enough to change the boundary: flexibility comes at a price. One might think that K-NN has a single parameter (k), but realistically it has about l/k **effective parameters** (Hastie-Tibshirani-Friedman): it is as if it divided the space into l/k regions, each with its own “mean”.

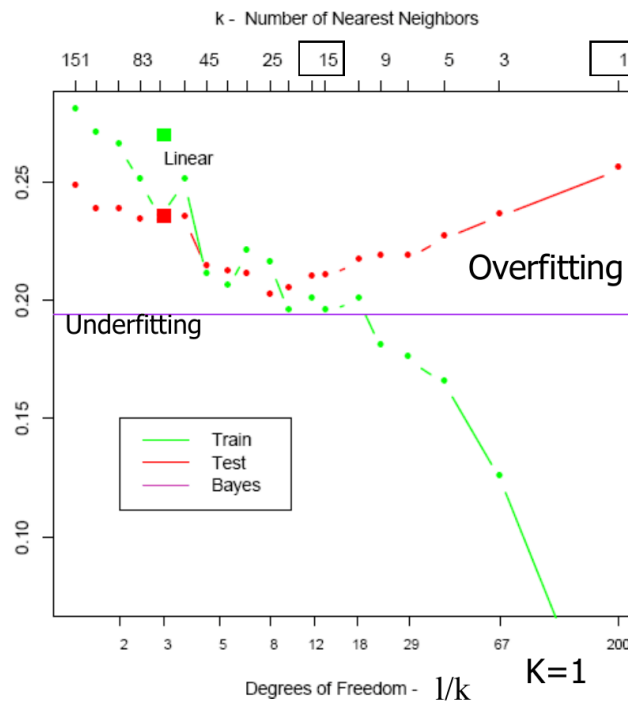


Figure 5.20: K-NN error curves as k varies (training of 200 points, test of 10000). The squares are the linear model, the purple line the optimal Bayes error.

Moving from large k to $k = 1$ (i.e. increasing l/k) we go from **underfitting** to **overfitting**: the training error drops to zero, the test error has a minimum for intermediate values. It is again the SLT bound plot, with k in the role of complexity control: more flexibility makes it possible to find the best result, **if** complexity is controlled.

5.8.8 The Bayes optimal classifier

Definition — Bayes classifier and Bayes error

If the density $P(\mathbf{x}, y)$ is known, the optimal choice is to assign the most probable class:

$$h(\mathbf{x}) = \arg \max_v P(v | \mathbf{x}), \quad v \in \{C_1, \dots, C_K\}.$$

Its error rate, called the **Bayes rate**, is the **minimum achievable error** given the data distribution (assuming the generating density is known).

K-NN directly approximates this solution: the majority vote in a neighbourhood is exactly an estimate of $\arg \max_v P(v | \mathbf{x})$, with two relaxations: the conditional probability **at a point**

becomes the conditional probability **in a neighbourhood** of the point (local approximation), and probabilities are estimated with the **proportions** in the training sample.

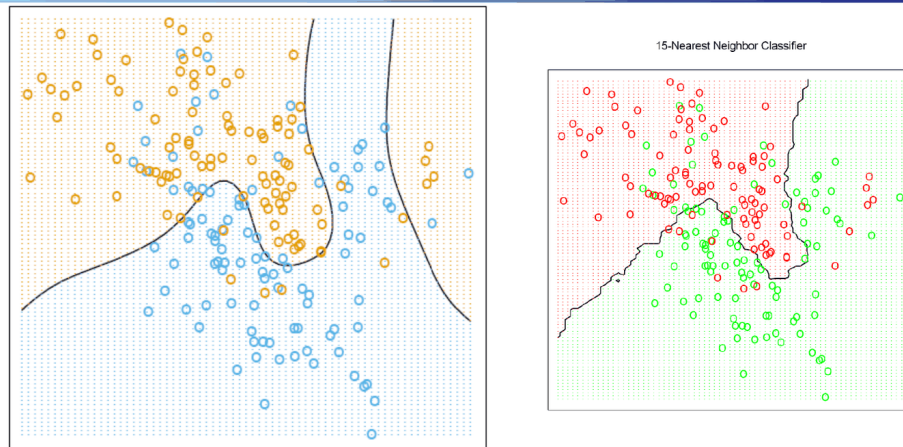


Figure 5.21: Optimal Bayes boundary (left) and 15-NN boundary (right): the 15-NN is very close to the optimum, consistently with the minimum of the test error in the previous figure.

5.8.9 Inductive bias of K-NN

1. **The assumed distance** establishes which examples are similar: it is assumed that the classification of a point is similar to that of its neighbours **according to the chosen metric**. Metrics other than the Euclidean one can be used; symbolic data require ad hoc metrics (e.g. the **Hamming distance** between strings of the same length: the number of positions in which they differ).
2. **Locality**: local approximation, with an assumption of local *smoothness* (to be compared later with the bias of deep learning).

The bias related to the metric is often underestimated in books, but it is crucial.

5.8.10 Limits of K-NN

Variable scale and metric

The choice of scale depends on domain knowledge. If the variables must contribute equally, one must pay attention to differences in range: **rescaling the data** (e.g. zero mean and unit variance) is equivalent to **changing the metric**. Scaling can completely change which is the nearest neighbour: K-NN is fragile even with respect to elementary pre-processing.

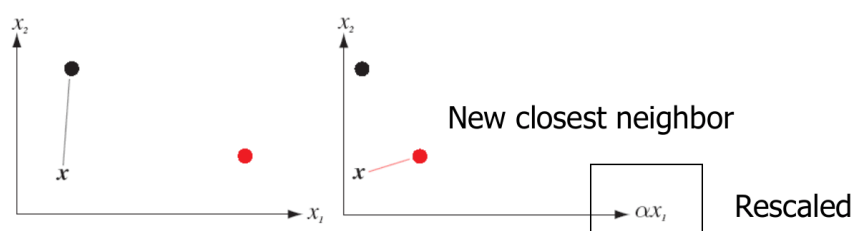


Figure 5.22: Rescaling a variable changes the nearest neighbour.

Computational cost

K-NN builds the local approximation **for each new example** to be predicted: the computational cost is **shifted to the prediction phase**. For each input the distances from all stored vectors are computed (time proportional to the number of patterns), although there are ad hoc proximity search algorithms, such as pattern indexing or **approximate nearest neighbor search** techniques, which give up always finding the exact neighbour in exchange for a much faster search. The **space** cost is also high (all the training data).

Interpretability

K-NN models offer little interpretation, subjective and dependent on the metric.

The curse of dimensionality

K-NN works well if a significant set of data close to each $\mathbf{x}$ can be found, i.e. with **dense sampling**. When there are many input variables (high dimension n) it often fails because of the **curse of dimensionality**, which has many manifestations. Three in particular directly affect generalisation.

1. **In high dimension it is hard to find close points.** It becomes difficult to collect k observations “close” (i.e. similar) to a query point: neighbourhoods tend to be spatially large and estimates are no longer local; reducing the size of the neighbourhood means reducing k , and the variance of the estimate increases (overfitting).

Consider data uniformly distributed in the unit cube and a sub-cube that must contain a fraction r of the data. Its side must be $r^{1/n}$ (since volume = side ^{n}).

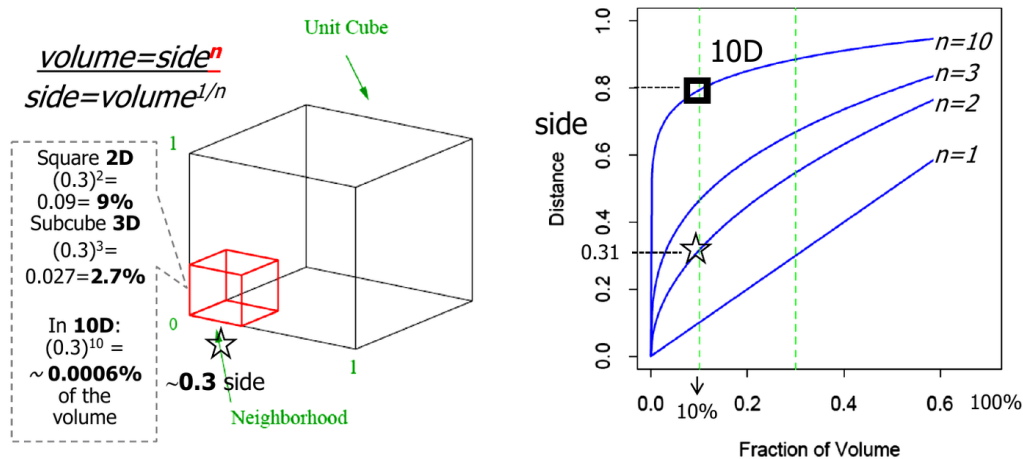


Figure 5.23: Curse of dimensionality: side of the sub-cube needed to capture a fraction r of the volume, as the dimension n varies.

- To capture 10% of the data: in 2D a side of $0.1^{1/2} \approx 0.32$ is enough (30% of the range of each variable); in 10D $0.1^{1/10} \approx 0.8$ is needed, i.e. **80% of the range** of each coordinate! Even for 1% of the volume in 10D, 63% of the range is needed. The neighbourhood is no longer “local” and the neighbours are no longer similar.
- Conversely, fixing the side at 0.3: in 1D we capture 30% of the data, in 2D 9%, in 3D 2.7%, in 10D about **0.0006%**. That side may not be enough to find k data points, unless a small k is used (and thus risking overfitting).

2. Low sampling density. The sampling density is proportional to $l^{1/n}$. If 100 points are enough to estimate a function in $\mathbb{R}^1$, to obtain similar accuracy in $\mathbb{R}^{10}$ $100^{10} = 10^{20}$ are needed.

3. Irrelevant features (curse of noisy). If the target depends only on a few of the many features (e.g. 2 out of 20), the similarity between patterns can be dominated by the large number of irrelevant features, and the “neighbour” found is not really similar. The problem grows with dimensionality. Remedies: **weighting the features** according to their relevance (stretching the axes along some dimensions; the weights can be searched with an expensive model selection or other methods) or performing **feature selection** (eliminating variables, reducing the input dimension).

5.8.11 K-NN design choices

- The **metric** d (Euclidean, Hamming, Manhattan, feature weights...): it is often the key to the success of an application.
- The value of k , which controls underfitting and overfitting.
- It is often necessary to select a **subset of the data** (a set of prototypes, for example via clustering) and a **subset of the features**.

Extensions to other local models: *kernel smoothers*, local linear regression, prototype methods, *case-based reasoning*.

Summary — General lessons from K-NN

- Too low a variance is poor (rigid linear models), too high a variance is dangerous (K-NN).
- **Smoothing**: in K-NN it is obtained by increasing k .
- **Curse of dimensionality**: the volume of the space grows so fast that the data become sparse.
- The error curves as k varies are another instance of the SLT bound plot.
- The inductive bias of K-NN is related to the metric.

5.8.12 What next?

K-NN does not build a real “learned model”: it does not extract regularities or synthesise knowledge, and it does not satisfy the goal of learning (being able to “forget” the examples after building the model). In the next lectures we look for models that are **compact** like the LTU (all knowledge in a few parameters) but **flexible** like K-NN, with adequate support for complexity control: **neural networks**.

The professor suggests looking at the slides of the following lectures in advance, and in particular at the **proof of the Perceptron convergence theorem** (06 - Neural networks (part 1) - from the neuron to the MLP).

Questions — Possible exam questions

- Formulate the learning problem for a linear model with LMS, for regression and classification.
- Derive the gradient of the squared error and the normal equations.
- Describe the gradient descent algorithm; difference between batch and on-line; role of η .
- Explain the delta rule as an error-correction rule.

- Why is the squared error on $\mathbf{w}^T \mathbf{x}$ used for classification instead of the 0/1 loss?
- What is linear separability? Why is XOR not linearly separable?
- What is linear basis expansion? Pros and cons.
- Write the Tikhonov loss, the direct solution and the weight decay rule; role of λ .
- Difference between L1 and L2 regularisation.
- Describe K-NN; compare it with the linear model; what is the Bayes classifier?
- Explain the curse of dimensionality.

Chapter 6

Neural networks (part 1): from the neuron to the MLP

With neural networks we enter the heart of the course. This first part starts from the biological inspiration, introduces the artificial neuron and the **Perceptron** with its learning algorithm and the **convergence theorem**, compares the Perceptron with LMS, introduces sigmoidal activation functions and arrives at the **Multi-Layer Perceptron (MLP)**, analysed as a flexible function, as an adaptive basis expansion and as a universal approximator. It closes with the problems that learning in a network raises, which will be solved by **backpropagation**.

6.1 Neural networks as an ML tool

Neural networks are studied with two different goals:

- **modelling biological systems** and learning processes (philosophy, psychology, neurobiology, cognitive science, computational neuroscience): here **biological realism** is essential;
- **building effective ML systems and algorithms** (statistics, AI, physics, mathematics, engineering), often losing strict biological realism: here the **computational and algorithmic** properties matter.

In the course we are interested in **ANNs** (*Artificial Neural Networks*) as a **flexible ML tool**, in the sense of function approximation: a network implements a mathematical function $h(\mathbf{x})$ with special properties.

Despite being a “historical machine”, neural networks remain a powerful approach:

- they learn from examples;
- they are **universal approximators** (Cybenko’s theorem): flexible approaches for arbitrary functions, including non-linear ones;
- they handle noise and incomplete data, with performance that **degrades gracefully** under adverse conditions;
- they deal with real and discrete data, for regression and classification;
- they are not a single model but a **paradigm** (in the class of sub-symbolic approaches).

The underlying “philosophy” is **connectionism**: complex (mental, behavioural) behaviours that **emerge** from the interaction of many simple interconnected computational units.

6.2 The biological inspiration

The human brain has more than 10^{10} neurons, each with 10^4 – 10^5 connections. A neuron responds in about one millisecond, yet we recognise an image in about 0.1 seconds: at most a hundred **serial** computation steps. It follows that the brain must perform a **massively parallel**

computation. (Note that face recognition, so easy for us, is complex for conventional computers.)

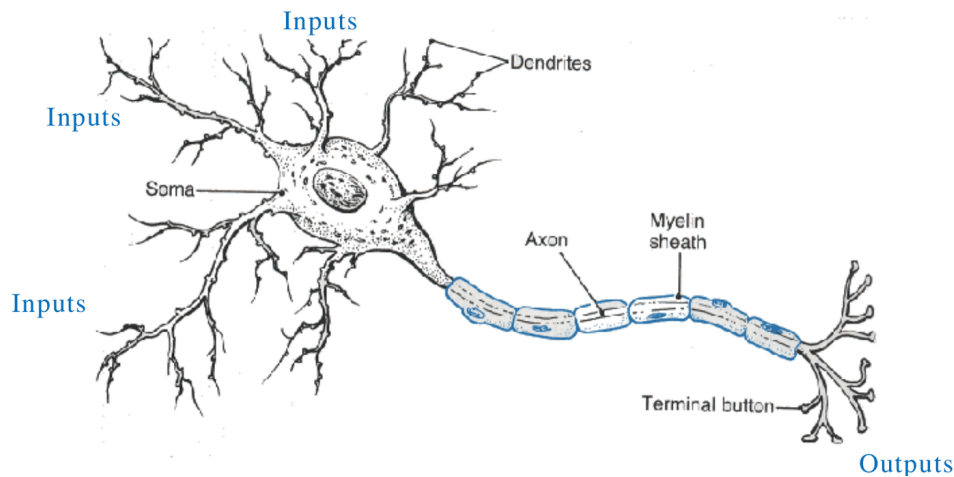


Figure 6.1: The biological neuron: dendrites (input), soma, axon and terminals (output).

How it works, in short: the neuron goes through cycles of charge and discharge (sodium and potassium ions); the inputs from the dendrites modify the potential of the cell; once a **threshold** is exceeded, a **spike** (a voltage pulse) is generated, and information is also encoded in the time intervals between spikes.

The signal propagates along the axon up to the **synapses**, where it passes (in chemical form, through **neurotransmitters**) to the next neuron. Synapses can be **excitatory** or **inhibitory**, and above all they change with learning: their strength (the “weight”) is reinforced in response to stimuli. This is the **plasticity** of the nervous system.

Definition — Hebbian learning (Hebb, 1949)

A synapse is strengthened when the inputs of the neuron (and the correlated outputs) are repeated. The incoming stimulus reinforces the synapse, hence **the weights move closer to the inputs**.

6.3 The artificial neuron

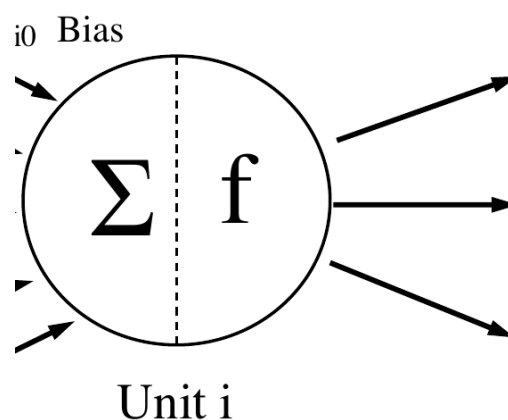


Figure 6.2: The processing unit (artificial neuron).

A **unit** (node, neuron) receives inputs from external sources or from other units. Each connection has a **weight** w , a free parameter that can be modified by learning (the analogue of

synaptic strength). Unit i computes:

$$net_i(\mathbf{x}) = \sum_j w_{ij} x_j, \quad o_i(\mathbf{x}) = f(net_i(\mathbf{x})).$$

- The weighted sum net_i is the **net input** of unit i ; it includes the bias w_{i0} through the constant input $x_0 = 1$.
- f is the **activation function** (linear, threshold, sigmoidal...).

Warning — Weight notation

w_{ij} is the weight **of unit i** , i.e. of the connection **from input/unit j to unit i** (not the other way round). Some sources (for example Wikipedia in the section on backpropagation, and some libraries) use the opposite convention; the course always uses the traditional one: the first index is the receiving unit.

Three classic activation functions:

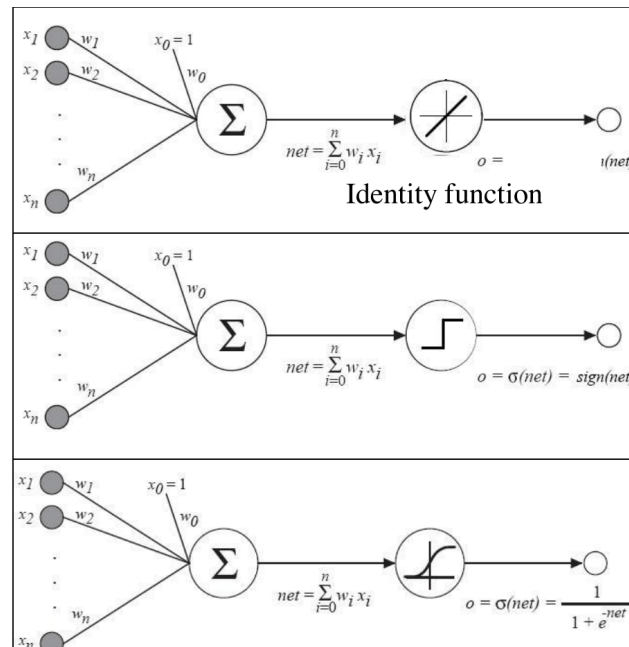


Figure 6.3: Linear (identity), threshold (Perceptron) and logistic (sigmoid) activation functions.

1. **linear** (identity): $h(\mathbf{x}) = \sum_i w_i x_i$;
2. **threshold** (step): this is the **Perceptron**, i.e. the LTU already seen;
3. **logistic** (sigmoid): see later.

6.4 The Perceptron

The **Perceptron** was proposed by Frank Rosenblatt (1957–1960) for pattern classification, simulating human perception: an “eye” of photocells connected to units that compute a weighted combination with a threshold.

The single neuron is a very simple computational unit. Minsky: “for the kind of recognition it can do, it is such a simple machine that it would be surprising if nature did not use it somewhere”. The Perceptron is a model of historical importance and **paradigmatic** for the transition

- Frank Rosenblatt (1957-1958, 1960, ...)

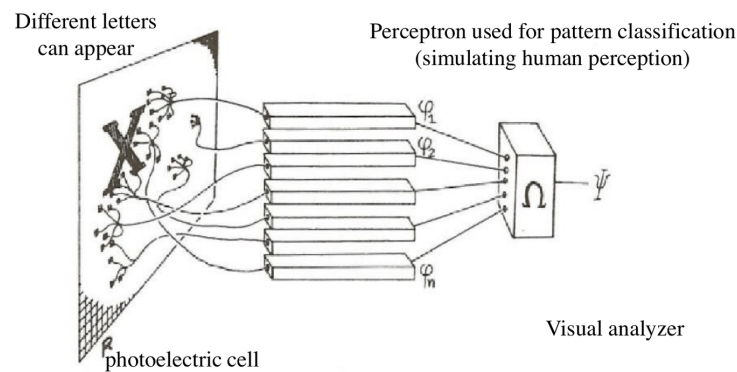


Figure 6.4: The original Perceptron, used to recognise letters.

from linear to non-linear models: by composing and connecting perceptrons one obtains **MLP** networks (*Multi-Layer Perceptron*).

6.4.1 McCulloch and Pitts networks (1943)

The first model, “*A logical calculus of the ideas immanent in nervous activity*” (1943), asked what a given network computes and whether a network can compute a given logical formula.

- Neurons have two states: active (1) and inactive (0).
- All synapses are equivalent and characterised by a real number (the strength w), positive for **excitatory** connections and negative for **inhibitory** ones.
- A neuron i becomes active when the sum of the weights w_{ij} coming from the active neurons j , plus a bias, exceeds zero.

Binary inputs and outputs: it is essentially the LTU/Perceptron.

Representing Boolean functions

With $net = \mathbf{w}^T \mathbf{x} = \sum_{i=0}^n w_i x_i$ and output 1 if $net > 0$, 0 (or -1) otherwise:

- **AND:** $w_1 = w_2 = 1$, $w_0 = -1.5$ (both inputs must be 1 to exceed 1.5);
- **OR:** $w_1 = w_2 = 1$, $w_0 = -0.5$ (one input at 1 is enough).

(Exercise: NOT is obtained with $w_1 = -1$, $w_0 = 0.5$: for $x_1 = 0$ the net is $0.5 > 0$, for $x_1 = 1$ it is -0.5 .)

XOR and two-layer networks

XOR is not linearly separable: no single perceptron can represent it (Minsky and Papert, 1969). It can however be decomposed:

$$x_1 \oplus x_2 = x_1 \bar{x}_2 + \bar{x}_1 x_2 = \bar{h}_1 \cdot h_2, \quad \text{with } h_1 = x_1 \cdot x_2 \text{ (AND), } h_2 = x_1 + x_2 \text{ (OR).}$$

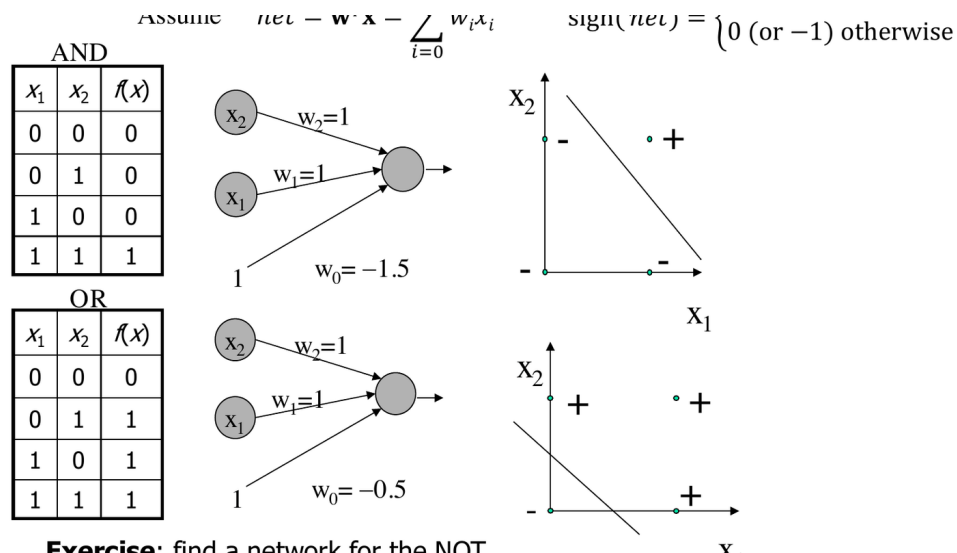


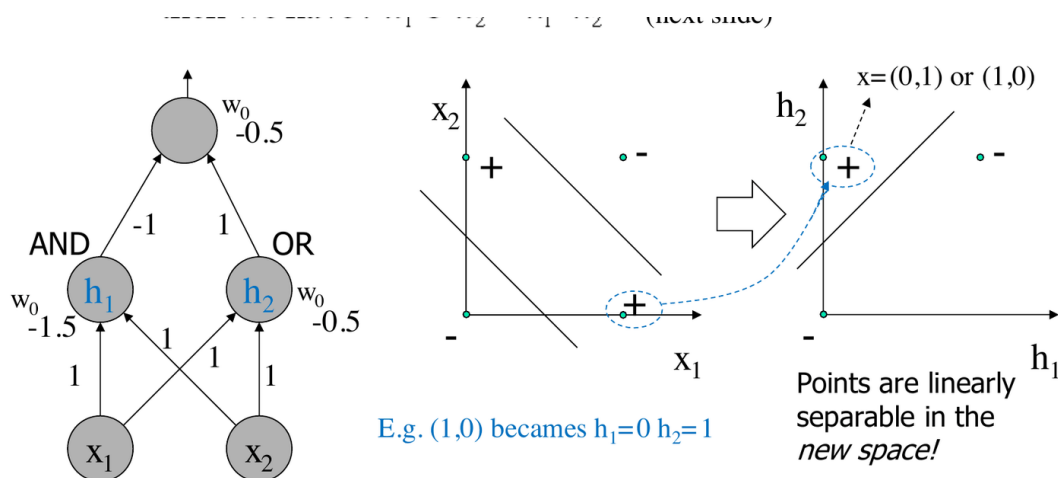
Figure 6.5: AND and OR implemented by a single perceptron.

Theorem — XOR as a composition of AND and OR

By De Morgan, $\overline{h_1} = \overline{a \wedge b} = \bar{a} \vee \bar{b}$. Hence

$$\bar{h}_1 \wedge h_2 = (\bar{a} \vee \bar{b}) \wedge (a \vee b) = (\bar{a} \wedge a) \vee (\bar{a} \wedge b) \vee (a \wedge \bar{b}) \vee (b \wedge \bar{b}) = (\bar{a} \wedge b) \vee (a \wedge \bar{b}) = a \oplus b.$$

We thus build a **two-layer** network: two hidden units compute h_1 (AND) and h_2 (OR), and an output unit computes $\bar{h}_1 \wedge h_2$ (weights -1 from h_1 and $+1$ from h_2 , bias -0.5).

Figure 6.6: XOR with a two-layer network: in the space of the hidden units (h_1, h_2) the points become linearly separable.

6.4.2 The hidden layer as a representation

The figure shows the key concept: the hidden layer produces an **internal re-representation** of the inputs. The point $(1, 0)$, which in the original space lies in the bottom-right corner, becomes $h_1 = 0$, $h_2 = 1$: the two positive points $(0, 1)$ and $(1, 0)$ are mapped **to the same point** $(0, 1)$ of the hidden space, and the problem becomes linearly separable for the output unit.

Tip — The most important concept of neural networks

Developing **high-level features** in the hidden layers is the key factor of neural networks: the representation in the hidden layer makes the task of the output layer easier. This composition of intermediate operations can be extended over **many layers of abstraction**, and — unlike what we saw here, where the weights are chosen by hand — in neural networks these representations **are learned**. This is the idea behind *representation learning* and **deep learning**.

Summary — Properties of perceptron networks

- Perceptrons represent AND, OR, NOT (and hence NAND, NOR) → **every Boolean function** can be represented by a network of perceptrons.
- **Two levels** are enough (more levels can be more efficient).
- A single layer is not enough (Minsky and Papert, 1969): limits due to linear separability.
- So far, however, **we have no learning algorithm**, not even for a single unit.

6.5 Learning for a single unit

Historically there are two families of methods:

1. **Adaline** (*Adaptive Linear Neuron*, Widrow and Hoff): during training the unit is **linear**; the direct LMS solution or gradient descent is used (exactly as in the lecture on linear models). It serves for regression and, with the threshold, classification. It is the approach we will generalise to MLPs.
2. **Perceptron** (Rosenblatt): during training the unit is **non-linear** (with a threshold function). Classification only; its capabilities and convergence are studied.

To build a linear classifier we thus have three algorithms: direct LMS, gradient LMS and now the Perceptron algorithm (and others will come, such as SVMs).

6.5.1 The Perceptron learning algorithm

The goal is to **minimise the number of misclassified patterns**: find $\mathbf{w}$ such that $\text{sign}(\mathbf{w}^T \mathbf{x}) = d$. It is an **on-line** algorithm (one step per pattern).

Summary — Perceptron Learning Algorithm

1. Initialise the weights (to zero or to small random values).
2. Choose a learning rate $\eta \in (0, 1)$.
3. Until the stopping condition is met (e.g. the weights no longer change), for each pattern $(\mathbf{x}, d)$ with $d \in \{+1, -1\}$:
 - compute $\text{out} = \text{sign}(\mathbf{w}^T \mathbf{x})$;
 - if $\text{out} = d$, **do not change the weights**;
 - if $\text{out} \neq d$, update: $\mathbf{w}_{\text{new}} = \mathbf{w} + \eta d \mathbf{x}$.

In practice $\eta \mathbf{x}$ is added if $\mathbf{w}^T \mathbf{x} \leq 0$ and $d = +1$ (false negative), and $\eta \mathbf{x}$ is subtracted if

$\mathbf{w}^T \mathbf{x} > 0$ and $d = -1$ (false positive). Equivalently:

$$\mathbf{w}_{new} = \mathbf{w} + \frac{1}{2} \eta (d - out) \mathbf{x},$$

because $d - out$ is 0 if the classification is correct and ± 2 if it is wrong.

Warning — Difference from LMS

The formula looks like the LMS one, but here *out* contains the **sign**: $out = \text{sign}(\mathbf{w}^T \mathbf{x})$, whereas in LMS the error was computed on $\mathbf{w}^T \mathbf{x}$ (without threshold).

Geometric view

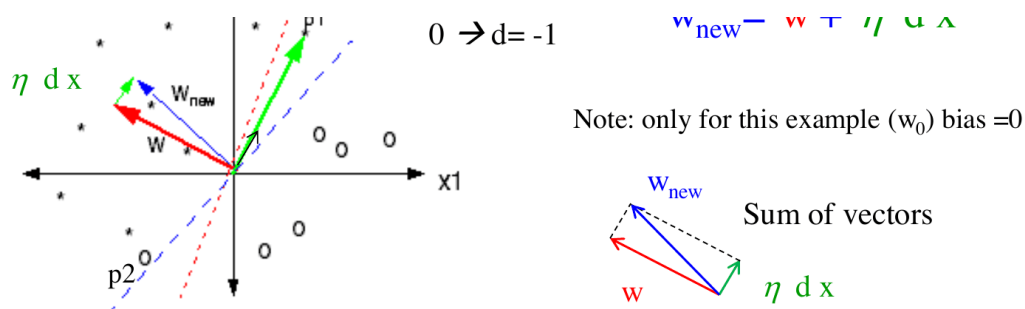


Figure 6.7: Perceptron update: $\mathbf{w}$ is moved in the direction of the misclassified pattern p_1 .

Before the update $\mathbf{w}$ (which points towards the positive region) misclassifies both p_1 and p_2 . Choosing p_1 , which has target $d = +1$, $\mathbf{w}$ is moved a little in the direction of p_1 (vector sum $\mathbf{w} + \eta d \mathbf{x}$), and the new boundary is better. (Exercise: if p_2 is chosen, with $d = -1$, $\mathbf{w}$ is moved away from p_2 .)

6.5.2 Delta rule: two readings

- The form $\mathbf{w}_{new} = \mathbf{w} + \eta d \mathbf{x}$ can be read as **Hebbian learning**: $\mathbf{w}$ moves towards $\mathbf{x}$.
- The form $\mathbf{w}_{new} = \mathbf{w} + \eta (d - out) \mathbf{x} = \mathbf{w} + \eta \delta \mathbf{x}$ is an **error-correction** rule (delta rule, Widrow-Hoff): the weight changes in proportion to the error.

In terms of neurons: *the change of a synaptic weight is proportional to the product of the error signal and the input signal that excites the synapse*. The computation is easy when the error signal δ is **directly measurable**, i.e. when we know the desired response for the unit. (This will be the problem in networks with hidden layers.)

6.5.3 Representing and learning

The Perceptron can **represent** linear decision boundaries, so it solves linearly separable problems. But can it always **learn** the solution? Yes: the Perceptron with its algorithm is always able to learn what it can represent.

Note — A historical result

The convergence theorem is a milestone: a **biologically inspired** model, with well-defined and **mathematically proven** computational capabilities.

6.5.4 The Perceptron convergence theorem

Theorem — Perceptron convergence theorem

If the problem is **linearly separable**, the Perceptron algorithm converges (correctly classifying all patterns) in a **finite number of steps**, regardless of the starting point (although the final solution is not unique and depends on the starting point).

If the problem is not separable, the algorithm can be unstable: it develops **cycles**, with weights that are not necessarily optimal.

In the proof we omit the T in dot products between vectors.

Preliminaries 1: reducing to all-positive patterns

Let $(\mathbf{x}_i, d_i)$ be the training examples, $d_i \in \{+1, -1\}$, $i = 1, \dots, l$. If the problem is linearly separable there exists a solution $\mathbf{w}^*$ such that

$$d_i(\mathbf{w}^* \mathbf{x}_i) \geq \alpha, \quad \text{with } \alpha = \min_i d_i(\mathbf{w}^* \mathbf{x}_i) > 0.$$

Hence $\mathbf{w}^*(d_i \mathbf{x}_i) \geq \alpha$. Defining $\mathbf{x}'_i = d_i \mathbf{x}_i$, $\mathbf{w}^*$ is a solution of the original problem if and only if it is a solution of the problem $(\mathbf{x}'_i, +1)$ with all positive targets:

- (if) $\mathbf{w}^*$ solves the original problem $\Rightarrow d_i(\mathbf{w}^* \mathbf{x}_i) \geq \alpha \Rightarrow \mathbf{w}^* \mathbf{x}'_i \geq \alpha \Rightarrow \mathbf{w}^*$ solves $(\mathbf{x}'_i, +1)$;
- (only if) the reverse reasoning.

It is therefore enough to prove the theorem for a problem with **only positive patterns**: “flipping” the negative patterns changes nothing.

Preliminaries 2: form of the weights after q errors

Assume $\mathbf{w}(0) = \mathbf{0}$, $\eta = 1$ and define $\beta = \max_i \|\mathbf{x}_i\|^2$. With only positive patterns, the rule becomes $\mathbf{w}(j) = \mathbf{w}(j-1) + \mathbf{x}_{i_j}$ at each error. After q errors (all false negatives):

$$\mathbf{w}(q) = \sum_{j=1}^q \mathbf{x}_{i_j},$$

where i_j denotes the misclassified patterns (for example indices 2, 8, 9, 14 of the training set; the same pattern can appear several times).

Proof idea

We find a **lower bound** on $\|\mathbf{w}(q)\|^2$ that grows as q^2 and an **upper bound** that grows as q . Since a quadratic function sooner or later exceeds a linear one, the two bounds are compatible only for q up to a certain maximum value: the number of errors (and thus of updates) is finite.

Lower bound

$$\mathbf{w}^{*T} \mathbf{w}(q) = \mathbf{w}^{*T} \sum_{j=1}^q \mathbf{x}_{i_j} \geq q\alpha,$$

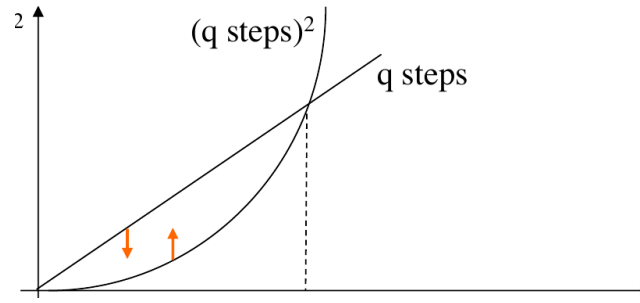
n 2^{me} ed. pages 139-141]

Figure 6.8: Proof idea: the quadratic lower bound and the linear upper bound meet at q_{max} .

because each term $\mathbf{w}^{*T} \mathbf{x}_{i_j}$ is at least α . By the **Cauchy-Schwarz inequality**, $\|\mathbf{a}\|^2 \|\mathbf{b}\|^2 \geq (\mathbf{a}^T \mathbf{b})^2$; with $\mathbf{a} = \mathbf{w}^*$ and $\mathbf{b} = \mathbf{w}(q)$:

$$\|\mathbf{w}^*\|^2 \|\mathbf{w}(q)\|^2 \geq (\mathbf{w}^{*T} \mathbf{w}(q))^2 \geq (q\alpha)^2 \Rightarrow \|\mathbf{w}(q)\|^2 \geq \frac{(q\alpha)^2}{\|\mathbf{w}^*\|^2}.$$

Upper bound

Using $\|\mathbf{a} + \mathbf{b}\|^2 = \|\mathbf{a}\|^2 + 2\mathbf{a}^T \mathbf{b} + \|\mathbf{b}\|^2$:

$$\|\mathbf{w}(q)\|^2 = \|\mathbf{w}(q-1) + \mathbf{x}_{i_q}\|^2 = \|\mathbf{w}(q-1)\|^2 + 2\mathbf{w}(q-1)^T \mathbf{x}_{i_q} + \|\mathbf{x}_{i_q}\|^2.$$

Since $\mathbf{x}_{i_q}$ was misclassified (it is the q -th error), $\mathbf{w}(q-1)^T \mathbf{x}_{i_q} \leq 0$: dropping this non-positive term we get

$$\|\mathbf{w}(q)\|^2 \leq \|\mathbf{w}(q-1)\|^2 + \|\mathbf{x}_{i_q}\|^2,$$

and iterating (with $\mathbf{w}(0) = \mathbf{0}$):

$$\|\mathbf{w}(q)\|^2 \leq \sum_{j=1}^q \|\mathbf{x}_{i_j}\|^2 \leq q\beta.$$

Conclusion

Putting the two bounds together:

$$q\beta \geq \|\mathbf{w}(q)\|^2 \geq \frac{(q\alpha)^2}{\|\mathbf{w}^*\|^2} \Rightarrow q \leq \frac{\beta \|\mathbf{w}^*\|^2}{\alpha^2}.$$

The number of errors (and thus of updates) is bounded by a constant: the algorithm converges in a **finite number of steps**. ■

Tip — Interpretation of the bound

The number of steps grows with β (how “large” the patterns are) and decreases with α^2 : α measures how “well separated” the problem is (the minimum margin of the solution $\mathbf{w}^*$). Problems with classes close to the separating hyperplane require more steps. The idea of **margin** will become central in SVMs.

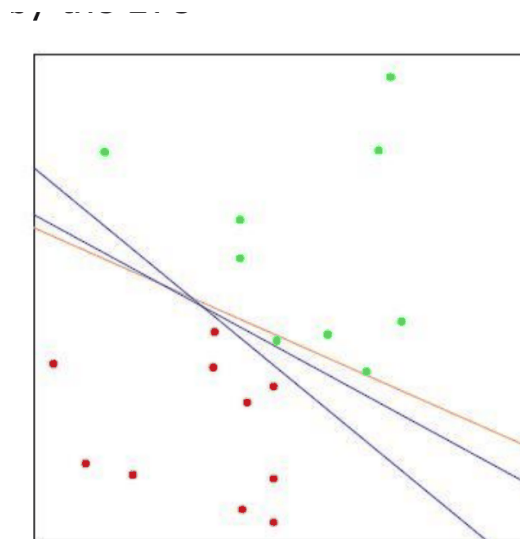
6.6 Perceptron versus LMS

The two rules look similar ($\mathbf{w}_{new} = \mathbf{w} + \eta \delta \mathbf{x}$), but:

- LMS is derived (through the gradient) **without** the threshold function, minimising the error of the linear unit: $\delta = d - \mathbf{w}^T \mathbf{x}$;
- the Perceptron uses the thresholded output: $\delta = d - \text{sign}(\mathbf{w}^T \mathbf{x})$.

Of course the model trained with LMS can be used to classify by applying the threshold ($h(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x})$, LTU).

An important consequence: **LMS can misclassify even linearly separable problems**. LMS does not minimise the number of errors of the LTU, but the squared error, and it modifies the weights **even for patterns already classified correctly**. A correct pattern that is “far” from the hyperplane (with $\mathbf{w}^T \mathbf{x}$ much larger than 1) still produces a large squared error that “pulls” the hyperplane.



4. A toy example with two classes separable by a linear function. The orange line is the least squares solution, which misclassifies one point. The two blue lines are two different separating hyperplanes found by the Perceptron algorithm.

Figure 6.9: A separable problem: the LMS solution (orange) misclassifies one point, the Perceptron (blue, two different initialisations) separates everything (Hastie et al., Fig. 4.14).

Perceptron Learning Algorithm	LMS
Minimises misclassifications ($out = \text{sign}(\mathbf{w}^T \mathbf{x})$)	Minimises $E(\mathbf{w})$ with $out = \mathbf{w}^T \mathbf{x}$
Always converges for separable problems, in a finite number of steps, to a perfect classifier	Asymptotic convergence, also for non-separable problems
Does not converge if the problem is not separable	Not always zero classification errors, even on separable problems
Hard to extend to networks of units	Extendable to networks with the gradient approach

The last row is decisive: to build neural networks we need a **gradient**-based approach, which requires differentiable functions.

6.7 Sigmoidal activation functions

Besides the linear and threshold functions, **non-linear “squashing”** functions are used, such as the **logistic sigmoid**, which takes continuous values in the bounded interval $[0, 1]$:

$$f_{\sigma}(x) = \frac{1}{1 + e^{-ax}}.$$

It is a **smooth and differentiable version of the threshold function**. The parameter a is the slope.

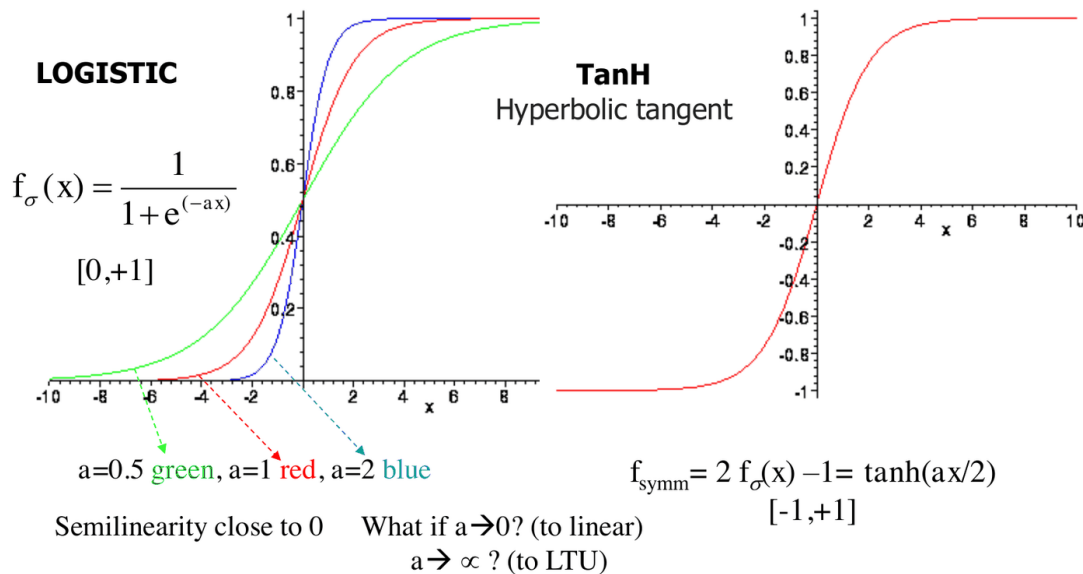


Figure 6.10: Logistic sigmoid (left) for different values of a and hyperbolic tangent (right).

The symmetric version is the **hyperbolic tangent**, with values in $[-1, +1]$:

$$f_{\text{symm}}(x) = 2f_{\sigma}(x) - 1 = \tanh(ax/2).$$

Near zero the sigmoid is almost linear (**semi-linearity**). As a varies:

- for $a \rightarrow 0$ the function tends to a **linear** function (very flat);
- for $a \rightarrow \infty$ it tends to the **step** function (LTU).

For classification, with the logistic function an output ≥ 0.5 corresponds to the positive class and < 0.5 to the zero/negative class (for tanh the threshold is 0). The threshold can be moved (studying the effect on false positives/negatives or with a ROC curve), and a **rejection zone** can also be defined around the threshold to avoid fragile decisions.

6.7.1 Other activation functions

- **Radial basis functions** (RBF): $f(x) = e^{-ax^2}$ applied to $\|\mathbf{w} - \mathbf{x}_{\text{input}}\|$, i.e. a Gaussian centred on the weights $\rightarrow$ **RBF networks**.
- **Softmax**: for the case with multiple outputs (see later lectures).
- **Stochastic neurons**: output +1 with probability $P(\text{net})$, -1 otherwise $\rightarrow$ **Boltzmann machines** and statistical mechanics models.
- **Piecewise linear** approximations of tanh, for efficient computation (even with binary shifts).

- **ReLU** (*Rectified Linear Unit*): $f(x) = \max(0, x)$. It has become the default choice for deep models; it will be discussed in deep learning.
- **Softplus**, its smooth approximation: $f(x) = \ln(1 + e^x)$.

Many activation functions have similar performance in practice.

6.7.2 Derivatives

- The derivative of the identity is 1.
- The derivative of the step function is **not defined** (at zero) and is zero elsewhere: this is exactly why it is not used with LMS.
- For sigmoids (with $a = 1$):

$$\frac{df_{\sigma}(x)}{dx} = f_{\sigma}(x)(1 - f_{\sigma}(x)), \quad \frac{d \tanh(x)}{dx} = 1 - \tanh^2(x).$$

(With generic a : $\frac{df_{\sigma}}{dx} = a f_{\sigma}(x)(1 - f_{\sigma}(x))$.)

Note — Derivation of the derivative of the logistic

$f_{\sigma}(x) = (1 + e^{-x})^{-1}$, hence $f'_{\sigma}(x) = \frac{e^{-x}}{(1+e^{-x})^2} = \frac{1}{1+e^{-x}} \cdot \frac{e^{-x}}{1+e^{-x}} = f_{\sigma}(x)(1 - f_{\sigma}(x))$, because $\frac{e^{-x}}{1+e^{-x}} = 1 - \frac{1}{1+e^{-x}}$. It is convenient: the derivative is computed from the value of the function itself.

6.7.3 LMS with a sigmoidal unit

Since the logistic is a smooth and differentiable threshold, we can derive an LMS algorithm by computing the gradient of the squared error as for linear units: we go from $o(\mathbf{x}) = \mathbf{x}^T \mathbf{w}$ to $o(\mathbf{x}) = f_{\sigma}(\mathbf{x}^T \mathbf{w})$, and minimise

$$E(\mathbf{w}) = \sum_{p=1}^l (d_p - f_{\sigma}(\mathbf{x}_p^T \mathbf{w}))^2.$$

The gradient is obtained with the **chain rule**, $\frac{\partial f}{\partial x} = \frac{\partial f}{\partial g} \frac{\partial g}{\partial x}$, using as auxiliary variables $g = \text{net}$ and then $g = \text{out}$. For a pattern p with $\text{net}_p = \mathbf{x}_p^T \mathbf{w}$:

$$\frac{\partial E_p}{\partial w_j} = \frac{\partial E_p}{\partial o_p} \cdot \frac{\partial o_p}{\partial \text{net}_p} \cdot \frac{\partial \text{net}_p}{\partial w_j} = -2(d_p - o_p) \cdot f'_{\sigma}(\text{net}_p) \cdot x_{p,j}.$$

Summing over all patterns:

$$\frac{\partial E(\mathbf{w})}{\partial w_j} = -2 \sum_{p=1}^l (d_p - o(\mathbf{x}_p)) f'_{\sigma}(\text{net}(\mathbf{x}_p)) x_{p,j} = -2 \sum_{p=1}^l \delta_p x_{p,j}.$$

This derivative answers the question: **how much does a change in w_j affect the change in E ?**

Gradient descent is identical to that for the linear unit, with a new delta:

$$\mathbf{w}_{\text{new}} = \mathbf{w} + \eta \delta_p \mathbf{x}_p, \quad \delta_p = (d_p - o(\mathbf{x}_p)) f'_{\sigma}(\text{net}(\mathbf{x}_p)).$$

It is still an **error-correction** rule, with some remarks:

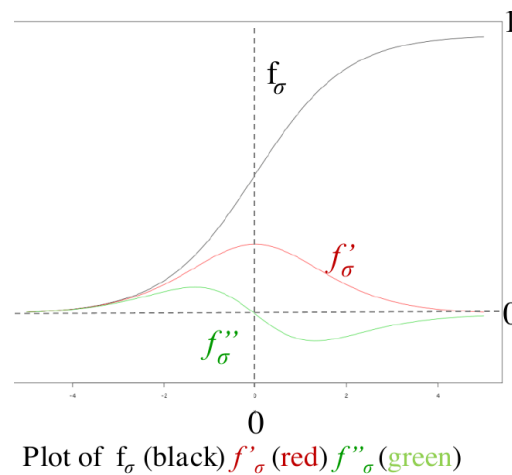


Figure 6.11: The logistic f_σ (black), its derivative f'_σ (red) and the second derivative f''_σ (green).

- the slope a affects the size of the descent step;
- f'_σ is **maximal for net close to 0** (almost linear zone): there the deltas can be large;
- f'_σ is **minimal in saturated cases** (f close to 0 or 1): small deltas, weights that change very slowly. One must **avoid premature saturation**, for example by **starting with small weights**.

Tip — A bridge between LMS and Perceptron

When a pattern is classified correctly “with confidence” (output saturated close to the target), $f'_\sigma \approx 0$ and the correction is almost zero, just as in the Perceptron, which does not correct correct patterns. Using f_σ therefore approximates not only the LTU, but also the Perceptron algorithm.

6.8 Neural networks: the Multi-Layer Perceptron

All the ingredients are ready. An **MLP** can be seen in two ways:

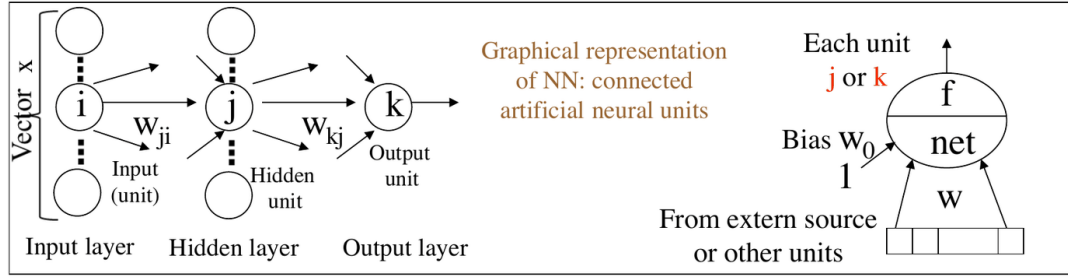
- **A.** as a **network of interconnected units**;
- **B.** as a **flexible function** $h(\mathbf{x})$, composed of nested non-linear functions.

6.8.1 View A: a network of units

In an MLP architecture:

- units are connected by weighted links and organised in **layers**;
- the **input layer** is just the source of the input $\mathbf{x}$: it loads (copies) the pattern, without computing net or f ;
- the **hidden layer** projects onto the output layer (or onto another hidden layer);
- the **output layer** produces $h(\mathbf{x})$.

A. A network of units



B. A flexible function

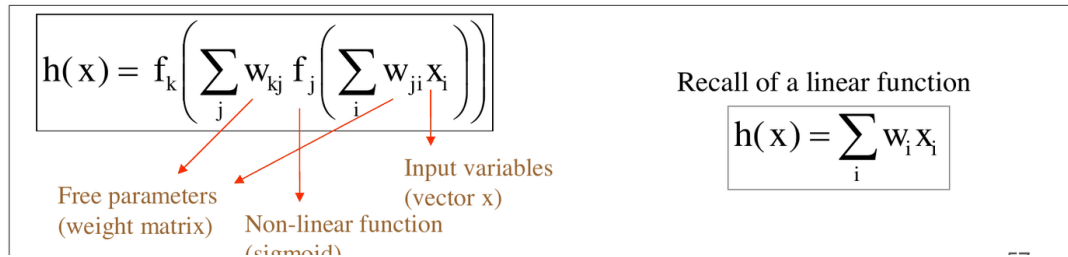


Figure 6.12: The two views of a feedforward network with one hidden layer: network of units (A) and flexible function (B).

6.8.2 View B: a flexible function

A two-layer network (one hidden and one output layer) computes

$$h(\mathbf{x}) = f_k \left(\sum_j w_{kj} f_j \left(\sum_i w_{ji} x_i \right) \right),$$

where x_i are the input variables, w the free parameters (weight matrices) and f non-linear functions (sigmoids). Compare with the linear function $h(\mathbf{x}) = \sum_i w_i x_i$: the network is a composition of linear and non-linear functions.

6.8.3 Components of a neural network

A network is traditionally described by:

- the type of **unit** (net and activation function);
- the **architecture** (number of units, topology, number of layers);
- the **learning algorithm**.

6.8.4 Uniform notation for units

For a generic unit t (which can be hidden, j , or output, k):

$$net_t(\mathbf{x}) = \sum_u w_{tu} o_u, \quad o_t(\mathbf{x}) = f_t(net_t(\mathbf{x})),$$

where u denotes a generic input source (which can be i or j). By loading the pattern into the input layer, the symbol o can be used both for the inputs and for the outputs of the hidden units: the input to unit t from source u (through the connection w_{tu}) is o_u . This notation will be used in the derivation of backpropagation.

Example — Exercise (from the lecture)

1. For the weights w_{ji} (input $\rightarrow$ hidden), the input o_u is $o_i = x_i$, the component of the pattern.
2. For the weights w_{kj} (hidden $\rightarrow$ output), the input o_u is o_j , the output of hidden unit j .
3. w_{t0} is the **bias** of each unit t , with constant input $o_0 = 1$.
4. Hidden unit: $o_j = f_j(\sum_i w_{ji} x_i)$; output unit: $o_k = f_k(\sum_j w_{kj} o_j)$.

6.8.5 Architecture and feedforward processing

The **architecture** defines the connection topology. The two-layer network described above is the standard, **fully connected MLP**; there can be several hidden layers and also connections that “skip” layers.

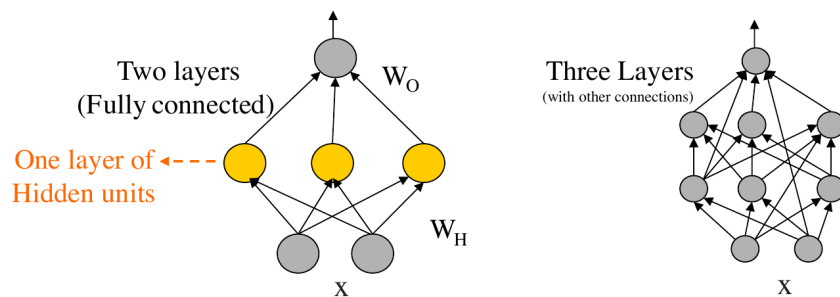


Figure 6.13: MLP architectures: two fully connected layers (left), three layers with other connections (right).

The **feedforward** processing of a pattern proceeds from input to output:

1. the pattern $\mathbf{x}$ is loaded into the input layer;
2. the outputs of all units of the first hidden layer are computed;
3. then those of the second hidden layer, and so on;
4. the outputs of the output layer are computed, i.e. $h(\mathbf{x})$;
5. the error (delta) at the output can now be computed.

Note — Recurrent networks

Feedforward networks have a single direction input $\rightarrow$ output. **Recurrent networks** add feedback connections (cycles): self-loops give the network dynamic properties and a **memory** of past computations, extending the model to the processing of **sequences** (and structured data). They will be covered in a dedicated lecture.

6.9 Why neural networks are flexible

Three questions guide the analysis: why are networks a flexible model (and can they be seen as LBE)? Is the flexibility theoretically founded? How are the weights learned?

6.9.1 Which tasks

The hypothesis space is the **continuous space of all functions** that can be represented by assigning values to the weights of a given architecture. Depending on the output units, the

network handles:

- **classification**, with sigmoidal output;
- **regression**, with linear output.

With **multiple output units** one obtains multiple regression or multi-class classifiers (for example three outputs giving 0.2; 0.7; 0.1 for three classes).

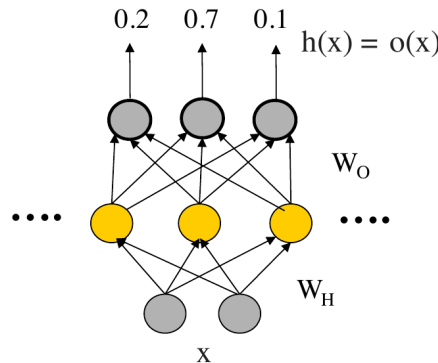


Figure 6.14: A network with multiple outputs for a multi-class problem.

6.9.2 The network as an adaptive basis expansion

The network computes $h(\mathbf{x}) = f_k \left(\sum_j w_{kj} f_j \left(\sum_i w_{ji} x_i \right) \right)$. Units and architecture are just a graphical representation of the data flow. Each term

$$\phi_j(\mathbf{x}, \mathbf{w}) = f_j \left(\sum_i w_{ji} x_i \right)$$

can be seen as computed by an independent hidden unit, or as a **basis function ϕ of an LBE**. Let us compare:

- **LBE:** $h(\mathbf{x}) = \sum_j w_j \phi_j(\mathbf{x})$ (regression) or $h(\mathbf{x}) = f \left(\sum_j w_j \phi_j(\mathbf{x}) \right)$ (classification), with the ϕ **fixed a priori** and the model linear in the parameters.
- **Neural network:** $h(\mathbf{x}) = f_k \left(\sum_j w_{kj} \phi_j(\mathbf{x}, \mathbf{w}) \right)$, where now **the ϕ depend on the weights** and therefore **adapt to the data** during training.

Tip — Neural networks as adaptive bases

Neural networks are an **adaptive basis function** approach: the bases themselves are adapted to the data (by learning the $\mathbf{w}$ inside the ϕ). All bases are of the same type (given by the activation function). $h(\mathbf{x})$ is a non-linear function of weighted sums of non-linearly transformed linear models, with the crucial improvement of **adaptivity**. (To be compared later with SVMs, another form of LBE.)

Each hidden unit computes a new **non-linear derived feature**, in an adaptive way. In other words: the representational capacity of the model is tied to the presence of a **hidden layer with non-linear activations**, which transforms the input into the **internal representation** of the network. Learning defines a suitable internal representation, i.e. new hidden features of the data, which allow the model to extract the **higher-order statistics** relevant for approximating the target function.

Warning — Hidden units must be non-linear

An MLP with **linear** units is equivalent to a **single layer** network: the composition of linear functions is linear. With f the identity, $h(\mathbf{x}) = \sum_j w_{kj} \sum_i w_{ji} x_i = \sum_i (\sum_j w_{kj} w_{ji}) x_i = \sum_i w'_i x_i$.

The price of non-linearity: the model is **non-linear in the parameters $\mathbf{w}$** , and training becomes a **non-linear optimisation** (non-convex) problem.

6.9.3 Universal approximation**Theorem — Universal approximation theorem (Cybenko 1989, Hornik et al.)**

A network with a **single hidden layer** (with logistic activations) and linear output can approximate arbitrarily well **any continuous function** (on hypercubes), provided there are enough hidden units: given f and $\varepsilon > 0$, there exists h such that $|f(\mathbf{x}) - h(\mathbf{x})| < \varepsilon$ for all $\mathbf{x}$ in the hypercube. More generally, an MLP can approximate arbitrarily well any input-output mapping, with enough hidden units.

(It can be thought of as a generalisation of approximation with finite Fourier series.)

Warning — It is an existence theorem

The theorem says that the network **exists**, but it provides neither the **algorithm** to find it nor the **number of units** needed.

Two fundamental questions therefore remain: **how to learn** (backpropagation, next lecture) and **how to choose the architecture**.

6.9.4 Expressive power and VC-dimension

The expressive power of a network depends on the **number of units** and on their **configuration**. The number of units is related to the VC-dimension: the capabilities of the network depend on the number of parameters (in a fully connected MLP, roughly the number of inputs times the number of hidden units, plus the hidden units times the outputs), but also on the **value** of the weights:

- weights = 0 $\rightarrow$ minimal VC-dim;
- small weights $\rightarrow$ we work in the linear part of the sigmoid $\rightarrow$ small VC-dim;
- large weights $\rightarrow$ more complex model.

Since training starts from weights close to 0 and the weights grow during training, the network is a **variable-dimension hypothesis space**: the effective VC-dim grows during learning. (This will be the basis of *early stopping*.)

6.9.5 How many layers? (preview)

The universal approximation theorem says that one hidden layer is enough, but it does not guarantee that a **small** number of units is enough. There are “*no flattening*” results (on efficiency, not on expressiveness): functions that with a single hidden layer would require an **exponential**

number of units (with respect to the input dimension), while with more layers far fewer are needed. But is it easy to train an MLP with many layers? We will see it in deep learning.

6.9.6 The inductive bias of neural networks

For networks trained with backpropagation, the inductive bias is related to the **smoothness** properties of functions: small changes in the input produce small changes in the output (for example, locally bounded first derivative). It is a very common assumption in ML, and it is reasonable: if the target function is not smooth (for example a random number generator), generalisation is impossible anyway.

6.10 Towards the learning algorithm

The learning algorithm adapts the weights of the network to obtain the best approximation of the target function, minimising as always an error (or loss) function on the training data.

6.10.1 The credit assignment problem

- The Perceptron has a learning algorithm, but it does not represent all Boolean functions.
- A network of perceptrons represents every Boolean function, but an algorithm is needed to train it.

What changes? The **credit assignment problem**: how much “responsibility” for the error should be attributed to the hidden units? For the output units we know the desired response and hence the error; for the hidden units **we do not know what their output should be**, so the error signal is not directly measurable. Moreover, the model is non-linear in the weights.

Minsky and Papert (1969) considered the problem too difficult; it was solved by several researchers with the **backpropagation** algorithm, popularised by Rumelhart, Hinton and Williams in the *PDP* book (1986): it was the **renaissance** of neural networks.

6.10.2 The loading problem

Definition — Loading problem

Given a network and a set of examples, does there exist a set of weights for which the network is consistent with the examples? (“Loading” the training data into the free parameters of the network.)

The loading problem is **NP-complete** (Judd, 1990): no polynomial algorithm is known. In practice, however, networks are trained in reasonable time with backpropagation, even though the optimal solution is not guaranteed.

6.10.3 The key idea: extending gradient descent

Gradient descent minimising a loss **extends to MLPs**, provided that the loss and the activation functions are **differentiable**: the task is to find the delta for **every** unit of the network. The

problem remains the same: given l examples $(\mathbf{x}_p, d_p)$ and a loss L (for example $L(h(\mathbf{x}_p), d_p) = (d_p - h_{\mathbf{w}}(\mathbf{x}_p))^2$), find $\mathbf{w}$ minimising

$$E(\mathbf{w}) = R_{emp} = \frac{1}{l} \sum_{p=1}^l (d_p - h(\mathbf{x}_p))^2$$

by computing its gradient. We need: a differentiable loss, differentiable activations and a network to follow the flow of information.

Properties of backpropagation:

- it is simple thanks to the **compositional form** of the model;
- it uses only quantities **local** to each unit, preserving modularity;
- it is **efficient**: $O(\#W)$ instead of $O(\#W^2)$, thanks to the factorisation of the deltas;
- regarding biological plausibility the debate is open (it has been hypothesised that the brain uses a sub-optimal local approximation of BP).

The full derivation is in the next lecture (and in the notes [07 - Notes on backpropagation](#)).

6.10.4 Practical issues in training

The model is often **over-parameterised** and the optimisation is **non-convex** and potentially unstable. The issues to address (after backpropagation) are: initial weight values, learning rate, on-line/batch; overfitting and regularisation; input scaling and output representation; number of hidden units; multiple minima; stopping criteria.

Summary — Summary

The artificial neuron computes $o = f(\sum_j w_j x_j)$. The **Perceptron** (threshold) has a learning algorithm that **converges in a finite number of steps** for linearly separable problems, but it does not represent XOR. Two-layer perceptron networks represent every Boolean function thanks to the **internal re-representation** of the hidden layer. Replacing the threshold with a differentiable **sigmoid**, gradient descent can be used. The **MLP** is a nested function $h(\mathbf{x}) = f_k(\sum_j w_{kj} f_j(\sum_i w_{ji} x_i))$, interpretable as an **adaptive basis expansion**, and it is a **universal approximator**. To train it, the **credit assignment** of the hidden units must be solved: backpropagation does this.

Questions — Possible exam questions

- Describe the artificial neuron and the main activation functions.
- How are AND, OR, NOT represented with a perceptron? How is XOR solved?
- Write the Perceptron learning algorithm.
- State and prove the Perceptron convergence theorem.
- Differences between the Perceptron algorithm and LMS; why can LMS fail on separable problems?
- Derive the update rule for a single sigmoidal unit.
- Why must hidden units be non-linear?
- In what sense is a neural network an adaptive LBE?
- State the universal approximation theorem and discuss its limits.
- What is the credit assignment problem? And the loading problem?

Chapter 7

Notes on backpropagation

These notes contain the **complete derivation of the backpropagation algorithm**, the algorithm that solves the *credit assignment* problem in multilayer networks introduced in [06 - Neural networks \(part 1\) - from the neuron to the MLP](#). The professor insists on one point: **the derivation must be redone by oneself, not memorised**. Only in this way is one able to recompute it for different losses, activations or architectures — and this is the real goal. Implementing it in the project is the best way to understand it thoroughly.

References: Rumelhart, Hinton, Williams, *Parallel Distributed Processing* (1986), ch. 8; Mitchell, *Machine Learning*, ch. 4; Haykin, *Neural Networks*.

7.1 Differential calculus tools

Gradient descent requires derivatives of composite functions. Three rules are needed:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial g} \cdot \frac{\partial g}{\partial x} \quad (\text{chain rule}), \quad D[f(g(x))] = f'(g(x)) \cdot g'(x), \quad D[f \cdot g] = f' \cdot g + f \cdot g'.$$

The chain rule is useful because it **decomposes** the computation, highlighting what can be computed immediately without losing the general thread.

7.2 The problem

The basic architecture is a fully connected feedforward network (MLP). Different indices are used for the layers: i for the input, j for the hidden layer, k for the output.

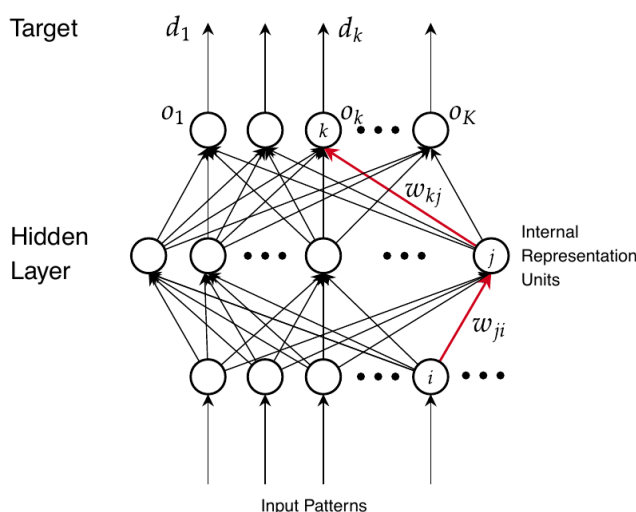


Figure 7.1: The reference network: input i , hidden units j , output units k .

Definition — The backpropagation problem

Estimate the **contribution of the hidden units to the error** at the output. This is done by computing the **Generalized Delta Rule**.

The training set is $TR = \{(\mathbf{x}^1, \mathbf{d}^1), \dots, (\mathbf{x}^l, \mathbf{d}^l)\}$ (supervised learning). Each input unit i loads the component x_i of the pattern: $o_i = x_i$.

The **total error** of the network on the training set is

$$E_{tot} = \sum_p E_p, \quad E_p = \frac{1}{2} \sum_{k=1}^K (d_k - o_k)^2,$$

where E_p is the error on pattern p computed over all K output units. (The factor $\frac{1}{2}$ only serves to simplify the derivative.) To lighten the notation, from now on the index p is omitted: o_k stands for $o_k(\mathbf{x}_p)$ and d_k for $d_{p,k}$.

Goal: find the weights $\mathbf{w}$ that minimise E_{tot} (for now only the data term). It is still a least squares minimisation (LMS). **Idea:** go down the surface of E_{tot} following the gradient.

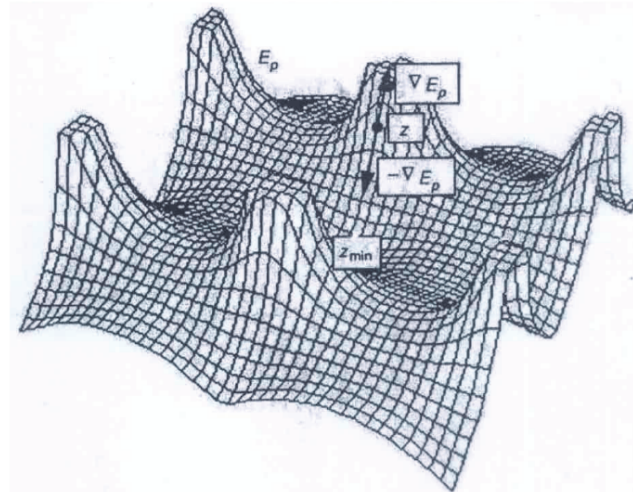


Figure 7.2: A landscape of E_{tot} in the space of two weights: following $-\nabla E$ from point Z we reach Z_{min} . Unlike the linear model, the surface is not convex.

7.3 The iterative algorithm

The loss is computed, its gradient is computed, all weights are updated; this is repeated until convergence or another stopping criterion.

Summary — Algorithm 1 — Backpropagation

Initialise all the weights $\mathbf{w}$ of the network and η . Compute the outputs of the units and E_{tot} . **While** $E_{tot} > \epsilon$ (or another criterion): - for each $w \in \mathbf{w}$: $\Delta w = -\frac{\partial E_{tot}}{\partial w}$ (**step 1**); - $w_{new} = w + \eta \Delta w + \dots$ (**step 2**); - recompute the outputs and E_{tot} .

There are stochastic (on-line) and batch versions. Two aspects must be distinguished:

1. the **computation of the gradient** $\partial E_{tot} / \partial w$, for each weight of the network: this is what backpropagation does;

2. the weight **update rule**: standard backprop, Quickprop, R-prop, ...

We focus on step 1:

$$\Delta w = -\frac{\partial E_{tot}}{\partial w} = -\sum_p \frac{\partial E_p}{\partial w} \stackrel{def}{=} \sum_p \Delta_p w.$$

7.4 The derivation

7.4.1 The gradient for a generic weight

Consider a generic weight w_{tu} , which connects the input coming from a generic unit u (which can be i or j) to the generic unit t . Since

$$net_t = \sum_s w_{ts} o_s, \quad o_t = f_t(net_t),$$

applying the chain rule through net_t :

$$\Delta_p w_{tu} = -\frac{\partial E_p}{\partial w_{tu}} = -\underbrace{\frac{\partial E_p}{\partial net_t}}_{\delta_t} \cdot \underbrace{\frac{\partial net_t}{\partial w_{tu}}}_{o_u} = \delta_t \cdot o_u.$$

Indeed $\frac{\partial net_t}{\partial w_{tu}} = \frac{\partial \sum_s w_{ts} o_s}{\partial w_{tu}} = o_u$: all terms of the sum are constant with respect to w_{tu} except the one with $s = u$. (The o_s are the inputs of unit t .)

We have defined the **delta of unit t** :

$$\delta_t = -\frac{\partial E_p}{\partial net_t}.$$

Let us expand it further with the chain rule, going through $o_t = f_t(net_t)$:

$$\delta_t = -\frac{\partial E_p}{\partial net_t} = -\frac{\partial E_p}{\partial o_t} \cdot \frac{\partial o_t}{\partial net_t} = -\frac{\partial E_p}{\partial o_t} \cdot f'_t(net_t).$$

It remains to compute $-\frac{\partial E_p}{\partial o_t}$, and here two cases are distinguished: t is an **output unit k** , or a **hidden unit j** .

7.4.2 Case 1: output unit ($t = k$)

The output o_k appears directly in E_p :

$$-\frac{\partial E_p}{\partial o_k} = -\frac{\partial \frac{1}{2} \sum_{r=1}^K (d_r - o_r)^2}{\partial o_k} = (d_k - o_k),$$

because only the term $r = k$ depends on o_k , and its derivative is $\frac{1}{2} \cdot 2(d_k - o_k) \cdot (-1)$. Hence:

$$\boxed{\delta_k = (d_k - o_k) \cdot f'_k(net_k)}$$

This is exactly the delta already seen for the single sigmoidal unit: the error is directly measurable because we know the target.

7.4.3 Case 2: hidden unit ($t = j$)

For a hidden unit there is no target: o_j does not appear directly in E_p . However, o_j affects E_p **through all the output units** it is connected to: each net_k depends on o_j . Applying the chain rule with a sum over all k :

$$-\frac{\partial E_p}{\partial o_j} = \sum_{k=1}^K \underbrace{-\frac{\partial E_p}{\partial net_k}}_{\delta_k} \cdot \frac{\partial net_k}{\partial o_j} = \sum_{k=1}^K \delta_k w_{kj},$$

since $\frac{\partial net_k}{\partial o_j} = \frac{\partial \sum_s w_{ks} o_s}{\partial o_j} = w_{kj}$. Hence:

$$\delta_j = \left(\sum_{k=1}^K \delta_k w_{kj} \right) \cdot f'_j(net_j)$$

Tip — The crucial point

The term $-\frac{\partial E_p}{\partial net_k}$ **has already been computed**: it is δ_k ! The delta of a hidden unit is thus obtained by **propagating backwards** the deltas of the output units, weighted by the same weights w_{kj} used in forward propagation. Hence the name **back-propagation**: the error “climbs back” through the network. This solves credit assignment: the “blame” of a hidden unit is the sum of the blames of the units it contributes to, weighted by how much it contributes.

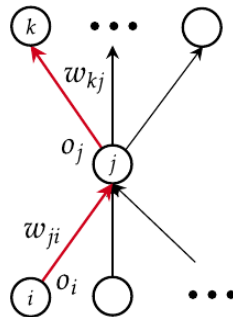


Figure 7.3: Using the network graph to localise the computation: each term involves only adjacent units and weights.

Some remarks on case 2:

- it expresses the variation of E_p considering **all** the output units o_k ;
- each o_k (and net_k) depends on o_j : this is why the sum over k appears;
- the same result can be obtained directly from the definition of E_p (as an exercise), but this form is more instructive because it shows the flow of partial derivatives **over the network**.

7.5 Summary of the formulas

For two layers we have derived:

$$\Delta_p w_{tu} = \delta_t \cdot o_u,$$

where δ_t is the **error signal** available at unit t and o_u is the input to t coming from u (through the connection w_{tu}). In particular:

Weight	Update	Delta
w_{kj} (hidden $\rightarrow$ output)	$\Delta_p w_{kj} = \delta_k \cdot o_j$	$\delta_k = (d_k - o_k) f'_k(net_k)$
w_{ji} (input $\rightarrow$ hidden)	$\Delta_p w_{ji} = \delta_j \cdot o_i$	$\delta_j = (\sum_k \delta_k w_{kj}) f'_j(net_j)$

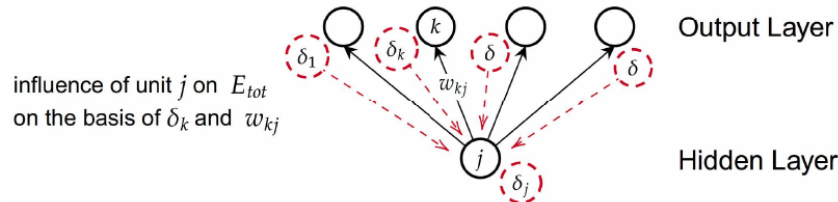


Figure 7.4: Backpropagation of the deltas: the influence of hidden unit j on E_{tot} is computed from the δ_k of the upper layer and the weights w_{kj} .

7.5.1 Generalisation to m hidden layers

The derivation holds for **any number of hidden layers**: the deltas do not come only from the output layer, but from any upper layer h the unit is connected to. In general, **the deltas come from all the units the current unit is connected to**:

$$\delta_j = \left(\sum_{h \in \text{successors}(j)} \delta_h w_{hj} \right) f'_j(net_j).$$

The update rule for each pattern (omitting p) is

$$w_{tu}^{new} = w_{tu} + \eta \delta_t o_u.$$

Using $\Delta_p w$ for each pattern gives the **on-line** version; for the **batch** version the contributions are summed over all patterns before updating.

7.6 The complete training cycle

1. **Forward computation**: outputs of all units, layer by layer.
2. Computation of the **errors and deltas in the output layer**.
3. **Backward propagation** of the deltas from the output layer to the hidden layers.
4. **Weight update**.
5. Start again, until the stopping criterion.

Warning — Do not forget the biases

Computation and update must also be applied to the **biases** of all units: the first component of the input of each unit is assumed to be 1, with the bias as weight w_{t0} . Hence $\Delta w_{t0} = \delta_t \cdot 1$.

7.7 Properties and interpretation

- **Only local computations:** each unit directly uses only nearby information (the units below and above), not distant layers; the rest arrives indirectly through the flow of deltas. In other words: even though the computation is local, changing a weight has a **global effect** on the network. What is the influence of each w_{tu} on the global error? This complex effect has been **decomposed** into simple components through the deltas local to each unit.
- **View as a diffusion process:** backpropagation is a local spatio-temporal propagation from the “parents” of the unit to the unit and from the unit to its “children”.
- **Efficiency — the “magic factorisation”:** the delta δ_t of each unit is computed **only once** and reused for **all** its weights w_{tu} . The cost is therefore proportional to the **number of weights**, not to its square. With models of billions of weights the difference is enormous: 10^9 versus $(10^9)^2 = 10^{18}$ operations, which would be infeasible.

Tip — Three levels of reading of the derivation

1. The individual **mathematical steps**.
2. The **interpretation** of the local variations of one quantity with respect to the others (the meaning of the partial derivatives).
3. The **big picture**: the decomposition serves to find the delta values of each level, decomposing the error **over the network** at hand.

Warning — Correct terminology

One does not say “backprop network”: the neural network is the **model**. Backpropagation is a **training algorithm** based on the backward propagation of errors, i.e. on the direct computation of the gradient through the network.

Example — Numerical example (to check understanding)

Network with one input $x = 1$, one sigmoidal hidden unit and one linear output unit, no bias; weights $w_{ji} = 0.5$, $w_{kj} = 1$, target $d = 1$, $\eta = 0.1$. - Forward: $net_j = 0.5$, $o_j = \sigma(0.5) \approx 0.622$; $o_k = 1 \cdot 0.622 = 0.622$. - Output delta (linear, $f' = 1$): $\delta_k = (1 - 0.622) \cdot 1 = 0.378$. - Hidden delta: $\delta_j = (\delta_k w_{kj}) \sigma'(net_j) = 0.378 \cdot 0.622 \cdot (1 - 0.622) \approx 0.089$. - Updates: $\Delta w_{kj} = \eta \delta_k o_j \approx 0.1 \cdot 0.378 \cdot 0.622 \approx 0.0235$; $\Delta w_{ji} = \eta \delta_j x \approx 0.0089$. Both weights increase, making the output grow towards the target.

Questions — Possible exam questions

- Derive backpropagation for a two-layer MLP with squared error.
- What is δ_t ? How is it computed for an output unit and for a hidden unit?
- Why does backpropagation have a cost linear in the number of weights?
- How does the derivation change with a different loss or with more hidden layers?

Chapter 8

Neural networks (part 2): training in practice

With backpropagation (07 - Notes on backpropagation) we know how to compute the gradient for each weight of the network. This lecture addresses all the practical issues needed to actually train an MLP: initialisation, multiple minima, on-line/batch/mini-batch versions, learning rate and momentum, stopping criteria, overfitting and regularisation, number of units (with Cascade Correlation), input and output representation. It closes with a summary of the strengths and weaknesses of neural networks and the first advice for the project (MONK benchmark).

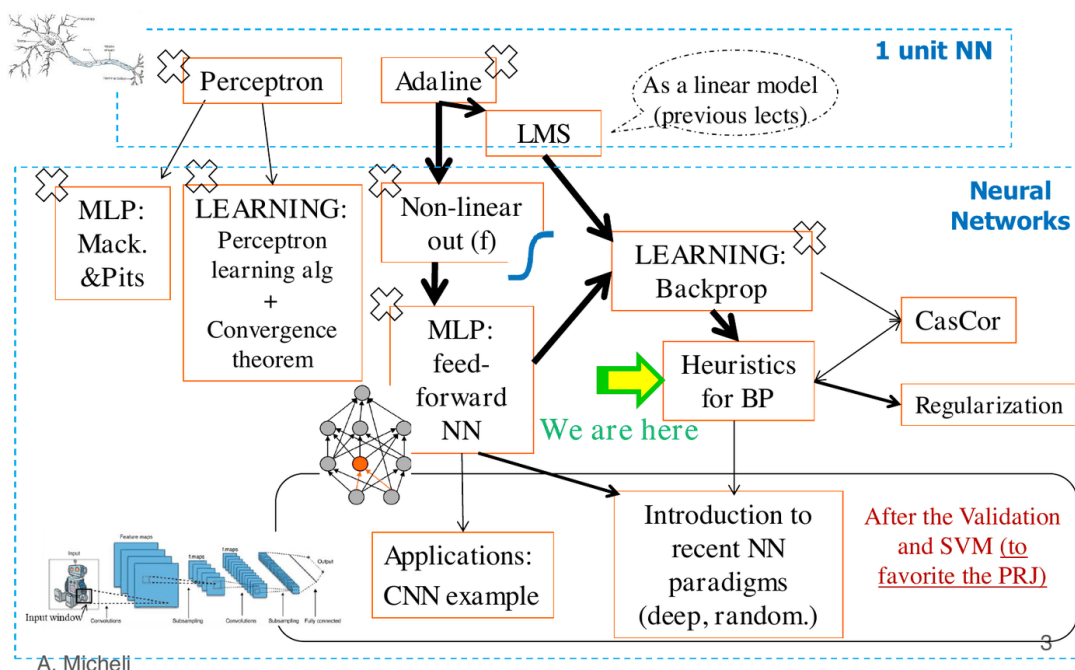


Figure 8.1: Zoom on the “neural networks” block of the course.

8.1 Philosophy

ML has evolved from a collection of heuristic tricks to general conceptual frameworks: the professor prefers not to introduce too many heuristic details. Many of the techniques that follow can be explored experimentally in the project; some have a theoretical foundation (regularisation) and apply in a more general framework. Those marked with (!) are very common and **must be used** in the project.

Warning — Beware of “rules of thumb”

Avoid the *rules of thumb* found on blogs, unless they are proven in scientific papers or their advantage has been verified experimentally.

Tip — A useful interpretation

Backpropagation is a **path in weight space** (on the loss surface). The path depends on: the data, the model, the **starting point** (initial weights), the **convergence speed** and the **end point** (stopping rule). All these choices constitute a **control of the search** in the hypothesis space.

The basic algorithm is still gradient descent: small initial weights, $\Delta \mathbf{w} = -\partial E / \partial \mathbf{w}$ (computed with backprop: forward phase up to the outputs, then backward phase for the deltas), $\mathbf{w}_{new} = \mathbf{w} + \eta \Delta \mathbf{w}$, repeat. The model is often **over-parameterised** and the optimisation is **non-convex** and potentially unstable: hence the issues that follow. The **hyperparameters** are the values that must be fixed to run the training.

8.2 Initial weight values

Tip — (!) Initialisation

Initialise the weights with **random values close to zero**, for example in the interval $[-0.7; +0.7]$ (for standardised data).

To be avoided:

- **all zeros**: the deltas of the hidden units would all be zero (they depend on the weights towards the output) and the network would not learn;
- **high values**: the sigmoids start **saturated**, with almost zero derivatives and very slow learning;
- **all equal**: the hidden units remain **symmetric** (they receive the same updates and all compute the same thing). Randomness serves to **break the symmetry**.

The **fan-in** (number of inputs of a unit) can be taken into account, for example with an interval equal to $\text{range} \cdot 2 / \text{fan-in}$, but not if the fan-in is too large and not for the output units (otherwise the deltas start close to zero). Very popular is the **Glorot and Bengio** initialisation (2010): biases at 0 and weights from a uniform distribution in $[-1/a, +1/a]$ with $a = \sqrt{\text{fan-in}}$ (or, in later results, a also dependent on the fan-out).

Note — (!) For the project

A very small network (very few units) can depend strongly on the initialisation.

8.3 Multiple minima

The loss is **not convex**: there are many local minima, and the result depends on the initial weights. Therefore:

Tip — (!) Multiple initialisations

Try several random initial configurations (for example 5–10 or more *trials*). To evaluate the model, report the **mean** of the results (mean of the errors) and the **variance**. If a single answer is wanted, one can choose the solution with the lowest validation error (or

the median), or exploit the different end points with a **committee** answer (average of the outputs or vote). Later we will see why *ensembles* are advantageous in general.

Stopping in a local minimum with too high an error is not a big problem: one sees the final error and can start again. Often a “**good**” **local minimum is enough**.

8.3.1 Do we really need the global minimum?

Two fundamental remarks:

1. In ML **we look for neither the local nor the global minimum of R_{emp}** , but for the minimum of R , which we cannot compute. Often we stop at a point that does not even have zero gradient.
2. The network is a **variable-dimension hypothesis space**: during training the VC-dim grows, the training error goes down towards zero (or the global minimum) while the network becomes too complex. One must stop **before** this condition of *overtraining* (and hence overfitting), regardless of the local/global minima issue.

8.4 On-line, batch and mini-batch

Let us recall (see linear models):

- **Batch**: the gradients of all patterns of an **epoch** (a complete cycle of presentation of the training set) are summed and then the weights are updated.
- **On-line/stochastic**: $\mathbf{w}$ is updated after each pattern p with $\Delta_p \mathbf{w}$, without waiting for the sum. It makes progress at every example and can be faster, but it requires a smaller η .

In formulas, for the weight w_{tu} :

$$\text{batch: } -\frac{\partial E}{\partial w_{tu}} = -\sum_{p=1}^l \frac{\partial E_p}{\partial w_{tu}} = \sum_{p=1}^l \delta_{p,t} o_{p,u}, \quad \text{on-line: } -\frac{\partial E_p}{\partial w_{tu}} = \delta_{p,t} o_{p,u}.$$

Summary — Batch and on-line compared

Batch — 1) fix $\mathbf{w}_{init}$ and η ; 2) compute $\Delta \mathbf{w}$ on the whole training set (the sum over p is inside this step); 3) $\mathbf{w}_{new} = \mathbf{w} + \eta \Delta \mathbf{w}$; 4) repeat until convergence.

On-line — 1) fix $\mathbf{w}_{init}$ and η ; for each pattern p : 2) compute $\Delta_p \mathbf{w}$; 3) $\mathbf{w}_{new} = \mathbf{w} + \eta \Delta_p \mathbf{w}$; 4) repeat (epoch after epoch) until convergence.

Batch gives a **more accurate** estimate of the gradient. In on-line learning the gradient of a single pattern is a **noisy** approximation of the overall gradient, hence the name *stochastic gradient descent*: the descent proceeds in a zig-zag, which **can help escape local minima**.

Tip — (!) Shuffling

In the on-line version the patterns must be presented in a **different order** at each epoch (usually by **shuffling them randomly**), to avoid a systematic drift in the descent.

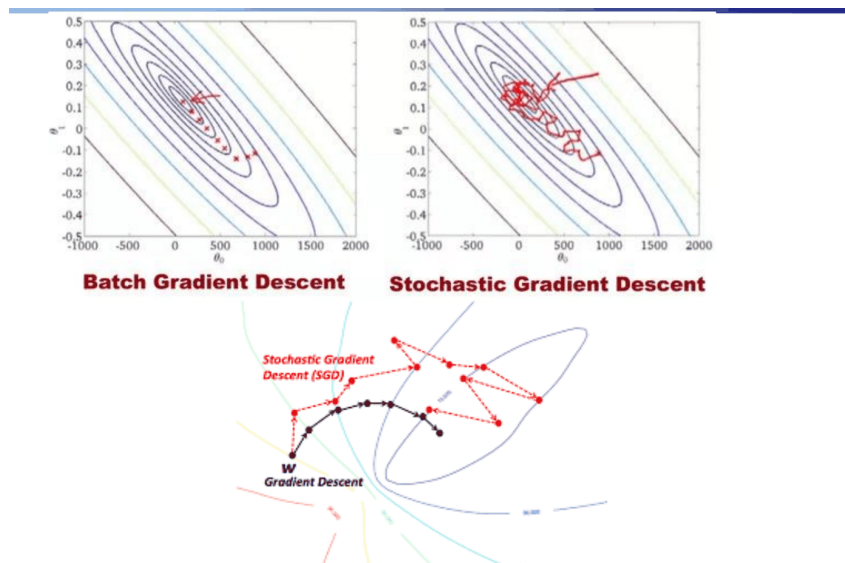


Figure 8.2: Batch and stochastic gradient descent.

8.4.1 Mini-batch

In **mini-batch** each epoch is divided into parts: the gradients of mb patterns ($1 < mb < l$) are summed before updating the weights, and this is repeated until all data of the epoch have been used. The update thus takes place at each mini-batch instead of at each pattern or each epoch.

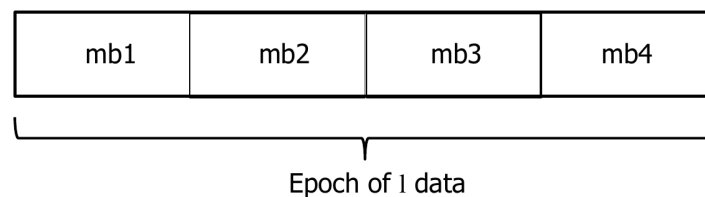


Figure 8.3: An epoch split into mini-batches.

On-line	Mini-batch (SGD)	Batch
does not exploit parallelism across examples	trade-off; fits the GPU memory	more accurate gradient estimate
can be unstable		slower (waits for all data)
noise can have a regularising effect		

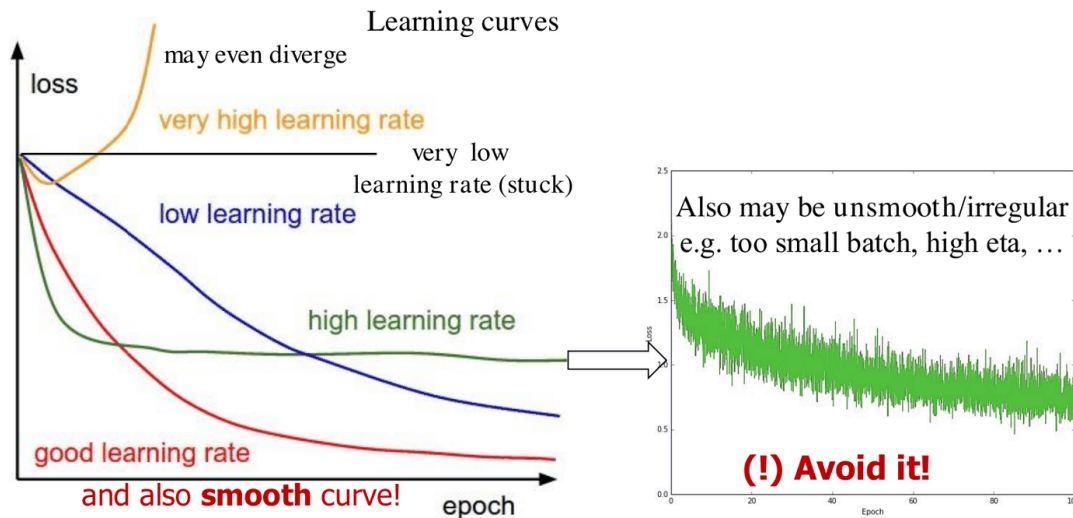
Mini-batch SGD uses random sampling (shuffling) for an unbiased estimate of the gradient. With huge training sets or streaming data one can even do without epochs. Since the cost of an update depends on mb and not on l , mini-batch SGD is a key factor for scaling learning to datasets of millions of examples.

8.5 The learning rate η

- **Batch:** more accurate gradient $\rightarrow$ a higher η can be used.
- **On-line:** faster but more unstable $\rightarrow$ smaller η .
- In general: **high** η is fast but can be unstable; **low** η is stable but can be too slow. Typically it is searched in $[0.01; 0.5]$ with LMS.

Tip — (!) Always look at the learning curve

The plot of the error during training makes it possible to check the behaviour in the early design phases. Here the **quality** of the curve and the **speed** of convergence are evaluated (the absolute value of the error also depends on the capacity of the model and on the other hyperparameters).



Here just looking to the optimization “failures” (on the TR set),

Figure 8.4: Effect of the learning rate on the learning curves (training error).

An irregular curve may be due to batches that are too small or to η too high: it must be avoided. We look for a curve that decreases quickly **and smoothly**. (Here we only look at the “failures” of the optimisation on the training set, not at the generalisation error.)

8.5.1 LMS: dividing by l

Tip — (!) Use the mean of the gradients

It is convenient to use the **mean** of the gradients over the epoch (sum divided by l): it makes the approach uniform with respect to the number of data (it is *least mean squares*). It is equivalent to using η/l with $0 < \eta < 1$.

Beware: when comparing on-line and batch with LMS, on-line requires a **much smaller** η (by the same factor $1/l$) to be comparable. For mini-batch (if dividing by mb) η must be re-tuned, or one always divides by l .

8.5.2 Improving the descent

Momentum (!)

$$\Delta \mathbf{w}_{new} = -\eta \frac{\partial E(\mathbf{w})}{\partial \mathbf{w}} + \alpha \Delta \mathbf{w}_{old}, \quad \mathbf{w}_{new} = \mathbf{w} + \Delta \mathbf{w}_{new},$$

with $0 < \alpha < 1$ (for example 0.5–0.9). Here η is included in $\Delta \mathbf{w}$, and after each step $\Delta \mathbf{w}_{old} = \Delta \mathbf{w}_{new}$ is saved.

It is the **“heavy ball” method**: at each step a fraction of the previous displacement is added, as if the weights had inertia.

- **Faster on plateaus**: if the gradient has the same sign in consecutive iterations, momentum increases the step.
- **Dampens oscillations**: in directions in which the gradient changes sign the contributions cancel out.
- **Inertia effect**: allows higher η to be used.

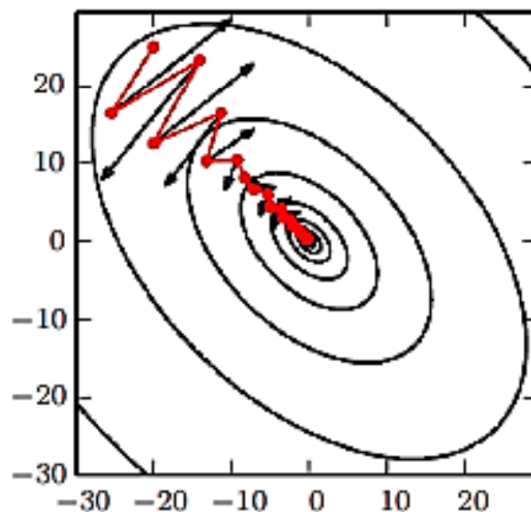


Figure 8.5: Effect of momentum in a “canyon” of the error surface (ill-conditioned Hessian): the red path goes along the valley instead of bouncing between the walls.

Momentum was born for batch. In on-line learning $\Delta \mathbf{w}_{old}$ is that of the **previous pattern**, and momentum becomes a **moving average of past gradients** that smooths the stochastic gradients within the epoch (similarly for mini-batch). The meaning is therefore different from batch, where gradients computed on the same examples in different epochs are combined.

Nesterov momentum: momentum is applied first ($\mathbf{w} + \alpha \Delta \mathbf{w}_{old}$), the gradient is evaluated at this intermediate point, and then the $\Delta \mathbf{w}$ (momentum plus new gradient) is applied to the original weights. It improves the convergence speed in batch (not in the stochastic case).

Variable learning rate

With mini-batch the gradient **does not tend to zero** near the minimum (because of sampling noise), so a fixed η should be avoided. For example it is decreased linearly until iteration τ :

$$\eta_s = \left(1 - \frac{s}{\tau}\right) \eta_0 + \frac{s}{\tau} \eta_\tau, \quad s \leq \tau,$$

and then a constant, small $\eta = \eta_\tau$ is used (for example $\eta_\tau \approx 1\%$ of η_0 , τ a few hundred steps). η_0 is chosen with the instability/stall trade-off (preliminary trials or grid search).

Adaptive learning rates

Algorithms that automatically adapt the learning rate during training, even separately **for each parameter**: **AdaGrad**, **RMSProp**, **Adam** (very popular and often robust with the default values, but with caution). They are combined with momentum. In networks with many layers, η can also depend on the layer or on the fan-in.

8.5.3 Beyond basic SGD

SGD is efficient (linear in the number of parameters) but depends on the number of data and epochs. There are many heuristics: **Quickprop** (jumps to the minimum of a parabola that locally approximates the loss), **R-prop** (uses only the **sign** of the gradient, not its value, useful when the gradient vanishes in deep layers).

Second-order methods use the Hessian (second derivatives) to obtain information about the curvature and descend better (Newton's method), but the Hessian is extremely expensive. There are approximate versions: Gauss-Newton, Levenberg-Marquardt, *Hessian-free* methods such as **conjugate gradient**, quasi-Newton methods such as **BFGS**.

Note — Saddle points

A saddle point has (almost) zero gradient but is not a minimum. In high-dimensional spaces saddle points grow exponentially with respect to local minima. Newton's method looks for (and “jumps towards”) zero-gradient points, so it is more sensitive to saddle points: this may explain why second-order methods have not replaced gradient descent for neural networks, while gradient descent empirically manages to escape saddle points. In practice we find low loss values fairly quickly, which are mainly useful for generalisation.

Tip — In practice

For optimisation, the basic approaches (those marked with (!)), if applied correctly, are usually enough. Experience with well-known techniques matters more than sophisticated methods one does not know how to handle. The optimisation technique is relevant only if there are obstacles to training: **validation matters more than optimisation on the training set**, because optimising on the training data is not our goal. The starting point (also for teaching purposes) is **SGD with momentum** (and regularisation).

8.6 Stopping criteria

- The basic criterion is the error used (mean error $< \epsilon$): it is the best if the tolerance of the data is known (expert knowledge), but often this is not the case. Variants: maximum tolerance instead of mean; for classification, the number of errors.
- “Internal” criteria: weights that no longer change, almost zero gradient (Euclidean norm $< \epsilon$), error that no longer decreases significantly within an epoch (e.g. less than 0.1%). Beware: they can be **premature** (for example with η too small), so k epochs are observed (**patience**).
- (!) Stop anyway after an excessive number of epochs, to escape overly slow convergence, but **do not** stop at an arbitrary number of epochs fixed a priori.
- And do not necessarily stop with a very low training error...

8.7 Overfitting and regularisation

8.7.1 Stop at the minimum?

Typically we do **not** want the global minimiser of $R_{emp}(\mathbf{w})$, because it is probably an overfitting solution: this is a fundamental difference with respect to optimisation methods (CM). Our main goal is **complexity control** to obtain the best generalisation, through regularisation (explicit

with a penalty term, or implicit with early stopping), plus model selection (cross-validation) to find the trade-off.

8.7.2 How overfitting arises in a network

- We start with **small random weights** (symmetry breaking).
- The input→output mapping is almost **linear** (the sigmoids work in the linear zone): the effective number of free parameters (and the VC-dim) is almost that of a perceptron.
- As optimisation proceeds, the hidden units tend to **saturate**, increasing the effective number of parameters and the non-linearities: the VC-dim grows.

It is again the **variable-dimension hypothesis space**.

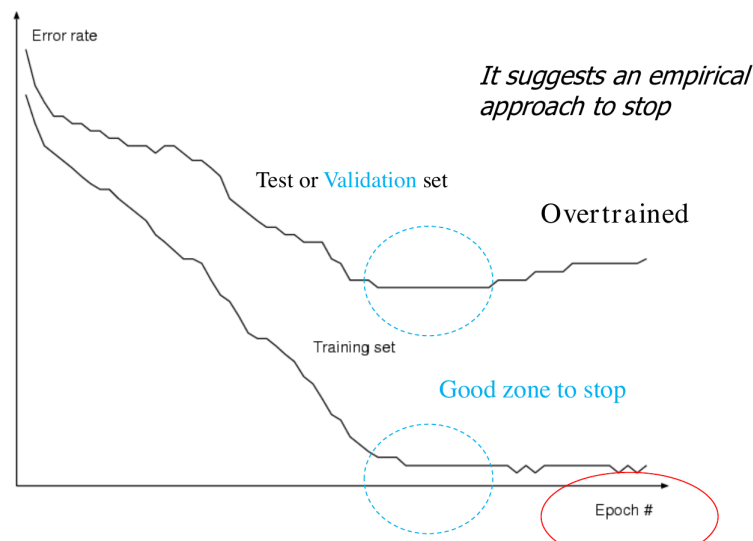


Figure 8.6: Typical behaviour of backpropagation: it suggests an empirical criterion for stopping.

The curve must be read together with the VC-bound plot (theoretical foundation). How to intervene:

1. **Early stopping:** use a **validation set** to decide when to stop, i.e. when the validation error starts to rise. It is vague: several epochs must be observed before deciding (*patience*) and possibly one must go back (*backtracking*). Since the effective number of parameters grows during training, **stopping earlier is equivalent to limiting the effective complexity**.
2. **Regularisation** of the loss: our main tool.
3. **Pruning** methods (later).

8.7.3 Regularisation (!)

During training the weights grow: we can optimise the loss taking their value into account. The principled approach (Tikhonov again) adds a penalty term:

$$\text{Loss}(\mathbf{w}) = \sum_p (d_p - o(\mathbf{x}_p))^2 + \lambda \|\mathbf{w}\|^2, \quad \|\mathbf{w}\|^2 = \sum_i w_i^2 \text{ over all the weights of the network.}$$

The effect is **weight decay**: about $2\lambda w$ is added to the gradient of each weight,

$$\mathbf{w}_{new} = \mathbf{w} + \eta \Delta \mathbf{w} - 2\lambda \mathbf{w}$$

(the 2 can be omitted). λ is usually very small (e.g. 0.01) and is chosen in **model selection** (cross-validation). Applied to a linear model it is ridge regression; there are more sophisticated penalties (for example for *weight elimination*).

Note — Error and Loss

With a penalty term it is worth distinguishing: **Loss** is the objective function used for training (MSE + penalty); **Error/Risk** is the data term (MSE), which measures the error of the model. (!) It is the error that must be reported in tables and plots, because it is the useful measure for those who use the model.

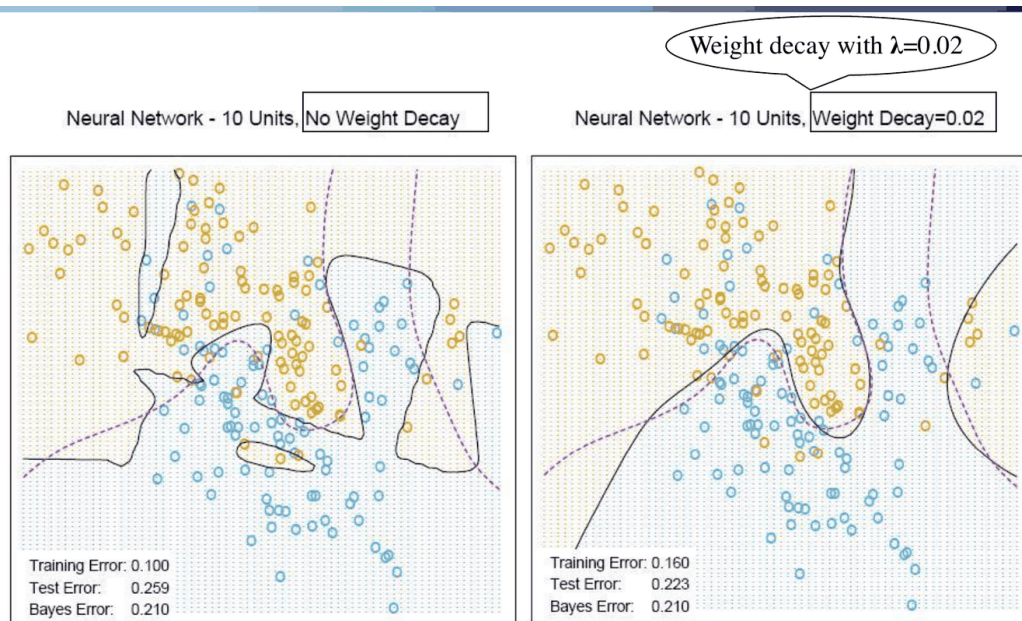


Figure 8.7: Effect of weight decay on a 10-unit network (Hastie-Tibshirani-Friedman): the boundary becomes more regular and the test error gets closer to the Bayes error (0.210).

Common misconceptions

Warning — Regularisation is not for stabilising convergence

Regularisation is **not** a technique to make the learning curve more stable: it serves to **control the complexity** of the model (measured by the VC-dim, related to the number and value of the weights). If the curve is unstable, the problem lies elsewhere (learning rate, batch...).

Questions — Early stopping or regularisation?

Early stopping is an **empirical** approach that requires a validation set to decide when to stop, sacrificing part of the data. Regularisation in the loss is a **principled** approach: it allows the validation curve to follow the training curve (as long as there is no underfitting due to a too high λ), and in that case early stopping is not (strictly) needed and one can stop at convergence. Both can also be used: early stopping does not intervene if the validation error does not rise.

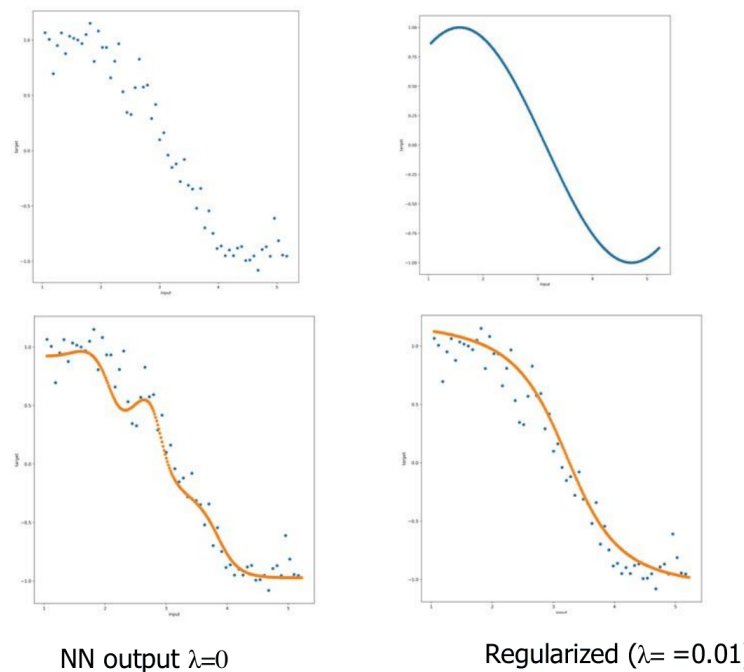


Figure 8.8: Univariate regression: network without regularisation ($\lambda = 0$) and regularised ($\lambda = 0.01$).

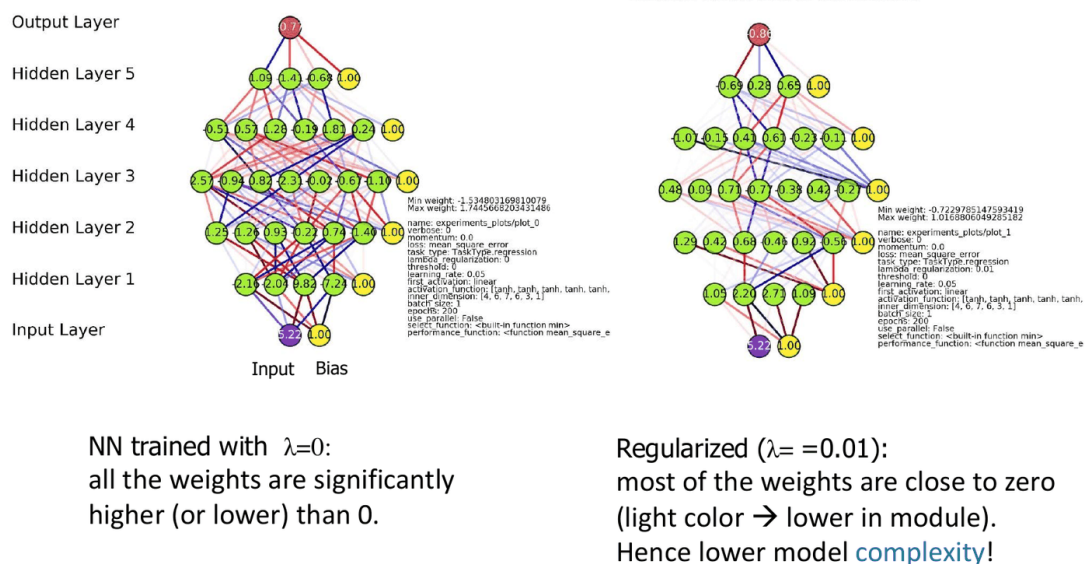


Figure 8.9: The weights of the network without regularisation (left) and with regularisation (right): with $\lambda = 0.01$ most weights are close to zero, so the complexity of the model is lower.

Details

- Often the **bias** w_0 is **excluded** from the regulariser (including it would make the results dependent on translations/scalings of the target), or it has its own coefficient.
- It is typically applied in the batch version. In on-line/mini-batch the penalty is applied at every step, hence many times per epoch: for a fair comparison with batch one would use $\lambda \cdot mb/l$. (If λ is chosen with model selection, this happens automatically.)
- Another technique is **dropout** (later).

8.7.4 Putting everything together: momentum and weight decay

With the regularised loss, the rule with momentum becomes (separating the two terms of the loss and using η only for the error term, as already done for linear models):

$$\Delta w_{tu} = -\eta \frac{\partial \text{Loss}}{\partial w_{tu}} + \alpha \Delta w_{tu}^{\text{old}} \doteq \eta \delta_t o_u - \lambda w_{tu} + \alpha \Delta w_{tu}^{\text{old}}, \quad w_{tu}^{\text{new}} = w_{tu} + \Delta w_{tu}.$$

In this way η does not multiply λ : changing one does not change the other.

Should momentum also include the λ part? Often yes (Matlab, Caffe...). But to make **all three hyperparameters** η , α , λ **independent** one can write:

$$\Delta w_{tu} = \eta \delta_t o_u + \alpha \Delta w_{tu}^{\text{old}}, \quad w_{tu}^{\text{new}} = w_{tu} + \Delta w_{tu} - \lambda w_{tu},$$

separating weight decay from the momentum memory. Only the gradient of E (where the sum over patterns is) is divided by l .

8.8 Number of hidden units

The number of units is related to **complexity control**, to the input dimension and to the size of the training set. In general it is a **model selection** problem: the number of units, together with λ , is chosen with cross-validation.

- Too few units $\rightarrow$ underfitting; too many $\rightarrow$ overfitting (it can happen).
- The number of units can be higher if adequate regularisation is used.

Alternatives to manual search:

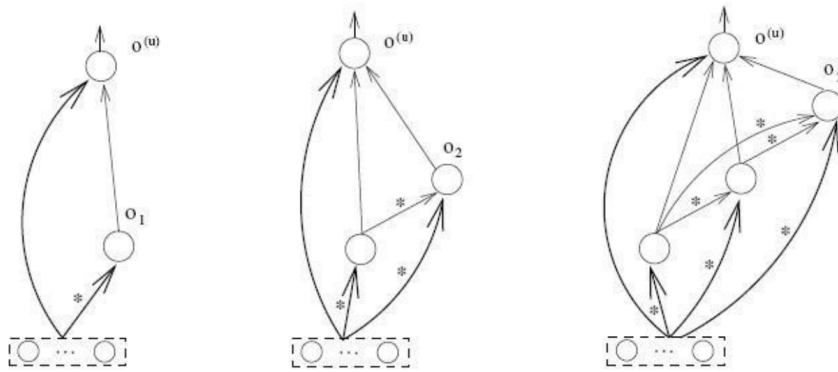
- **constructive approaches**: the algorithm decides the number of units starting from a small network and adding more;
- **pruning methods**: one starts from large networks and progressively removes weights or units.

8.8.1 Cascade Correlation

Among constructive approaches (Tower, Tiling, Upstart for classification) the best known is **Cascade Correlation** (CC) by Fahlman and Lebiere (1990), for regression and classification. It learns **both the weights and the topology** (number of units): the hypothesis space has a flexible dimension, decided by the algorithm, and at each step only one unit is trained.

Summary — Cascade Correlation algorithm

1. Start from a network N_0 **without hidden units** and train it. If it does not solve the problem, move to N_1 .
2. In N_1 a hidden unit is added whose weights are trained to **maximise the correlation** between its output and the **residual error** of network N_0 .
3. After training, the input weights of the new unit are **frozen**; the remaining weights (output layer) are retrained.
4. If the network does not solve the problem, more units are added, each connected to **all the inputs and all the previous hidden units** (hence “cascade”).
5. Continue until the residual error satisfies the stopping criterion.



© A. Micheli 2003

* = weights frozen after candidate training

Figure 8.10: Construction of a Cascade Correlation network with up to 3 hidden units.

The algorithm alternates minimisation of the total error (LMS, for example backprop on the output layer) and **maximisation of the covariance** S between the output o_p of the candidate and the residual error $E_{p,k} = o_{p,k} - d_{p,k}$:

$$S = \sum_k \left| \sum_p (o_p - \bar{o})(E_{p,k} - \bar{E}_k) \right|.$$

The gradient (with $d|f|/dx = \text{sign}(f) df/dx$ and neglecting the dependence of the means on o_p) is

$$\frac{\partial S}{\partial w_j} = \sum_k \text{sgn}(S_k) \sum_p (E_{p,k} - \bar{E}_k) f'(net_{p,h}) I_{p,j},$$

where h is the candidate and $I_{p,j}$ its input j . Gradient **ascent** is performed (S is maximised).

Notes on CC:

- the role of the hidden units is to **reduce the residual error**: each unit solves a sub-problem and becomes a **permanent feature detector**;
- **candidate pool**: the surface of S has many maxima, so a group of candidates is trained and the best one is chosen, avoiding local maxima;
- it is a **greedy** algorithm: it converges easily and can find a minimal number of units, but it can overfit (regularisation is needed);
- the loss does not necessarily have to be “max S ” (for regression LMS can be used).

8.9 Input and output

Input: pre-processing has a big effect.

- **Standardisation** (often useful): for each feature mean 0 and standard deviation 1, $(v - \text{mean})/\text{std. dev.}$.
- **Rescaling** to $[0, 1]$: $(v - \min)/(\max - \min)$.
- Categorical inputs: **1-of-K** encoding ($A \rightarrow 100$, $B \rightarrow 010$, $C \rightarrow 001$).
- Missing data: beware, **0 does not mean “no input”** if 0 is within the range of values.

Output:

- **(!) Regression: linear** output units (one or more).
- **Classification:** one unit (binary) or 1-of-K with several outputs. With the sigmoid, the threshold for assigning the class is chosen (with a possible rejection zone); with 1-of-K the highest output wins. Often **tanh** (symmetric) learns faster. As target one can use 0.9 instead of 1 (and -0.9 or 0.1 instead of -1 or 0) to avoid asymptotic convergence towards saturation. Obviously the range of the targets must lie within the range of the outputs.
- **Softmax** for multi-class 0/1 targets: the outputs sum to 1 and can be interpreted as probabilities $p(\text{class} = k \mid \mathbf{x})$:

$$o_k(\mathbf{x}) = \frac{e^{\text{net}_k}}{\sum_{j=1}^K e^{\text{net}_j}}.$$

- **Cross-entropy** can be used as an alternative loss (maximum likelihood estimation); for one unit:

$$- \sum_{i \in TR} \{d_i \log(\text{out}(\mathbf{x}_i)) + (1 - d_i) \log(1 - \text{out}(\mathbf{x}_i))\}.$$

Questions — Exercise for the project

Reflect on the role of the different hyperparameters and relate them to the theory: for example relate the loss with penalty to the VC-bound, to understand how the number of units, λ and the others control underfitting and overfitting. Which are the most relevant for the project (and for winning the competition)?

8.10 Strengths and weaknesses of neural networks

Summary — Strengths

- Flexible method: it extends linear methods to the estimation of arbitrary functions and builds an **explicit hypothesis** (unlike K-NN).
- Expressive power given by number of units, architecture and training.
- Rich, continuous hypothesis space, differentiable loss $\rightarrow$ training with gradient descent.
- Universal approximators; they handle noise and incomplete data with graceful degradation; continuous and discrete data, regression and classification.
- A broad **paradigm** (also unsupervised learning, associative memories, stochastic approaches).
- Neural plausibility; **distributed representation** of knowledge (compact in weight matrices); **fault tolerance**; adaptation to non-stationary environments with on-line

learning (*continual learning*, with the stability-plasticity dilemma).

Warning — Critical points

- **Black box problem:** it is difficult to interpret the acquired knowledge and extract it for human experts. It is related to the current topic of **explainable AI** (XAI). Approaches: projection/clustering of internal representations, sensitivity analysis, **Layer-wise Relevance Propagation** (LRP 2015, also for deep networks, which produces heat maps on the parts of the input relevant to the decision), extraction of symbolic rules.
- **Architecture design:** constructive/pruning methods or different approaches (SVM) are needed.
- **Dependence on the training process:** the many variants and hyperparameters affect the final solution.
- Not ideal with **missing data** (removal, imputation... are used).

Neural networks are appropriate when: the input is high-dimensional (discrete or real), the task is classification or regression, the data may be noisy, training time is not critical, the form of the target function is unknown, readability of the result is not critical, and the output computation must be fast.

Note — Historical notes

McCulloch and Pitts (1943), Hebb (1949), Minsky (1954, 1969), von Neumann (1956), Rosenblatt (1958), Widrow and Hoff (1960), Kohonen (1972, 1982), Grossberg (ART, 1980), Hopfield (1982), Rumelhart, Hinton, Williams, Werbos, Parker, LeCun (1975–1985, backprop), Poggio and Girosi (RBF theory, 1990), Vapnik (SVM, 1990s), deep learning (Hinton, Bengio, LeCun, since 2007). **Hopfield and Hinton** received the **2024 Nobel Prize in Physics**; Hinton, Bengio and LeCun the 2018 Turing Award.

8.11 Towards the project: the MONK benchmark

We are not yet ready for the project (**validation** and the dedicated lecture are still missing), but one can start implementing the network (backprop and update rules) or exploring the simulators.

Verifying the correctness of an implementation is difficult: a network often “works” (even if badly) despite small errors. The first test is the **MONK** dataset (UCI), whose results must be reported in the report:

- 3 binary classification tasks, small and artificial, “not difficult”: a small network with few units reaches very high accuracies (up to 100%) in little time;
- there is a report with previous results of MLPs and other models;
- **input:** 6 variables with 2, 3 or 4 symbolic values each → **1-of-k** encoding → **17 input units**;
- the test set includes the training set (bad practice, but irrelevant here).

Tip — Tips for the first trials on MONK

- One output unit for classification (targets and outputs consistent, 0/1 or $-1/+1$); **tanh** at the output is suggested.
- **One-hot encoding of the inputs**: a very frequent mistake if forgotten!
- Batch with gradient divided by the number of patterns: it converges in a few hundred epochs.
- Few hidden units (2–5), as in the manual: the test works if state-of-the-art results are reached with few units.
- MONK is sensitive to initialisation (few units): try small initial intervals (or fan-in based).
- With on-line approaches use **shuffling** (the patterns are ordered by class).

Good results on MONK **do not guarantee** the correctness of the simulator; bad results **surely** indicate that code or settings must be revised. In the plots what matters is the **shape** of the curves (smooth, converging), not the values.

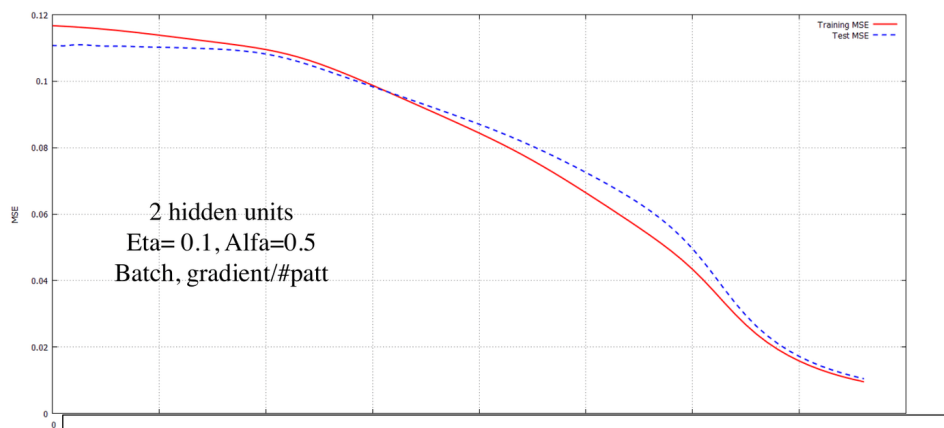


Figure 8.11: MONK2: training (red) and test (dashed blue) MSE with 2 hidden units, $\eta = 0.1$, $\alpha = 0.5$.

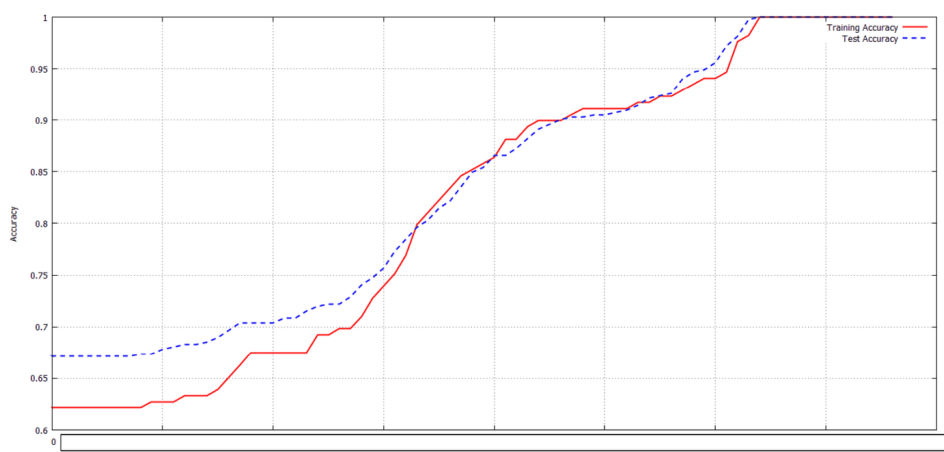


Figure 8.12: MONK2: the accuracy reaches 100% also on the test set (an unusual case in general).

Questions — What happens in Fig. 8.13?

In the final part the training error keeps decreasing while the test error rises slightly: it is the onset of **overfitting** (MONK3 has noisy labels). Regularisation mitigates it; a λ chosen

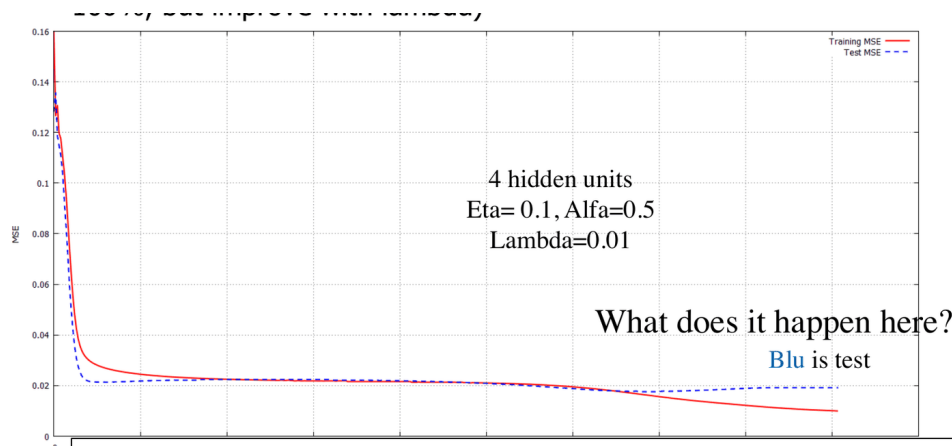


Figure 8.13: MONK3 with regularisation ($\lambda = 0.01$): MONK3 contains noise and can over-fit; the test does not reach 100%.

with model selection or early stopping would help.

To verify backpropagation one can use a step-by-step example (beware if it omits the biases) or a reliable library as an “oracle” (Keras, scikit-learn, PyTorch...). Other useful datasets can be found in the UCI Machine Learning Repository.

Note — Project preview (A.Y. 2026/27 rules)

In past years the project was mandatory (groups of 2–3, type A with a simulator implemented from scratch or type B with libraries, ML-CUP competition). From A.Y. 2026/27 the project is **optional**: groups of **2 students**, implementation/application/analysis of an ML method with experimental evaluation, presented as a **poster** in one of the last lectures (December). It can give a small bonus but it is not necessary for the exam, which is based on the written test (see [01 - Introduction to Machine Learning](#)). The advice on MONK remains valid as a test of any implementation.

Questions — Possible exam questions

- How are the weights initialised and why?
- Differences between on-line, batch and mini-batch.
- Role of the learning rate; what is momentum and why does it work?
- Is it necessary to find the global minimum? Why?
- How does overfitting manifest itself in a neural network? What is early stopping?
- Write the loss with weight decay and the update rule with momentum.
- Does regularisation improve the stability of convergence?
- Describe Cascade Correlation.
- How are inputs and outputs represented for regression and classification?

Chapter 9

Validation (part 1): model selection and model assessment

Validation was introduced in [04 - Generalisation and validation \(introduction\)](#); these lectures go into more depth, because it is indispensable for the project and for any serious application of ML. The two goals (model selection and model assessment) are restated, a counterexample shows how easy it is to obtain wrong estimates, **grid search** and **K-fold cross-validation** are introduced, and sampling, scarce data and error measures are discussed.

9.1 Preamble: bias and variance

The **bias-variance** decomposition (which will have a dedicated lecture) is a useful framework for understanding why estimating the performance of a model is difficult: it shows the role of the different **realisations of the training set** and, once again, the trade-off between fitting capability (bias) and model flexibility (variance).

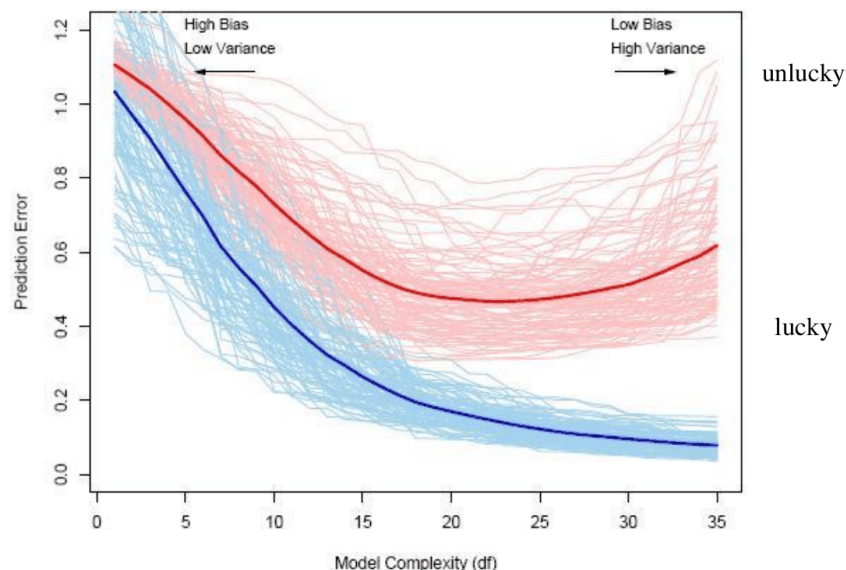


Figure 9.1: Training (light blue) and test (red) error on 100 different training sets as complexity grows (Hastie et al., Fig. 7.1). With a complex model the bias is low but the variance is high: depending on the sample one can be “lucky” or “unlucky”.

The figure shows an important fact: for the same complexity, the test error varies a lot from one training set to another, especially for complex models. A single measurement can be lucky or unlucky.

9.2 Motivations

We look for the solution with the minimum prediction (test) error, balancing **fit to the training data** and **model complexity**. The training error **is not a good estimate** of the test error: at first there is too much bias (underfitting, high training error), then too much variance (low training error but overfitting).

Assuming we have a hyperparameter θ (implicit or explicit) that controls the complexity of the model, we want the value of θ that minimises the test error. We therefore need methods to **estimate the expected error** of a model (or of each model of a class, or of a set of models).

9.2.1 How to estimate

- **Analytically:** AIC and BIC criteria (*Akaike/Bayesian Information Criterion*, limited to models linear in the parameters), MDL (*Minimum Description Length*), **SRM** (*Structural Risk Minimization*) with the VC-dimension (see the lecture on SLT).
- **Empirically**, on the data, through **resampling** (direct estimation of the out-of-sample error): **cross-validation** (hold-out, K-fold, ...) and **bootstrap**.

9.3 The two goals of validation

Warning — The most important slide of the course

After training the models on the training set:

- **Model selection:** estimate the performance (generalisation error) of different models in order to **choose the best one**. It includes searching for the best **hyperparameters** (degree of the polynomial, number of units of a network, λ , η , ...). *Returns a model.*
- **Model assessment:** having chosen the final model (or a class of models), **estimate its prediction error** (risk) on new test data, as a measure of the quality of the chosen model. *Returns an estimate.*

Golden rule: keep the goals separate and use **separate data sets** in each phase.

9.3.1 Hold-out

If there are enough data, the dataset is split into three **disjoint** sets, for example 50% TR, 25% VL, 25% TS:

- **TR** (*training set*): to fit the model, i.e. minimise the SLT R_{emp} [**training**];
- **VL** (*validation or selection set*): to choose the best model among different models and hyperparameter configurations [**model selection**];
- **TS** (*test set*): to estimate the generalisation error of the final model, i.e. estimate the SLT R [**model assessment**].

TR and VL together form the **development/design set**, used to build the final model.

Note — Two clarifications

1. The estimate made on VL serves **only** for model selection: it is not a good estimate for assessment.
2. The results on TS **cannot** be used for model selection. If this is done, it is no longer a test set: let us call it a validation set.

9.3.2 Test or model selection?

What happens if the test set is used in a repeated design cycle (try, look at the test, change, try again)? One is doing **model selection**, not a reliable evaluation, and we could not do this on future examples. The error on the test set used in this way is a **too optimistic estimate** of the true error. It is like an exam exercise whose solutions have been seen: it is no longer a test! Hence the concept of **blind test set**, used in ML competitions.

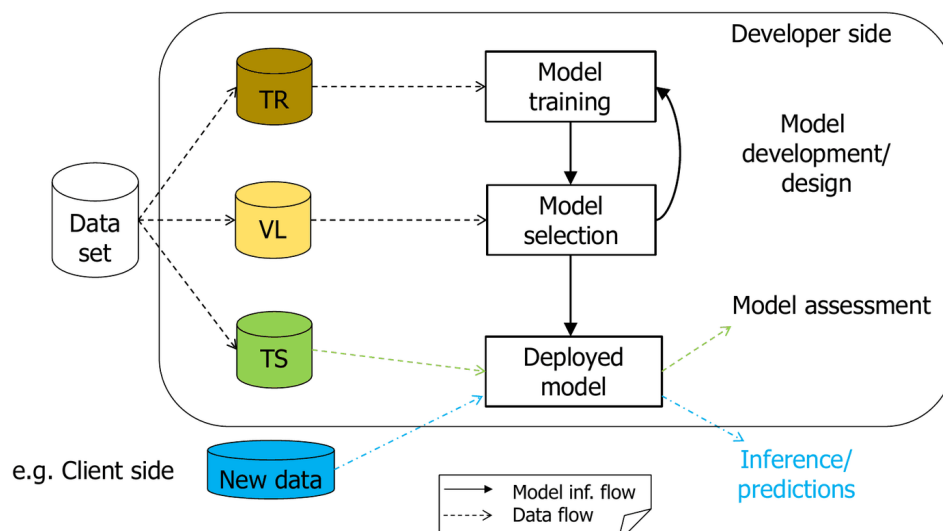


Figure 9.2: Role of TR, VL and TS (taken from lecture 4).

9.4 An instructive counterexample**Example — The random target counterexample**

- 20–30 examples, **1000 input variables** with random values;
- **random** 0/1 target.

A model is selected that uses **a single** input variable, the one that by pure chance “guesses” the target at 99% (or 100%) on any subsequent split into TR, VL and TS. Perfect result? What is wrong?

The 99–100% is **not** a good estimate of the test error: the true one is **50%** (the target is random, no model can do better than a coin). On a truly new external test set one would get 50%.

There are two errors:

1. the error estimate on TR and VL is **not** a good estimate of the risk;

2. using **the whole dataset** for feature or model selection **compromises the estimate** (biased estimate, called *subset selection bias*): the test set was used **implicitly** at the beginning, in the choice of the variable. The test set must be separated **in advance**, before **any** model selection, including **feature selection**.

With 1000 random variables and only 20 patterns, it is almost certain that one variable will coincide by chance with the target on most patterns. Since the choice is made looking at **all** the data, that variable “works” on any subsequent partition. On truly new patterns (TS1, TS2, TS3 in the table of the slide) the same variable has accuracies of 100%, 33%, 66%: pure chance.

Warning — Warning

Here we are discussing the **correctness of the estimate**, not the possibility of solving the task. K-fold CV and other techniques **do not** solve the problem if the selection is made before the split (on the contrary, they can confuse things even more).

9.5 Grid search

Hyperparameters are parameters not learned directly by the algorithm; they are fixed by searching in the **hyperparameter space**, naturally through model selection on a validation set.

Exhaustive grid search generates all candidates from a grid of values. Trivial example with number of units and λ :

Units λ	0.1	0.01	0.001
1 unit	Res1	Res4	Res7
10 units	Res2	Res5	Res8
100 units	Res3	Res6	Res9

If the best result (on validation) is Res3, the configuration (100 units, $\lambda = 0.1$) wins. It must be **automated**, and it is easy to **parallelise** (the trials are independent).

The cost can be high: it is the Cartesian product of the values, i.e. $(\text{\#values})^{\text{\#hyperparameters}}$. With 3 values for 2 hyperparameters there are $3^2 = 9$ trials; with 5–6 hyperparameters of 10 values each it becomes prohibitive. Therefore:

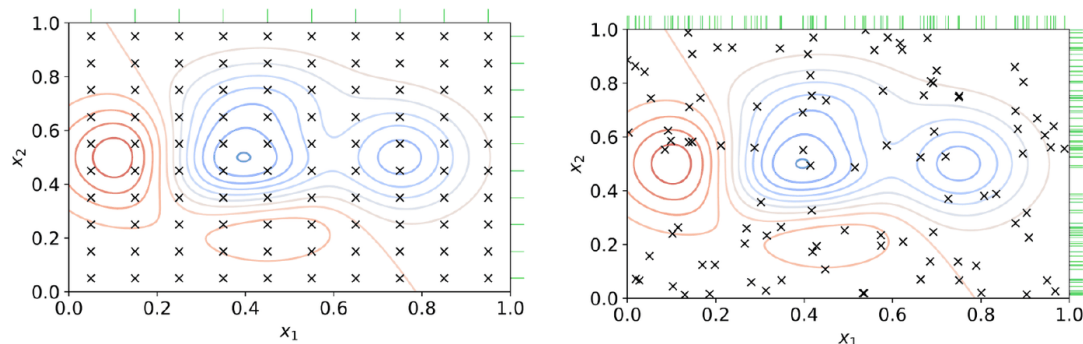
- some hyperparameters can be fixed in a preliminary experimental phase, if they prove to be of little relevance (with caution, see the sequential selection error in part 3);
- **two (or more) levels of nested grid search** are used: first a **coarse** search over all combinations (for example with exponentially growing values) to find the good regions, then **finer** searches over smaller and smaller intervals, always considering **all** the significant hyperparameters together.

9.5.1 Alternatives to grid search

Grid search is very useful for **observing the behaviour** of the model as the hyperparameters vary (fundamental for understanding), but it is expensive and scales poorly with the number of hyperparameters. Alternatives:

- **random search** (Bergstra and Bengio, 2012): it avoids resampling the same values of an influential hyperparameter when others are not influential, and it allows fixing the **budget** of trials independently of the number of hyperparameters;
- **automatic** search: Bayesian and evolutionary approaches, configuration evaluation methods (e.g. Hyperband).

Libraries such as Keras Tuner and scikit-learn include many alternatives (towards **AutoML**), but they should not be used uncritically: grid search teaches more at first approach.



Grid search for 2 hyperparameters: a total of 100 (10x10) different combinations are evaluated and compared. Blue contours indicate regions with strong results, whereas red ones show regions with poor results.

Random search for 2 hyperparameters: 100 different random choices are evaluated. The green bars show that more individual values for each hyperparameter are considered compared to a grid search.

Figure 9.3: Grid search and random search with 100 trials each: random search explores many more distinct values of each hyperparameter (green bars).

Tip — Why random search works

If only one of the two hyperparameters really matters, the 10x10 grid tries only **10 distinct values** of it (the other 90 trials repeat the same values while varying the useless hyperparameter), whereas random search tries **100**.

9.6 K-fold cross-validation

9.6.1 How much data is needed?

Hold-out requires enough data for three meaningful partitions. How many? It depends on the complexity of the model and on the signal-to-noise ratio (few are needed for a linear model on a noise-free linearly separable task). What to do if there are not enough data? And how to avoid depending on the **particular partition** chosen? K-fold CV helps.

Note — Terminology

“Cross-validation” denotes both an **approach** to model selection based on direct estimation of the error through resampling (rather than analytical), and more specifically the **implementations** that establish how to split the data for selection and for evaluation (for example K-fold CV).

9.6.2 Recap

D is split into K mutually exclusive subsets $D_1, \dots, D_K$; for each i one trains on $D \setminus D_i$ and evaluates on D_i . In this way all data are used both for training and for evaluation. One does not obtain a **single model** (there is one per fold), but one also obtains a **variance** (standard deviation) over the folds for one's class of models: the result is reported as **mean $\pm$ standard deviation**. It can be used both for the validation set and for the test set.

Issues: how many folds (3, 5, 10, ..., *leave-one-out*)? The computational cost is often high. It can be combined with a validation set or a double K-fold CV can be performed.

9.6.3 A complete example of selection and assessment

1. Split the data into **TR** and **TS** (here with hold-out, but it could be a K-fold).
2. **Model selection**: use a K-fold CV **internal** to TR (which in each fold produces new TR and VL) to find the best hyperparameters with a **grid search**. For each cell of the grid (e.g. $\lambda = 0.1$, 20 units) an entire K-fold CV is run; the configuration with the **best mean validation error** over the folds is chosen.
3. Retrain the final model on the **whole** original **TR**.
4. **Model assessment**: evaluate it on the **external TS**.

(Can one then retrain again on all the data? We will see this in the next part.)

9.7 Special cases

9.7.1 Lucky or unlucky sampling

To avoid depending on the particular partition:

- **Stratification**: grouping the population into homogeneous subgroups before sampling. For classification: in each partition (TR and TS in hold-out or CV) each class is represented roughly in the same proportions as in the full dataset.
- Check the composition of the folds: the **order of the data** could have a meaning (e.g. data sorted by class or by time).
- **Repeated hold-out or CV**: repeat the split with different random samplings (e.g. repeat the CV 10 times) and average the results. Useful above all for **comparing different models**.

9.7.2 Very few data

With very few data it is hard to say whether a sample is representative. Besides stratification, avoid (or take into account in the evaluation): classes or features missing in the training data; special classes not sampled in TR or TS; known outliers that affect the mean of the test results; conversely, selecting only “easy” cases.

Even a **blind test set** can be misleading if it comes from a different distribution, is measured with different scales or tolerances, is not cleaned or pre-processed, or requires **extrapolation** (out-of-range data).

9.8 Error measures for evaluation

Classification (see the first lectures): accuracy (or error rate, often in %), confusion matrix, specificity, sensitivity, ROC curve.

Regression, with residual $r_i = y_i - o_i$ (y target, o output):

- **MSE** = $\text{mean}_i[r_i^2]$ and its square root **RMS** (or S);
- **mean absolute error** (MAE) = $\text{mean}_i[|r_i|]$;
- **maximum absolute error** = $\max_i |r_i|$;
- **correlation coefficient** R (and **coefficient of determination** R^2), for example $R = \sqrt{1 - S^2/S_y^2}$, with $S_y^2 = \text{mean}_i[(y_i - \bar{y})^2]$ the variance of the target: it measures the degree of linear dependence between y and o , in $[0, 1]$, where 1 is best;
- **outputs versus targets** plots;
- statistical tests and significance analysis.

Note — Bootstrap

Another approach is the **bootstrap**: random resampling **with replacement**, repeated to obtain different (validation or test) subsets.

Summary — The two golden rules

- The result on **TR** is not a good estimate of that on VL and TS.
- The result on **VL** is not a good estimate of that on TS.

Therefore: **do not use validation set results for estimating the risk** (model assessment), and **do not use test set results for model selection**, in any form. In a single rule: *keep the goals separate and use separate sets in each phase.*

Questions — Possible exam questions

- Difference between model selection and model assessment; why are separate sets needed?
- Describe the random target counterexample: what are the two errors?
- What is grid search? How can its cost be reduced? Advantages of random search.
- Describe K-fold CV and a complete example of selection and assessment.
- What is stratification?
- Which error measures are used for regression?

Chapter 10

Validation (part 2): formal schemes

This second part formalises the correct use of cross-validation for model selection and/or risk estimation, with precise notation (adapted from S. Bengio's slides). The concepts are simple, but rigorous notation avoids ambiguity. It is not a cookbook: it serves to **rationalise** and systematically understand a rigorous approach, so as to then be reasonable. The time needed to run the experiments depends mainly on the validation choices: one must avoid both an overly coarse evaluation and an endless evaluation process. When in doubt, these schemes are in any case better than imagination.

We talk about selection among hyperparameter configurations (denoted by θ), but the same schemes apply to choosing among **different models** (θ can also represent the type of model).

Tip — Keep the counterexample in mind

From part 1: (1) error estimates on TR and/or VL are **not** good estimates of the risk; (2) using the whole dataset for feature or model selection **compromises** the estimate. In the schemes that follow we see exactly where these errors would be made.

10.1 Notation

Data: $Z_1, \dots, Z_l$ is a random sample from an unknown distribution $p(z)$, with all Z_i **independent and identically distributed** (i.i.d.). $D_r = \{z_1, \dots, z_r\}$ is a particular instance with r elements.

Task: classification, $Z = (X, Y) \in \mathbb{R}^n \times \{-1, +1\}$; regression, $Z = (X, Y) \in \mathbb{R}^n \times \mathbb{R}$.

Loss ("inner"): classification $L((x, y), h) = 0$ if $h(x) = y$, 1 otherwise; regression $L((x, y), h) = (h(x) - y)^2$.

Goal: minimise the **expected risk** (true error) over H :

$$R(h) = \mathbb{E}_Z[L(z, h)] = \int_Z L(z, h) p(z) dz, \quad h^* = \arg \min_{h \in H} R(h).$$

The problem: $p(z)$ is unknown and we do not have access to all the $L(z, h)$.

Empirical risk on a finite set D_r :

$$R_{emp}(h, D_r) = \frac{1}{r} \sum_{i=1}^r L(z_i, h).$$

Warning — General use of R_{emp}

Here R_{emp} is meant in a general sense, as a **surrogate of R** on a finite set of data, used for training, model selection or assessment depending on the partition. Not to be confused with the SLT R_{emp} , which is specifically the training error: in this notation it is $R_{emp}(h, D_{TR})$.

10.2 Methodology: identifying the goal

1. Provide the **best model** obtainable from a dataset? → **model selection** is needed.
2. Provide the **expected performance** of a model obtained by empirical risk minimisation?
→ **model assessment** (risk estimation) is needed.
3. Provide **both**? → both are needed.

The original dataset D_l will be used for these different purposes.

10.3 Model selection

10.3.1 Hold-out

Summary — Model selection with hold-out

- Choose a hypothesis space with hyperparameter θ .
- Split D_l into $D_{TR} = \{z_1, \dots, z_{tr}\}$ and $D_{VL} = \{z_{tr+1}, \dots, z_{tr+vl}\}$, with $tr + vl = l$.
- For each value θ_m [**grid search**]:
 - $h_{\theta_m}^*(D_{TR}) = \arg \min_{h \in H_{\theta_m}} R_{emp}(h, D_{TR})$ [**training**];
 - estimate $R(h_{\theta_m}^*)$ with $R_{emp}(h_{\theta_m}^*, D_{VL}) = \frac{1}{vl} \sum_{z_i \in D_{VL}} L(z_i, h_{\theta_m}^*(D_{TR}))$.
- Choose $\theta_m^* = \arg \min_{\theta_m} R(h_{\theta_m}^*)$ [best result on VL].
- Return $h^*(D_l) = \arg \min_{h \in H_{\theta_m^*}} R_{emp}(h, D_l)$ [**retraining** on all l data].

The estimate of R on VL serves **only** for model selection and **is not returned** as a risk estimate: returning it would be exactly error (1) of the counterexample.

10.3.2 K-fold cross-validation

Summary — Model selection with K-fold CV

- Choose a hypothesis space with hyperparameter θ .
- Split D_l into K distinct, equal parts $D_1, \dots, D_K$; let $\bar{D}_k$ be the complement of D_k .
- For each value θ_m [**grid search**]:
 - for each part D_k [**loop over folds**]:
 - * $h_{\theta_m}^*(\bar{D}_k) = \arg \min_{h \in H_{\theta_m}} R_{emp}(h, \bar{D}_k)$ [training];
 - * estimate $R(h_{\theta_m}^*(\bar{D}_k))$ with $R_{emp}(h_{\theta_m}^*(\bar{D}_k), D_k) = \frac{1}{|D_k|} \sum_{z_i \in D_k} L(z_i, h_{\theta_m}^*(\bar{D}_k))$;
 - estimate $R(h_{\theta_m}^*(D_l))$ with the mean $\frac{1}{K} \sum_k R(h_{\theta_m}^*(\bar{D}_k))$.
- Choose $\theta_m^* = \arg \min_{\theta_m} R(h_{\theta_m}^*(D_l))$ [best over all CVs].
- Return $h^*(D_l) = \arg \min_{h \in H_{\theta_m^*}} R_{emp}(h, D_l)$ [retraining].

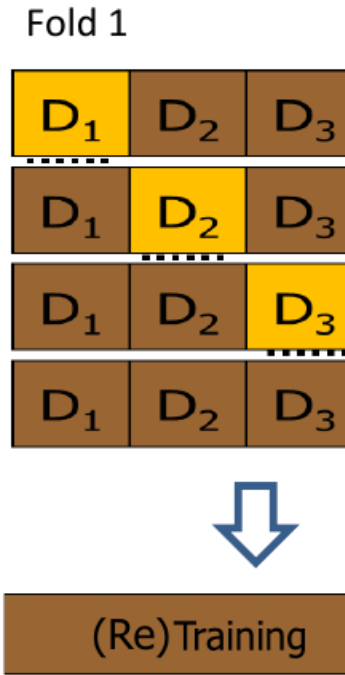


Figure 10.1: K-fold CV for model selection, followed by retraining.

Warning — Beware of the order

Each iteration **starts from scratch**, with training and estimation local to the split. If for example a model trained in split 1 were used to evaluate split 2, a part (D_2) that had been used for training in split 1 would be used as validation for split 2.

10.4 Risk estimation (model assessment)

10.4.1 Hold-out

Summary — Model assessment with hold-out

- Split D_l into D_{TR} and D_{TS} , with $tr + ts = l$. **The test set must be separated NOW** (not later): otherwise error (2) of the counterexample is made.
- $h^*(D_{TR}) = \arg \min_{h \in H} R_{emp}(h, D_{TR})$ [training; it may include an internal model selection on D_{TR}].
- Estimate $R(h^*(D_{TR}))$ with $R_{emp}(h^*(D_{TR}), D_{TS}) = \frac{1}{ts} \sum_{z_i \in D_{TS}} L(z_i, h^*(D_{TR}))$ [**test error**].

10.4.2 K-fold cross-validation

Summary — Model assessment with K-fold CV

- Split D_l into K distinct, equal parts.
- For each part D_k [loop over folds]:
 - $h^*(\bar{D}_k) = \arg \min_{h \in H} R_{emp}(h, \bar{D}_k)$ [training; it may include a model selection];

- estimate $R(h^*(\bar{D}_k))$ with $R_{emp}(h^*(\bar{D}_k), D_k)$ [test error of each iteration].
- Estimate $R(h^*(D_l))$ with the mean $\frac{1}{K} \sum_k R(h^*(\bar{D}_k))$.

With $K = l$ we have **leave-one-out CV**. The principles are not violated: test results are never used for training or selection, in any iteration.

10.5 Combining model selection and model assessment

When both the best model and its expected risk are wanted, the methods must be combined. For example:

- **hold-out** with training–validation–test (three separate sets are needed);
- **K-fold CV + test**: cross-validation on the training set, then test on a separate set;
- **external K-fold CV for the test**, with internal hold-out for model selection;
- **double (nested) cross-validation**: for each external fold, a nested cross-validation on the other $K - 1$.

Other aspects: take into account the **different trials** (different initialisations of a network) when computing the means on TR, VL, TS. More generally: compare the results with other methods, use **statistical tests** to check their significance, verify the model on several datasets.

10.5.1 Hold-out training–validation–test

Summary — TR–VL–TS scheme

- Split D_l into D_{TR} , D_{VL} , D_{TS} .
- For each θ_m : train $h_{\theta_m}^*(D_{TR})$ and compute $R_{emp}(h_{\theta_m}^*(D_{TR}), D_{VL})$.
- Choose θ_m^* with the best result on VL.
- Retrain $h^*(D_{TR} \cup D_{VL})$ with θ_m^* .
- Estimate R with $\frac{1}{ts} \sum_{z_i \in D_{TS}} L(z_i, h^*(D_{TR} \cup D_{VL}))$.

Retraining can use part of the data as VL for early stopping.

Questions — Can one retrain again on the whole D_l ?

The final retraining on all the data (TR+VL+TS) **does not use** the test results for any choice: the final model will in general have performance equal to or better than the estimate (more data), which remains valid as a (slightly pessimistic) estimate of the risk of the procedure. What matters is that the test has not influenced any decision.

10.5.2 K-fold CV for selection + hold-out for the test

Summary — CV + test scheme

- Split D_l into D_{TR} (the *design set*) and D_{TS} .
- For each θ_m : estimate $R(h_{\theta_m}^*(D_{TR}))$ with a K-fold CV on D_{TR} (mean validation error over the folds).
- Choose θ_m^* with the best estimate.

- Retrain $h^*(D_{TR})$.
- Estimate R with the error on D_{TS} .

10.5.3 K-fold CV for the test with internal hold-out for selection

In each row of the external K-fold, one fold acts as test and the rest is split (arbitrarily and with shuffling) into TR and VL for model selection. The description as a meta-algorithm is left as an exercise.

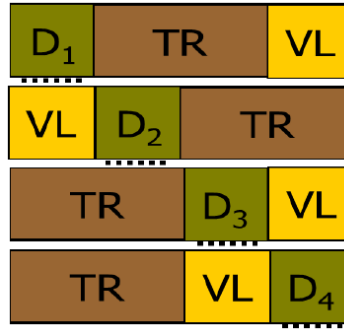


Figure 10.2: External K-fold CV for the test, with internal TR/VL hold-out for model selection (retraining is not shown).

Note — No single model

This process **does not provide a final model**: it only gives an estimate of the risk of the **class** of models considered, because potentially each row selects a different model.

10.5.4 Double (nested) K-fold CV

Summary — Double K-fold CV

- Split D_l into K distinct, equal parts.
- For each part D_k [**outer loop**]:
 - choose $h^*(\bar{D}_k)$ with an **internal K-fold CV** on $\bar{D}_k$ (with K' possibly different from K) [**inner loop**];
 - estimate $R(h^*(\bar{D}_k))$ with $R_{emp}(h^*(\bar{D}_k), D_k)$ [test error for each outer split].
- Estimate $R(h^*(D_l))$ with the mean over the test folds.

Here too **no single final model is obtained**, because each outer loop may choose different hyperparameters: one obtains an estimate of the risk of the class of models/algorithm, together with its **variance** (standard deviation over the folds). If a single final model is needed, a **separate model selection** (hold-out or K-fold CV) and a possible final retraining are carried out.

This does not violate the rules: the test error has already been estimated and test results are never used for selection; the final model will have an expected error within the estimated range. This is why it is not simply true that “the test always comes last”: here it takes place **before** the last model selection (even if it may seem counter-intuitive).

Double CV makes better use of **all** the data for training and evaluation (both for selection and for assessment), at the price of a high computational cost.

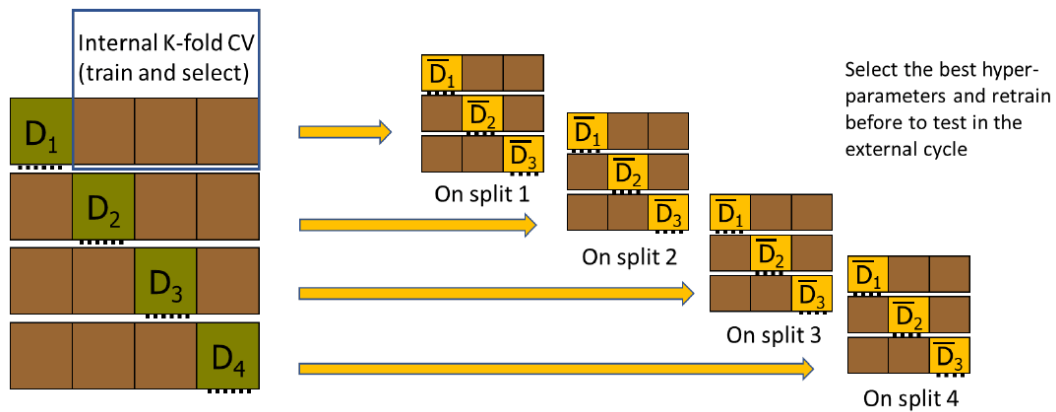


Figure 10.3: Double K-fold CV: in each outer split an inner CV selects the hyperparameters.

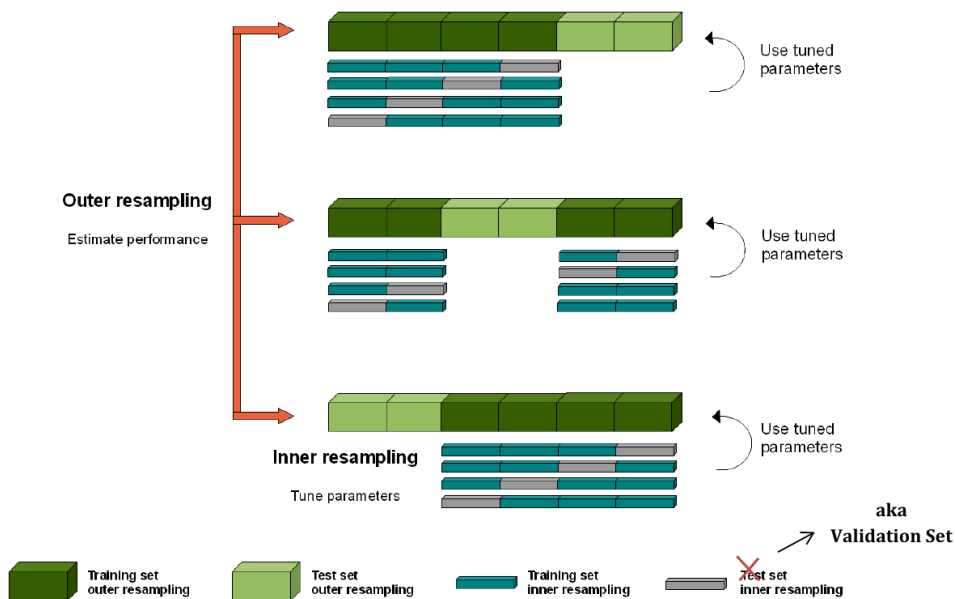


Figure 10.4: Another view of double CV: outer resampling to estimate the performance, inner resampling to tune the parameters.

Summary — Summary

First the goal is identified (best model, performance estimate, or both), then the scheme is chosen. In all correct schemes: the **test is separated before** any choice, the estimates on **VL are not returned** as risk estimates, each CV iteration **starts from scratch**. The most used combined schemes are TR-VL-TS, CV+test and double CV.

Questions — Possible exam questions

- Write the model selection algorithm with K-fold CV.
- Write the model assessment algorithm with K-fold CV.
- Describe double (nested) CV. Does it return a model?
- Relate errors (1) and (2) of the counterexample to the formal schemes.

Nested/Double CV (a further different view, also with the inner hold-out case)

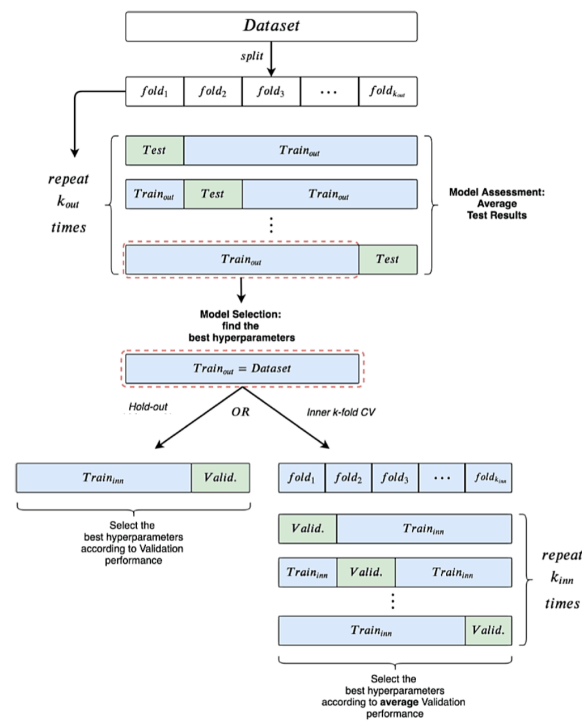


Figure 10.5: Nested CV, also in the variant with internal hold-out.

Chapter 11

Validation (part 3): common mistakes and FAQ

The last part on validation collects practical advice on the use of cross-validation, the **most frequent mistakes** (especially in projects) and some examples of model selection, partly taken from the slides of Andrew W. Moore (CMU).

11.1 Stopping training

11.1.1 Do not fix an arbitrary number of epochs

Warning — Frequent mistake

Avoid stopping the training of a network after a **fixed and arbitrary number of epochs**.

Why:

- if it is small, one may stop **too early** (underfitting, or favouring high learning rates that require fewer epochs);
- if it is large, one may stop **too late** (overfitting or wasted time);
- the same value cannot be right for all configurations and all iterations of a K-fold CV;
- and which number should be used in the final retraining?

Even **selecting the number of epochs via model selection** is not the best practice: it is better than a fixed number but still imprecise (because of the problems with K-fold CV and because better criteria exist). The solutions that work well with a **well-regularised** system are those seen in [08 - Neural networks \(part 2\) - training in practice](#): look at the convergence of training (threshold on the error decrease or on the gradient), or **early stopping** if one can/wants to use a validation set.

11.1.2 Early stopping

Warning — Early stopping and model selection

The early stopping (ES) criterion is **part of model selection** (it must not be applied on the TS): in principle it requires a **VL set every time** the model is trained.

It is delicate: it uses part of the data only to decide when to stop, without giving a concrete hyperparameter value, and it can become complex inside a CV. The problem: how to choose the stopping point for the **final retraining**, if each fold of the CV has a different stopping point?

- One could consider the number of epochs as a hyperparameter and take the **mean** over the folds. But changing the amount of data in the retraining can change the optimal stopping

point (for example with more data per epoch, and hence more updates per epoch in on-line mode, fewer epochs are needed): one risks under- or over-training.

- **Much better:** take the **mean training error** at the best validation point and use it as a threshold in the retraining, to reach the same level of fitting (avoiding under-training effects).
- Otherwise a VL set is needed every time: for example CV with ES active in each validation fold (together with the other hyperparameters); once the best hyperparameters are found, retrain on the recomposed dataset still using part of the data as VL for ES.

It is a choice: use ES or rely on **regularised models** with standard stopping criteria (e.g. no improvement on training). In any case regularisation helps avoid heavy over-training and reduces the dependence on ES.

11.2 Random initialisation and model selection

Different weight initialisations lead to **different models**. How to choose the final one (a choice that must be reported in the report)?

- The **mean and variance** of the error/accuracy over the different trials are computed: a random case will fall within that interval, so the risk is under control, unless the variance is high (which is in any case a negative signal, see the lecture on bias and variance).
- **All** the models can be used, with an **ensemble** (average of the outputs or vote).
- If **one** is chosen, that is a **model selection** and a VL set is needed: the **best** (minimum error on VL), the **median** (a more cautious choice), or a **random** one (in this case no VL set is needed).
- “Unlucky” cases can also be discarded by looking only at the training error.

11.3 Sequential hyperparameter selection

Warning — Frequent mistake

Selecting the hyperparameters **one at a time**, sequentially, introduces an **order-related bias**. Example: choosing the best η with 10 units and then using it also with 100 units.

The correct procedure:

1. initial trials to determine the right **ranges** (for example by looking at the stability of training) and the most **influential** hyperparameters (for example by looking at the variance of the results);
2. search on a **grid with all combinations** of the hyperparameters (more hyperparameters and values $\rightarrow$ larger table);
3. possibly **nested** grids, from coarse to fine, on smaller and smaller intervals, but considering **all the significant hyperparameters together**, so as to take cross-effects into account.

But: reason, select the relevant aspects, do not rely only on systematic, expensive automatic approaches. The time constraints of the project should push towards simple and fast schemes, but correct and reasonable ones.

11.4 What to do after selection and assessment?

Questions — Can I redesign the model based on the test result?

No: it is not the correct way and it is not really advantageous (unless new data are available for a new test set). By repeating this procedure on the same data one can obtain any result one wants, adapting the model to that dataset: the model can become perfect on it and not in future use, and **one is no longer estimating the risk**.

The predictor must not be tuned (by hand or automatically) on the basis of the test error, because we could not do so on future examples. In competitions the blind test prevents exactly this. The evaluation on an internal test is nonetheless useful to **check one's own approach**.

The temptation to decide based on the results is a “psychological” problem, external to ML methods. At least be aware of it, or better:

- to build a **good model** one must do a **good model selection**;
- to estimate the **real performance** one must do a **rigorous** estimate on the test.

11.5 Cross-validation can overfit

Can intensive use of cross-validation lead to overfitting? **Yes**, and it retains the problems already seen. Example with feature selection:

- dataset with 20–30 records and 1000 attributes;
- 1000 linear regression models are tried, each with a single attribute;
- the best of the 1000 looks good (on the validation set)... but it would have looked good even with a **purely random** output!

Or (wrong usage): approximating a random target, with a K-fold CV for the test one finds an error of 3% instead of the true 50%, after choosing the features by best correlation with the targets. **Why?** The selection was made on **all** the examples.

What to do? Keep samples aside **before** selecting:

- keep an **additional test set** before any model selection and check that the best model also performs well on it;
- for model selection with CV: use “**CV + test**” or **double CV** (redoing the selection for each fold);
- for assessment with CV: do the **feature selection inside each fold**. This gives the correct estimate, about 50% for the random task.

11.6 Which cross-validation?

	Disadvantages	Advantages
Test set (hold-out)	variance: unreliable estimate of future performance (luck/bad luck)	cheap
Leave-one-out	expensive; has strange behaviours	does not waste data

	Disadvantages	Advantages
10-fold	wastes 10% of the data; 10 times more expensive than the test set	wastes only 10%; only 10 times more expensive instead of R times
3-fold	wastes more data than 10-fold; more expensive than the test set	slightly better than the test set
R-fold ($R = l$)	identical to leave-one-out	

11.7 Examples of model selection

11.7.1 Example 1: different models

i	f_i	TRAINERR	10-FOLD-CV-ERR	VL	Choice
1	f_1				
2	f_2				
3	f_3				⊗
4	f_4				
5	f_5				
6	f_6				

Figure 11.1: Choice among six classes of models with 10-fold CV.

The class of models with the **best CV score** is chosen, it is retrained on all the data, and that is the predictive model to use. The choice is made looking **only at the validation error**, **not** at the training error (nor at an average of the two).

11.7.2 Example 2: number of hidden units

Algorithm	TRAINERR	10-FOLD-CV-ERR	VL	Choice
0 hidden units				
1 hidden units				
2 hidden units				⊗
3 hidden units				
4 hidden units				
5 hidden units				

Figure 11.2: Choice of the number of hidden units of a network with one hidden layer.

The training error always decreases with the number of units, but the CV error has a minimum (here 2 units). In practice the search is done over values such as 10, 20, 30... (or with other steps).

11.7.3 Example 3: K-NN

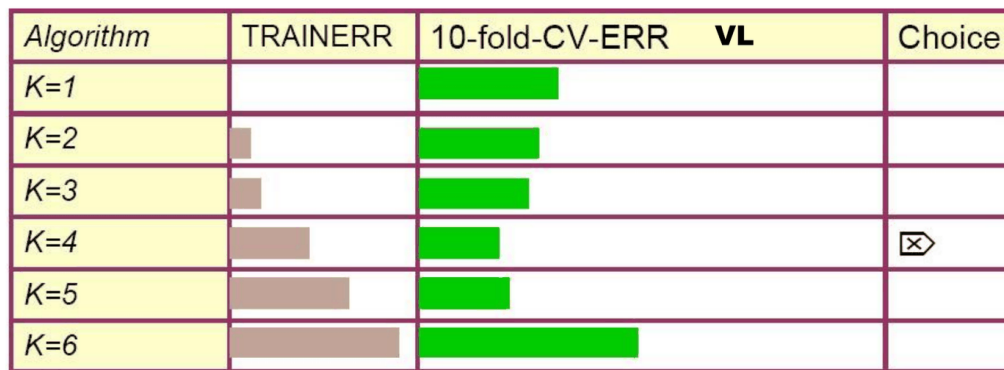


Figure 11.3: Choice of k for K-NN with leave-one-out CV.

Why **leave-one-out** for K-NN and 10-fold for networks? For **computational** reasons: for K-NN (and other non-parametric methods) LOOCV costs as much as a normal prediction (there is no model to train). And why stop at $K = 6$? Only because things seemed to get worse as K grew; and there is no guarantee that a local optimum of K with respect to LOOCV is the global one (the relationship can be very irregular). To explore many values one can use exponentially growing values ($K = 1, 2, 4, 8, 16, \dots$) or a *hill-climbing* search starting from an initial value.

11.8 Practical conclusions

- **Simple hold-out** is widely used: sometimes the test set is already given separately or “taken” from previous applications. Be careful with few data and check the error rates on TR and VL.
- **5- or 10-fold CV** is recommended as a good compromise (between bias and variance of the estimate) with respect to leave-one-out (Hastie et al.).

In the next lecture we go back to the **analytical** estimate of the expected error, with Structural Risk Minimization and the VC-dimension: it offers interesting insights for model selection and comparison (where the **relative size** of the errors matters), as an alternative to purely empirical evaluation.

Questions — Possible exam questions

- Why should training not be stopped after a fixed number of epochs?
- How is early stopping handled with cross-validation and final retraining?
- How to choose the final model with multiple random initialisations?
- What is the sequential hyperparameter selection mistake?
- Can cross-validation overfit? How can it be avoided?
- Compare hold-out, K-fold and leave-one-out.

Chapter 12

Statistical Learning Theory and VC-dimension

After the empirical validation techniques, we go back to the **analytical estimation** of the expected error (validation error, for model selection, or test error, for assessment), introduced in [04 - Generalisation and validation \(introduction\)](#). This lecture formally defines the **VC-dimension**, computes it for some hypothesis classes, presents the SLT bound and the principle of **Structural Risk Minimization** (SRM), which will lead to SVMs.

Let us recall the alternatives for estimating the error:

- **analytical**: AIC, BIC (limited to models linear in the parameters), MDL, **SRM with VC-dimension**;
- **by resampling**: cross-validation, bootstrap.

12.1 The VC-dimension

To obtain an analytical bound we must evaluate the hypothesis space H . We need a measure of its **complexity** (capacity, expressive power, flexibility) that also works for **infinite** H (such as linear models, with real weights): $|H|$ therefore cannot be used. The idea of Vapnik and Chervonenkis is to measure instead **the number of distinct instances that H can completely discriminate**.

Intuitively, in the case of classification: *how well can H discriminate points?* What is the **maximum number of points that can be classified correctly, without errors, for all possible labellings?**

12.1.1 Shattering

Let X be the instance space, with N points (beware: N here is the number of instances considered, which may differ from the number of data l), and let H be the hypothesis space for binary classification.

- There are 2^N possible **dichotomies** (partitions, or labellings of the points with -1 or $+1$).
- A dichotomy is **represented** in H if there exists a hypothesis $h \in H$ that realises it.

Definition — Shattering

H **shatters** X if and only if H can represent **all** possible dichotomies on X (with zero error): for every possible labelling of the points there exists a hypothesis $h \in H$ consistent with it.

Example — Three points in the plane and lines

Consider 3 points in $\mathbb{R}^2$ and as H the set of lines, $h(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x} + w_0)$. A specific dichotomy (for example two points at -1 and one at $+1$) is representable if there exists a line that correctly separates the points. The possible dichotomies are $2^3 = 8$.

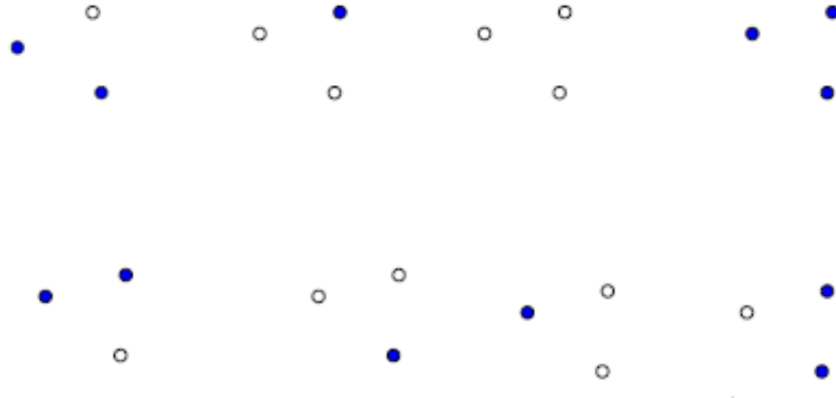


Figure 12.1: All the $2^N = 8$ dichotomies of 3 points in $\mathbb{R}^2$.

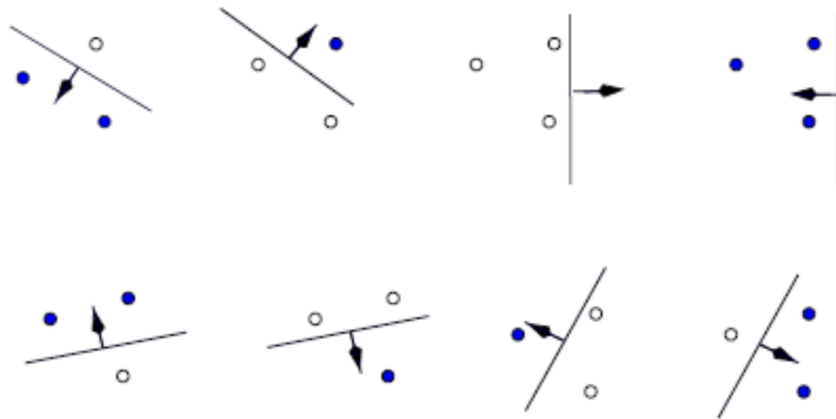


Figure 12.2: For each dichotomy there is a separating line: linear functions shatter these 3 points.

12.1.2 Definition of VC-dimension

Definition — VC-dimension

The **VC-dimension** of a class of functions H is the **maximum cardinality of a set of points** (in some configuration) of X that can be shattered by H . $VC(H) = p$ means that:

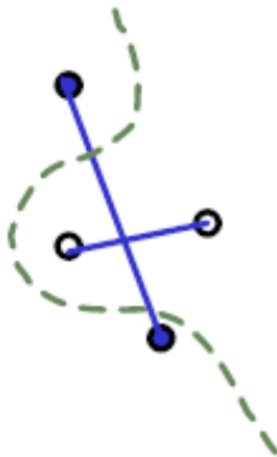
- H shatters **at least one** set (configuration) of p points;
- H does **not** shatter **any** set of $p + 1$ points.

If sets of arbitrarily large size can be shattered, $VC(H) = \infty$.

Note the asymmetry: for the lower bound **one** shatterable configuration is enough; for the upper bound one must show that **no** configuration of $p + 1$ points is shatterable.

12.1.3 VC-dimension of hyperplanes in the plane

- $VC(H) \geq 3$: we have just shown it with 3 non-collinear points. Not all configurations of 3 points are shatterable: three **collinear** points with the middle one of a different class are not linearly separable. But **one** shatterable configuration is enough.
- $VC(H) < 4$: given any 4 points in the plane, one can always draw the 6 segments connecting them in pairs, and two of these cross (if the 4 points form a convex quadrilateral they are the diagonals; if one is inside the triangle of the others, the labelling with the inner point of a different class is chosen). Putting in the same class the points connected by the crossing segments, they cannot be separated linearly: it is **XOR**.



Conclusion: $VC(H) = 3$

Figure 12.3: With 4 points there is always an XOR-like labelling that cannot be separated by a line.

Conclusion: $VC(H) = 3$ for hyperplanes (LTU) in $\mathbb{R}^2$.

Theorem — VC-dimension of hyperplanes

The VC-dimension of the class of separating hyperplanes (LTU) in an n -dimensional space is $n + 1$.

It is an upper bound: constraining the model (for example by bounding the norm of the weights, as will be seen in SVMs) it can decrease.

12.1.4 Other examples

Class H	VC-dim
Lines in $\mathbb{R}$ (thresholds, $\text{sign}(w_1x + w_0)$)	2
Intervals on $\mathbb{R}$	2
Circles centred at the origin in 2D, $\text{sign}(\mathbf{x} \cdot \mathbf{x} - b)$	1
Circles centred at the origin, $\text{sign}(k \mathbf{x} \cdot \mathbf{x} - b)$	2
Circles (free radius and centre) in 2D	3
Circles (fixed radius, free centre) in 2D	3

(For circles centred at the origin with a single parameter b , the circle includes points close to the origin: with two points at different distances one cannot label “+” the farther one and “−” the closer one. Adding k allows inside and outside to be swapped, and the VC-dim rises to 2.)

The VC-dimension can be hard to compute for models with non-linear parameters.

12.1.5 VC-dimension and number of parameters

Is the VC-dimension the number of free parameters? **They are related but do not coincide:**

- in the circle example, classes with the **same** number of parameters have different VC-dims, and vice versa;
- **redundant** parameters can be added without changing anything;
- there are models with **a single parameter and infinite VC-dim** (the classic example is $\text{sign}(\sin(\alpha x))$, which with a large enough α shatters any suitably chosen set of points);
- for the **nearest neighbor** the VC-dim is **infinite** (1-NN correctly classifies any labelling of the training set).

12.2 The analytical risk bound

Let N be the number of data (l).

Theorem — VC-bound

With probability at least $1 - \delta$, for every $h \in H$ (with $VC < N$):

$$R[h] \leq R_{\text{emp}}[h] + \varepsilon(VC, N, \delta),$$

where the right-hand side is the **guaranteed risk** and ε is the **VC-confidence**. For example, for the 0/1 loss:

$$\varepsilon(VC, N, \delta) = \sqrt{\frac{VC \left(\ln \frac{2N}{VC} + 1 \right) - \ln \frac{\delta}{4}}{N}}.$$

There are different formulations for different function classes and tasks. The interpretation is the one already seen: $\varepsilon \rightarrow 0$ as N grows, ε grows with the VC-dim, and the bound on R has a U-shaped trend as the VC-dim grows.

The formula shows it clearly: ε depends essentially on the ratio VC/N (up to logarithms). Doubling the data has the same effect as halving the complexity.

The bound makes it possible to **estimate the error on future data** based only on the training error and on the VC-dimension of H , and the VC-confidence can be computed **before** learning.

Warning — “Worst case” bound

The resulting bounds are *worst case*: they must hold for all functions and all training sets, except a fraction δ . This is why they are often very loose.

Tip — A practical consequence

For many reasonable classes (e.g. linear approximators) the VC-dim is **linear in the number of free parameters**. Therefore, to learn “well”, a number of examples **linear in the**

VC-dimension is needed (and hence, in this case, in the number of parameters). It is a theoretical explanation of practical heuristics such as “at least k examples per parameter are needed”.

12.3 Structural Risk Minimization

SRM uses the VC-dimension as a control parameter to minimise the risk bound. Assuming finite VC-dims, a **nested structure** of hypothesis spaces ordered by VC-dim is defined:

$$H_1 \subseteq H_2 \subseteq \dots \subseteq H_n, \quad VC(H_1) \leq \dots \leq VC(H_n).$$

Examples of structures:

- neural networks with an increasing number of **hidden units** (roughly), but also with an increasing number of **epochs** (we know that the size of the weights also matters: the network is a variable-dimension space);
- polynomials of increasing **degree** M ;
- increasing values of c , where $\|\mathbf{w}\| < c$ for **regularisation** (or equivalently controlled by λ);
- decision trees with an increasing number of nodes (roughly).

The **important hyperparameters** are precisely those related to the VC-dimension.

12.3.1 SRM for model selection

As the VC-dim grows, the empirical (training) error **decreases** and the VC-confidence **increases**. SRM looks for the trade-off on the bound

$$R \leq R_{emp} + \varepsilon(1/l, VC, 1/\delta),$$

i.e. it chooses the model h with the **best bound on the true risk**.

Warning — Warning

The VC-confidence can be **very conservative**: even hundreds of times larger than the overfitting effect observed empirically.

12.3.2 What the bound is for

- It provides a **theoretical foundation** for principled ML, independent of the details of the model or algorithm; it highlights the role of **complexity control**; it shows that the optimal choice of complexity (on the structure) minimises the expected risk: this is the **inductive principle of SRM**.
- As an **estimate of the predictive error** it is rarely used:
 - it can be very loose, but it remains useful for **model selection**, where the **relative size** (the *ranking*) of the errors matters;
 - it is a probabilistic upper bound valid for **all** functions in the class, so one can search within the class;
 - it is hard to compute the VC-dim of specific classes;

ε-off on the bound

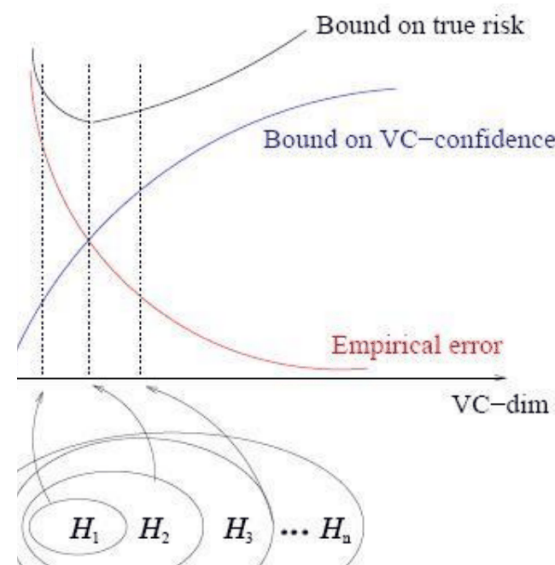


Figure 12.4: Structural Risk Minimization: on a nested structure of hypothesis spaces, the one that minimises the risk bound is chosen.

i	H_i	TR err	VC-Conf	Probable upper bound on R	Choice
1	H_1				
2	H_2				
3	H_3				x
4	H_4				
5	H_5				
6	H_6				

Figure 12.5: Model selection with SRM: H_3 is chosen, with the minimum probable bound on R .

- the raw bound (too pessimistic) may not be adequate for a reliable evaluation of the generalisation error (**model assessment**); tighter bounds are being developed.
- It indicates a direction for developing **new models** guided by SRM, towards principled approaches rather than empirical trial and error.

12.3.3 Two practical approaches to SRM

1. **Fix the structure/complexity** (and hence the VC-confidence) and **minimise the training error**. It can be used with neural networks. Training heuristics may already implement an **implicit SRM** (early stopping, ...); or SRM is implemented with **Tikhonov regularisation**, minimising a loss $R_{emp} + \text{complexity term}$, which considers both terms.
2. **Fix the training error and automatically minimise the VC-confidence**. How? This is the idea of **SVMs** (13 - Support Vector Machines).

Summary — Summary

The **VC-dimension** measures the capacity of H as the maximum number of points that H can shatter (classify correctly for every labelling). For hyperplanes in $\mathbb{R}^n$ it is $n + 1$; it does not coincide with the number of parameters (it can be infinite with a single parameter, and it is infinite for NN). The **VC-bound** $R \leq R_{emp} + \varepsilon(VC, N, \delta)$ links risk, training error, complexity and number of data. **SRM** chooses, in a nested structure of spaces, the one that minimises the bound. The bounds are often loose, but they provide the theoretical foundation of complexity control.

Questions — Possible exam questions

- Define shattering and VC-dimension.
- Prove that the VC-dim of lines in the plane is 3.
- Does the VC-dim coincide with the number of free parameters? Examples.
- Write the VC-bound and interpret its terms.
- What is Structural Risk Minimization? Examples of nested structures.
- What are the practical limits of the bound? What is it useful for anyway?

Chapter 13

Support Vector Machines

Support Vector Machines (SVMs) are the answer to the question left open by SLT (12 - [Statistical Learning Theory and VC-dimension](#)): is it possible to fix the training error and **automatically** minimise the VC-confidence? The SVM is still a linear machine (an LTU), but among all separating hyperplanes it chooses the one that **maximises the margin**, and this is equivalent to controlling the VC-dimension. The lecture proceeds in four steps: linear *hard margin* SVM, *soft margin* SVM for non-separable data, **kernels** for non-linear problems, and finally SVM for **regression**.

Note — Notation (from Haykin, ch. 6)

N is the number of examples (previously l), with indices i and j for the patterns; m is the dimension of the input vector (previously n); b is used instead of w_0 for the bias. The parts marked with * in the slides are useful for completeness but not essential for using the model (and for a standard oral exam): in particular the details of the solution with Lagrange multipliers.

13.1 Part I — SVM for binary classification

An SVM is:

- a **linear machine** (like the LTU or the Perceptron);
- that **maximises the separation margin**;
- and implements **Structural Risk Minimization**.

Initially (*hard margin* SVM) the problem is assumed to be **linearly separable** and the data are assumed to contain **no errors**.

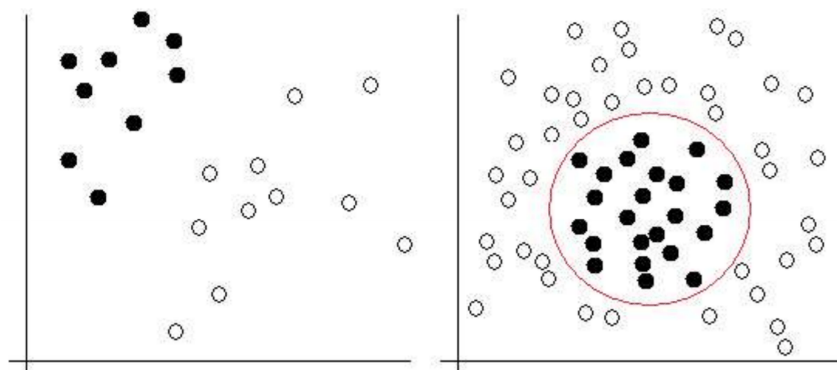


Figure 13.1: Linearly separable (left) and non-linearly separable (right) patterns.

13.1.1 The separating hyperplane

Given the training set $T = \{(\mathbf{x}_i, d_i)\}_{i=1}^N$, we look for a hyperplane $\mathbf{w}^T \mathbf{x} + b = 0$ that separates the examples:

$$\mathbf{w}^T \mathbf{x}_i + b \geq 0 \text{ if } d_i = +1, \quad \mathbf{w}^T \mathbf{x}_i + b < 0 \text{ if } d_i = -1.$$

$g(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$ is the **discriminant function** and $h(\mathbf{x}) = \text{sign}(g(\mathbf{x}))$ is the **hypothesis**.

13.1.2 The separation margin

Definition — Separation margin

If the hyperplane is at the same distance from the nearest positive example and from the nearest negative one, the **separation margin** ρ is twice the distance between the hyperplane and the nearest point. It can be thought of as a “**safety zone**” around the boundary.

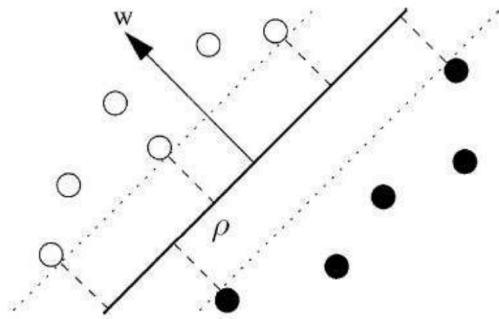


Figure 13.2: The separation margin ρ .

Not all hyperplanes that solve the problem are equal: as the separating hyperplane varies, the margin varies too.

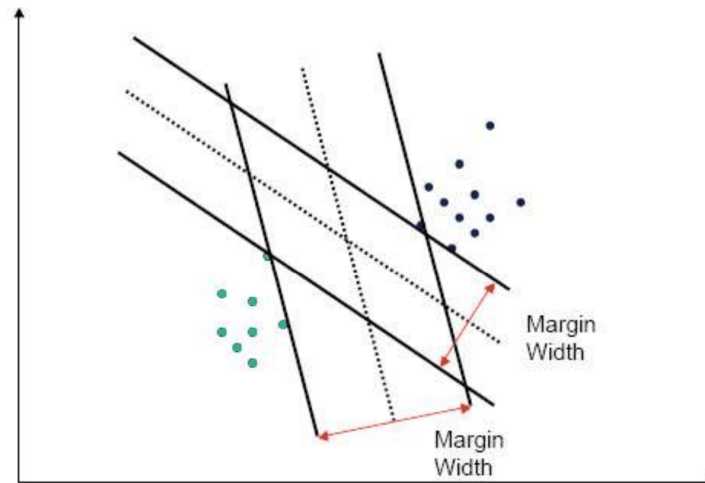


Figure 13.3: Different separating hyperplanes have different margins.

Definition — Optimal hyperplane

The **optimal hyperplane** $\mathbf{w}_o^T \mathbf{x} + b_o = 0$ (“o” stands for *optimal*) is the one that **maximises the margin** ρ . We will see that $\rho = \frac{2}{\|\mathbf{w}\|}$, hence

$$\text{maximising } \rho \iff \text{minimising } \|\mathbf{w}\|.$$

13.1.3 Canonical representation and support vectors

Thanks to **scale freedom** (multiplying $\mathbf{w}$ and b by a constant does not change the hyperplane) we can rescale $\mathbf{w}$ and b so that the points closest to the hyperplane satisfy $|g(\mathbf{x}_i)| = |\mathbf{w}^T \mathbf{x}_i + b| = 1$. Then:

$$\mathbf{w}^T \mathbf{x}_i + b \geq 1 \quad \text{if } d_i = +1, \quad \mathbf{w}^T \mathbf{x}_i + b \leq -1 \quad \text{if } d_i = -1,$$

i.e., in compact form:

$$d_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 \quad \forall i = 1, \dots, N.$$

Definition — Support vector

A **support vector** $\mathbf{x}^{(s)}$ satisfies the constraint **with equality**: $d^{(s)}(\mathbf{w}^T \mathbf{x}^{(s)} + b) = 1$. Support vectors are the points **closest** to the hyperplane, those lying on the edge of the margin.

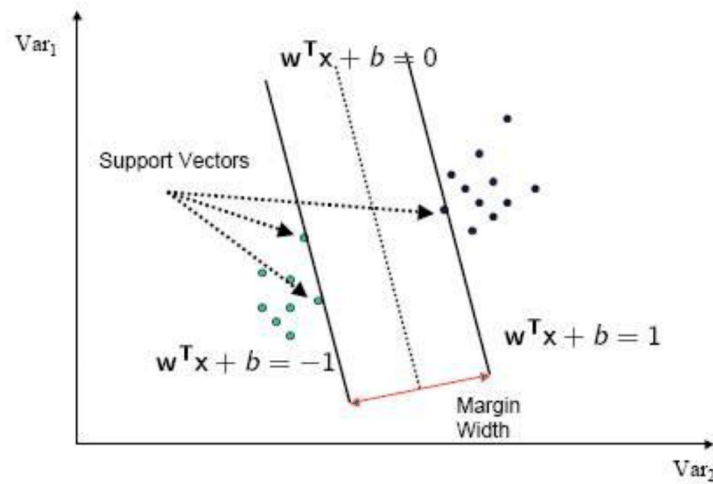


Figure 13.4: Support vectors lie on the lines $\mathbf{w}^T \mathbf{x} + b = \pm 1$.

13.1.4 Computing the margin

Distance of a point from the hyperplane. Let r be the distance between $\mathbf{x}$ and the optimal hyperplane. Since $\mathbf{w}_o$ is orthogonal to the hyperplane, we can write

$$\mathbf{x} = \mathbf{x}_p + r \frac{\mathbf{w}_o}{\|\mathbf{w}_o\|},$$

where $\mathbf{x}_p$ is the projection of $\mathbf{x}$ onto the hyperplane.

Evaluating $g(\mathbf{x}) = \mathbf{w}_o^T \mathbf{x} + b_o$:

$$g(\mathbf{x}) = \mathbf{w}_o^T \mathbf{x}_p + b_o + r \mathbf{w}_o^T \frac{\mathbf{w}_o}{\|\mathbf{w}_o\|} = \underbrace{g(\mathbf{x}_p)}_{=0} + r \frac{\|\mathbf{w}_o\|^2}{\|\mathbf{w}_o\|} = r \|\mathbf{w}_o\|,$$

because $\mathbf{x}_p$ lies on the hyperplane. Hence

$$r = \frac{g(\mathbf{x})}{\|\mathbf{w}_o\|}.$$

Margin. For a positive support vector $g(\mathbf{x}^{(s)}) = 1$, so its distance from the hyperplane is $r = \frac{1}{\|\mathbf{w}_o\|} = \frac{\rho}{2}$. Hence:

$$\rho = \frac{2}{\|\mathbf{w}_o\|}.$$

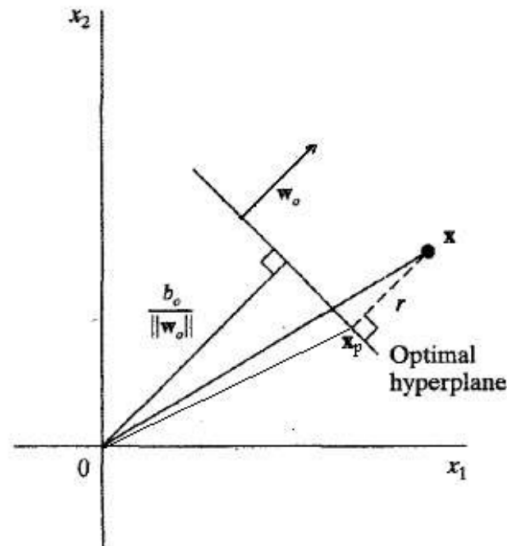


Figure 13.5: Distance r between a point $\mathbf{x}$ and the optimal hyperplane.

The optimal hyperplane maximises ρ , i.e. it **minimises** $\|\mathbf{w}\|$.

13.1.5 The quadratic optimisation problem (hard margin)

The formal derivation requires **constrained optimisation** techniques (assumed known, for example from the CM course); quadratic programming is considered solved by existing packages. Here we are interested in understanding how the problem is formulated, the relationship between support vectors and multipliers α , and the final result for $\mathbf{w}_o$.

Definition — Primal problem — hard margin SVM

Given $T = \{(\mathbf{x}_i, d_i)\}_{i=1}^N$, find $\mathbf{w}$ and b that minimise

$$\Psi(\mathbf{w}) = \frac{1}{2} \mathbf{w}^T \mathbf{w} \quad (\text{i.e. } \min \|\mathbf{w}\|)$$

subject to the constraints (zero classification errors)

$$d_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 \quad \forall i = 1, \dots, N.$$

- The objective function is **quadratic and convex** in $\mathbf{w}$.
- The constraints are **linear** in $\mathbf{w}$.
- Solving this problem scales with the dimension of the input space m (the actual cost depends anyway on the solver algorithm).

Solution with Lagrange multipliers*

The **Lagrangian** is built:

$$J(\mathbf{w}, b, \alpha) = \frac{1}{2} \mathbf{w}^T \mathbf{w} - \sum_{i=1}^N \alpha_i (d_i(\mathbf{w}^T \mathbf{x}_i + b) - 1),$$

with $\alpha_i \geq 0$ the **Lagrange multipliers**, one for each constraint of the primal. J must be **minimised with respect to $\mathbf{w}$ and b** and **maximised with respect to α** : the solution is a **saddle point** of J .

Optimality conditions:

$$\frac{\partial J}{\partial \mathbf{w}} = 0 \Rightarrow \mathbf{w} = \sum_{i=1}^N \alpha_i d_i \mathbf{x}_i, \quad \frac{\partial J}{\partial b} = 0 \Rightarrow \sum_{i=1}^N \alpha_i d_i = 0.$$

Kuhn-Tucker conditions. At the saddle point

$$\alpha_i (d_i (\mathbf{w}^T \mathbf{x}_i + b) - 1) = 0 \quad \forall i.$$

Therefore:

- if $\alpha_i > 0$, then $d_i (\mathbf{w}^T \mathbf{x}_i + b) = 1$: $\mathbf{x}_i$ is a **support vector**;
- if $\mathbf{x}_i$ is not a support vector, then $\alpha_i = 0$.

Tip — The hyperplane depends only on the support vectors

$\mathbf{w}_o = \sum_{i=1}^{N_s} \alpha_{o,i} d_i \mathbf{x}_i$: the sum is restricted to the N_s support vectors. **The hyperplane depends only on the support vectors!** All the other points could be removed without changing the solution.

The dual problem*

Substituting the optimality conditions into J we obtain the **dual problem**, in which only the multipliers appear:

Definition — Dual problem — hard margin SVM

Find $\{\alpha_i\}_{i=1}^N$ that maximise

$$Q(\alpha) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j d_i d_j \mathbf{x}_i^T \mathbf{x}_j$$

subject to $\alpha_i \geq 0$ for all i and $\sum_{i=1}^N \alpha_i d_i = 0$.

The α are found by solving the **quadratic programming** (QP) problem or with more recent and efficient approaches (for example **SMO**, *Sequential Minimal Optimization*). The dual scales with the **number of examples** N , less with the dimension of the space. Note that the data appear **only through dot products** $\mathbf{x}_i^T \mathbf{x}_j$: this will be the key to kernels.

Obtaining $\mathbf{w}_o$ and b_o

1. Solve the dual obtaining $\{\alpha_{o,i}\}$.
2. Compute $\mathbf{w}_o = \sum_{i=1}^N \alpha_{o,i} d_i \mathbf{x}_i$.
3. Compute $b_o = 1 - \mathbf{w}_o^T \mathbf{x}^{(s)}$ for a positive support vector $\mathbf{x}^{(s)}$, i.e. $b_o = 1 - \sum_{i=1}^N \alpha_{o,i} d_i \mathbf{x}_i^T \mathbf{x}^{(s)}$.

13.1.6 How it is used

There is no need to know $\mathbf{w}_o$ explicitly: the multipliers (from the dual) and the bias are enough. The decision surface is

$$\mathbf{w}_o^T \mathbf{x} + b_o = 0 \iff \sum_{i=1}^N \alpha_{o,i} d_i \underbrace{\mathbf{x}_i^T \mathbf{x}}_{\text{dot product}} + b_o = 0.$$

Given a pattern $\mathbf{x}$: (1) compute $g(\mathbf{x}) = \sum_{i=1}^N \alpha_{o,i} d_i \mathbf{x}_i^T \mathbf{x} + b_o$; (2) classify with the sign of $g(\mathbf{x})$. The sum can be restricted to the N_s support vectors.

13.1.7 Why does it improve generalisation?

On linearly separable problems we have **fixed the training error** (at zero). Minimising the norm of $\mathbf{w}$ is then equivalent to **minimising the VC-dimension**, and hence the capacity term (VC-confidence) ε in

$$R[h] \leq R_{emp}[h] + \varepsilon(VC, N, \delta).$$

Theorem — Vapnik's theorem

Let D be the diameter of the smallest sphere containing the data $\mathbf{x}_1, \dots, \mathbf{x}_N$. For the class of separating hyperplanes $\mathbf{w}^T \mathbf{x} + b = 0$ with margin ρ , the VC-dimension is bounded by

$$VC \leq \min \left(\left\lceil \frac{D^2}{\rho^2} \right\rceil, m_0 \right) + 1.$$

Since $\frac{D^2}{\rho^2} \propto \text{Radius}^2 \|\mathbf{w}\|^2$, the VC-dim can be **smaller than** $m_0 + 1$ (that of generic hyperplanes) by restricting the search to “regularised” hyperplanes with maximum margin. **The larger the margin, the smaller the VC-dim**, regardless of the dimension of the space.

13.1.8 Why it is an elegant approach

For linearly separable data there are many solutions; Vapnik proposes the maximum-margin **optimal separating hyperplane**, which provides:

- a **unique solution** (unlike the iterative Perceptron algorithm) with **zero errors** (unlike LMS) for the binary classifier;
- an **automated SRM**, which minimises the VC-confidence (by maximising the margin) directly within the optimisation process, **without hyperparameters** in the separable case;
- the use of **constrained quadratic programming** solvers (instead of gradient descent), with a nice dual form that shows the support vectors and the dot products between patterns;
- a solution focused on “**selected**” training data, the support vectors: a strength, because it does not depend on points far from the boundary; a weakness, because one hopes that the points on the boundary are not the noisiest.

But what to do with **noisy** or **non-linearly separable** data?

13.2 Soft margin SVM: allowing errors

If at least one point violates the exact separation condition $d_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1$, we move to the **soft margin**.

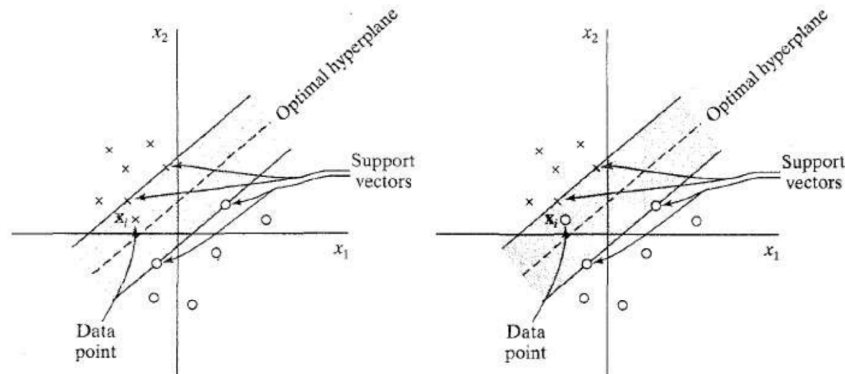


Figure 13.6: Patterns violating the margin, without (left) and with (right) classification error.

Tip — The advantage of the soft margin

Allowing points inside the margin makes it possible to have a **wider margin**: one obtains noise tolerance and a simpler model. (Exercise: find the margin in the figures without allowing points inside it.)

13.2.1 Slack variables

Non-negative variables $\xi_i \geq 0$, called **slack variables**, are introduced, and the constraints become

$$d_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i \quad \forall i.$$

- $\xi_i = 0$: the point is outside the margin or on its edge; - $0 < \xi_i \leq 1$: the point is inside the margin but correctly classified; - $\xi_i > 1$: the point is misclassified.

A support vector now satisfies $d_i(\mathbf{w}^T \mathbf{x}_i + b) = 1 - \xi_i$.

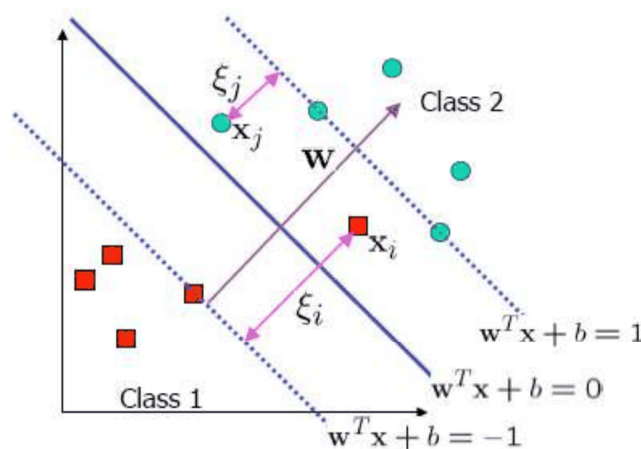


Figure 13.7: Soft margin and slack variables.

Beware: **Vapnik's theorem no longer holds** (the hard margin does not allow points inside the margin).

13.2.2 The soft margin primal problem

Definition — Primal problem — soft margin SVM

Find $\mathbf{w}$ and b that minimise

$$\Psi(\mathbf{w}, \xi) = \frac{1}{2} \mathbf{w}^T \mathbf{w} + C \sum_{i=1}^N \xi_i$$

subject to $d_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i$ and $\xi_i \geq 0$ for all i .

C is a **regularisation hyperparameter** (fully automatic SRM is lost!): it controls the trade-off between minimising the empirical risk and minimising the capacity term (VC-confidence).

- **Low** C : many training errors allowed $\rightarrow$ possible **underfitting**.
- **High** C : no errors allowed $\rightarrow$ smaller margin $\rightarrow$ possible **overfitting**.

(For $C \rightarrow \infty$ we recover the hard margin.)

13.2.3 The soft margin dual*

Definition — Dual problem — soft margin SVM

Find $\{\alpha_i\}$ that maximise

$$Q(\alpha) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j d_i d_j \mathbf{x}_i^T \mathbf{x}_j$$

subject to $\sum_{i=1}^N \alpha_i d_i = 0$ and $0 \leq \alpha_i \leq C$ for all i .

It is **identical** to the hard margin case, except for the upper constraint $\alpha_i \leq C$. The Kuhn-Tucker conditions become

$$\alpha_i (d_i(\mathbf{w}^T \mathbf{x}_i + b) + \xi_i - 1) = 0, \quad \mu_i \xi_i = 0,$$

where the μ_i are the multipliers that enforce $\xi_i \geq 0$. It follows that:

- $0 < \alpha_i < C \Rightarrow \xi_i = 0$: the point is **on the edge** of the margin;
- $\alpha_i = C \Rightarrow \xi_i \geq 0$: the point is **inside** the margin (or on the wrong side).

Solution: $\mathbf{w}_o = \sum_i \alpha_{o,i} d_i \mathbf{x}_i$ and $b_o = d_j - \sum_i \alpha_{o,i} d_i \mathbf{x}_i^T \mathbf{x}_j$ for a pattern j with $0 < \alpha_j < C$ (or the average over all of these, for numerical stability). The non-zero multipliers correspond to the support vectors. Usage is as before: $g(\mathbf{x}) = \sum_i \alpha_{o,i} d_i \mathbf{x}_i^T \mathbf{x} + b_o$, $h(\mathbf{x}) = \text{sign}(g(\mathbf{x}))$.

13.3 Kernels: SVM for non-linear problems

13.3.1 Mapping into a high-dimensional space

Definition — Idea

Map the data from the input space to a high-dimensional **feature space**, where they become **linearly separable**.

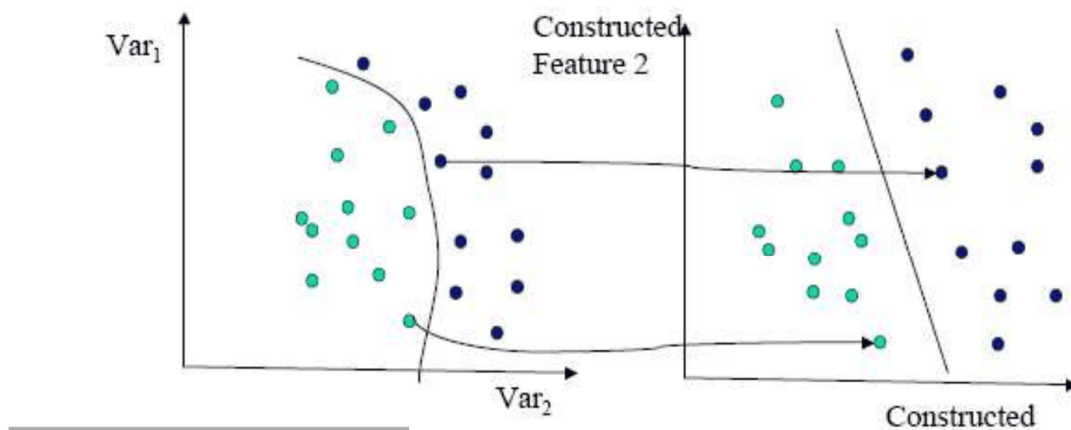


Figure 13.8: A function $\Phi(\mathbf{x})$ maps the data into a space where they are linearly separable (recall the LBE of linear models).

Example — From $\mathbb{R}^2$ to $\mathbb{R}^3$

With $\Phi : \mathbb{R}^2 \rightarrow \mathbb{R}^3$, $(x_1, x_2)^T \mapsto (x_1^2, \sqrt{2}x_1x_2, x_2^2)^T$ (second-degree monomials), two classes separated by an ellipse in the input space become separable by a **plane** in the feature space.

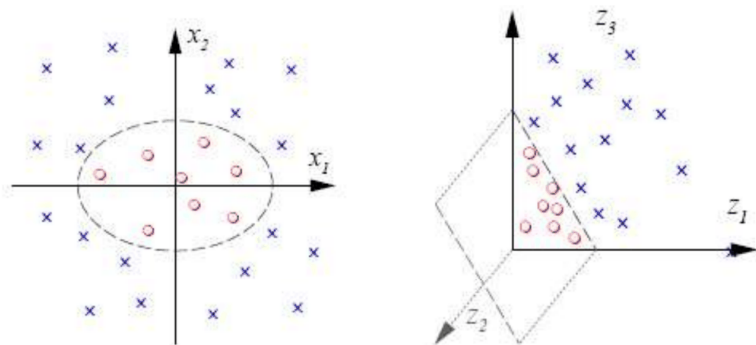


Figure 13.9: In the input space a complex surface is needed; in the space of second-degree monomials a linear function is enough.

The mapping in the example is built on purpose to solve the problem (the features correspond to ellipsoids in the original space). But finding the “best” transformation **is not automatic** (unless there is prior knowledge); general transformations are often used. **The choice of Φ is the critical point of SVMs.**

13.3.2 High-dimensional feature spaces

The procedure is: (1) non-linear mapping of the patterns into a high-dimensional feature space (by **Cover’s theorem**, the patterns are linearly separable there with high probability); (2) search

for the optimal hyperplane in the feature space.

But we know that very large feature spaces (wide basis expansions) can be **computationally infeasible** and lead to **overfitting** without controlling the size of the space and the complexity of the classifier. The **kernel approach** makes it possible to handle the feature space **implicitly**, in a regularised context (where it is the **margin** that controls complexity, not the dimension of the input space).

With $\Phi : \mathbb{R}^{m_0} \rightarrow \mathbb{R}^{m_1}$, the training set becomes $\{(\Phi(\mathbf{x}_i), d_i)\}$ and the hyperplane $\mathbf{w}^T \Phi(\mathbf{x}) + b = 0$. Including the bias ($w_0 = b$, $\phi_0(\mathbf{x}) = 1$), the decision surface $\mathbf{w}^T \Phi(\mathbf{x}) = 0$ is a **linear basis expansion** in which the dimension of the space can be very large and the complexity depends on the **margin**, not on the dimension.

The weight vector is $\mathbf{w} = \sum_i \alpha_i d_i \Phi(\mathbf{x}_i)$, and the hyperplane becomes

$$\sum_{i=1}^N \alpha_i d_i \underbrace{\Phi^T(\mathbf{x}_i) \Phi(\mathbf{x})}_{\text{dot product}} = 0.$$

The dot product is now in the feature space, and computing $\Phi(\mathbf{x})$ could be intractable.

13.3.3 The kernel trick

Under certain conditions **there is no need to compute** $\Phi(\mathbf{x})$, nor even to know the feature space! A function that directly computes the dot product in the feature space is enough:

Definition — Kernel (inner product kernel)

$$k : \mathbb{R}^{m_0} \times \mathbb{R}^{m_0} \rightarrow \mathbb{R}, \quad k(\mathbf{x}_i, \mathbf{x}) = \Phi^T(\mathbf{x}_i) \Phi(\mathbf{x}).$$

Property: it is **symmetric**, $k(\mathbf{x}_i, \mathbf{x}) = k(\mathbf{x}, \mathbf{x}_i)$.

Example — The kernel of the example

With $\Phi(\mathbf{x}) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)^T$, for $\mathbf{x} = (x_1, x_2)^T$ and $\mathbf{y} = (y_1, y_2)^T$:

$$\Phi^T(\mathbf{x})\Phi(\mathbf{y}) = x_1^2y_1^2 + 2x_1x_2y_1y_2 + x_2^2y_2^2 = (x_1y_1 + x_2y_2)^2 = (\mathbf{x}^T\mathbf{y})^2 = k(\mathbf{x}, \mathbf{y}).$$

Very important: the dot product in the (3D) feature space is computed **without going through the mapping**, directly in terms of the (2D) input patterns.

13.3.4 The kernel matrix

The dot products between the images of the training patterns are organised in an $N \times N$ matrix, the **kernel matrix** (or Gram matrix):

$$K = \{k(\mathbf{x}_i, \mathbf{x}_j)\}_{i,j=1}^N,$$

symmetric because the kernel is symmetric.

Does every kernel function compute a dot product in some space? No: the property holds only for kernels that produce **positive semidefinite** kernel matrices (**Mercer's theorem**), i.e. with non-negative eigenvalues.

Closure properties: if k_1 and k_2 are kernels, so are $k_1 + k_2$, αk_1 (with $\alpha > 0$) and $k_1 k_2$.

13.3.5 SVM with kernels

The primal in the feature space is the same as the soft margin one with $\Phi(\mathbf{x}_i)$ instead of $\mathbf{x}_i$ (but $\mathbf{w}$ lives in the feature space, which can make the problem intractable). The **dual**, instead, is tractable:

Definition — Dual problem in the feature space

Find $\{\alpha_i\}$ that maximise

$$Q(\alpha) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i,j=1}^N \alpha_i \alpha_j d_i d_j \underbrace{k(\mathbf{x}_i, \mathbf{x}_j)}_{K_{ij}}$$

subject to $\sum_i \alpha_i d_i = 0$ and $0 \leq \alpha_i \leq C$.

The kernel incorporates the mapping into the feature space. The dependence on N (instead of the dimension of the space) now becomes an **advantage**, because the feature space can have a very high or infinite dimension.

Summary — Summary of the procedure

1. We have the training set $T = \{(\mathbf{x}_i, d_i)\}$.
2. The parameter C and the kernel function k are chosen.
3. The kernel matrix K is computed.
4. The optimisation problem (quadratic programming) is solved, finding $\{\alpha_i\}$.
5. The bias b is computed from the multipliers and the kernel matrix.
6. For a new pattern $\mathbf{x}$: $\mathbf{w}^T \Phi(\mathbf{x}) = \sum_i \alpha_i d_i k(\mathbf{x}, \mathbf{x}_i) + b$ (without ever computing $\mathbf{w}$!), and it is classified with the sign.

Sparsity: the solution is sparse in the α , because all the multipliers of non-support vectors are zero.

In the test phase: $h(\mathbf{x}) = \text{sign}(\sum_i \alpha_i d_i k(\mathbf{x}, \mathbf{x}_i) + b)$. The $\mathbf{x}_i$ (the support vectors) are stored for the test phase: does it resemble K-NN?

13.3.6 “Architectural” view of an SVM

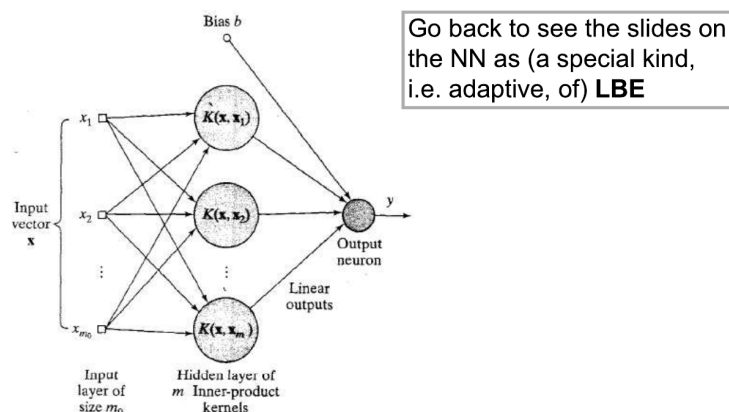


Figure 13.10: “Architecture” of an SVM: number of hidden units = number of support vectors.

The number of “hidden units” is equal to the **number of support vectors**, but they are not exactly the same units as in a neural network: the hidden units of a network have **adaptive free parameters**, whereas here the “centres” are the support vectors themselves and the function is fixed by the kernel. It is only an illustrative view: to be compared with neural networks (an adaptive form of LBE) in terms of complexity control, efficiency and other characteristics.

13.3.7 Examples of kernels

- **Polynomial:** $k(\mathbf{x}, \mathbf{x}_i) = (\mathbf{x}^T \mathbf{x}_i + 1)^p$, with p chosen by the user.
- **RBF (Radial Basis Function, also called Gaussian kernel):** $k(\mathbf{x}, \mathbf{x}_i) = e^{-\frac{1}{2\sigma^2} \|\mathbf{x} - \mathbf{x}_i\|^2}$, with σ^2 chosen by the user. With very “narrow” kernels (small σ) the response on $\mathbf{x}_i$ is just d_i (it resembles 1-NN!).
- **Two-layer perceptron:** $k(\mathbf{x}, \mathbf{x}_i) = \tanh(\beta_0 \mathbf{x}^T \mathbf{x}_i + \beta_1)$, with $\beta_0 > 0$ and $\beta_1 < 0$.

Warning — Important

For the polynomial and the RBF kernel one **always** obtains a valid kernel (dot product), whereas for the two-layer perceptron Mercer’s theorem holds only for some choices of β_0 and β_1 . An RBF kernel always corresponds to a feature space of **infinite dimension**.

13.4 Part II — SVM for non-linear regression

13.4.1 The problem

We have $d = f(\mathbf{x}) + v$, with f unknown and noise v independent of $\mathbf{x}$. d is estimated with a linear expansion of non-linear functions:

$$y = h(\mathbf{x}) = \mathbf{w}^T \Phi(\mathbf{x}), \quad \mathbf{w} = (b, w_1, \dots, w_{m_1})^T, \quad \Phi(\mathbf{x}) = (1, \phi_1(\mathbf{x}), \dots, \phi_{m_1}(\mathbf{x}))^T.$$

13.4.2 The ε -insensitive loss

$$L_\varepsilon(d, y) = \begin{cases} |d - y| - \varepsilon & \text{if } |d - y| \geq \varepsilon \\ 0 & \text{otherwise.} \end{cases}$$

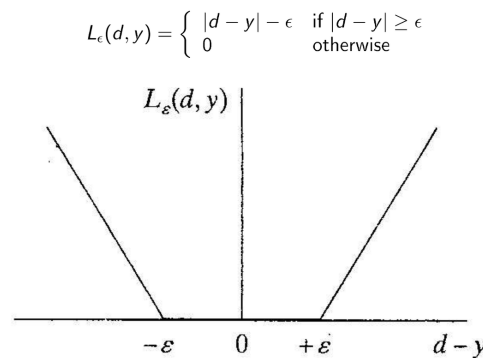


Figure 13.11: The ε -insensitive loss.

Errors smaller than ε **cost nothing**: a **tube** of width ε forms around the prediction, and only the points outside the tube contribute to the loss. Again, **only the support vectors drive the**

solution: in classification they were the points beyond the margin, here they are the points outside the tube.

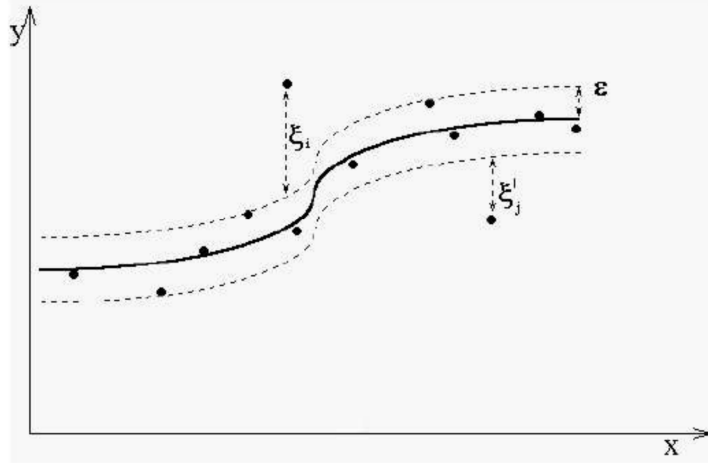


Figure: ϵ -tube and slack variables.

Figure 13.12: The ϵ tube and the slack variables for regression.

13.4.3 Formulation

Two sets of non-negative slack variables ξ_i and ξ'_i (above and below the tube) are introduced:

$$-\xi'_i - \epsilon \leq d_i - \mathbf{w}^T \Phi(\mathbf{x}_i) \leq \epsilon + \xi_i.$$

Definition — Primal problem — SVM for regression

Minimise

$$\Psi(\mathbf{w}, \xi, \xi') = \frac{1}{2} \mathbf{w}^T \mathbf{w} + C \sum_{i=1}^N (\xi_i + \xi'_i)$$

subject to $d_i - \mathbf{w}^T \Phi(\mathbf{x}_i) \leq \epsilon + \xi_i$, $\mathbf{w}^T \Phi(\mathbf{x}_i) - d_i \leq \epsilon + \xi'_i$, $\xi_i \geq 0$, $\xi'_i \geq 0$.

Definition — Dual problem — SVM for regression*

Find $\{\alpha_i\}$ and $\{\alpha'_i\}$ that maximise

$$Q(\alpha, \alpha') = \sum_{i=1}^N d_i(\alpha_i - \alpha'_i) - \epsilon \sum_{i=1}^N (\alpha_i + \alpha'_i) - \frac{1}{2} \sum_{i,j=1}^N (\alpha_i - \alpha'_i)(\alpha_j - \alpha'_j) k(\mathbf{x}_i, \mathbf{x}_j)$$

subject to $\sum_i (\alpha_i - \alpha'_i) = 0$ and $0 \leq \alpha_i, \alpha'_i \leq C$.

From the solution:

$$\mathbf{w} = \sum_{i=1}^N \underbrace{(\alpha_i - \alpha'_i)}_{\gamma_i} \Phi(\mathbf{x}_i), \quad h(\mathbf{x}) = \sum_{i=1}^N \gamma_i \Phi^T(\mathbf{x}_i) \Phi(\mathbf{x}) = \sum_{i=1}^N \gamma_i k(\mathbf{x}_i, \mathbf{x}).$$

The **support vectors** correspond to the non-zero γ_i .

Summary — Summary for regression

1. Choose the parameters C and ε .
2. Choose a kernel k .
3. Compute the kernel matrix K .
4. Solve the dual and obtain the γ_i .
5. Compute the optimal bias.
6. The estimated function is a linear combination of dot products in a feature space that we can ignore: $h(\mathbf{x}) = \sum_i \gamma_i k(\mathbf{x}_i, \mathbf{x})$.

Questions — What you need to know

- The definition of a **maximum margin** classifier.
- How maximum margin turns into a **quadratic programming** problem.
- What QP does for us (not how it does it).
- Why the SVM approximates **SRM**.
- How **noisy** (non-separable) data are handled: soft margin.
- How **non-linear boundaries** are obtained: kernels.
- How kernels make it possible to work with very high-dimensional basis expansions without computing them.
- How SVM for regression works (ε -insensitive loss).

References: Haykin, *Neural Networks* (2nd ed.), ch. 6 (“necessary and sufficient”); Burges, *A Tutorial on Support Vector Machines for Pattern Recognition*; Schölkopf and Smola, *Learning with Kernels*; Vapnik, *The Nature of Statistical Learning Theory* and *An Overview of Statistical Learning Theory* (1999).

Chapter 14

SVM and kernels: practical aspects and critical view

This slide deck complements [13 - Support Vector Machines](#) with a critical view: strengths and weaknesses of SVMs, examples of results, the crucial role of hyperparameters, some myths to debunk and the generalisation to **kernel methods**.

14.1 Main features

Summary — Strengths

- **Regularisation built into** the optimisation problem (the margin).
- **Approximation of** theoretical **SRM** (the bound on the VC-dim of hyperplanes is minimised by the solution, in the hard margin case without kernels).
- **Convex problem**: training always finds the **global minimum** (unlike neural networks).
- **Implicit feature transformation** through kernels.

Warning — Weaknesses

- One must **choose the kernel and its parameters**.
- It is a **batch** algorithm.
- Very large problems used to be computationally intractable: beyond about 20,000 examples it was difficult to solve them exactly with standard approaches (the kernel matrix is $N \times N$). Today there are many solutions, including gradient-descent-based ones.

In short, the advantages:

- a **linear model** (simple, compact) with a bound on complexity in terms of the **margin**, which is optimised directly;
- a rich set of **non-linear decision functions** in the input space through kernels: it is still a linear separation, but in a different space, with an **efficient** treatment of LBE. A rich set does not necessarily imply a high VC-dim, which for SVMs depends on the margin, the kernel parameters, C , etc.;
- the **kernel trick** extends to other models: **kernel methods**.

14.2 Results

14.2.1 The first famous application

Handwritten digit recognition (**MNIST**) was a historic success case for SVMs: about 0.8% errors in 1998 with a degree-9 polynomial SVM, a remarkable result, comparable to LeNet and obtained without years of architecture development. Interest in SVMs and SLT (whose studies dated back to the 1960s–70s) grew considerably: an example of the relevance of applications, or of the importance of good theory for successful applications.

14.2.2 The Gaussian mixture example

Let us go back to the two-class problem (mixture of Gaussians) already seen with linear models, K-NN and neural networks.

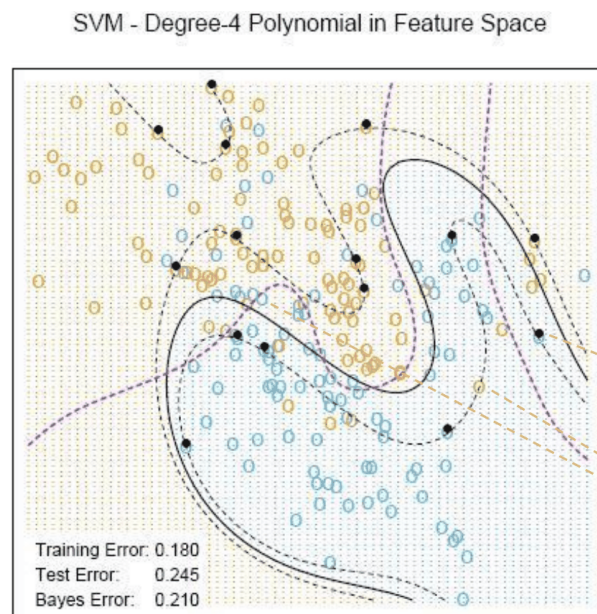


Figure 14.1: SVM with a degree-4 polynomial kernel ($C = 1$): 42% of the data turn out to be support vectors.

Support vectors are the points **on the margin** (slack = 0) or **on the wrong side** of the margin (slack > 0). The radial kernel gives the best result, as can be expected given that the data come from Gaussian mixtures. Compare with the neural network with 10 units and weight decay 0.02 (test error 0.223): similar results.

Warning — Many support vectors

In theory support vectors are assumed to be few, but **this is not always true**: here they are more than 40% of the data. Sparsity of the solution is not guaranteed.

14.3 SVMs in practice

Note — No guarantee

“There is currently no theory that guarantees that a given family of SVMs will have high accuracy on a given problem” (Burges, 1998).

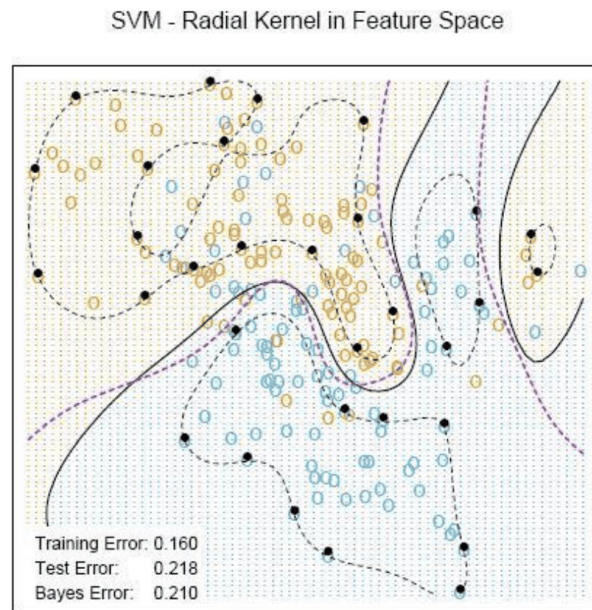


Figure 14.2: SVM with an RBF kernel ($\gamma = 1/(2\sigma^2) = 1$): 45% of the data are support vectors.

The nice properties of hard margin classifiers **do not extend directly** to the soft margin and to kernels: the parameter C and the kernel can lead to a VC-dim that is even **infinite**. For example:

- an SVM with a Gaussian kernel of **sufficiently small width** σ can correctly classify an arbitrarily large number of points (like 1-NN, only the support vector closest to $\mathbf{x}$ contributes to the solution): infinite VC-dim, unless regularised;
- conversely, with a **large** width all support vectors contribute and one obtains a sort of “global average”, with low VC-dim.

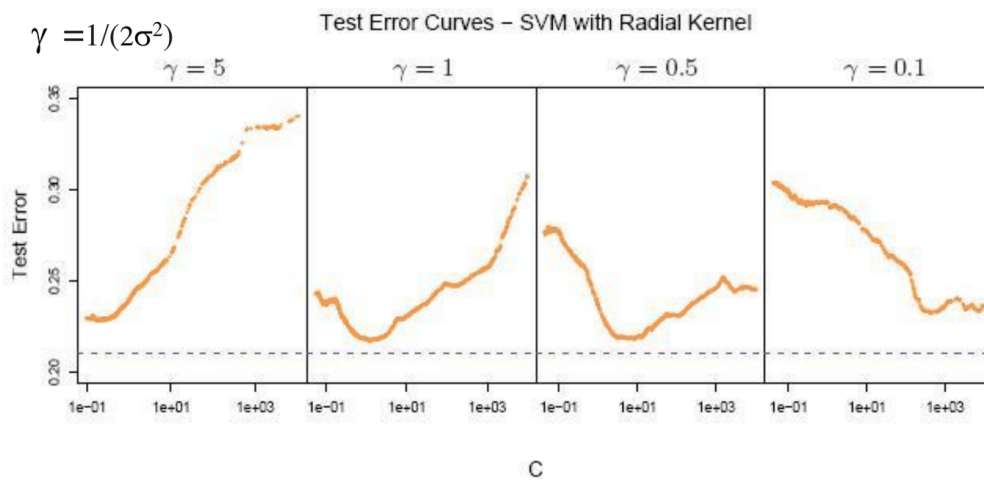
Therefore **by controlling the kernel width one controls the VC-dim**.

14.3.1 The critical role of hyperparameters

With $\gamma = 5$ (small σ , very narrow kernels) a low C (maximum regularisation) is needed; with small γ the optimum moves towards higher C . **The best value of C also depends on the kernel hyperparameters:** they must be selected **together** (grid search), usually with cross-validation, carefully.

To apply an SVM one must therefore choose the **type of kernel** (and its parameters) and the value of C (and of ϵ for regression). A rigorous selection would require an estimate of the complexity (VC-dim); the hyperparameters that affect complexity are those to be used in model selection. In practice one performs, once again, a **careful empirical evaluation**: tuning the hyperparameters ($C, \gamma, \dots$) can introduce overfitting, hence validation set + test set, or double CV.

Performance: often good, but the comparison with other methods is not always favourable. Since 2012–2013 **deep networks** have clearly beaten the previous records in image and speech recognition.



$\gamma = 5$ (small σ , narrow peaked Kernels) require lower C value (max regularization). Hence, the best value of C depends also on the kernel hyper-parameters !

Often selected using cross-validation: take care !

Figure 14.3: Test error as a function of C for different values of $\gamma = 1/(2\sigma^2)$.

14.3.2 Myths about SVMs

Questions — Exercise: what is wrong?

1. “The SVM can be used with the default hyperparameter values.” → No: C and the kernel parameters are critical and must be selected.
2. “I choose the SVM because it does not overfit.” → No: with soft margin and kernels the VC-dim can be very high; overfitting depends on C and on the kernel.
3. “The SVM has no problems with high-dimensional inputs (does it solve the curse of dimensionality?).” → No: the SVM handles well the high dimension of the **feature space**, not of the **input space**. The curse of dimensionality of the input remains.
4. “With the SVM the number of units is found automatically.” → The number of support vectors is determined by the solution, but it depends on C and on the kernel, which must be chosen; and it is not necessarily small.

14.4 Kernel methods

The **kernel trick** leads to **kernel methods**, even without SVMs: the “**kernelisation**” of traditional approaches. Whenever a **dot product** or a **similarity measure** appears in a model, it can be replaced with a kernel, making the model operate in a new implicit high-dimensional space simply by specifying (and changing) the kernel.

- It is a fixed, high-dimensional **linear basis expansion**, but very **efficient**.
- The choice of the basis functions is **modular**: the kernel is changed without changing the learning machine.
- It provides a unified theoretical framework for general dot-product functions.

14.4.1 Kernel as a similarity measure

The kernel is related to a **similarity** measure between $\mathbf{s}$ and $\mathbf{t}$; indeed it induces the distance in the feature space:

$$d_k(\mathbf{s}, \mathbf{t})^2 = k(\mathbf{s}, \mathbf{s}) - 2k(\mathbf{s}, \mathbf{t}) + k(\mathbf{t}, \mathbf{t}) = \|\Phi(\mathbf{s}) - \Phi(\mathbf{t})\|^2.$$

Similar objects (small distance) $\rightarrow$ high kernel value.

Tip — The idea of kernels

Focus on representing and processing **pairwise comparisons** of objects, rather than on the representation of a single object. In some domains it is easier to compare two objects than to define an abstract feature space in which to represent one of them.

An SVM is characterised largely by the choice of the kernel. Compare with **neural networks**, where the basis functions **adapt** to the data (by learning the weights of the hidden units): in SVMs the choice of the best kernel for a problem is still a research topic.

(Exercise: rethink the kernel as in distance-based methods, like K-NN. When is a kernel good? When it groups objects of the same class in the feature space, reflecting the similarity relevant to the task. When is it bad? When the similarity it induces has nothing to do with the task, for example if all objects turn out to be equally similar or equally different.)

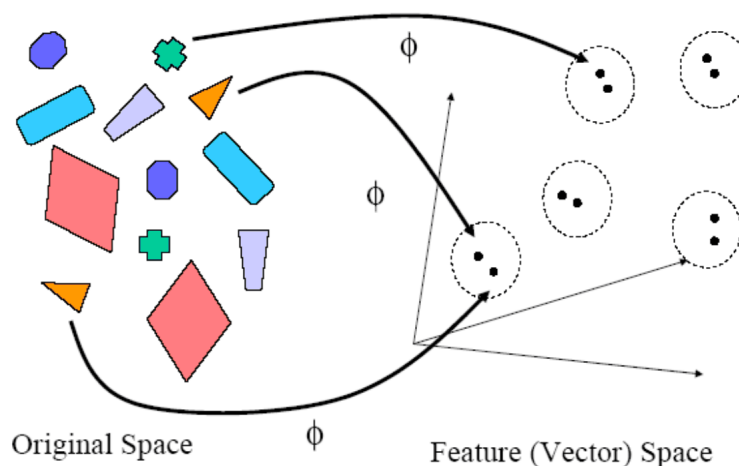


Figure 14.4: A good kernel for “objects”: the map ϕ brings similar objects close together in the feature space.

14.4.2 Advanced topics

Designing new kernels requires choosing a “good” function for the specific domain, and it can be extended to **generic objects** as input, not only vectors: kernels for **strings, trees, graphs**; ad hoc kernels for the domain and the task (for example kernels based on *edit distance* for bioinformatics, or on *suffix trees* for strings in NLP), with attention to algorithmic efficiency. **Adaptive kernels** are a research topic.

A conceptual advantage: **domain knowledge is separated from the solver**, because it lies entirely in the kernel.

Summary — Summary

SVMs combine a linear model, a convex problem with a global minimum, regularisation via the margin and the power of kernels. But they are not “magic”: the choice of kernel and hyperparameters (C , σ , ε) is critical and must be made with rigorous validation; support vectors can be many; they do not solve the curse of dimensionality of the input space. The kernel trick is a general idea (kernel methods) that makes it possible to work with similarities between objects, even structured ones.

Questions — Possible exam questions

- Strengths and weaknesses of SVMs compared to neural networks.
- Why can the VC-dim of an SVM with an RBF kernel be infinite? How is it controlled?
- Why must C and the kernel parameters be selected together?
- Does the SVM solve the curse of dimensionality?
- What are kernel methods? What is the relationship between kernels and similarity?

Chapter 15

Bias-variance and ensembles

SLT explained underfitting and overfitting through model complexity and the number of data. The **bias-variance decomposition** offers another, complementary point of view: it considers the fact that the training set is **only one** of the possible realisations of the data, and decomposes the expected error into three components — **bias**, **variance** and **noise**. From here we naturally arrive at **ensembles** (bagging, boosting), which exploit precisely the reduction of variance.

15.1 The idea

A training set is only one possible realisation from the universe of data: **different training sets produce different estimates**. The expected error (over the various training sets) at a point $\mathbf{x}$ decomposes into:

- **Bias**: quantifies the discrepancy between the true function and $h(\mathbf{x})$ (averaged over the data). It is high if H is too small (model too rigid).
- **Variance**: quantifies the variability of the response of the model h for different realisations of the training data. It is due to excessive flexibility.
- **Noise**: the labels contain a random error (for example, for a given $\mathbf{x}$ there can be several possible values of d).

We assume the **regression** scenario with target y and squared loss.

15.2 Analysis in regression

Suppose examples $\langle \mathbf{x}, y \rangle$ with true function $y = f(\mathbf{x}) + \varepsilon$, where ε is Gaussian noise with zero mean and standard deviation σ . With linear regression, given a set of examples $\langle \mathbf{x}_i, y_i \rangle$, $i = 1..l$, a linear hypothesis $h(\mathbf{x}) = \mathbf{w}\mathbf{x} + w_0$ is fitted by minimising $\sum_{i=1}^l (y_i - h(\mathbf{x}_i))^2$. Two facts:

1. for the chosen hypothesis class (linear), for some f there will be a **systematic prediction error**;
2. depending on the dataset, the parameters $\mathbf{w}$ found **will be different**.

Example — 20 points and 50 fits

True function $y = x + 2 \sin(1.5x) + \mathcal{N}(0; 0.2)$. With 20 points a line is fitted. Repeating with 50 different datasets (each of 20 points), 50 different lines are obtained.

15.2.1 The expected prediction error

Given a new point $\mathbf{x}$, what is the **expected prediction error**? The data are assumed to be drawn **i.i.d.** (independent and identically distributed) from a single distribution P . The goal is

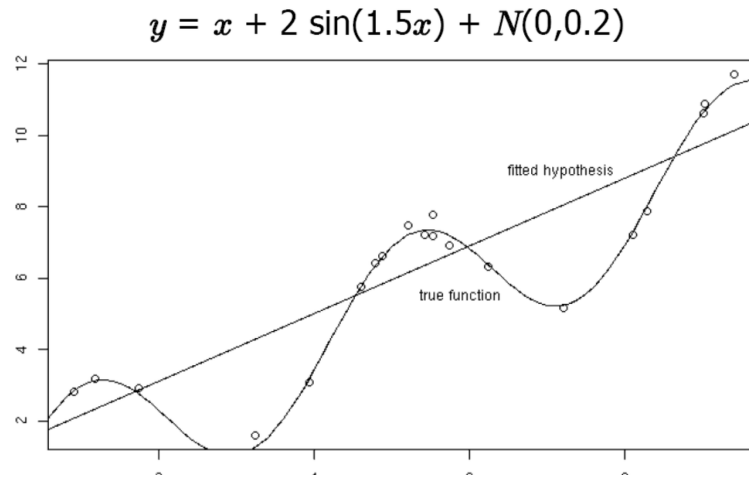


Figure 15.1: A dataset of 20 points, the true function and the fitted linear hypothesis.

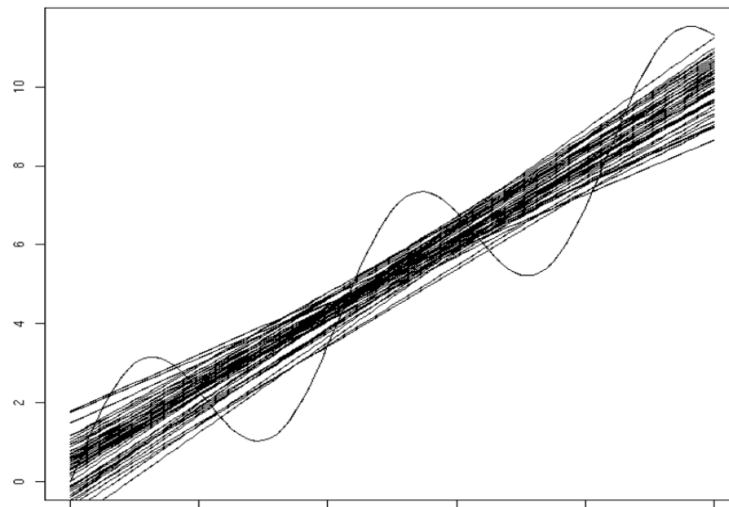


Figure 15.2: 50 fits, each on 20 points: the lines vary from one dataset to another.

to compute, for an arbitrary $\mathbf{x}$,

$$E_P[(y - h(\mathbf{x}))^2],$$

where y is the value of $\mathbf{x}$ that could appear in a dataset, and the expectation is **over all training sets** drawn according to P . Beware: for each “drawn” training set there is a different h (and a different y).

15.2.2 Statistics recap

For a discrete random variable Z : $\bar{Z} = E_P[Z] = \sum_i z_i P(z_i)$ and $\text{Var}[Z] = E[(Z - \bar{Z})^2]$.

Theorem — Variance lemma

$$\text{Var}[Z] = E[Z^2] - \bar{Z}^2 \quad \Longleftrightarrow \quad E[Z^2] = \bar{Z}^2 + \text{Var}[Z].$$

Proof. $\text{Var}[Z] = \sum_i (z_i - \bar{Z})^2 P(z_i) = \sum_i z_i^2 P(z_i) - 2\bar{Z} \sum_i z_i P(z_i) + \bar{Z}^2 \sum_i P(z_i) = E[Z^2] - 2\bar{Z}^2 + \bar{Z}^2 = E[Z^2] - \bar{Z}^2$.

15.2.3 The decomposition

Expanding the square and using the linearity of expectation (and the fact that y and $h(\mathbf{x})$ are independent, because y is a new data point and h depends on the training set):

$$E_P[(y - h(\mathbf{x}))^2] = E_P[h(\mathbf{x})^2 - 2yh(\mathbf{x}) + y^2] = E_P[h(\mathbf{x})^2] + E_P[y^2] - 2E_P[y]E_P[h(\mathbf{x})].$$

Let $\bar{h}(\mathbf{x}) = E_P[h(\mathbf{x})]$ be the **mean prediction** of the hypothesis at $\mathbf{x}$, when h is trained on data drawn from P .

- **First term**, with the lemma: $E_P[h(\mathbf{x})^2] = E_P[(h(\mathbf{x}) - \bar{h}(\mathbf{x}))^2] + \bar{h}(\mathbf{x})^2$.
- Note that $E_P[y] = E_P[f(\mathbf{x}) + \varepsilon] = f(\mathbf{x})$ (the noise has zero mean).
- **Second term**, with the lemma: $E_P[y^2] = E_P[(y - f(\mathbf{x}))^2] + f(\mathbf{x})^2$.

Putting everything together:

$$\begin{aligned} E_P[(y - h(\mathbf{x}))^2] &= E_P[(h(\mathbf{x}) - \bar{h}(\mathbf{x}))^2] + \underbrace{\bar{h}(\mathbf{x})^2 - 2f(\mathbf{x})\bar{h}(\mathbf{x}) + f(\mathbf{x})^2}_{(\bar{h}(\mathbf{x}) - f(\mathbf{x}))^2} + E_P[(y - f(\mathbf{x}))^2] \\ &= \underbrace{E_P[(h(\mathbf{x}) - \bar{h}(\mathbf{x}))^2]}_{\text{variance}} + \underbrace{(\bar{h}(\mathbf{x}) - f(\mathbf{x}))^2}_{\text{bias}^2} + \underbrace{E_P[(y - f(\mathbf{x}))^2]}_{\text{noise}^2} \\ &= \text{Var}[h(\mathbf{x})] + \text{Bias}[h(\mathbf{x})]^2 + E_P[\varepsilon^2] = \text{Var}[h(\mathbf{x})] + \text{Bias}[h(\mathbf{x})]^2 + \sigma^2. \end{aligned}$$

Theorem — Bias-variance decomposition

$$\text{Expected prediction error} = \text{Variance} + \text{Bias}^2 + \text{Noise}^2.$$

15.2.4 Interpretation of the three components

- **Bias** $(\bar{h}(\mathbf{x}) - f(\mathbf{x}))^2$: the discrepancy between the true function and the **mean** prediction over the different training sets. It is a **systematic error**, due for example to a space H that is too small or a model that is too rigid.
- **Variance**: the variability of the response of the model for different realisations of the training set. It is due to **excessive flexibility**.
- **Noise**: even the optimal solution can be wrong (for example if for a given $\mathbf{x}$ several values of d are possible). It is **irreducible**: it does not depend on the model and represents a tolerance on the response (σ).

Going back to the example: the **bias** is evident in the mean $\bar{h}$ of the 50 lines, which remains far from the wavy true function (the linear model is too rigid); the **variance** is the spread of the 50 lines around $\bar{h}$; the **noise** is the spread of the data around the true function.

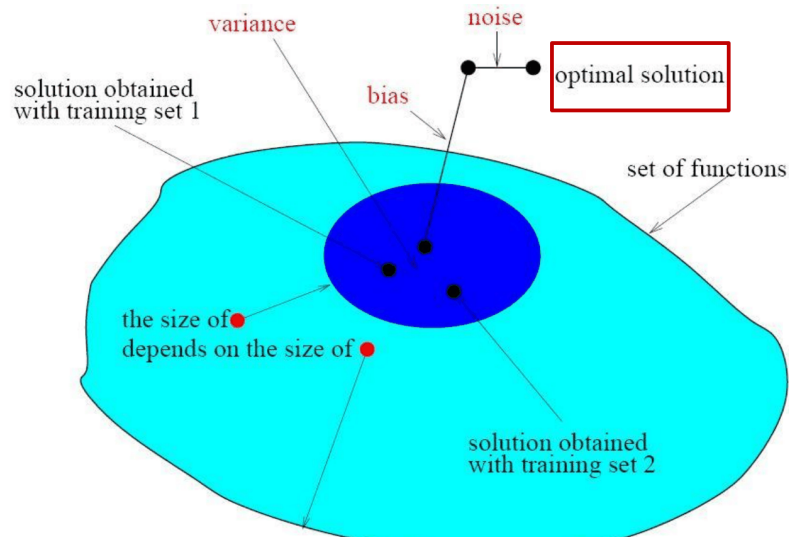


Figure 15.3: Graphical view: different training sets produce different solutions (variance); their mean is far from the optimum (bias); the optimum itself is far from the data because of noise.

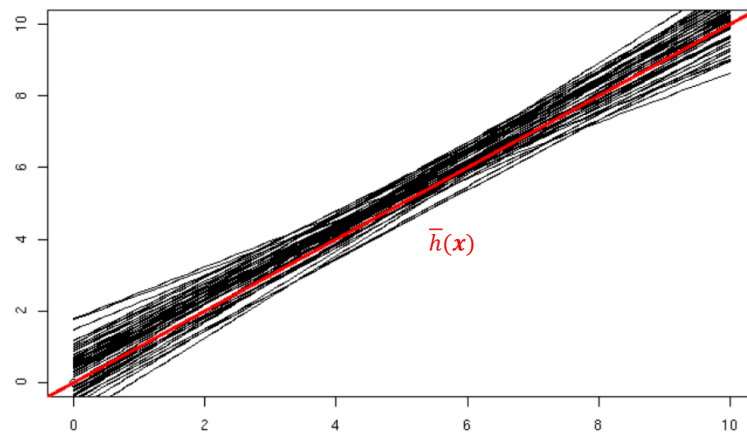


Figure 15.4: The variance: the spread of the hypotheses around the mean prediction $\bar{h}(x)$ (in red).

Example — The darts analogy



The centre is the true function, each dart is a model trained on a different training set. **High bias and low variance** (bottom left) corresponds to **underfitting**; **low bias and high**

variance (top right) to **overfitting**. The analogy is not perfect for ML, but it helps.

15.3 Bias, variance and regularisation

Recall the regularised loss:

$$\text{Loss}(\mathbf{w}) = \underbrace{\sum_p (d_p - o(\mathbf{x}_p))^2}_{\text{data term}} + \underbrace{\lambda \|\mathbf{w}\|^2}_{\text{penalty}}.$$

Varying λ we obtain less regularised solutions (complex, low λ) or more regularised ones (less complex, high λ). (The intercept is typically excluded from the penalty.)

Bishop's example: 25 datasets drawn from the sinusoid, a model (an RBF network) trained on each with different values of λ .

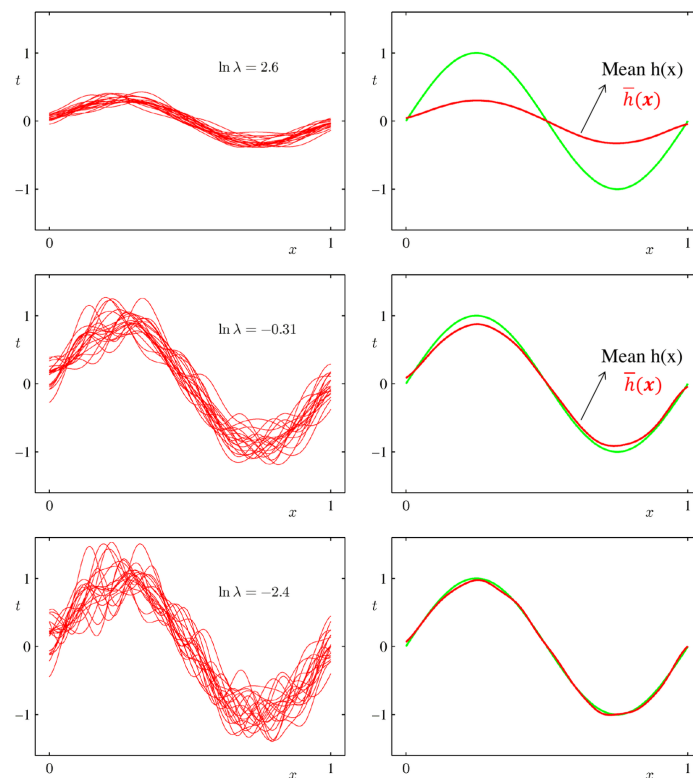


Figure 15.5: Effect of λ on bias and variance: $\ln \lambda = 2.6$ (top), -0.31 (middle), -2.4 (bottom). On the left the 25 hypotheses, on the right their mean $\bar{h}(x)$ (red) and the true function (green).

- **High λ : high bias, low variance.** The penalty dominates, the model depends little on the data and is “too rigid”: the curves are similar but their mean is far from the sinusoid.
- **Medium λ :** lower bias, higher variance. Decreasing λ one depends more on the particular training set.
- **Very low λ : low bias, high variance.** The test error for a single training set can be high (because of the variance component), but even though the curves are very different from each other, **on average** they are correct (plot on the right).

An **over-regularised** model (large λ) has high bias; an **under-regularised** one (small λ) has high variance. Again, **the trade-off**: the minimum of $\text{bias}^2 + \text{variance}$ roughly coincides

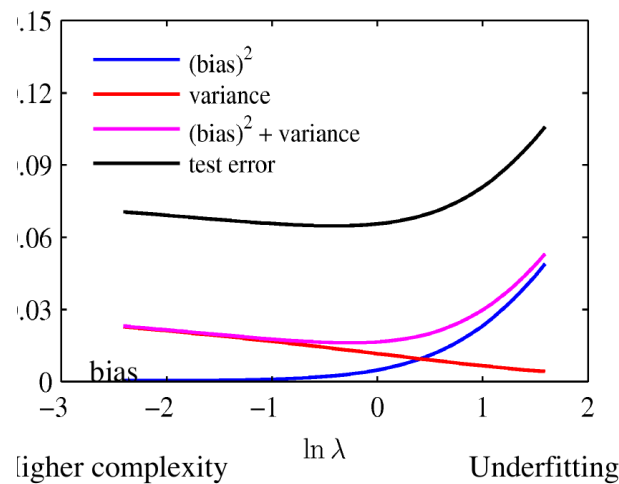


Figure 15.6: The bias-variance trade-off as $\ln \lambda$ varies. Beware: here complexity is higher on the **left** (the opposite of the usual plots).

with the minimum of the test error.

The same figure seen in validation ([09 - Validation \(part 1\) - model selection and assessment](#)) shows, as complexity grows, training and test errors on 100 training sets: a complex model has **low bias and high variance**, hence a risk of overfitting, and depending on the training set one can be lucky or unlucky.

Tip — The link with SLT

Bias and variance are another face of the trade-off between fitting the data and complexity: bias corresponds to the inability of the model to represent the function (high R_{emp} , underfitting), variance to the sensitivity to the specific data (high VC-confidence, overfitting).

15.4 Ensemble learning

The idea of **ensembles** is to exploit **several models** by combining their responses.

In the simplest case (**voting scheme**):

- **Regression:** simple averaging committee, $o(\mathbf{x}) = \frac{1}{K} \sum_{i=1}^K h_i(\mathbf{x})$.
- **Classification:** vote over many classifiers (as long as they give **diversified** answers), or classify after averaging the continuous outputs.

Alternatively (**stacking**) the combiner is itself an ML model.

Theorem — The committee is not worse than the average of its members

For a convex loss such as the squared error, the error of the committee is less than or equal to the average of the errors of the individual models:

$$\text{loss}(E_i[h_i]) \leq E_i[\text{loss}(h_i)].$$

(It is Jensen's inequality.) Example with target $t = 5$ and two models predicting 2 and 4:

the average of the errors is $\frac{(5-2)^2 + (5-4)^2}{2} = 5$, while the error of the committee ($\sigma = 3$) is $(5 - 3)^2 = 4$. With predictions 6 and 4: average of the errors $\frac{1+1}{2} = 1$, committee error $(5 - 5)^2 = 0$.

15.4.1 Bagging (bootstrap aggregating)

- K classifiers are trained on **different subsets** of the training set, differentiated with the **bootstrap** (resampling with replacement).
- Regression: the average is used; classification: vote of the K classifiers.

The link with bias-variance is direct (recall the “low λ ” case): **high-variance** and **low-bias** models can work well **on average**. In other words, **averaging the h reduces variance** without increasing bias. Moreover bagging tends to increase the **margin** of a classifier: the average of K separation curves tends to lie “in the middle” (*in medio stat virtus*).

15.4.2 Boosting (e.g. AdaBoost)

If the models make the **same errors**, combining them is useless. Boosting differentiates them on purpose:

1. it trains the classifiers in sequence focusing on the **errors**: the second gives more weight to the instances misclassified by the first (for example they are more likely in its training set), and so on;
2. it combines the results with a **weighted vote**, giving more weight to the classifiers with lower error.

If not stopped, boosting learns to correctly classify **all** the training instances using **weak learners** (classifiers just better than chance, with error $< 1/2$ in the binary case), incrementally building complex models. It can greatly improve the performance of weak learners; it reduces bias without increasing variance (it resists overfitting) and, like bagging, it tends to produce separation curves “in the middle” (maximum margin). However, it **suffers with noisy data** (which are weighted more and more), and it does not by itself solve the underfitting/overfitting problem.

15.5 Feature selection (outline)

Finding the subset of the most informative features for the problem brings great benefits: **dimensionality reduction** (effects on the VC-dim of networks, on the curse of dimensionality...), filtering of irrelevant information and noise (with possibly strong performance improvements), **interpretability** (finding useful descriptors). But:

- it is **computationally hard** (huge number of possible subsets, and each evaluation requires retraining);
- a **heuristic search** is typically used (greedy, genetic algorithms or other optimisation techniques), including the learner or not;
- the results are particularly sensitive to the validation method: beware of the **feature subset selection bias**, i.e. the error of making the selection on the whole dataset (see the slides on cross-validation).

Other related topics: PAC learning; comparison between inductive principles (SRM, MDL, Bayesian inference, regularisation) with respect to complexity control.

Summary — Summary

The expected error (over training sets) decomposes into **variance** + **bias**² + **noise**. Rigid (or heavily regularised) models have high bias and low variance (underfitting); flexible (lightly regularised) models have low bias and high variance (overfitting); noise is irreducible. **Ensembles** exploit the decomposition: **bagging** averages high-variance models, reducing it; **boosting** combines weak learners by focusing on errors.

Questions — Possible exam questions

- Derive the bias-variance decomposition for the squared loss.
- What do bias, variance and noise represent? Which component is irreducible?
- How do bias and variance vary with the regularisation parameter λ ?
- Why is a committee not worse than the average of its members?
- Difference between bagging and boosting; why does bagging reduce variance?

Chapter 16

Convolutional neural networks (CNN)

Neural networks have a huge number of successful applications, thanks to their performance and to the fact that they are easily set up as a first approach for learning arbitrary functions of unknown form, without assuming an a priori statistical model. **Sub-symbolic** tasks (sensory data, motor control, visual processing) are their workhorse.

This lecture presents a historical and paradigmatic example: **convolutional networks** (CNN, *ConvNet*, *LeNet*). It shows the **flexibility in the design** of networks, how the design is changed to deal with a specific application, and how a **not fully connected** feedforward network is built for a precise problem. CNNs are also a historical instance of a **deep** network, and they act as a bridge towards deep learning.

16.1 The problem: recognising characters

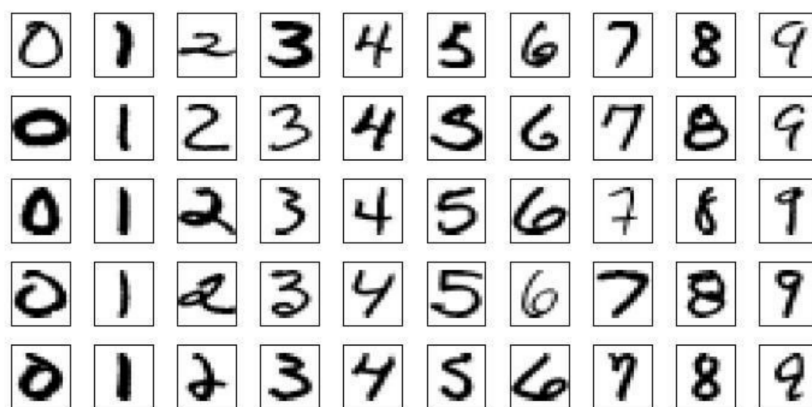


Figure 16.1: Examples from the ZIP code dataset: normalised 16×16 8-bit grey-scale images (Le Cun et al., 1990). The MNIST benchmark is “the drosophila of ML”.

A first approach is a standard network with **256 inputs** (one per pixel, 16×16): an error of 5–20% is obtained, with many errors due to the **lack of invariance** to rotations, translations, etc. A fully connected network treats each pixel as an independent input: it does not “know” that nearby pixels are correlated nor that the same stroke can appear in different positions.

Tip — The basic idea

Exploit the **architecture** to include **prior knowledge** in the network: - **local connections** (local receptive fields): each unit sees only a small area of the image, and extracts **local features**; - **weight sharing** among different units: the **same operation** is applied to different parts of the input, because the features of a character can appear anywhere in the image (regardless of position, small rotations...). The number of free parameters is reduced while keeping a complex connectivity.

16.2 Convolution

The name comes from the **convolution operator**: a weighted average of a function f , weighted by another function g that slides over time:

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau) g(t - \tau) d\tau.$$

The support of g is bounded (non-zero only on an interval): g acts as a **sliding window**.

16.2.1 1D convolution: a unit on a stream

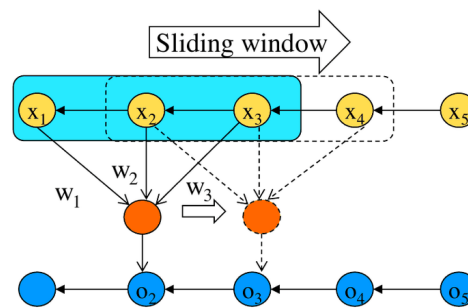
A unit with three weights sliding over an input sequence:

$$out_t = \sum_{i=1}^3 w_i x_{t+i-2}.$$

$$out_t = \sum_{i=1}^3 w_i x_{t+i-2}$$

Exercise: compute o_2 and o_3
according to the equation above
(start with $t=2$)

Weights are tuned as usual by learning:
An adaptive filter



Weight sharing
for the two instances of units
More units operate as a “filter bank”

Figure 16.2: 1D convolution: the same unit (with the same weights) slides over the input.

(Exercise: with $t = 2$, $o_2 = w_1x_1 + w_2x_2 + w_3x_3$; with $t = 3$, $o_3 = w_1x_2 + w_2x_3 + w_3x_4$.) The weights are **shared** among the instances of the unit and are learned as usual: the unit is an **adaptive filter**. Several different units form a **filter bank**. This network is also called a *Time-Delay Neural Network*.

16.2.2 2D convolution

On an image a 2D **kernel** is used, for example 3×3 (the **local receptive field** of the unit), which moves one pixel at a time (**stride 1**). **Padding** handles the borders. The kernel slides over the image and produces a **feature map** (the features extracted by the filter).

In neural network terms: the image is scanned with **the same neuron**, which uses local connections.

Note — Form used in libraries

The 2D convolution of an image I with a kernel K is $S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(m, n) K(i - m, j - n)$. Many CNN libraries actually implement **cross-correlation**: $S(i, j) = \sum_m \sum_n I(i + m, j + n) K(m, n)$ (for details: *Deep Learning book*, sec. 9.1; not required for the oral exam).

- **One kernel** (and local receptive field), 1 pixel stride (stride 1 for downsizing)
- **Padding** is used to treat the border

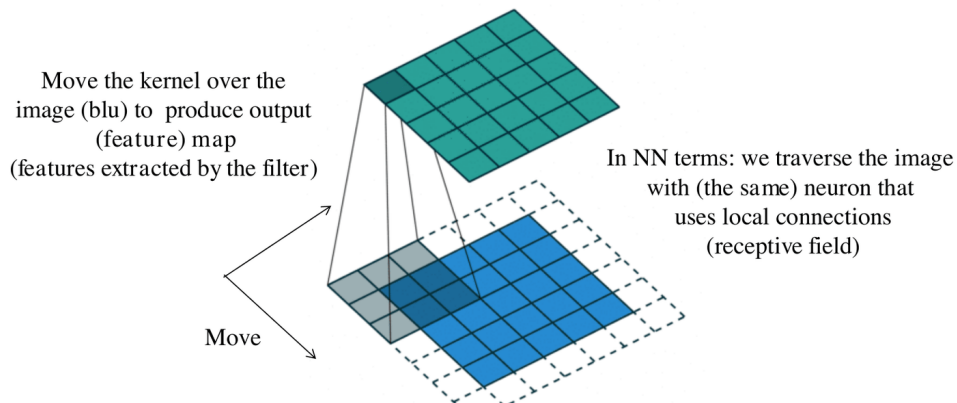


Figure 16.3: 2D convolution: the kernel slides over the image and produces the feature map.

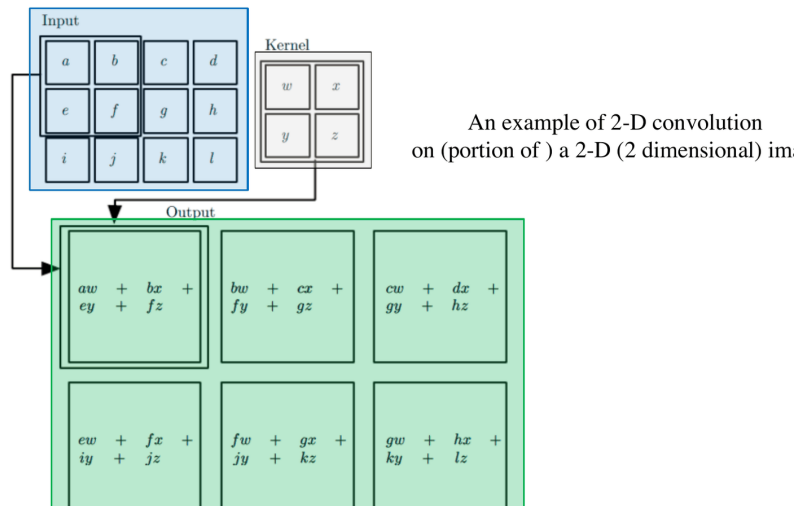


Figure 16.4: Example of 2D convolution of a 3×4 input with a 2×2 kernel.

16.2.3 Stride greater than 1

With **stride** > 1 the kernel “skips” pixels (for example two at a time): it is a **sub-sampling**, and it produces a reduced feature map.

stride

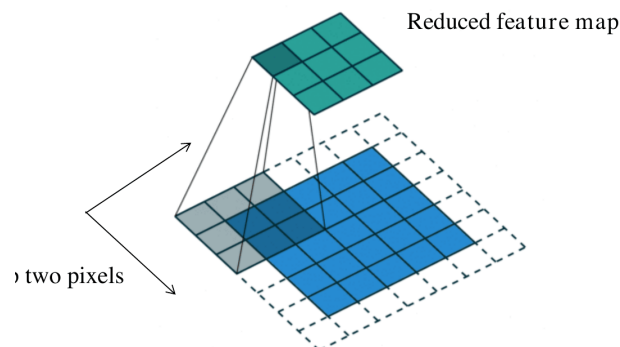


Figure 16.5: Convolution with stride 2: the resulting feature map is reduced.

16.3 A network of “filters”

The weights of the units are a **trained filter** that detects certain features or patterns in the image. The architecture design provides:

- **small, local receptive field:** thanks to this constraint, the learned filters respond maximally to **spatially local** patterns (as in the visual cortex of animals);
- **the same filter applied over the whole image:** features are detected **regardless of position**, obtaining **translation invariance**;
- **learned filters:** the filters that in traditional image processing algorithms were designed by hand are **learned**. Independence from prior knowledge and from human effort in designing features is a great advantage;
- **many stacked layers:** the units of the feature maps represent increasingly larger areas of the original image (by assembling areas of the previous maps).

16.4 Pooling

Pooling reduces the feature map:

- with sub-sampling (stride > 1, seen above);
- with an **average** (simple or weighted);
- with **max pooling** (the most common).

In practice, instead of producing a value for each input pixel, one is produced for a (rectangular) set of pixels, taking the mean or the maximum of the outputs.

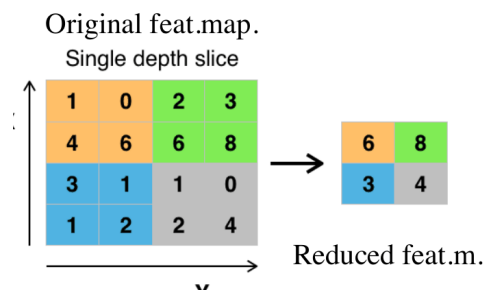


Figure 16.6: Max pooling with a 2×2 filter and stride 2.

Pooling also helps make the representation **approximately invariant to small translations** of the input: the maximum (or the mean) changes little if the input shifts slightly, so a high output stays high for a slightly translated version.

16.5 The CNN as a whole

A CNN exploits weight sharing to implement a sliding window of local fields over a signal, extended to 2D images and repeated over many layers (feature maps), alternated with pooling operations. The four ingredients:

1. **local connections;**
2. **shared weights;**
3. **pooling;**
4. **many layers.**

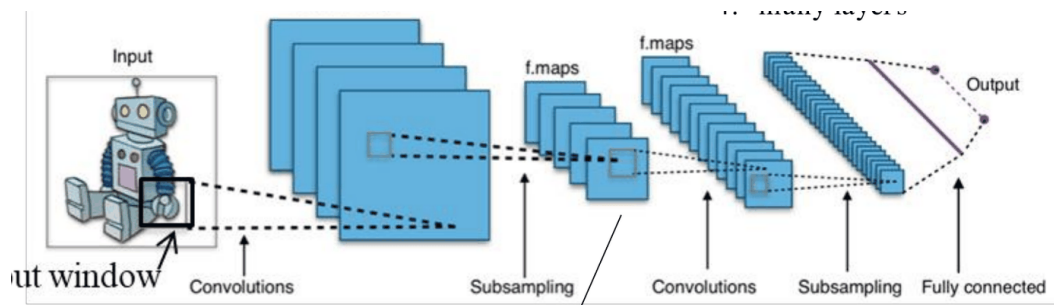


Figure 16.7: A complete CNN: alternating convolutions and sub-samplings, followed by fully connected layers. The number of filters (feature maps) can grow in the higher layers.

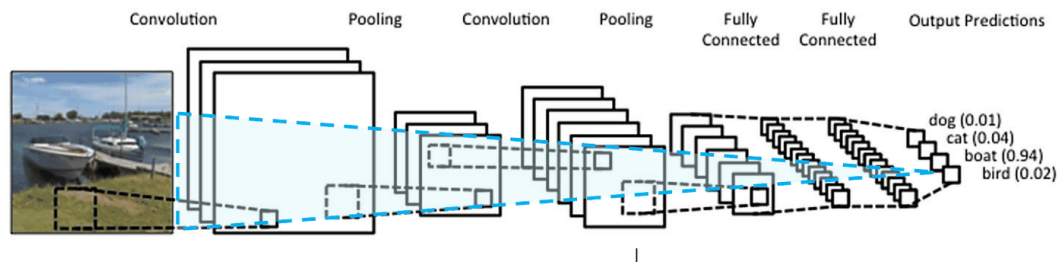


Figure 16.8: The “cone” from the image to the final maps: the receptive field of the final units is much wider than that of the units of the first layers, because they are indirectly connected to a large part of the image.

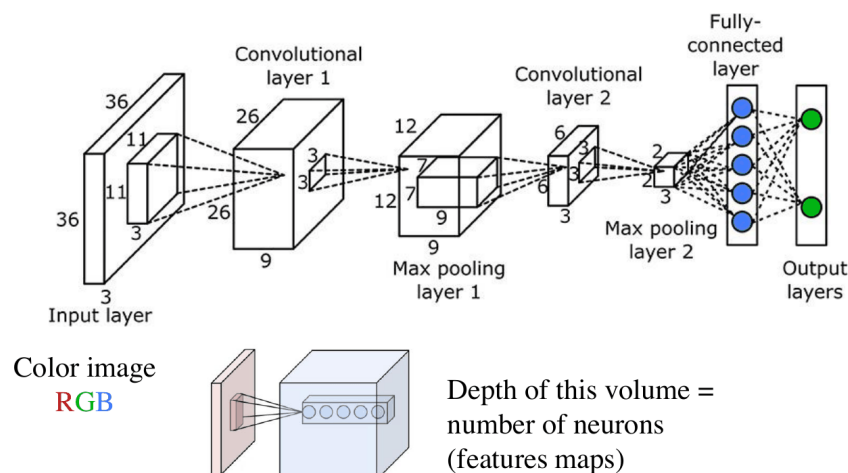


Figure 16.9: An example with the dimensions; for a colour (RGB) image the depth of the volume is the number of feature maps.

Summary — Advantages of CNNs

- **Local connections and shared weights:** they detect local motifs/patterns in images, in a **translation-invariant** way (with respect to the position of the pattern); they **reduce the number of free parameters** while keeping the connections, which is a form of **regularisation**.
- **Pooling:** reduces the size of the representation at each layer (a **pyramid** of layers) and helps invariance to small shifts and distortions.
- **Many layers:** operations 1 and 2 applied to all the feature maps produce a **progressive abstraction** of the features, making it possible to **compose primitives** (low-level features, learned only once) in all possible combinations in the subsequent layers.
- They are a historical instance of a **deep network**, and analogies with the visual system

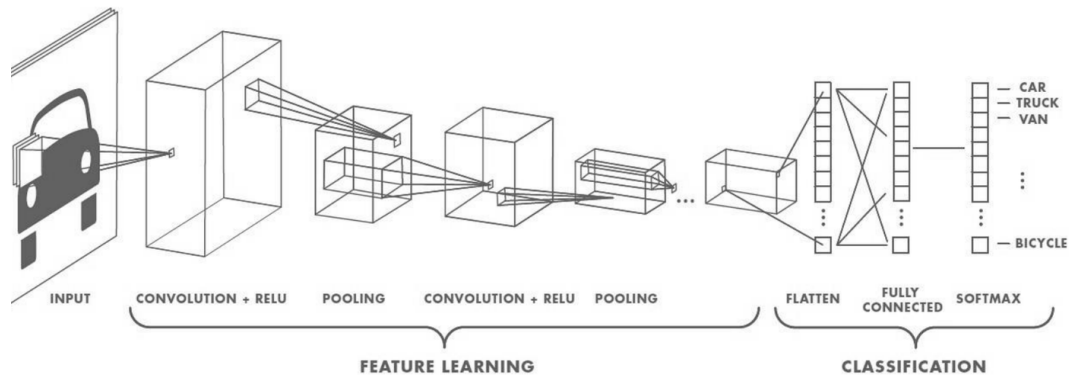


Figure 16.10: An AlexNet-like architecture.

have been studied (neuroscience).

16.6 How they are used

Training is typically done with **backpropagation**, with the well-known heuristics and some specialisations (for weight sharing, pooling...). Given the typical use of huge networks and large amounts of data, many hyperparameters are fixed **by experience** or on the advice of experts, because it would be too expensive to cross-validate over large hyperparameter spaces. Training, regularisation effects and the good behaviour of these huge networks will be discussed in the framework of deep learning.

16.7 Historical results

16.7.1 LeCun's architectures on the ZIP code dataset

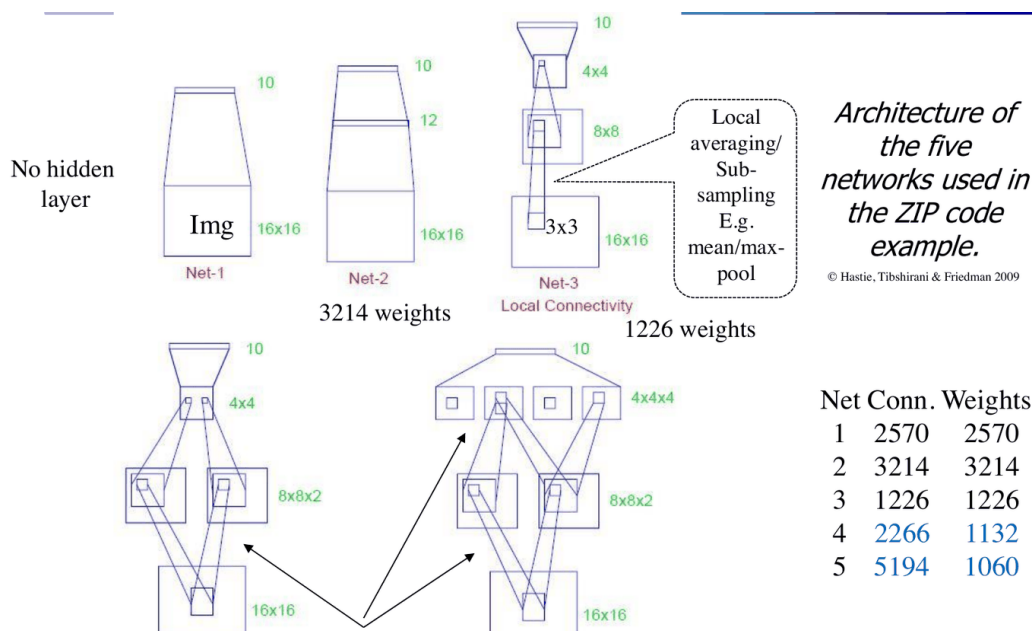


Figure 16.11: The five networks used by LeCun on the ZIP code problem: from a network without hidden layers (Net-1) to networks with local connections (Net-3) and shared weights (Net-4, Net-5).

Network	Connections	Weights
Net-1	2570	2570
Net-2	3214	3214
Net-3	1226	1226
Net-4	2266	1132
Net-5	5194	1060

With shared weights the number of **connections** can grow while the number of **weights** (free parameters) stays low.

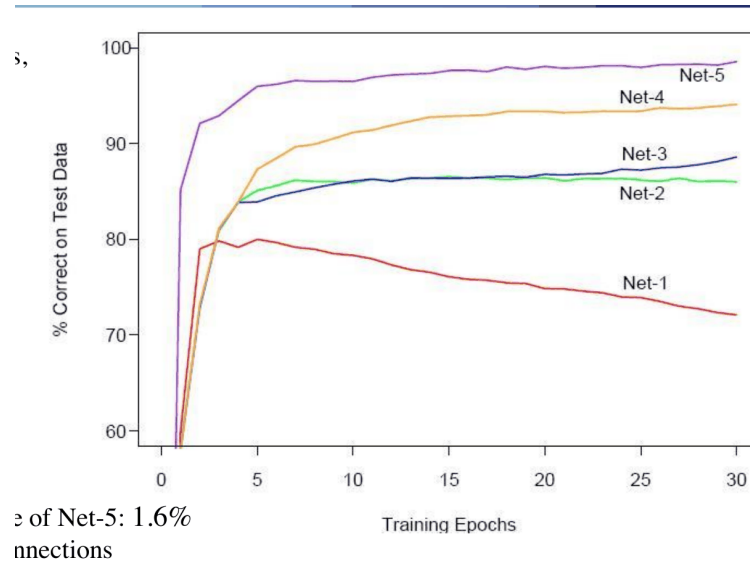


Figure 16.12: Test performance curves for the five networks (Le Cun, 1989). The training error is zero for all of them, on a reduced set of about 300 images.

Net-5 reaches an error of **1.6%** with 5194 connections and only 1060 weights. Today 0.8% (with networks and SVMs) and even less is achieved.

16.7.2 MNIST

The complete dataset has 60,000 training images and 10,000 test images. Progress (1998–2012) is documented on LeCun’s website. With deep learning (MLPs or CNNs with many layers, unsupervised layer-wise pre-training) 0.39% (2006) and 0.35% (2011) were reached; the best in 2012 is 0.23% (committee of convolutional networks), 0.21% in 2016.

Warning — Warning

The MNIST test set has been used so many times that improvements of the order of 0.1% are no longer significant: it has de facto become a validation set for the community.

Note — Does a deep MLP work too?

A 2010 paper (*Neural Computation*) shows that “good old” on-line backpropagation on a simple MLP achieves 0.35% on MNIST: all that is needed is **many layers**, **many neurons** per layer, **many deformed images** for training and **GPUs** to speed things up. The 784-2500-2000-1500-1000-500-10 network has 12 million weights and trains in 2 hours. Complexity control is obtained by deforming the images to have many more examples (*data*

augmentation).

16.7.3 Face recognition and ImageNet

- **DeepFace** (CVPR 2014): 9-layer convolutional networks trained on 4 million images of more than 4000 people; it recognises whether two photos show the same person with 97.25% accuracy (humans: 97.53%).
- **AlexNet** (Krizhevsky, Sutskever, Hinton, NIPS 2012): a deep CNN trained on 1.3 million high-resolution images from ImageNet (ILSVRC-2010) in 1000 classes. Top-1 and top-5 errors of 39.7% and 18.9% (from the previous $\sim 26\%$), much better than the state of the art. The network has **60 million parameters** and 500,000 neurons: five convolutional layers (some followed by max pooling) and two fully connected layers with a final 1000-class softmax. To speed up training: **non-saturating** neurons (ReLU) and a very efficient GPU implementation. To reduce overfitting in the dense layers: a new regularisation method, **dropout**.

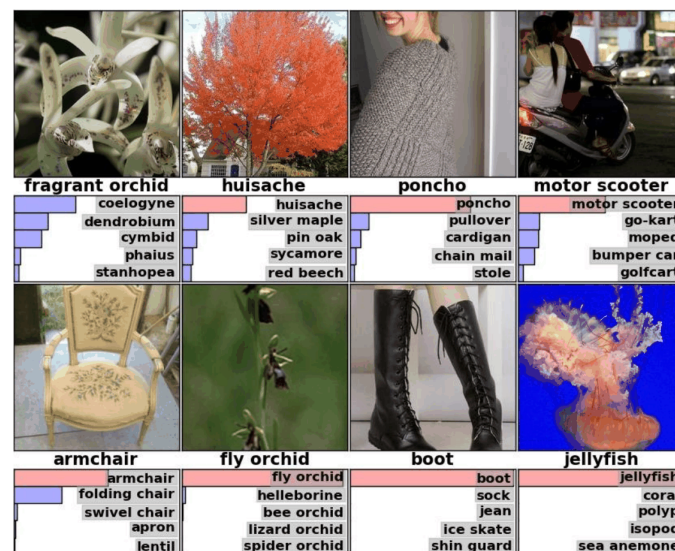


Figure 16.13: AlexNet top-5 predictions on random test set images.

16.8 Modern CNNs and GPUs

There are many variants and specialised architectures for images (course *Intelligent Systems for Pattern Recognition*), with a great impact on Pattern Recognition and Computer Vision; the neuroscientific foundations are covered in the *Computational Neuroscience* course. Features of models **already trained** on large benchmarks (AlexNet, GoogLeNet, VGG, ResNet...) can also be reused: many pre-trained CNNs are available for classification, segmentation, face recognition, text detection.

Efficient implementations exploit **GPUs** through the **tensor** representation. Matrix multiplication (i.e. dot products) is a linear operation that GPUs perform very efficiently, parallelising the independent sums and products. Many kernel operations can be written as a single multiplication between block matrices; the **tensor** notation (multidimensional arrays with the *tensordot* operation, for example in TensorFlow or NumPy) makes it possible to do so without building complex block matrices by hand.

Note — Beyond CNNs: Capsule Networks

Capsule Networks (Sabour, Frosst, Hinton, 2017) use specialised sub-networks: a “capsule” is a group of neurons that outputs both a parameter indicating whether an entity is present and a vector of **pose parameters** (position, orientation) relative to a canonical version. Instead of max pooling they use a **routing** mechanism (the idea that the brain routes low-level visual information to the most suitable capsule). They achieve excellent results on MNIST, recognising overlapping digits much better than a CNN.

Questions — Possible exam questions

- What are the key ideas of CNNs (local connections, shared weights, pooling, many layers) and what does each imply?
- What is convolution in a neural network? What are kernel, stride, padding, feature map?
- What is pooling for? How does max pooling work?
- Why is weight sharing a form of regularisation?
- How does the receptive field grow with depth?

Chapter 17

Deep learning

This lecture introduces **deep learning** (DL) from the point of view of the course: not the details of the architectures for applications (which will be seen in the subsequent courses of the AI curriculum, such as Pattern Recognition and NLP), but an **overview** of networks with many layers, the concepts that make them effective (**are many layers really needed?**, **representation learning**, **distributed representations**) and the main training **techniques**, many of which are also useful in the “basic” case.

17.1 What is deep learning

Note — LeCun, Bengio, Hinton, *Nature* (2015)

“Deep learning allows computational models that are composed of multiple processing layers to learn representations of data with multiple levels of abstraction. These methods have dramatically improved the state-of-the-art in speech recognition, visual object recognition, object detection and many other domains.”

17.1.1 A bit of history

Since about 2010 deep learning has had a strong impact on industry, due to three factors:

- **large datasets** available (millions of images);
- **computing power** (GPUs are the enabling factor);
- **new ideas** to make training easier.

Learning with deep architectures is no longer “infeasible”. The new state-of-the-art results in speech recognition, machine translation (Google Neural Translator since 2016, DeepL) and image recognition have attracted a great many researchers. In the past an improvement was expected by changing the scale of models and data, but not necessarily such a leap in performance. All the big IT companies today base their research and development strategies on these approaches.

17.1.2 The general framework

DL is a **general framework** that includes different models:

- **deep neural networks** (MLPs with many layers);
- **convolutional networks** (winners in computer vision);
- *deep belief networks* and generative approaches;
- **recurrent** and **recursive** networks;
- ...

It is contrasted with “**shallow**” **models**, for example networks with fewer than 3 layers. The aspects common to the various approaches are:

- **several layers** of **non-linear** processing units;
- learning (supervised or unsupervised) of **feature representations** in each layer, with the layers forming a **hierarchy** from low-level to high-level features (different levels of abstraction);
- hence, a **hierarchical, sparse and distributed representation** of the data.

(The differences depend on the specific models: max pooling in CNNs, energy-based training in RBMs, etc.)

17.1.3 The conceptual ingredients: hierarchical abstraction

Recall XOR solved with two layers (06 - Neural networks (part 1) - from the neuron to the MLP): the hidden layer creates an **internal re-representation** that makes the task easy for the output layer, and this composition of intermediate operations can be extended over many levels of abstraction.

- Unsupervised or semi-supervised **feature learning** and hierarchical feature extraction, **instead of feature engineering** (features extracted by hand, from experience, such as texture in images): we move to the concept of **learning representations of the data**.
- **Increasing levels of abstraction** through the layers. An image can be represented as a vector of pixel intensities, or more abstractly as a set of edges, regions of particular shape, etc.: **edges form motifs, motifs assemble into parts, parts form objects...** over many levels.
- It works best when the input has some form of **structure** (spatial, temporal, ...).

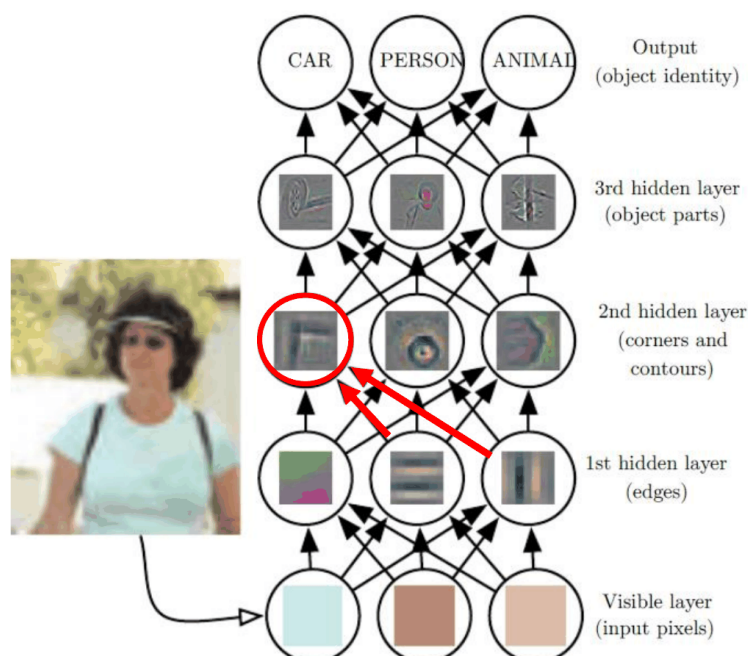


Figure 17.1: From pixels to the identity of the object: a series of hidden layers extracts increasingly abstract features (edges → corners and contours → object parts).

Deep learning solves the difficulty of a complicated mapping (pixels → identity) by **breaking it into a series of simple nested mappings**, each described by a different layer. The new

representation **simplifies classification** in the last layer. There are analogies with the biological visual system.

In networks these abstract features **are learned from examples**: with backpropagation, the error on the output layer (for example to distinguish cars and people) propagates through the deltas across the layers, and the units specialise to provide high-level features useful for that purpose (the wheel for the car, the hand for the person...). The link between the automatically developed features and reality is a fascinating aspect; and surprisingly the first filters extract horizontal or vertical lines, just as a simple linear convolutional filter would.

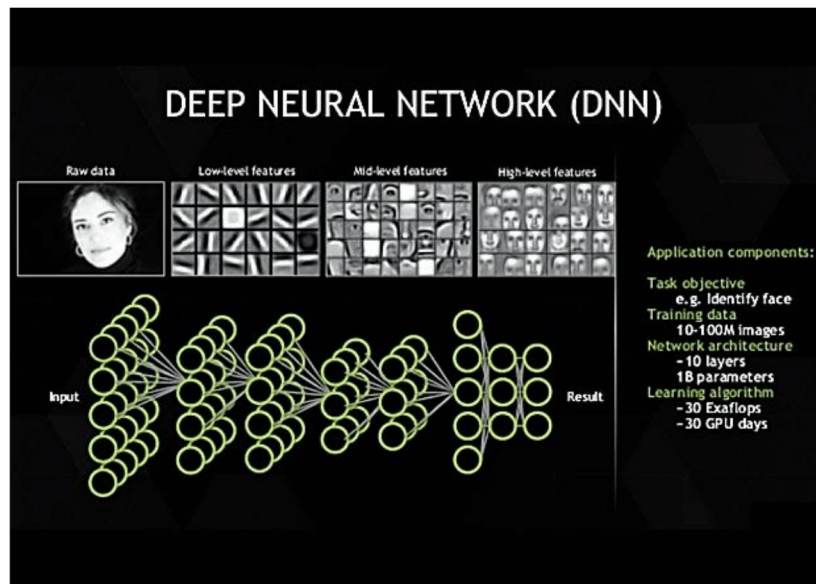


Figure 17.2: Another example: low-, mid- and high-level features in a deep network for faces.

17.1.4 Combining to generalise

Abstract features can be **reused** at higher levels (e.g. gender, glasses...) and **combined** to generalise to cases never seen in training: if the network has seen women without glasses and men with glasses, it can easily represent a woman with glasses as well.

Operations on vector representations learned by the model and then used to generate new images.

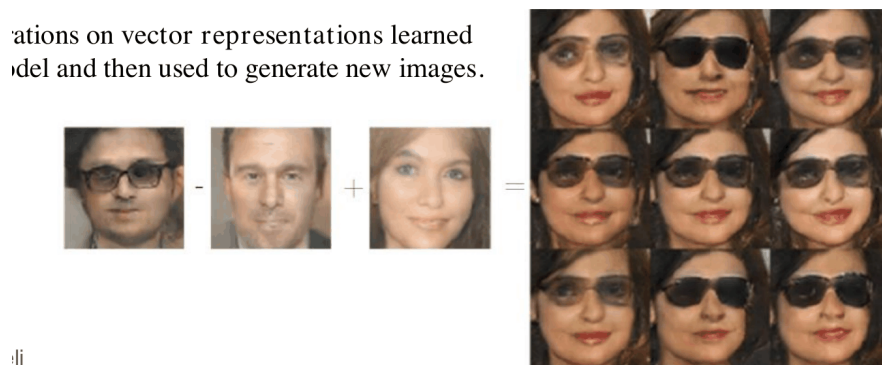


Figure 17.3: Operations on the vector representations learned by the model and used to generate new images: “man with glasses – man + woman = woman with glasses”.

The same happens in language: in the vector space developed by the inner layers of a deep model,

- $\text{Rep}(\text{king}) - \text{Rep}(\text{male}) + \text{Rep}(\text{female}) \approx \text{Rep}(\text{queen})$: the difference between “king” and “male” captures the concept of monarchy;

- $\text{Rep}(\text{Paris}) - \text{Rep}(\text{France}) + \text{Rep}(\text{Poland}) \approx \text{Rep}(\text{Warsaw})$: the difference captures the concept of capital.

17.2 Are many layers really needed?

17.2.1 An example from logic circuits

The general idea (**no-flattening**): *when a function is represented compactly by a deep architecture, it might require a very large architecture to be represented by an insufficiently deep one.*

A **two-layer** logic circuit can represent **any** Boolean function, written as a sum of products (disjunctive normal form): AND gates in the first layer (with possible negation of the inputs) and an OR gate in the second. But **with depth-two circuits, most Boolean functions require an exponential number of gates** (with respect to the input size).

Example — The parity function

Odd parity extends XOR to N inputs: it returns 1 if and only if the number of 1s among the N bits is odd. (NOTs are also assumed to be available; with 2 inputs 3 gates are enough.)

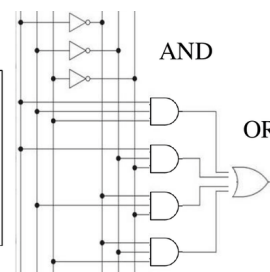
With 2 layers all the positive configurations must be listed: for 3 inputs, $x_1x_2x_3 + x_1\bar{x}_2\bar{x}_3 + \bar{x}_1x_2\bar{x}_3 + \bar{x}_1\bar{x}_2x_3$ (1 if the input is 111, 100, 010 or 001), i.e. 4 AND + 1 OR = 5 gates. For 4 inputs 9 gates are needed, for 8 inputs **129 gates**, for N inputs $2^N/2 + 1 = 2^{N-1} + 1$ gates: an **exponential** number.

With $\log N$ layers a **complete binary tree** of XORs is used: a tree with N leaves has $N - 1$ internal nodes, each an XOR of 3 AND/OR gates, for a total of $3(N - 1)$ gates, a **polynomial** number. For 8 inputs $3 \times 7 = 21$ gates are enough against the 129 of the 2-layer solution.

- For 3 inputs: 4 AND + 1 OR: **5 gates**

$$x_1x_2x_3 + x_1\bar{x}_2\bar{x}_3 + \bar{x}_1x_2\bar{x}_3 + \bar{x}_1\bar{x}_2x_3$$

1 iff inputs are 111 or 100 or 010 or 001
 $\Rightarrow$ All the combinations of the positive patterns



- For 4 inputs: 8 AND + 1 OR: **9 gates**

Figure 17.4: Parity with 2 layers: an exponential number of gates.

This, however, **does not hold for all classes of functions**: characterising the classes that gain an (exponential) advantage from a multi-layer structure is an open research problem.

17.2.2 The case of neural networks

Recall that the **universal approximation theorem** shows that one hidden layer is enough in general, but it does not guarantee that a **small** number of units is enough. There are, however, **no-flattening** results (on efficiency, not on expressiveness):

1. **shallow networks**: there are cases in which the implementation with a single hidden layer requires an **exponential** number of units (with respect to the input dimension n) or of non-zero weights. It is easy to see in the binary case: Boolean functions on $\{0, 1\}^n$ are

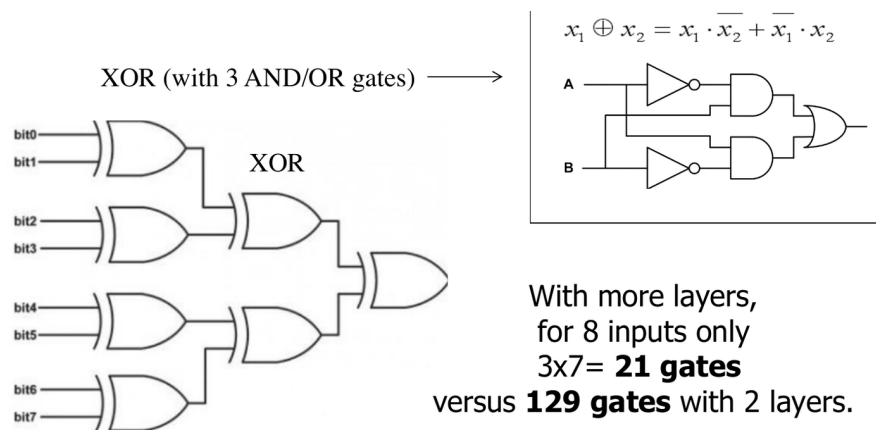


Figure 17.5: Parity with $\log N$ layers: a tree of XORs with a polynomial number of gates.

2^{2^n} , and in the worst case one hidden unit may be needed for each input configuration to be distinguished (as in the parity example). It is not only a problem of network size: it also means that it becomes **hard to learn with few examples** (and this is the most relevant point for ML);

2. **no-flattening of deep networks:** there are families of functions that can be approximated efficiently with depth greater than d , but that require a much larger model (up to an exponential gain) if the depth is at most d . Examples for logic gates (1986), networks with standard activations (1990–1994), ReLU (2014)...

The question remains: is it easy to train an MLP with many layers? (See the techniques later.)

Note — Theoretical analysis (open research)

No-flattening theorems identify compositional functions that a deep network implements well and that cannot be implemented with the same efficiency by “flattening” the network (for example going from a linear to an exponential number of units). But there is no guarantee that a given task has this property: a complete list of no-flattening theorems would say exactly when deep networks are more efficient than shallow ones.

17.2.3 Why: compositionality

Deep networks can exploit the **compositionality of internal representations**: there is an exponential gain in representational power because the simple concepts represented in one layer are used as **primitives** by the next layer to represent more complex concepts, avoiding the explicit and combinatorial representation (and learning) of the features.

For example, one must learn the individual words or sub-patterns, but not directly all their possible combinations: one can learn without seeing all the configurations. And perhaps nature itself (not only images and language) has a hierarchical structure (phenomena observed at different scales in space and time).

Tip — Less is more!

Deep networks should not be seen as **huge** models compared to small shallow models (just because in applications one sees gigantic instances: deep does not mean large). They can be seen as **compact** models compared to potentially much larger shallow models for the same task (even exponentially). So they are “something less” and more efficient. And not

only for computational reasons: **fewer units and fewer weights help learning** on complex tasks, making it possible to generalise well with **fewer examples** (i.e. making it feasible). A “small deep network” of 50,000 units can replace a “huge shallow network” of one million units.

17.2.4 The inductive bias

Choosing a deep model encodes a **very general belief**: that the function to be learned is a **composition of simpler functions** (or of nested factors of variation). If the task matches this bias, the deep form is suitable, and generalisation turns out better precisely thanks to the many layers.

Typically hierarchical tasks: the structure of **images** (composition of graphical parts), the structure of **language** (text and speech), perhaps music; new fields are being discovered. But not necessarily all data and tasks! It is in any case a much more **general** bias than ad hoc constraints for specific tasks (such as hand-chosen ϕ in LBE or engineered features). But, again, nothing is magic without assumptions.

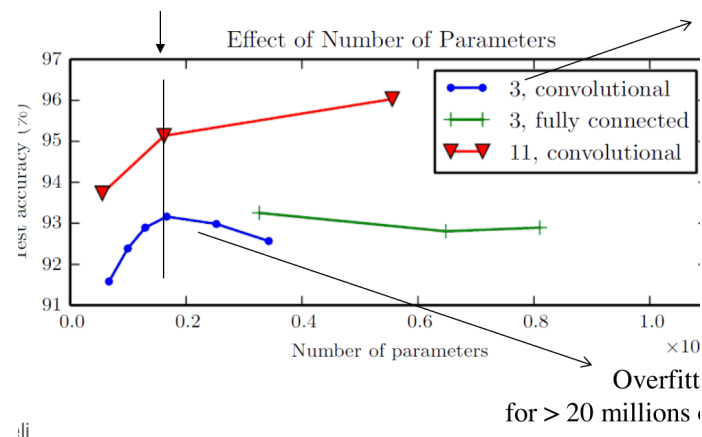


Figure 17.6: Empirical results (SVHN, ICLR 2014): with the same number of parameters, deeper networks generalise better.

When the task matches the bias, deeper models tend to work better: generalisation is better than a model with fewer layers and **the same number of units/weights**.

17.2.5 Curse of dimensionality and bias

Other arguments in favour of deep models concern the **curse of dimensionality** and *manifold learning* (the distribution of real images and texts seems concentrated on a manifold that occupies only a small fraction of the volume of all possible random images or texts).

- Many learners rely on **local approximations** (a local *smoothness* prior): K-NN, local kernels, decision trees... But often this is not enough: examples are needed to generalise in the neighbourhood of each region, and in high dimension a huge number of examples is needed to cover the whole space. To distinguish $O(k)$ regions, all these methods require $O(k)$ examples.
- To work with $O(k)$ examples on an **exponential** (in k) number of regions, i.e. to generalise **non-locally**, additional assumptions on the data-generating distribution (an inductive bias) are needed.

- One could make strong, specific assumptions for each task (losing generality). Deep learning instead chooses a **general** bias, related to the **composition of functions**: the data are assumed to be generated by the composition of factors or features, potentially over several levels of a hierarchy. When the assumption is true, one obtains a potentially exponential gain between the number of examples and the number of distinguishable regions, one generalises non-locally, and one learns with fewer examples.

17.2.6 Practical issues

How many layers? How many units? There is still no general answer (it is a model selection problem). In general deep networks often use fewer units per layer, hence fewer parameters and less data to generalise well; but many layers can be **harder to optimise**. The hyperparameters of these very large models may be too many: **experience** makes the difference in fixing many architectural aspects.

17.3 Representation learning

Definition — Representation learning

“Representation learning is a set of methods that allows a machine to be fed with raw data and to automatically discover the representations needed for detection or classification.” (LeCun, Bengio, Hinton, 2015)

Deep learning methods are representation learning methods with **multiple levels** of representation (representations expressed in terms of other, simpler ones). But the concept is more abstract and applies to many models (for example neural networks in general). Note that we speak of learning the representation mainly for **raw data** (images, text strings), not when a small set of features is already provided.

17.3.1 Basic ideas

- Many information processing tasks are easy or very hard **depending on how the information is represented** (this holds across computer science).
- In ML, **a good representation is one that makes the subsequent learning task easier**.
- Designing features by hand is hard: whole decades of work for some communities (language, images). It is equivalent to hand-engineering the $\phi(\mathbf{x})$ of LBE.
- Supervised learning of an MLP leads to an **automatic representation** in each hidden layer, with properties that make the task of the output layer easier (XOR!). That is, $\phi(\mathbf{x}, \mathbf{w})$ is learned by finding the best $\mathbf{w}$.
- In CNNs the raw image is used as input and the model learns to extract the features salient for the task, without patterns or features specified in advance.

How to obtain or exploit the hidden representations (besides the full training of MLPs and CNNs)?

- **Obtaining them** (historical case): with **semi-supervised** learning a representation is learned from unlabelled data and used for supervised tasks → **pre-training**.
- **Exploiting them**: the learned representation is used for other tasks → **transfer learning**.

17.3.2 Pre-training

Greedy layer-wise unsupervised pretraining (around 2006) was the first approach that made it possible to train deep supervised networks (apart from CNNs and RNNs). **Unsupervised** learning is used to capture the shape of the input distribution, **layer by layer**, and the network is initialised this way, making overall training easier (instead of immediately doing *end-to-end* training with gradient descent, which was difficult for deep models).

Why does it help? Each layer is optimised independently in an unsupervised way, constituting a pre-training for the final **fine-tuning**. It works both as a **good initialisation strategy** and as **regularisation**: not in the sense of a simpler model, but of discovering features that capture the regularities of the data. If the true functions are complicated and shaped by the regularities of the input distribution, unsupervised learning can be a more appropriate regulariser. It also reduces the **variance** of the estimation process (pre-trained models tend to end up in a smaller region).

Autoencoder

Definition — Autoencoder

An **autoencoder** is a neural network trained to **copy its input to its output**. Internally it has a hidden layer $\mathbf{h}$ that describes a **code** to represent the input. It consists of an **encoder** $\mathbf{h} = f(\mathbf{x})$ and a **decoder** that produces the reconstruction $\mathbf{r} = g(\mathbf{h})$.

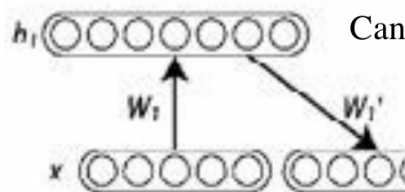


Figure 17.7: An autoencoder: encoder (W_1) and decoder (W_1').

- **Undercomplete**: the hidden layer is **smaller** than the input, and the architectural constraint forces the network to capture the most salient features. (With a linear decoder and squared loss one obtains PCA.)
- **Overcomplete**: the hidden layer is **larger** than the input, but a regularisation is used that imposes sparsity, robustness to noise or other properties, beyond the trivial ability to copy.
- It is **unsupervised** learning.

Example of use: **denoising autoencoders** and auto-associative memories. It is trained on images looking for a good representation (“a good representation is one that can be obtained robustly from a corrupted input and that will be useful for recovering the clean input”), then the original image is recovered by presenting a noisy version of it and repeating the encoding and decoding phases.

The pre-training algorithm (Bengio)

Summary — Layer-wise pre-training

1. Train the first layer as an **auto-associator** to minimise the reconstruction error of the raw input (unsupervised: unlabelled examples are enough).
2. Use the outputs of the hidden units as input for another layer, also trained as an auto-associator.
3. Repeat until the desired number of layers.
4. Use the output of the last hidden layer as input for a supervised layer, initialised (randomly or with supervised training, keeping the rest fixed).
5. **Fine-tune** all the parameters with respect to the supervised criterion.

The hope is that the unsupervised initialisation has brought the parameters into a region from which local descent reaches a good local optimum.

Autoencoders are used as RBMs (*Restricted Boltzmann Machines*, energy-based models, in Hinton’s *deep belief networks*) or as stacked (denoising) autoencoders (Bengio, 2007).

Note — Is pre-training still needed?

Pre-training made it possible to **kick-start** deep learning and still gives improvements in some tasks (especially NLP, where it allows exploiting large text corpora to learn distributed word representations). But it is hard to manage (hyperparameters split into two phases) and today “*unsupervised pre-training is largely abandoned*” (DL book, ch. 15): modern techniques are enough for good **end-to-end** training from scratch with backpropagation.

Note — Encoders and decoders today

- **Variational autoencoders** use a Bayesian framework: the latent space is a mixture of distributions instead of a fixed vector.
- **Neural machine translation** is implemented with encoder-decoder architectures in which the output does not coincide with the input but is in another language.
- **Transformers** (such as GPT, *Generative Pre-trained Transformer*) are encoder/decoder networks with an **attention** mechanism. One can use only the encoder (BERT, for various NLP tasks) or only the decoder, for generative purposes (the GPT series for autoregressive text generation).

17.3.3 Transfer learning

The representation discovered by a model is used to **improve another model**, assuming that there are useful features in different contexts or tasks, corresponding to common underlying factors. Different forms (they are real sub-fields of ML, not just terminology):

- from autoencoder to classifier (as above);
- **multi-task learning**: same inputs, different targets (an implicit regularisation);
- **domain adaptation**: different input domain but shared features (e.g. sentiment analysis on video and smartphone reviews);
- reuse of **pre-trained models** on larger datasets.

Example — Transfer learning with AlexNet

AlexNet is a CNN trained on about 1.2 million ImageNet images in 1000 categories: it has learned **rich representations** for a wide range of images, with low-level primitives shared across different images. It has 5 convolutional layers and 3 fully connected ones. For a new task one **replaces the last 3 layers** and trains only those with a few new images, or does **fine-tuning** of the whole network. Many pre-trained CNNs (VGG, ResNet, GoogLeNet...) are available in the libraries.

17.4 Distributed representations**Definition — Distributed representation**

“In a distributed representation the elements (the features) are not mutually exclusive and their many configurations correspond to the variations seen in the observed data.” (LeCun, Bengio, Hinton, 2015)

Deep learning methods exploit distributed representations over several levels, but the concept applies to many models (neural networks in general, on the hidden units; probabilistic graphical models, with several latent variables).

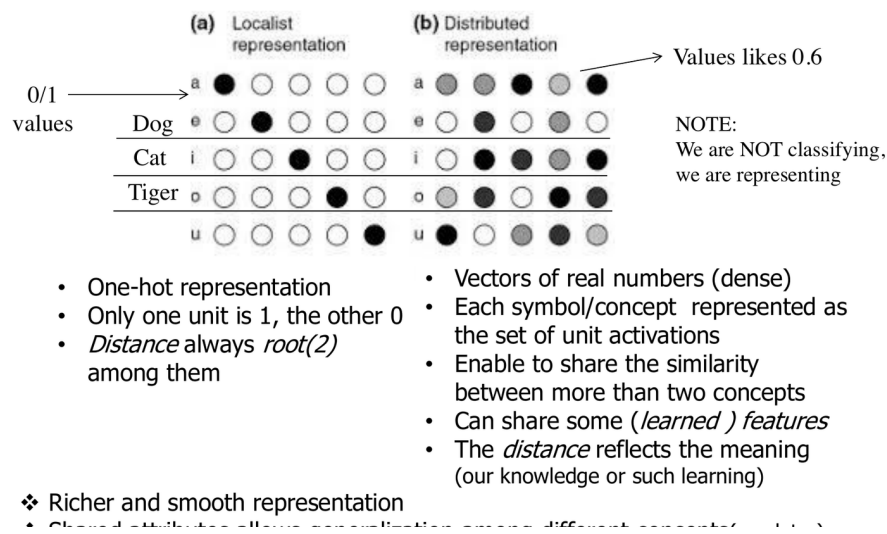


Figure 17.8: Symbolic (one-hot) and distributed representation.

Symbolic (one-hot)	Distributed
only one unit is 1, the others 0	(dense) vectors of real numbers
distance always $\sqrt{2}$ between different concepts	each concept is the set of activations of all the units
	similarity can be shared among more than two concepts
	(learned) features can be shared
	distance reflects meaning

Beware: here we are **not classifying**, we are **representing**. Distributed representations are richer and smoother, and shared attributes make it possible to generalise across different concepts.

Input or internal representation? The discussion is general, but learning acts on the **internal** representation. A distributed representation at the **input** is used only if domain knowledge allows it to be fixed (rare and difficult; fixing it arbitrarily can harm the task). In general a (non-informed) one-hot input is used anyway and the model is left to develop **internally** the necessary distributed representation.

17.4.1 Counting the difference

Four concepts: blue car, blue bike, red car, red bike. 4 neurons are not needed: **2** are enough (2^2 concepts), one for blue/red and one for car/bike. And the neuron describing “red” learns red from images of both cars and bikes, not just from a specific category.

In general, with n features of k values a distributed representation describes k^n different concepts (for example 2^n binary configurations), versus the n of a symbolic one-hot representation. A unique code can be assigned to an **exponential** number of regions: models with distributed representations (such as networks with hidden units) distinguish exponentially more regions than non-distributed ones.

Models based on **non-distributed** representations: K-NN (one or a few prototypes per input, without sharing information with the whole training set), RBF kernels (they reduce to K-NN, as seen for SVMs), Gaussian mixtures, decision trees (only one leaf is activated).

17.4.2 Shared attributes: disentangling concepts

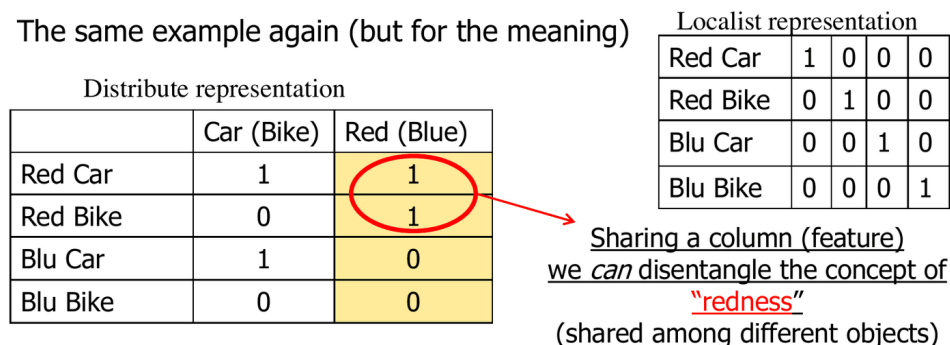


Figure 17.9: By sharing a column (feature) the concepts are disentangled.

By separating the concepts of colour and type, one can learn the car/bike distinction or the colour **without having to see examples of all combinations**. This **statistical separability** makes it easy to generalise to configurations never seen: having seen the other three cases, the blue bike is easily classified (for example “I don’t like it” because it is not red, if the model has learned that I like red objects); with the localist representation the blue bike would be completely new.

With **words**: instead of a one-hot symbol per word (dimension equal to the vocabulary), a distributed representation in which “dog” and “cat” share features allows the model to:

1. **treat them similarly**: sentences with “cat” inform the predictions for sentences with “dog” and vice versa, transferring information from each training sentence to an exponential number of semantically related sentences (for example “both have a tail” → animal, not human);
2. **automatically learn that they are similar**, bringing their representations closer because they appear in similar contexts: this is **word embedding** (for example *word2vec*). In the

one-hot space every word is at distance $\sqrt{2}$ from all the others; in the learned space words with similar meaning become close.

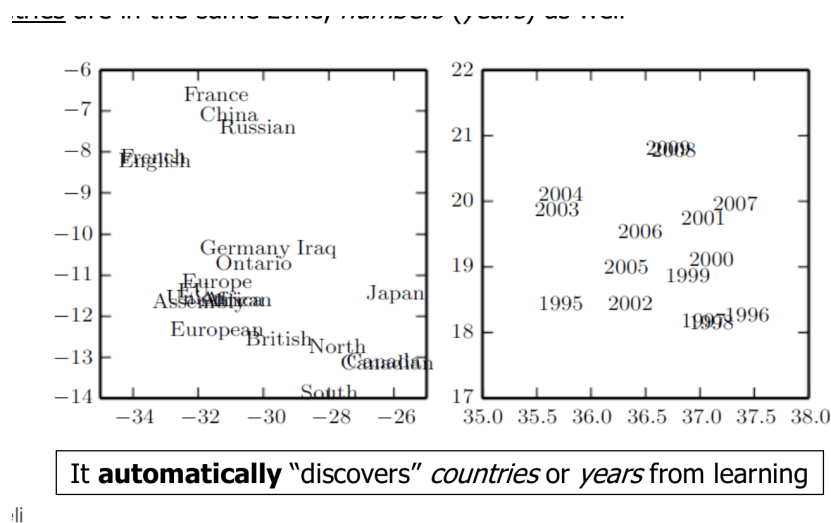


Figure 17.10: Word embedding: semantically similar words (countries, years) end up close together. The model automatically “discovers” countries and years through learning.

Tip — A consideration on the results

The model learns not only grammar (not surprising) but **semantically meaningful** representations, and it does so automatically, just by reading text. Why does our semantics coincide with what the network learns? Probably language (and images) intrinsically have a **hierarchical** structure and the deep network has the right inductive bias to learn it in a distributed way (real images are a tiny fraction of white-noise images). It is still an open field.

The debate on distributed representations extends to that between **logical-symbolic** and **neural** paradigms for cognition: the successes in translation cast doubt on the need for symbolic approaches to understand sentences.

Warning — Interpretability

The other side of the coin: a distributed representation is **less easy to interpret** than a localist one.

17.4.3 Deep distributed representations

DL exploits distributed representations **across many layers**, obtained by composing levels of abstraction or a hierarchy of reused features. Compositionality brings a further exponential gain in statistical efficiency, in addition to that of distributed representations: **two potentially exponential advantages** over non-distributed and shallow models. Overall, DL models learn a distributed representation of the data by finding (disentangling) the **shared causal factors** that generate them, over different levels of abstraction.

Application examples: the combination of CNNs and RNNs to generate **image captions** (with an attention mechanism that focuses on the part of the image related to each generated word); the classification of **skin cancers** with a deep network trained on 130,000 cases, with competence comparable to that of dermatologists (*Nature* 2017).

17.5 Further insights and recent results

- **Smoothness through structure:** with the same number of parameters, deeper networks impose more smoothness than shallow ones (each layer works on the already smooth surface produced by the previous one), and they seem to learn better with the same total number of neurons: **implicit** smoothness constraints, rather than explicit ones.
- **The non-overfitting puzzle:** huge, over-parameterised networks, able to drive the training error to zero even on **random** labels, still generalise. One explanation: gradient descent imposes a form of **implicit regularisation**, with convergence towards the **maximum margin** solution (Poggio et al., 2017–2019). SGD variants with batch/weight normalisation maximise the normalised margin, and even standard gradient descent maximises the L^2 margin without explicit normalisation or regularisation.
- **Double descent:** in CNNs, ResNets and transformers, as the size of the model (or of the data, or the training time) grows, performance first improves, then worsens (the classic U) and then **improves again** beyond the interpolation threshold (where the training error becomes almost zero). The effect is often avoided with careful regularisation; it is not yet fully understood and is an important research direction. (It does not necessarily concern the small models of the project.)

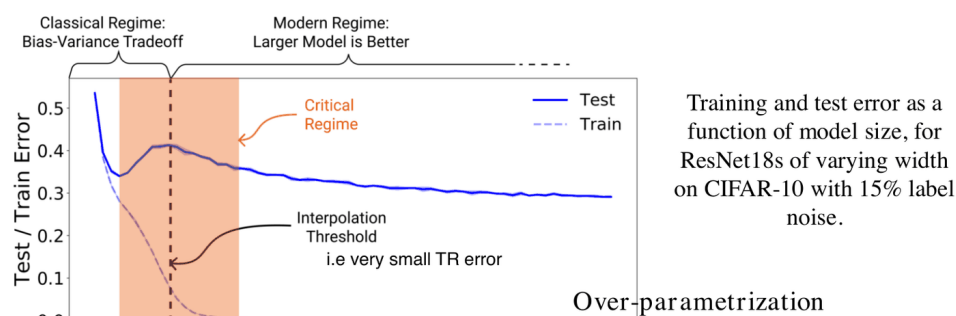
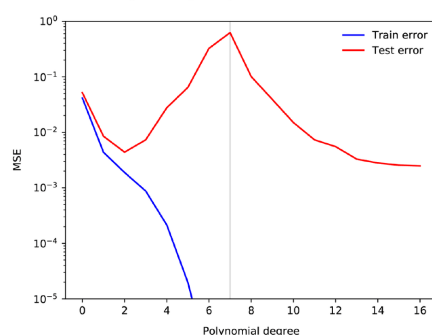


Figure 17.11: The double descent phenomenon.

If we look at the [median over 250 runs] of the MSEs, we get the pretty "double descent"!



Credits: François Fleuret

111

Figure 17.12: A toy example with regularised polynomial regression: double descent appears here too. Beyond zero training error, as the degree increases the fit is driven only by the penalty term and the polynomial becomes more and more “piecewise linear”.

- **Redundancy:** in deep models there is significant redundancy in the parameters; given a few weights per feature the others can be predicted, up to more than 95% of the weights without loss of accuracy (Denil et al., 2013). Parameters can also be **compressed** (pruning, quantisation, Huffman coding) without loss (Han et al., 2015): it seems that small

networks are enough!

- **Lottery ticket hypothesis** (Frankle and Carbin, 2018): why then do we train large networks? Because good performance depends on a **lucky initialisation** of one or more sub-networks, and large networks contain exponentially more of them. Retraining the pruned network with the same initialisation gives similar performance.
- The role of **randomness** (randomly initialised networks and random-weight networks) is receiving more and more attention (see [18 - Randomized neural networks](#)).

17.6 Techniques

Many layers can be hard to optimise, so the emphasis is on methods to improve **gradient descent** (also against the *vanishing gradient*), **regularisation** (large networks) and the **exploitation of data** (semi-supervised, reinforcement learning, multi-task, adversarial learning...). To this are added larger datasets, public software infrastructures and hardware improvements.

Summary — Key techniques for DL

- Originally: **pre-training** approaches (still used in domains such as NLP for word embeddings).
- Today (in the most popular libraries), mostly already known:
 - **SGD with momentum** (with decay of η , mini-batch) or **Adam**;
 - **ReLU** activations in the hidden units;
 - **maximum likelihood** loss (cross-entropy) with **softmax** at the output (also to avoid saturation and small gradients);
 - regularisation: early stopping, weight decay, **dropout**, **batch normalization**.

17.6.1 Gradient problems

What happens to the norm of the gradient when it is backpropagated through many layers?

- if the weights are **small**, the gradients **shrink exponentially** (*vanishing gradient*), also because of the derivative of the sigmoids;
- if the weights are **large**, the gradients **grow exponentially** (*exploding gradient*).

The vanishing gradient

Traditional activations such as tanh have gradients in $(-1, 1)$, and backpropagation computes the gradients with the chain rule repeated through the layers. In a network with d layers, d of these small numbers are multiplied to compute the gradients of the layers close to the input: the gradient (the delta) **decreases exponentially with d** and the layers close to the input train very slowly.

Note — Formal sketch

If $\mathbf{h}_l = F_l(\mathbf{h}_{l-1})$ and $F = F_M \circ \dots \circ F_1$, then

$$\frac{\partial F}{\partial w_l} = \frac{\partial F_M}{\partial \mathbf{h}_{M-1}} \dots \frac{\partial F_l}{\partial w_l}, \quad \left| \frac{\partial F}{\partial w_l} \right| \approx \rho^{M-l+1} \text{ if } \left\| \frac{\partial F_i}{\partial \mathbf{h}_i} \right\| \approx \rho.$$

With $\rho < 1$ the gradient vanishes, with $\rho > 1$ it explodes.

Approaches to tackle it range from R-prop to *shortcut* connections, to randomised networks, and ReLUs.

Gradient clipping

Repeated multiplication across layers can introduce **cliffs** in the cost function: very high derivatives that “catapult” the weights very far away. **Clipping** bounds the norm of the gradient while keeping its direction:

$$\text{if } \|\mathbf{g}\| > v \text{ then } \mathbf{g} \leftarrow \frac{v}{\|\mathbf{g}\|} \mathbf{g},$$

with v a threshold on the norm.

if $\|\mathbf{g}\| > v$ then $\mathbf{g} = v \mathbf{g} / \|\mathbf{g}\|$, $\mathbf{g}$: gradient and v : norm threshold
 we in the gradient direction, but bounding the weight update

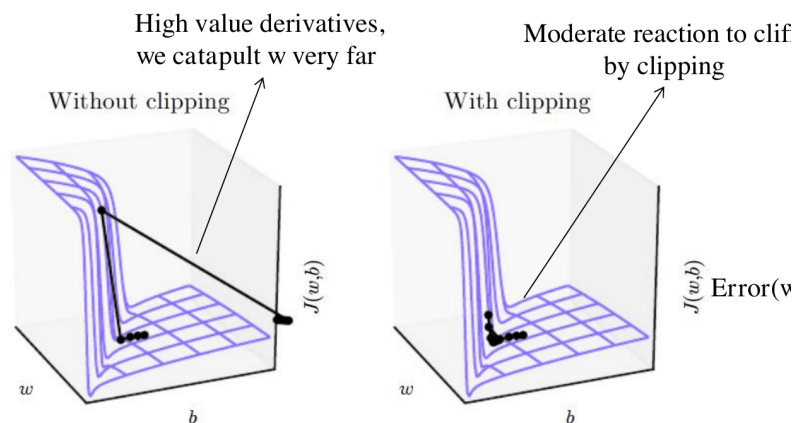


Figure 17.13: Without clipping (left) and with clipping (right) when facing a cliff of the cost function.

17.6.2 ReLU

Definition — ReLU (Rectified Linear Unit)

$$f(x) = \max(0, x) = \begin{cases} 0 & x < 0 \\ x & x \geq 0. \end{cases}$$

Why use it in DL (since 2009)?

- Better **gradient propagation** through many layers: it avoids the saturation of sigmoids, because $f'(x) = 1$ for every positive net.
- **Sparse activation**: in a randomly initialised network about 50% of the hidden units are active (non-zero output).
- **Efficient computation**: only comparisons, sums and products.
- It still has a neuroscientific inspiration, and it allows faster and more effective training of deep networks.

ReLU is **not differentiable at 0**, but in libraries the derivative is usually assumed to be 0 (left) or 1 (right): the approximation is acceptable and safe, because with numerical approximations it is unlikely to evaluate exactly $f(0)$. (A more rigorous approach uses non-differentiable optimisation for piecewise linear functions.)

A trick: start with **positive nets**, for example with small positive biases (0.1), so that ReLUs are initially active for most inputs and let the derivatives through.

Variants: **ELU** (*Exponential Linear Unit*), $f(x) = \alpha(e^x - 1)$ for $x \leq 0$ and x for $x > 0$; **Leaky ReLU**, $f(x) = 0.01x$ for $x < 0$ and x for $x \geq 0$. They also learn on the left side (neurons do not “die”) and have mean activation closer to zero. Many variants have comparable performance; in general they avoid some defects while keeping the linear part, which makes the model easier to optimise.

17.6.3 Batch normalization

Batch normalization normalises each mini-batch by computing its statistics (mean and variance) for each layer: the matrix [examples of the batch $\times$ activations of the units] is normalised to zero mean and unit variance, and the transformation is included in backpropagation. Just as normalising the input is standard practice, BN keeps the data flowing between the intermediate layers normalised. Effects: **regularisation** (due to the noise introduced by the mini-batch) and **faster learning** with higher accuracy (relevant because it made it possible to effectively train CNNs and deep models with sigmoids). (Details not required for the exam, unless it is used.)

17.6.4 Dropout

Dropout randomly selects a subset of the network during training. It is explained by two concepts already known: **ensembles** (an implicit bagging of different sub-networks, but computationally cheap) and **regularisation**.

Bagging trains n models on different subsets of the training set and averages their outputs: high-variance, low-bias models work well on average. Dropout approximates and exploits this process with an **exponential** number of sub-networks, while having **a single network** at test time, and it maximises the diversity of the ensemble by avoiding complex co-adaptations on the training data.

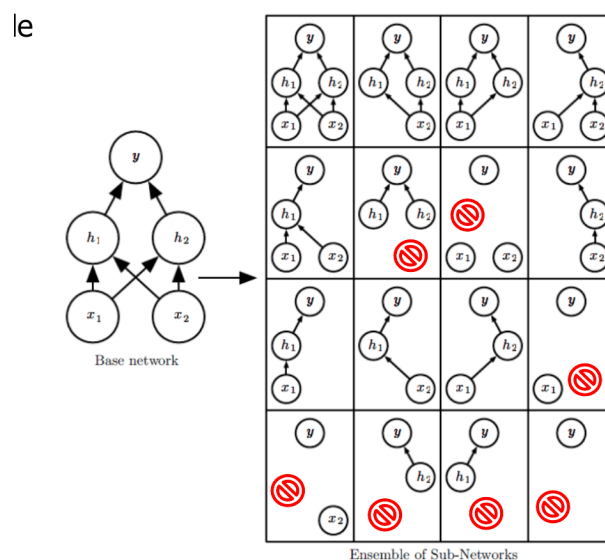


Figure 17.14: Dropout trains the ensemble of all the sub-networks obtainable by removing non-output units from the base network. Some sub-networks do not work, but in large networks it is unlikely that there is no path from input to output.

Summary — Dropout algorithm

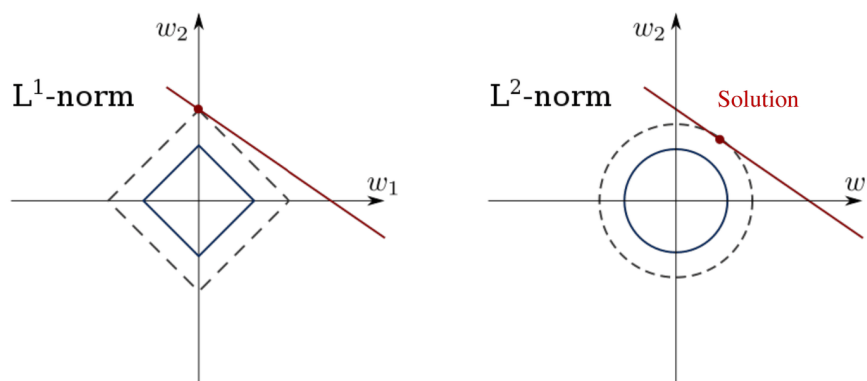
- Each time an example is loaded into a mini-batch, a different **binary mask** is sampled to be applied to all input and hidden units, independently for each unit. The probability of including a unit is a hyperparameter (typically 0.8 for the input and 0.5 for the hidden units).
- Each unit is multiplied by its mask (zero is equivalent to removing it, *drop out*): a sub-network is obtained.
- Forward pass, backpropagation and update are performed as usual, training a subset of units at a time.
- The removed units are then reinserted with their original weights.

Only a small fraction of the sub-networks is trained (each for a single step), but **parameter sharing** means that the others also reach good parameters: an exponential number of models is handled with tractable memory. To approximate the prediction of the ensemble at test time the *weight scaling inference rule* is used: the weights are multiplied by the inclusion probability (typically 0.5), so that the expected output of each unit is the same as in training.

Regularising effect: it avoids training all units on all data by reducing their interactions (and diversifying them, as committees require); it reduces variance like bagging; it injects **structured noise** (for example recognising a face without the unit that detects the nose). Above all, it regularises each unit to be not only a good feature, but **a feature that is good in many contexts** (different sub-networks). For linear regression it is equivalent to L^2 weight decay with a different λ for each input. It can be used with any distributed-representation model trained with SGD.

17.6.5 L1 regularisation

Using the L^1 norm (sum of absolute values) instead of L^2 in the penalty (for linear models it is **LASSO**) favours the **elimination** of features (weights exactly at zero), whereas L^2 usually only reduces the magnitude of the weights: simpler models are obtained. The solution requires non-differentiable optimisation techniques (sub-gradient, smooth approximations of the L^1 norm...).



A. Micheli

149

Figure 17.15: Why L1 produces sparse solutions: the level curves of the L1 norm have “corners” on the axes, where one coordinate is zero.

17.6.6 Adversarial learning

Generative Adversarial Networks (GANs) are two networks in competition: one **generates** candidates, the other **evaluates** them (distinguishing real from generated ones). They are used for example to generate photorealistic images of people who do not exist.

17.7 Applications, challenges and conclusions

DL is successful in **NLP** (speech recognition, translation, sentiment analysis, question answering...), in **computer vision** (images and video: object recognition, medical applications, image synthesis...) with performance never reached before, and in the game of **Go**. It is discussed in more depth in the NLP and ISPR courses.

Open challenges: theoretical analysis and framework; optimal architecture design (how many layers? which units? cross-validation may be too expensive); efficiency for big data; stream (on-line) processing; structured data (sequences, trees, graphs).

Hardware has changed too: Google TPUs (initially only for inference), Nvidia GPUs, NPUs in smartphone chips (Huawei Kirin 970), Apple's *Neural Engine* (since 2017).

Summary — Summary

- Technical progress and the availability of data have made it possible to learn with deep and very large networks, with state-of-the-art results especially where the **compositionality** of features is relevant (vision, language).
- Two potentially exponential advantages: **depth** (composition of functions, no-flattening) and **distributed representations** (shared attributes).
- The most general concept to emerge is **representation learning**.
- Techniques: ReLU, cross-entropy with softmax, SGD with momentum/Adam, dropout, batch normalization, gradient clipping.
- ML makes it relatively easier to develop sophisticated AI systems (and software in general) for real problems.

Questions — Possible exam questions

- What is meant by deep learning? What are the aspects common to the various approaches?
- Are many layers really needed? Discuss the parity example and the no-flattening results.
- What is the inductive bias of deep models?
- What is representation learning? Pre-training and transfer learning.
- What is an autoencoder?
- Symbolic versus distributed representations: advantages of the latter.
- What is the vanishing gradient? How is it tackled (ReLU, clipping...)?
- Describe dropout and explain its regularising effect.

Chapter 18

Randomized neural networks

After deep learning, the other recent paradigm presented in the course is that of **random-weight neural networks** (“deep learning and *extreme* learning”, the professor jokes). The idea seems paradoxical: what if the hidden layer were not trained at all? The lecture shows how randomness is a useful component of ML and how randomised networks connect to concepts already seen: basis expansion, Cover’s theorem, regularisation.

18.1 Randomness in Machine Learning

A **randomised** approach uses some degree of randomness as part of its constructive strategy. Randomness can improve predictive performance and alleviate the difficulties of classical methodologies, and it is present at different levels:

- **data processing:** data splitting (hold-out, K-fold CV, leave-one-out), data generation (bootstrap, boosting, balancing techniques);
- **learning:** order of observations (shuffling for mini-batch), random initialisation of network weights;
- **hyperparameter selection** (including random search);
- **ensembles:** random differentiation among models (different initialisations, sampling of training sets in bagging and boosting);
- **at the heart of the model.**

18.1.1 Intrinsically random models

- The stochastic neurons of **Restricted Boltzmann Machines**.
- **Extremely Randomized Trees** and **Random Forests:** an ensemble of randomised decision trees, each built on samples drawn randomly from the training set and with a random choice of the input variables on which to split the nodes. They combine the advantages of ensembles and randomness.
- **Dropout** as regularisation (although here randomness is not necessary per se).

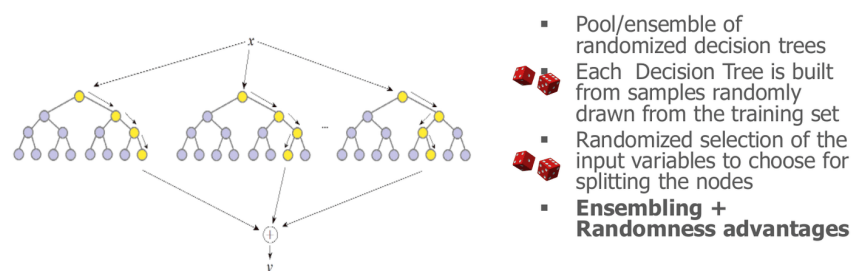


Figure 18.1: Random Forest: an ensemble of randomised decision trees.

18.2 Random-weight neural networks

Questions — The question

Is it possible to exploit MLP (or recurrent) architectures **without long training cycles** of the hidden layers?

There is a long tradition of randomised, random-weight, random projection networks: for example the **Random Vector Functional Link** (RVFL), from the very beginning. Recently they have been popularised under various names, including **ELM** (*Extreme Learning Machine*).

Definition — Basic idea

- A network with one (or more) **randomly connected hidden layers**: the weights are **fixed after random initialisation** and are no longer trained.
- **Only the output weights** are trained (a linear model: with the pseudoinverse or ridge regression).

This overcomes the problems of the complex and computationally expensive training algorithms traditionally used for neural networks.

The roots lie in Rosenblatt’s pioneering work on the **Perceptron**, in which a “projection” area randomly connected to the retina fed an “association” area, and only the final responses were learned. Other pioneering works had stressed the importance of learning in the output layer compared to the hidden one.

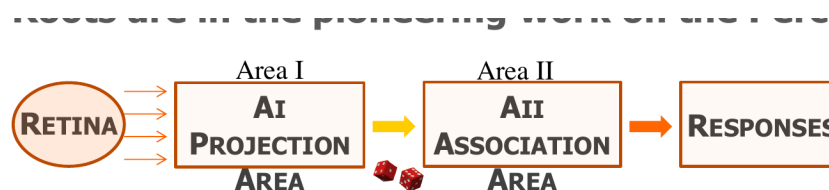


Figure 18.2: The original Perceptron: the connections from the retina to the projection area were random.

Randomised networks are today a growing line of research, particularly suitable when **efficiency** is fundamental (training is limited to the last layer).

18.2.1 General structure

- **Hidden layer (untrained)**: it **non-linearly** embeds the input in a **high-dimensional feature space**, where the problem is more likely to be linearly solvable (it is an **LBE!**). It has a theoretical basis in Cover’s theorem.
- **Readout (trained)**: it combines the features of the hidden space to compute the output; typically it is a **linear model**.

Theorem — Cover’s theorem (1965)

“A complex pattern-classification problem, cast in a high-dimensional space non-linearly, is more likely to be linearly separable than in a low-dimensional space, provided that the space is not densely populated.”

For example, with a deterministic mapping: given l examples, they are lifted onto the vertices of an $(l - 1)$ -dimensional simplex (the generalised “triangle”). Now **every** binary par-

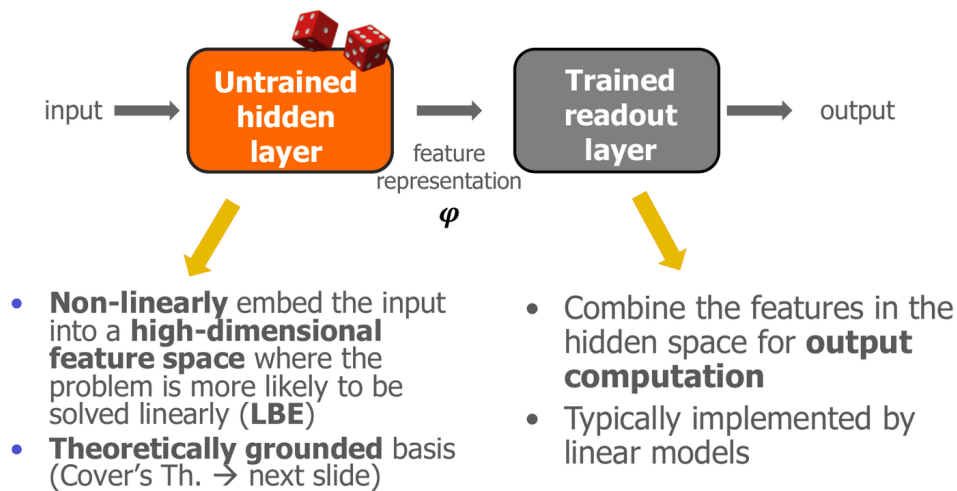


Figure 18.3: General structure: an untrained (random) hidden layer and a trained readout layer.

tion of the samples is linearly separable.

It is exactly the same property exploited by **kernels** in SVMs.

18.2.2 Randomised feedforward networks

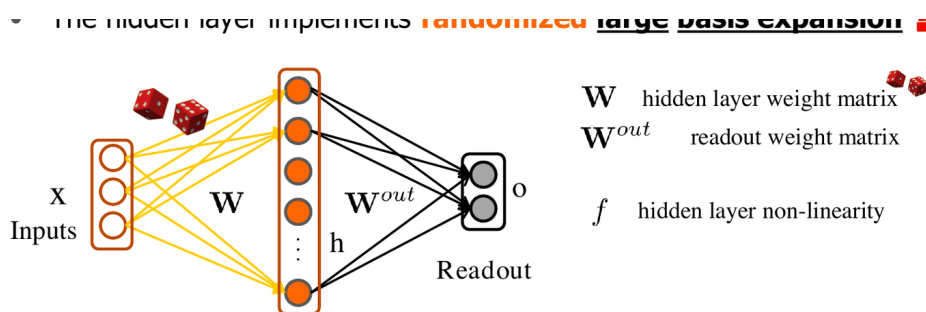


Figure 18.4: Randomised feedforward network: W (random, fixed) and W^{out} (trained).

The input-output relationship is implemented by an untrained hidden layer, which realises a **large random basis expansion**.

Summary — Construction, learning and use

Construction: fill W with random values (for example Gaussian noise), using **many units**.

Learning of the readout: given l examples $(\mathbf{x}_p, \mathbf{d}_p)$, as for linear models the squared error is minimised with the direct method. With $\mathbf{H} = f(\mathbf{W}\mathbf{X})$ (the hidden activations of all patterns):

$$\mathbf{W}^{out} = \mathbf{H}^+ \mathbf{d},$$

often with L^2 **regularisation** (useful because of the high number of units):

$$\mathbf{W}^{out} = (\mathbf{H}^T \mathbf{H} + \lambda \mathbf{I})^{-1} \mathbf{H}^T \mathbf{d},$$

or with L^1 .

Use: the hidden activations $\mathbf{h}$ are computed as for ordinary neurons and then the outputs

$$\mathbf{o}(\mathbf{x}) = \mathbf{W}^{out} f(\mathbf{W}\mathbf{x}),$$

possibly applying a threshold (or a non-linear function) for classification.

The main reference models are the **RVFL** (*Random Vector Functional-Link*), the **ELM** (*Extreme Learning Machine*) and **RBF networks with random centres**.

Tip — Why it works

Compare with “ordinary” neural networks: there the hidden layer is an **adaptive** basis expansion $\phi_j(\mathbf{x}, \mathbf{w})$ whose weights are learned; here it is a **random** but very **wide** basis expansion. If the bases are numerous enough, by Cover’s theorem the problem becomes (almost) linearly separable in the hidden space, and a linear output model is enough, trainable in a single step with least squares. The training cost is that of a regularised linear regression.

18.2.3 Pros and cons

- The large set of hidden units plays an interesting role: the projection non-linearly expands the input dimension, making the data more separable.
- A high number of units can provide a **sufficient** basis expansion for the task (with theoretical foundations, including universal approximation theorems).
- **Extremely efficient**.
- There are many variants: incremental learning by adding neurons (like Cascade Correlation), pruning, etc., often related to past research under different names.
- “Targeted” models, with fewer adaptive units trained on the specific task, nonetheless seem **more elegant**: there is a debate between **random feature** and **learned feature** approaches, and it is worth knowing both.
- They extend to recurrent networks: **Echo State Networks** (ESN), see [20 - Recurrent neural networks \(RNN\)](#).

18.2.4 Where they are useful

- **Big Data**, thanks to their extreme efficiency.
- Tiny implementations for **embedded** and distributed systems, on resource-constrained devices (sensor networks, IoT, edge computing).
- **Deep learning**, where training algorithms are particularly expensive: randomised networks alleviate the computational load (randomised deep networks as stacks of RVFL/ELM modules, randomised CNNs).
- **Randomised recurrent networks**: where training is even more expensive than for MLPs, and with solid theoretical foundations on the bias introduced by the system dynamics (stability, learnability).

Summary — Conclusions

Randomness is not an enemy: it is a friend, if it is controlled. (This also holds in other fields of computer science, such as randomised algorithms, and in mathematics.) Note how the same concepts keep coming back — Cover’s theorem, LBE, regularisation, feature representation: the course is now “mature”.

Note — FAQ: can ELMs be used in the project?

Yes, but as an **additional part** with respect to the standard implementation of an MLP fully trained with backpropagation and SGD. They are easy to implement as a sub-part of a feedforward network.

Questions — Possible exam questions

- At which levels does randomness appear in ML?
- Describe a randomised feedforward network: construction, readout training, use.
- State Cover's theorem and explain its link with randomised networks and with kernels.
- Pros and cons of random features compared to learned features.

Chapter 19

Unsupervised learning: K-means and SOM

This lecture is a light introduction to **unsupervised learning** with neural networks. The focus is on **clustering** seen as **vector quantization**: we start from the classic **K-means** and arrive at a historical neural model, Kohonen's **Self-Organizing Maps** (SOM), which add to clustering the property of preserving topology and thus of visualising the data.

19.1 Unsupervised learning

In **unsupervised learning** there is **no teacher**: the training set contains only **unlabelled** data $\langle \mathbf{x} \rangle$. The main tasks:

- **Clustering**: finding natural groupings in the data.
- **Dimensionality reduction**, visualisation, pre-processing: multidimensional data are projected into a lower-dimensional space (PCA, Multi-Dimensional Scaling, Independent Component Analysis...).
- Modelling the density of the data (not covered in the course).

What is it for? Examples: organising life forms into a taxonomy (what are mammals?); newborns learn to recognise (to “cluster”) familiar faces before understanding language. Above all, **unlabelled data cost much less**: for example 10 million YouTube frames made state-of-the-art results possible in object recognition across 22,000 categories.

19.1.1 Clustering

Definition — Clustering

Partitioning the data into **clusters** (subsets of “similar” data): the patterns in a cluster are more similar to each other than to a pattern in another cluster. Each cluster is represented by a **centroid** (also called prototype, reference vector, cluster centre, *codevector*; the set of centroids is the **codebook**).

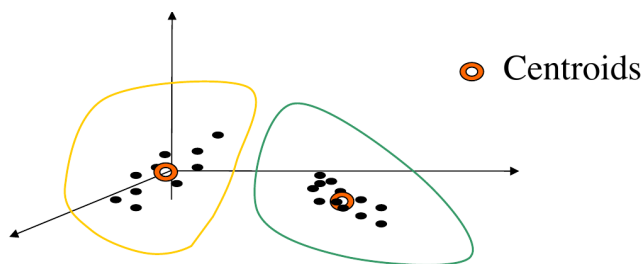


Figure 19.1: Clustering: partition of the data into groups represented by centroids.

Clustering has been tackled in a great many contexts and disciplines, with thousands of different algorithms: this reflects its usefulness as a step of exploratory data analysis. Here we adopt the perspective of **vector quantization** for **partitional clustering**.

Warning — The metric dominates

As for K-NN, it is the **assumed distance** that says which examples are similar, and the algorithm decides the groupings based on that metric. The Euclidean distance is popular but it is **just one choice**, and the choice can have deep implications (for example outliers can be weighted less by using 1 – a Gaussian centred on the prototype).

19.2 Vector quantization

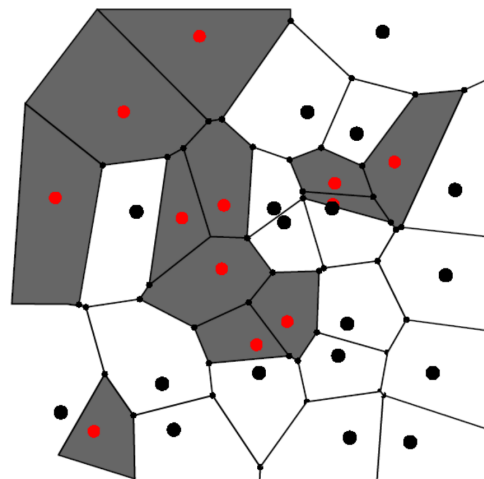
Definition — Vector Quantization (VQ)

Vector quantization techniques encode a data manifold $V \subseteq \mathbb{R}^n$ using only a finite set $\mathbf{w} = (\mathbf{w}_1, \dots, \mathbf{w}_K)$ of reference vectors (*codebook*) $\mathbf{w}_i \in \mathbb{R}^n$. A vector $\mathbf{x} \in V$ is described by the **winning** (*best matching*) reference vector $\mathbf{w}_{i^*(\mathbf{x})}$, the one with minimum distortion $d(\mathbf{x}, \mathbf{w}_{i^*(\mathbf{x})})$.

This divides the manifold into sub-regions, the **Voronoi polyhedra**:

$$V_i = \{\mathbf{x} \in V : \|\mathbf{x} - \mathbf{w}_i\| \leq \|\mathbf{x} - \mathbf{w}_j\| \forall j\},$$

and each $\mathbf{x}$ is described by the corresponding reference vector.



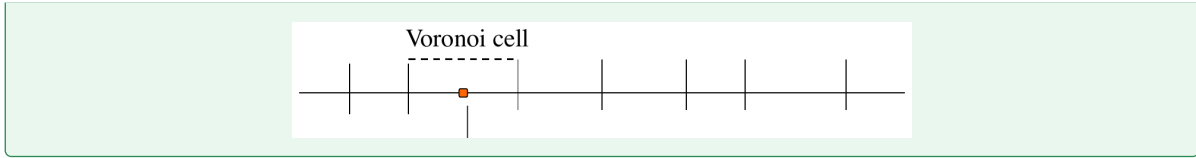
In general Hard to be co

Figure 19.2: Voronoi diagram: each cell contains the points closer to its centre than to any other.

Our problem is the opposite of that of K-NN: **given the dataset, find the centres** that minimise the loss. In general it is hard: finding the optimal codebook is **NP-complete**.

Example — Quantization in 1D

In digital signal processing, **quantization** approximates a continuous range of values (or a very large set of discrete values) with a relatively small set of discrete symbols, incurring a quantization error (distortion). This is what happens to music in digital devices.



VQ and clustering. Clustering has the more general goal of finding **interesting or useful** groupings, where “interesting” is often defined implicitly by the procedure itself and not necessarily by the minimum quantization error. However, VQ provides a useful framework, and its algorithms are often used for clustering.

19.2.1 The quantization error

The goal is to optimally partition an unknown distribution in the input space into regions approximated by prototypes: a set of quantizers $\mathbf{x} \mapsto c(\mathbf{x}) = \mathbf{w}_{i^*(\mathbf{x})}$, from a continuous space to a discrete one. The **squared distortion** $d(\mathbf{x}, c(\mathbf{x})) = \|\mathbf{x} - c(\mathbf{x})\|^2$ is considered; its mean value over the input distribution is the expected distortion (or reconstruction, or quantization error). **This is our loss:**

$$E = \int f(d(\mathbf{x}, \mathbf{w}_{i^*(\mathbf{x})})) p(\mathbf{x}) d\mathbf{x} = \int \|\mathbf{x} - \mathbf{w}_{i^*(\mathbf{x})}\|^2 p(\mathbf{x}) d\mathbf{x},$$

where $p(\mathbf{x})$ is the input distribution. In the discrete version, over l data and K prototypes:

$$E = \sum_{i=1}^l \sum_{j=1}^K \|\mathbf{x}_i - \mathbf{w}_j\|^2 \delta_{winner}(i, j), \quad \delta_{winner}(i, j) = \begin{cases} 1 & \text{if } j \text{ is the winner for } \mathbf{x}_i \\ 0 & \text{otherwise} \end{cases}$$

(δ_{winner} is the characteristic function of the receptive field of $\mathbf{w}_j$).

Minimising E with respect to $\mathbf{w}$ is the VQ problem. Beware: the integrand is **not continuously differentiable**, because the winner $\mathbf{w}_{i^*(\mathbf{x})}$ changes discretely as $\mathbf{x}$ varies; the derivatives are however computed **locally**, fixing $\mathbf{x}$ in a Voronoi cell.

19.3 K-means

19.3.1 On-line version

Differentiating E with respect to the $\mathbf{w}_j$ we obtain the VQ learning rule, i.e. the well-known **K-means** algorithm of Lloyd and MacQueen (on-line version): for each $\mathbf{x}_i$,

$$\Delta \mathbf{w}_{i^*} = \eta \delta_{winner}(i, i^*) (\mathbf{x}_i - \mathbf{w}_{i^*}).$$

(The derivative of $\|\mathbf{x}_i - \mathbf{w}_j\|^2$ with respect to $\mathbf{w}_j$ is $-2(\mathbf{x}_i - \mathbf{w}_j)$; moving in the direction opposite to the gradient brings $\mathbf{w}_j$ closer to $\mathbf{x}_i$.)

- **Only the winning prototype** is modified, moving it towards $\mathbf{x}$.
- It is a stochastic gradient descent $\rightarrow$ **local minima**; the method depends on the **initialisation**.
- The **number of clusters** must be given in advance.

19.3.2 Batch version

Summary — Batch K-means algorithm (Linde-Buzo-Gray, or generalised Lloyd)

1. Choose K centres: K randomly drawn patterns, or K random points in the hypervolume containing the patterns.
2. Assign each pattern to the closest centre (the **winner**):

$$i^*(\mathbf{x}) = \arg \min_i \|\mathbf{x} - \mathbf{w}_i\|^2, \quad \|\mathbf{x} - \mathbf{w}_i\|^2 = \sum_{j=1}^n (x_j - w_{ij})^2.$$

3. Recompute the centres as the **mean** (geometric centroid) of the cluster members:

$$\mathbf{w}_i = \frac{1}{|\text{cluster}_i|} \sum_{\mathbf{x}_j \in \text{cluster}_i} \mathbf{x}_j.$$

4. If the convergence criterion is not satisfied (no or minimal reassignment of patterns, minimal decrease of the squared error), go back to step 2.

(Exercise: from the on-line to the batch version. By summing — or averaging — the on-line delta with $\eta = 1$ over the patterns of a cluster, one obtains $\mathbf{w}_{new} = \mathbf{w}_{old} + \frac{1}{|C|} \sum_{\mathbf{x} \in C} (\mathbf{x} - \mathbf{w}_{old}) = \frac{1}{|C|} \sum_{\mathbf{x} \in C} \mathbf{x}$: exactly the mean.)

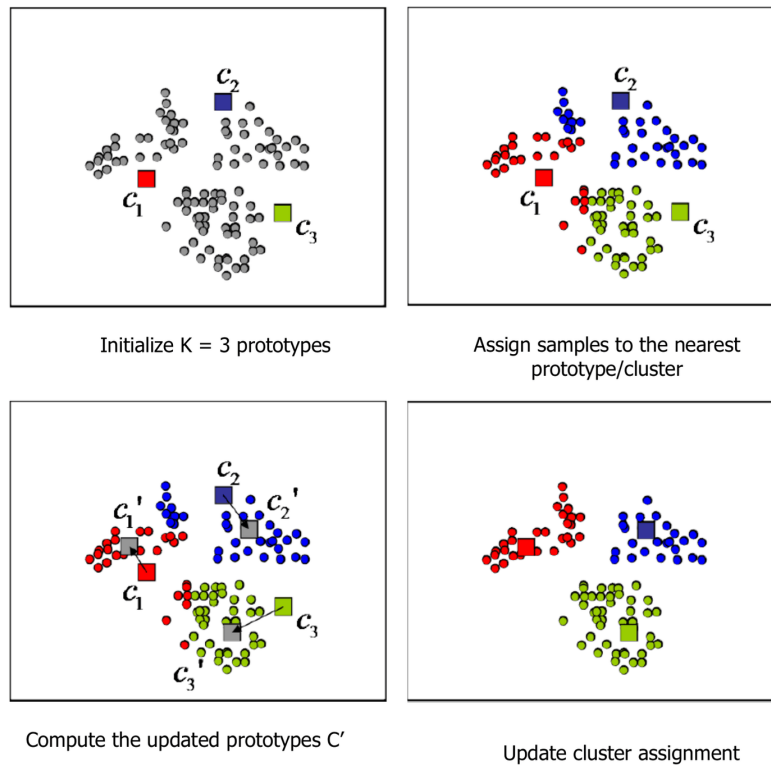


Figure 19.3: K-means example with $K = 3$: initialisation, assignment, prototype update, new assignment.

19.3.3 Strengths and weaknesses

- It is the simplest and most used algorithm with a squared error criterion; it is popular because it is easy to implement and generally efficient.
- The number K of clusters must be provided.
- The local minima of E make the method dependent on the initialisation, with only sub-optimal choices of the prototypes.
- It works very well for **compact, hyperspherical** clusters; with clusters of other shapes (for example two intertwined “spirals”) it fails.
- **No visualisation property**: it does not allow projecting the data into a lower-dimensional space, and the indexing of the $\mathbf{w}$ is arbitrary, so the mapping is not ordered.

19.3.4 Soft-max

To avoid local minima a **soft-max** adaptation rule is introduced: not only the winner is modified, but **all** the centres, according to their **proximity** to $\mathbf{x}$, with a step that decreases with the distance $d(\mathbf{x}, \mathbf{w}_i)$ (for example *maximum-entropy clustering* with Gaussian distances). An instance of this strategy in neural networks is **Kohonen’s SOM**, where however the proximity between reference vectors is defined on an **ordered map**, i.e. a neural grid on which they are arranged.

19.4 Self-Organizing Map

Self-Organizing Maps (SOM), or **Kohonen maps** (1981), are networks of N neurons arranged on a **regular low-dimensional grid** (usually 2D, a *lattice*).

- Each unit is identified by its **coordinates** on the map.
- Each unit receives **the same input $\mathbf{x}$** .
- Each unit has a **weight $\mathbf{w}$** , with $\dim(\mathbf{w}) = \dim(\mathbf{x}) = n$.

19.4.1 Goal

The SOM learns a map from the input space to a lattice of neural units that **preserves the topology**:

- nearby units on the map respond to similar input patterns;
- nearby points in the input space are mapped onto the same unit or onto nearby units;
- the (2D) output map has **visualisation** properties.

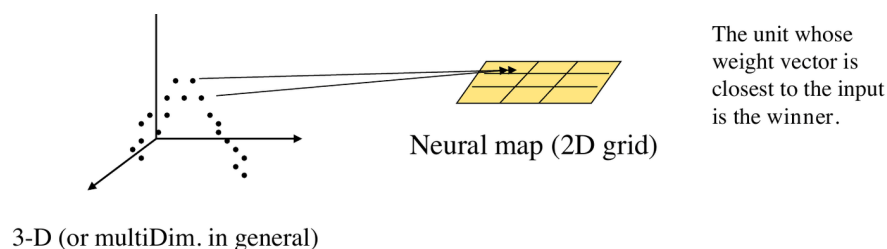


Figure 19.4: A SOM maps the input space (here 3D) onto a 2D neural grid.

Uses of the SOM:

- **clustering**: clusters are identified on the map (structures in the data are found quickly);

- **pattern recognition and VQ**: the input is transformed into the weight vector of the closest neuron (the codebook vector);
- **compression**: the input is transformed into the index (code) of the winning unit;
- **projection and exploration**: the distribution of multidimensional data is visualised in a lower-dimensional space (the 2D map).

19.4.2 Competitive learning and biological inspiration

Competitive learning is an adaptive process in which neurons gradually become sensitive (specialised) to different categories or sets of inputs: neurons **compete** for a data point, and the winner learns the most.

Nature exploits this mechanism: in the **somatosensory** (and motor) **cortex** there is a **topologically ordered** map of the body, the **homunculus**. Adjacent neurons represent nearby activation sources in the body, and the most sensitive parts occupy larger areas of the cortex. In a topology-preserving map, physically close units respond to classes of inputs that are in turn close.



Figure 19.5: The homunculus: a topologically ordered map of the body in the cortex.

19.4.3 The algorithm

Summary — SOM algorithm

- Randomly initialise the weights of the map.
- Draw an input $\mathbf{x}$.
- **Competitive phase**: the winner is the unit with $\mathbf{w}$ most similar to $\mathbf{x}$.
- **Cooperative phase**: update the weights of the units that have topological relations **on the map** with the winner (a form of soft-max).
- Continue until convergence (no change).

Key factor: the neighbours **on the map** (whereas standard soft-max uses proximity in the data space).

Competitive phase

The input $\mathbf{x}$ is compared with the weights of all units with the Euclidean distance. The standard SOM adopts a **winner-take-all** strategy (like K-means):

$$i^*(\mathbf{x}) = \arg \min_i \|\mathbf{x} - \mathbf{w}_i\|,$$

where i is an index on the map (for example the 2D coordinates).

Cooperative phase

The weights of the winner **and of the units in its neighbourhood** are moved towards the input (Hebbian learning). At iteration t :

$$\mathbf{w}_i(t+1) = \mathbf{w}_i(t) + \eta(t) h_{i,i^*(\mathbf{x})}(t) [\mathbf{x} - \mathbf{w}_i(t)],$$

where h is the **neighbourhood function** (*neighborhood kernel*), which decreases monotonically as the distance **on the map** between unit i and the winner increases. For example a Gaussian:

$$h_{i,i^*}(t) = \exp\left(-\frac{\|\mathbf{r}_i - \mathbf{r}_{i^*}\|^2}{2\sigma^2(t)}\right),$$

with $\mathbf{r}_i$ the coordinates of unit i on the grid and $\sigma(t)$ the width.

The **learning rate** η and the **neighbourhood radius** σ decrease with the iterations: the neighbourhood is **wide at the beginning** (global ordering of the map) and then shrinks slowly during learning (convergence, local refinement). In practice the shape of the neighbourhood is chosen among standard geometric shapes on the grid (rectangular, hexagonal...) that include a finite set of units.

19.4.4 The emergence of topological order

The cooperative phase rule is **fundamental** for the formation of topographically ordered maps. The weights are not modified independently, but **in topologically related subsets**: at each step the neighbourhood of the winner on the grid is selected, and all units in the subset receive similar updates. In this way topological information is transmitted to the map: the winner and its neighbours on the grid receive similar updates and, after learning, respond to similar inputs. Even though it seems natural, a formal proof of the ordering is difficult.

Tip — K-means versus SOM

- **K-means**: updates only the winner; the prototypes have no order; there is no visualisation.
- **SOM**: updates the winner **and its neighbours on the grid**, with a neighbourhood that shrinks over time; the prototypes are arranged on an ordered map that **preserves topology**, allowing visualisation; the initially wide neighbourhood also helps avoid local minima. (With a zero-radius neighbourhood, the SOM reduces to on-line K-means.)

During training the units “come to represent” the clusters of the data.

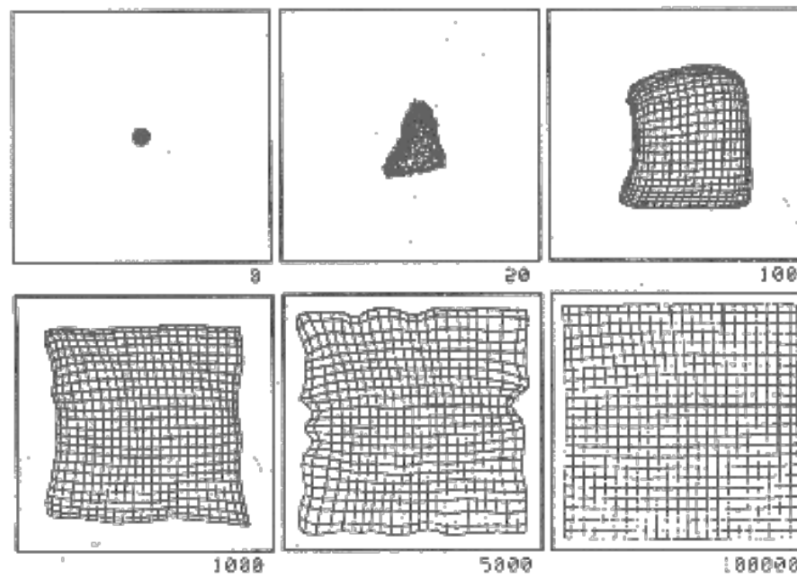


Fig. 3.4. Reference vectors during the ordering process, square array. The numbers at lower right-hand corner indicate learning cycles

Figure 19.6: Training a SOM on uniform 2D data: the reference vectors (connected according to the grid) progressively unfold over the distribution.

19.4.5 Visualisation

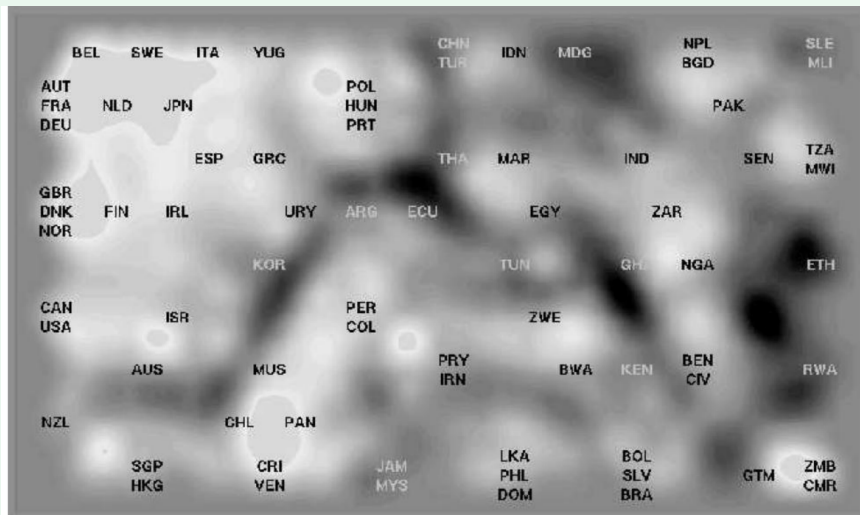
The map can be used to visualise various characteristics of the SOM and of the data:

- the **density** of the reference vectors (colour proportional to the number of “hits”);
- the **distances** between the prototypes of neighbouring units (**U-matrix**): dark colours indicate large distances between adjacent units, light colours small distances;
- Sammon maps (with lines)...

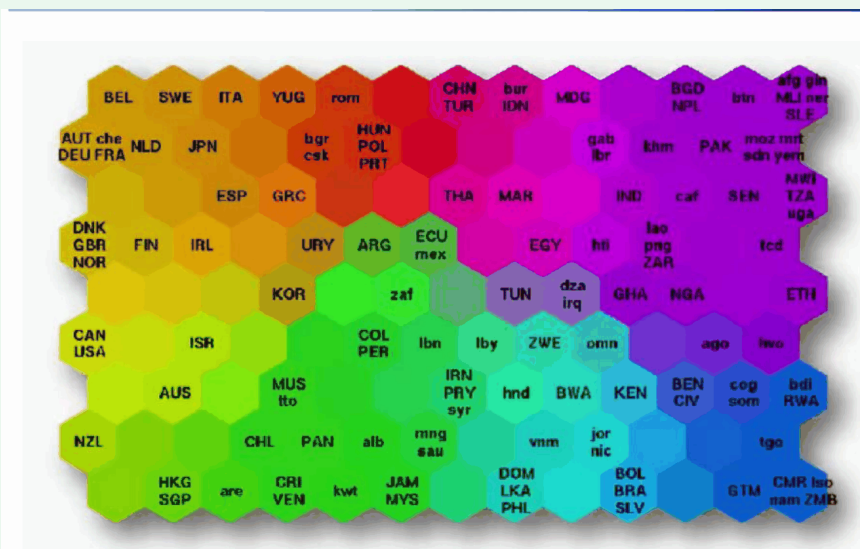
This makes the SOM easily **interpretable**.

Example — Welfare and poverty in the world (Kaski and Kohonen, 1995)

39 welfare indicators for 77 countries (World Development Report 1992). On the 13×9 map, the grey level between two units indicates the (scaled and smoothed) distance between their reference vectors: light areas are clusters of similar countries, dark areas the borders between clusters. No a priori assumption on the shape of the clusters is needed.



A smoothed distance matrix of 12 X-Y SOM cells (illustrating the clustering structure)



19.5 Evaluating a clustering (outline)

How is the result of a clustering evaluated? What distinguishes a good result from a poor one?

- Evaluation is often **subjective**: there are few “gold standards”, except in well-defined sub-domains.
- **Objective** measures, such as the quantization error (which however depends on the number of centroids).
- One does **not** trivially use the label of an underlying classification task (*purity*, with entropy, mutual information...): if labels are available, one might as well use a classifier.
- Quality of the model/clustering versus quality of the results (domain-dependent).

Other topics in unsupervised learning: hierarchical clustering (dendrograms), principal component analysis (and principal curves and surfaces), association rules, ICA, Multidimensional Scaling, t-SNE, Generative Topographic Mapping, ART networks.

Questions — Possible exam questions

- Define clustering and vector quantization. What is the quantization error?
- Derive the on-line K-means rule and move to the batch version.
- Strengths and weaknesses of K-means.
- Describe the SOM: architecture, competitive phase, cooperative phase, neighbourhood function.
- Why does the SOM preserve topology? Differences between K-means and SOM.
- How is a SOM visualised (U-matrix)?

Chapter 20

Recurrent neural networks (RNN)

This is a light introduction to **recurrent neural networks** (RNNs), which will be studied in depth in the ISPR and CNS courses. So far networks were **feedforward** (input $\rightarrow$ output direction) and worked on **vectors**. RNNs add **feedback connections** (cycles) to the topology: the presence of self-connections gives the network **dynamic properties** and a **memory** (a state) of past computations. This extends the representational capacity of the model to the processing of **sequences** (and structured data). RNNs are also biologically plausible: biological neural networks are recurrent.

20.1 Why sequential data

Models for sequences are needed when the output depends on the **history of the inputs** (for example over time: dynamic models), or when the domain consists of **variable-length sequences**:

- dynamic processes, signal processing (filters, control), robotics;
- **language**: speech recognition, NLP, formal languages, information retrieval;
- vision and reasoning about temporal events;
- **time series**: financial forecasting, signal processing;
- genomics and proteomics (bioinformatics): for example predicting the structure of a protein from its amino acid sequence;
- human activity recognition from sensor data.

RNNs (and related approaches) have been the reference for sequence processing, with state-of-the-art results in **speech recognition** and **text processing** (machine translation...), but also for “fun” applications such as music composition and text and speech generation.

20.2 From flat data to structured data

So far data were **flat** (vectors). **Structured** data are sequences, trees, graphs, multi-relational data: strings, proteins, time series, small molecules, networks.

20.2.1 The new domain: sequences and transductions

- **Data**: discrete sequences of **vectors** (with a serial order, for example time), of variable length. Each element $\mathbf{l}_t$ is a vector (e.g. $\mathbf{l}_t = [1, 0, 1, 0.7]$).
- **Task**: a **transduction** in the sequential domain, i.e. a mapping *input sequence* $\rightarrow$ *output value or sequence*.

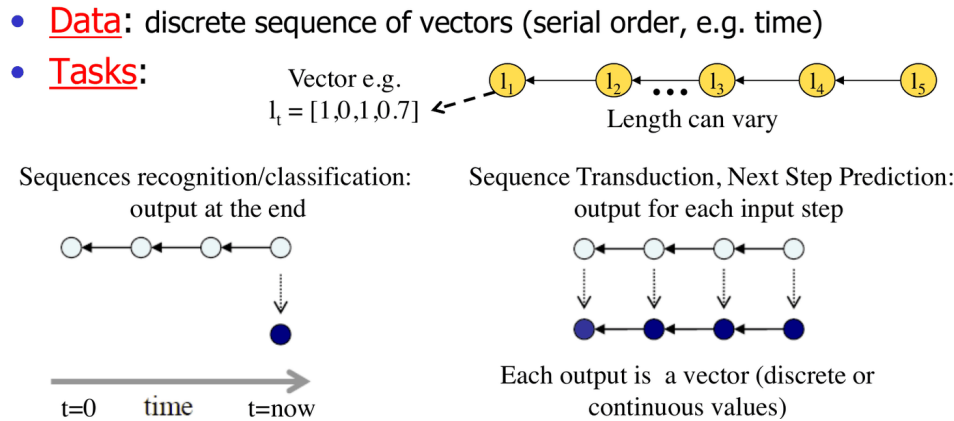
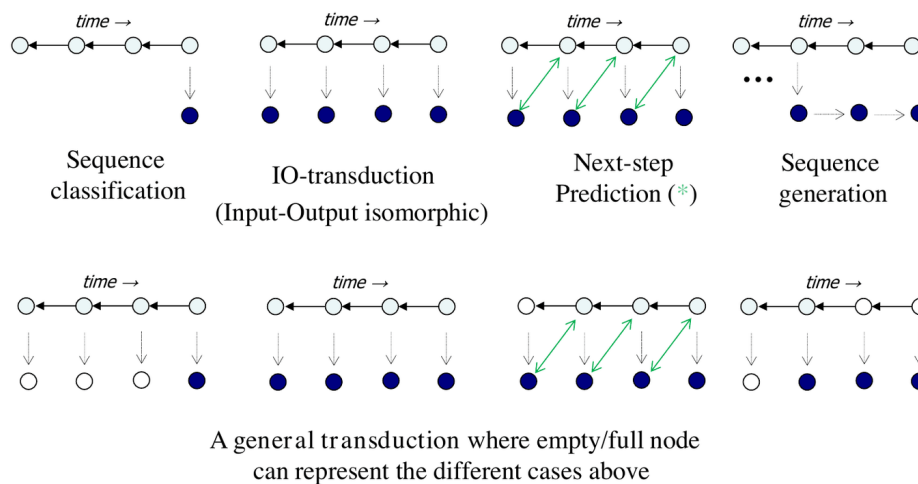


Figure 20.1: Sequence recognition/classification (output at the end) and sequence transduction (output at each step).



*Autoregressive: next-step prediction with just 1 sequence

Figure 20.2: Types of transduction: sequence classification, isomorphic IO transduction (one output per input), next-step prediction, sequence generation. Autoregressive prediction uses a single sequence.

20.3 Memory in neural networks

The goal is for the output to depend on **previous** inputs.

20.3.1 Approach 1: finite memory (IDNN)

The first approach is “spatial”, with **finite memory**: **Input Delay Neural Networks** (in the class of *Time-Delay NNs*, already seen at the beginning of CNNs). The delays are in the connections and act as a memory of the input: a finite-size **shift register**, a **sliding window** over the sequence. (CNNs extended this idea to 2D images.)

- The window size must be **known in advance**.
- The number of weights **depends on the buffer size**.
- If the buffer contains enough information, the problem is solved with normal MLP training (for example NETtalk, 1988).

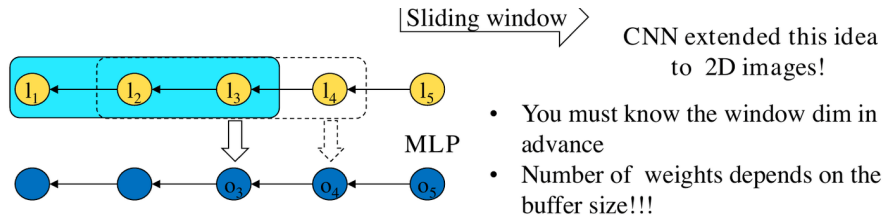


Figure 20.3: IDNN: a fixed-size sliding window over the sequence, processed by an MLP

20.3.2 Approach 2: recurrent units

A **recurrent neural unit** has a feedback connection (a *self-loop*) and uses both the current input and the **state** information:

$$x(t) = \tau(l(t), x(t-1)) = f(w^T l(t) + \hat{w} x(t-1) + \theta), \quad x(0) = 0,$$

where $l(t)$ is the input (label) at time t , f is the (sigmoidal) activation, $\hat{w}$ is the **recurrent weight**, θ the bias, and $x(t-1)$ is the state at the previous step, provided by a unit delay q^{-1} . The new part compared to an ordinary neuron is the term $\hat{w} x(t-1)$.

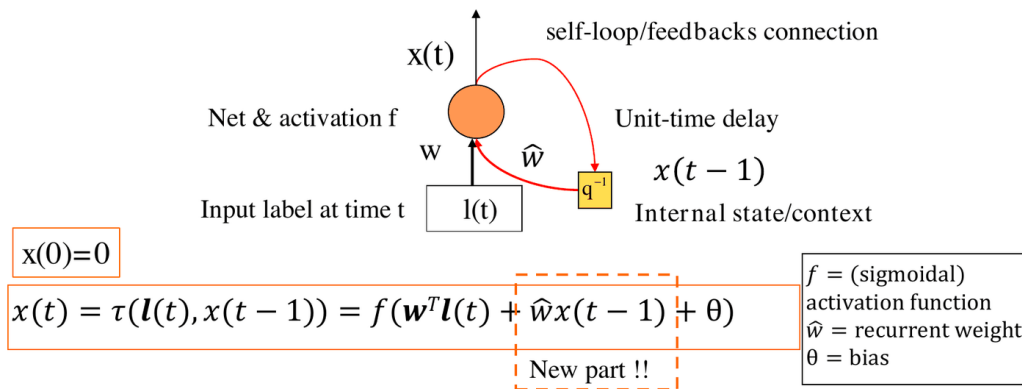


Figure 20.4: A recurrent unit: the self-loop with unit delay brings the previous state as an additional input.

- The **state** summarises past information (the **context**): it changes value following the flow of inputs.
- The **encoding of the past is adaptive**, because it depends on the free weights of the model.

Example — Exercise: counting the “1”s

Build an RNN with a single unit (finding the values of w and $\hat{w}$) that returns the number of “1”s received so far in a sequence of 0s and 1s.

Solution: a linear unit with $w = 1$, $\hat{w} = 1$, $\theta = 0$, i.e. $x(t) = l(t) + x(t-1)$. With input 1, 0, 1, 1, 0, 1 the state is 1, 1, 2, 3, 3, 4: **the state accumulates the sum of the “1”s** received. The RNN summarises the information of the past sub-sequence, something an IDNN **cannot do** for variable-length sequences (its memory is limited to the window).

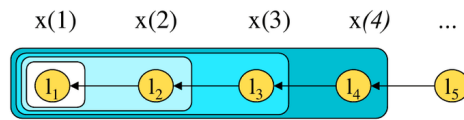
20.3.3 State transition systems

In general, a recurrent model is a **state transition system**:

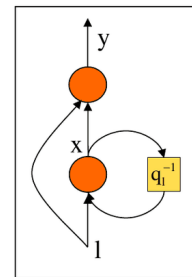
$$\begin{cases} x(t) = \tau(x(t-1), l(t)) \\ y(t) = g(x(t), l(t)) \end{cases} \quad x(0) = 0.$$

Given $x(0) = 0$

$$\begin{cases} x(t) = \tau(x(t-1), l(t)) \\ y(t) = g(x(t), l(t)) \end{cases}$$



State formation over time



Graphical model

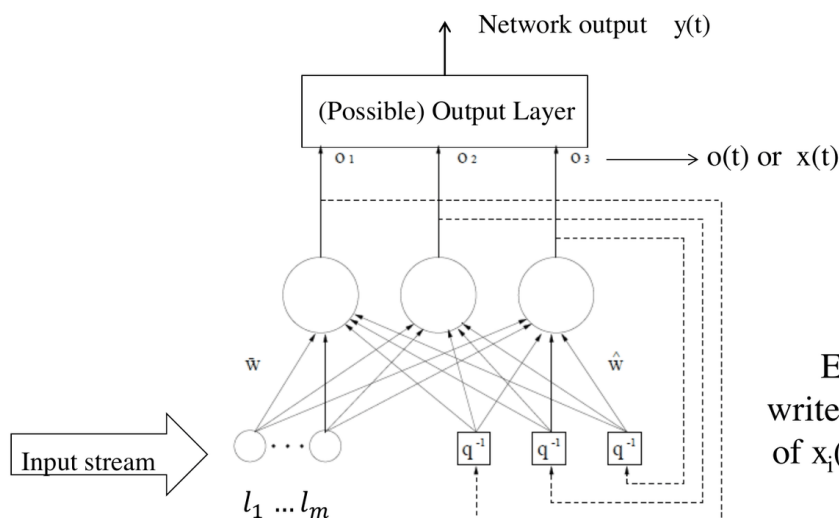
E.g. also HMM

Figure 20.5: State transition system: the state at time t encodes the past sub-sequence.

- τ is the **state transition function** (*next-state function*), implemented by a neural network.
- The state summarises past inputs: it is an **encoding** (embedding) of the sub-sequences.
- The state has **continuous** (real) values (unlike, for example, automata or HMMs, which have discrete states).
- $\mathbf{x}(t)$ can be a set of states, to compose a network of recurrent units.

20.3.4 A recurrent network (Simple RNN)

With many hidden recurrent units we obtain Elman's **Simple RNN**: each hidden unit receives the input and the previous states of **all** hidden units (through the delays), and an (optional) output layer produces $\mathbf{y}(t)$.



Exercise:
write the equation
of $x_i(t)$ (and then
of $\mathbf{x}(t)$)

Figure 20.6: A Simple RNN (Elman) with several hidden recurrent units.

Example — Exercise: the equations

For hidden unit i :

$$x_i(t) = f \left(\sum_{j=1}^m w_{ij} l_j(t) + \sum_k \hat{w}_{ik} x_k(t-1) + \theta_i \right),$$

and in vector form $\mathbf{x}(t) = f(\mathbf{W}\mathbf{l}(t) + \hat{\mathbf{W}}\mathbf{x}(t-1) + \boldsymbol{\theta})$.

20.3.5 Properties

Even the simple RNN is already very powerful:

- it is a **universal approximator** of non-linear dynamical systems;
- it is **Turing-equivalent** (it can simulate any automaton);
- it is a (non-autonomous) non-linear dynamical system, which can be studied with dynamical systems theory (chaos, fractals...).

Recurrent models rest on three assumptions:

- **Causality**: the output at time t_0 depends only on the inputs at times $t \leq t_0$. It is necessary and sufficient for having an internal state.
- **Stationarity**: time invariance (after training): the transition function τ is **the same** at every instant, regardless of the length of the sequences. This is what makes it possible to process variable-length data with a fixed-size model.
- **Adaptivity**: the transition functions are implemented by neural networks with free weights, hence **learned from data**.

20.4 Learning: unfolding

The same model is **unfolded** in time along the input sequence: a feedforward MLP (the **encoding network**) equivalent to the RNN over the k steps of the sequence is built, with a **replica of the model for each step**. By stationarity, the replicas **share the weights**.

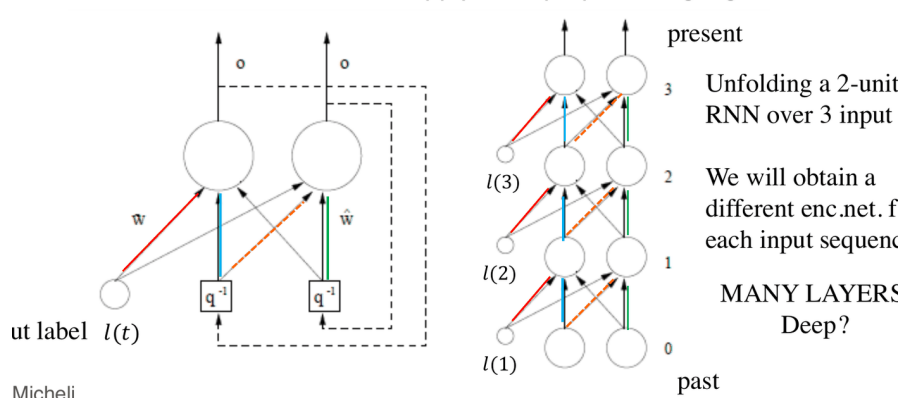


Figure 20.7: Unfolding of a 2-unit RNN over 3 input steps: a many-layer feedforward network is obtained, different for each input sequence.

Tip — The key point

Backpropagation can be applied to the unfolded encoding network! A different network is obtained for each input sequence, with **many layers** (as many as the steps): it is de facto a **deep** network, with weights shared across layers.

Supervised learning algorithms for RNNs must take into account all the encoding transitions developed by the model at each step:

- **BPTT** (*Back-Propagation Through Time*);
- **RTRL** (*Real-Time Recurrent Learning*).

They compute in different ways the same gradient of the output errors on the equivalent unfolded network.

Warning — Long-term dependencies

Since the unfolded network is very deep, the **vanishing gradient** makes it hard to learn **long-term dependencies** (when the output depends on inputs very far in the past).

20.4.1 Advanced models

It is a rapidly growing research field, with many variants:

- **LSTM** (*Long Short-Term Memory*): they try to solve the vanishing gradient with “**gate**” **units** able to select the flow of the past and of the gradient. They were born for **long** dependencies: they are not suited to short series, so, despite their popularity, they should not be used uncritically;
- **GRU** (*Gated Recurrent Units*): a simplified LSTM;
- **BRNN** (bidirectional RNNs): they consider both the left and the right context;
- deep networks and RNNs are often merged;
- against the vanishing gradient: Hessian-free optimisers, pre-training techniques...

They are included in the main libraries (PyTorch, Keras, Matlab...). Some data on their spread: in 2015 LSTMs were used in 2 billion Android phones and 1 billion iPhones (Siri); in 2016 almost 30% of the inference computing power of Google’s datacentres was used for LSTMs; in 2017 they were in Alexa and Facebook (30 billion translations per week). A curious example: generating the **handwritten** version of a text by predicting one (x, y) point of the pen trajectory at a time (Graves, 2013).

20.5 Related approaches

- SOMs and unsupervised networks for sequential domains.
- **Hidden Markov Models** (they model the probability distribution of state transitions).
- **Transformers**.
- **Randomised RNNs: Reservoir Computing** (ESN, *Liquid State Machine*).
- Distance-based models (string matching), string kernels, grammatical inference, ILP

20.5.1 Transformers and attention

A **transformer** (2017) is a deep learning architecture based on the parallel **multi-head attention** mechanism. Attention allows the model to **access any previous point** of the sequence; encoders and/or decoders plus feedforward networks complete the architecture for different purposes.

The **attention layer** weighs all previous states according to a **learned relevance** measure, potentially providing information even about very distant tokens. It has proved particularly useful in translation, where a distant context can be essential for the meaning of a word. First used to improve RNNs, then also on its own (“*Attention is all you need*”). All tokens are processed **simultaneously**, computing weights between their representations (embeddings) in successive layers: they are simply **scaled dot-product** units between *query* vectors and word vectors, which look for the highest correlations between the words of a sentence (for example in “see

that girl”, “that” is associated with “girl”). It is also applied to image *patches* instead of words (*vision transformer*).

20.5.2 Randomised recurrent networks: Echo State Networks

Can we directly exploit this state machine to encode sequences, and then use the resulting embedding to learn the output mapping? Yes, **even with randomly connected recurrent networks with fixed** (untrained) **weights**! This is the class of **Reservoir Computing (RC)**: Echo State Networks, *liquid state machines*, *fractal machines* (see [18 - Randomized neural networks](#)).

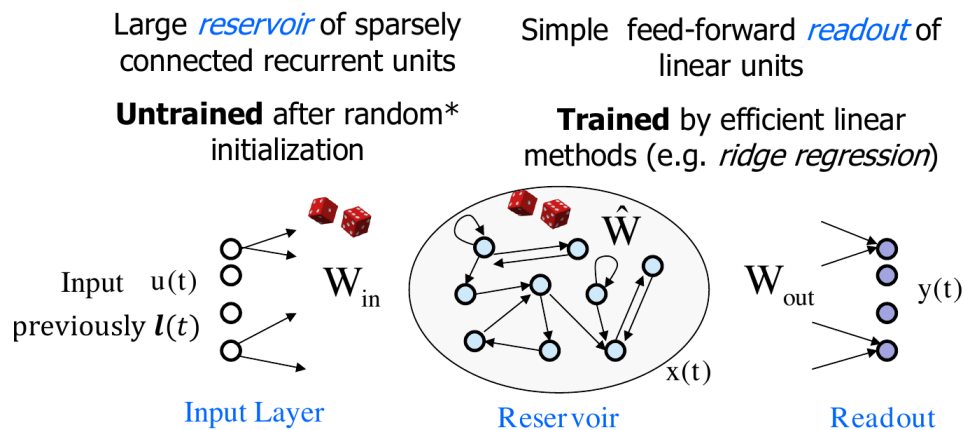


Figure 20.8: An Echo State Network: untrained random reservoir and trained linear read-out.

Definition — Echo State Network (ESN)

An ESN consists of: - a large **reservoir** of sparsely connected recurrent units, **not trained** after a random initialisation; - a simple feedforward **readout** of **linear** units, trained with efficient linear methods (e.g. **ridge regression**).

Echo State Property: the state transition function must be **contractive** (the spectral radius of the reservoir weights is bounded, i.e. the stability of the dynamical system is guaranteed). In this way the recurrent state depends asymptotically (like an “echo”) **only on the input history**, and not on the initial conditions of the state.

ESNs are **very efficient** (no training of the recurrent units) and solve tasks well under certain conditions.

Markovian organisation of the state space

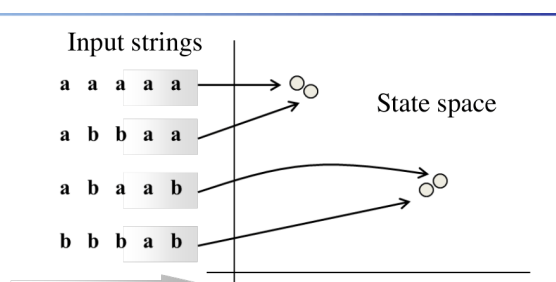


Figure 20.9: The reservoir projects the sequences into the state space in a “Markovian” way: sequences with the same suffix end up close together.

The reservoir acts as a **projection** (embedding) of the input sequences into the state vector $\mathbf{x}(t)$, and it is intrinsically able to **discriminate sequences based on their suffix**, without training the recurrent weights: the states are closer the longer the **common suffix** (recent inputs matter more, because of contractivity). This makes it easy to learn **Markovian** tasks (also for trained RNNs, but ESNs are much more efficient), and it is an **architectural bias** that also holds for recurrent models with learning (Gallicchio, Micheli, 2011).

Deep Reservoir Computing. Several reservoirs can be stacked (**DeepESN**, with many results from the CIML group in Pisa), combining the power of deep RNNs with the efficiency of ESNs. The layered organisation shows advantages in terms of **multiple time scales** and **richness of dynamics**: in frequency analysis, the higher layers focus on lower frequencies (wider time scales), forming a hierarchical temporal representation.

20.6 Towards structured domains

Can the recurrent approach be extended to **trees**? Yes: **recursive neural networks** (RecNN). Encoding proceeds **bottom-up** (from the leaves to the root), and the state of each vertex depends on its label and on the states of its **children** (extending the transition function to several states).

- Extending states for children of vertices of the tree

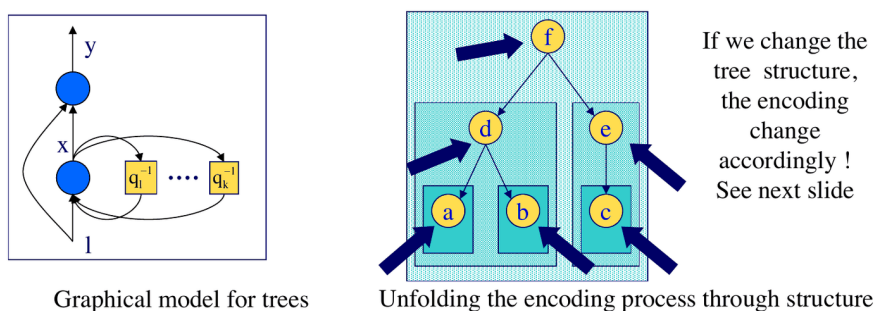


Figure 20.10: Graphical model for trees and unfolding of the encoding along the structure: if the structure of the tree changes, the encoding changes accordingly.

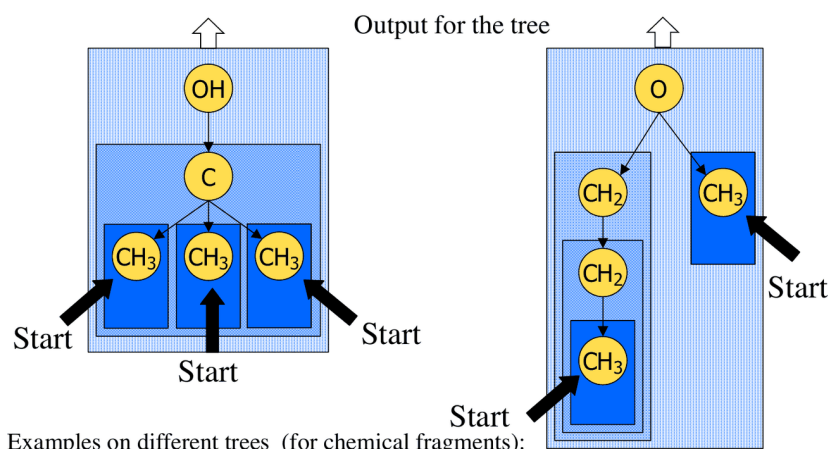


Figure 20.11: A recursive network applied to different trees (chemical fragments): the units are the same for all the vertices of a tree and for all the trees of the dataset (weight sharing).

Summary — Summary

RNNs add **states** and **feedback** to networks, becoming **state transition systems** able to process variable-length sequences. They rest on causality, stationarity and adaptivity; they are trained by **unfolding** them in time and applying backpropagation (BPTT, RTRL), but they suffer from the **vanishing gradient** on long dependencies (hence LSTM and GRU). Alternatives and developments: transformers (attention), Echo State Networks (random reservoir + linear readout, with a Markovian bias), recursive networks for trees and graphs.

Questions — Possible exam questions

- Why are models for sequential data needed? What is a transduction?
- Difference between IDNN and RNN; why can an IDNN not count the “1”s in arbitrary sequences?
- Write the equation of a recurrent unit and of a Simple RNN.
- What are the assumptions of causality, stationarity and adaptivity?
- What is unfolding and how does it allow backpropagation to be used?
- What is an Echo State Network and what is the Echo State Property?
- How are RNNs extended to trees?

Chapter 21

Learning on structured data and graphs

The last lecture of the course is an advanced introduction to **learning in structured domains** (SD, *Structured Domains*) and in particular to **learning on graphs** with **Deep Graph Networks** (DGNs). It is also an overview of the research of the CIML group in Pisa, which has made pioneering contributions to this field. The guiding questions: can deep learning on graphs be done **efficiently** (without fully *end-to-end* training)? Is **depth** in DGNs needed, and why does it cause problems? What is the relationship between model depth and training difficulty?

21.1 Why structured data and graphs

Because data have relationships. Examples of graphs and networks:

- abstractions of images and networks for scene understanding;
- **social networks**, **knowledge graphs**;
- syntactic analysis of language (parse trees);
- transport networks (for example traffic prediction in Google Maps, by DeepMind);
- **small molecules** (drug design), biological networks (proteins);
- logical terms, brain network modelling.

Examples. Graphs & Networks

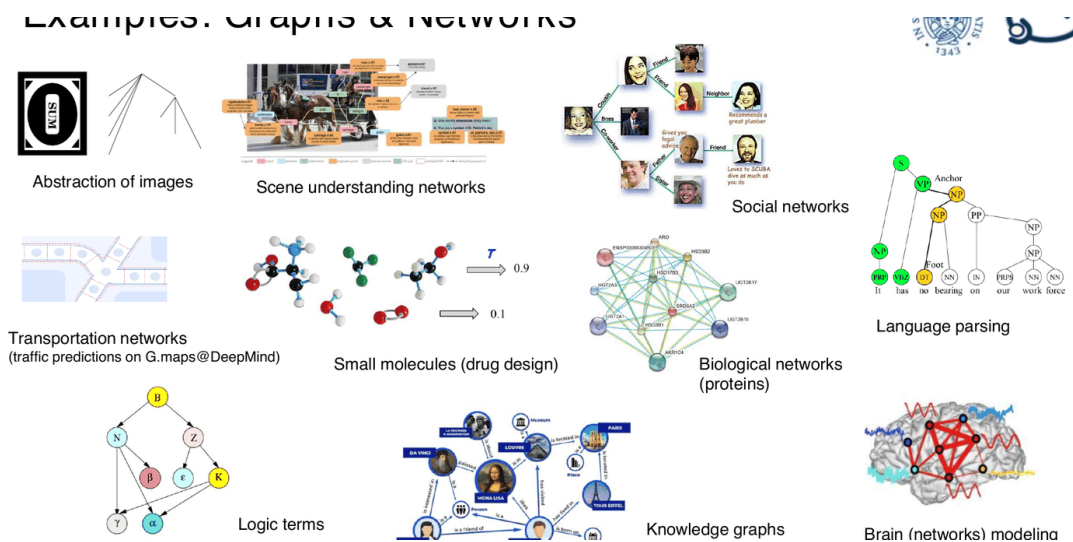


Figure 21.1: Examples of graphs and networks in different domains.

21.1.1 The scenario

The course has followed this progression: from **vectors** (feedforward networks) to **sequences** (RNNs) to **structures** (trees, graphs). Deep and “wide” (*deep and wide*) approaches represent the input at several levels of abstraction but have a high computational cost; **randomised** or **incremental** networks offer efficiency.

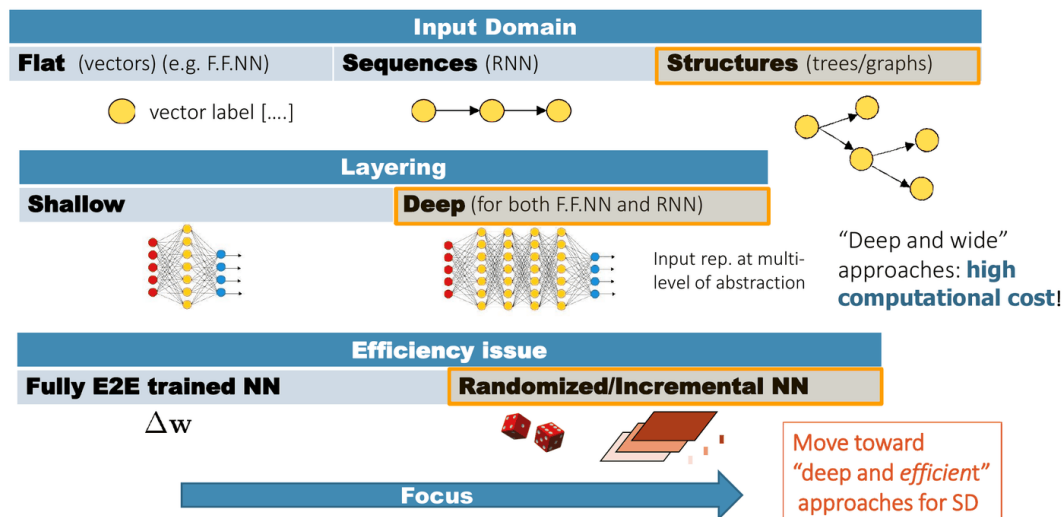


Figure 21.2: The scenario: from flat to structured input domains, from shallow to deep models, from end-to-end to randomised or incremental training. The goal is to obtain “deep and efficient” approaches for structured data.

21.1.2 A bit of history

Neural networks for structured data have a largely **Italian** history:

- since the late 1990s, **recursive neural networks** for trees and directed acyclic graphs (Sperduti, Starita, Frasconi, Gori, Hammer, Micheli, Bianchini, Scarselli, Hagenbuchner...): recursive networks (1997–2000), Cascade Correlation for structures, SOMs for structures (2003–2005), bottom-up Hidden Tree Markov Models (2012), Tree ESN and deep reservoir computing for trees (2010–2018);
- the extension to **general graphs**, pioneered between Pisa and Siena (2005–2009), with two approaches:
 1. **recursive**: **GNN** (Scarselli et al., 2009) and **GraphESN** (Gallicchio, Micheli, 2010);
 2. **layered / convolutional**: **NN4G** (Micheli, 2009) in a constructive version, later continued as convolutional networks for graphs (GCN, etc.).

Since 2015 it has been an **exploding** area worldwide, with dozens of papers at all the main conferences and real applications, under different names: *Graph Representation Learning*, *learning on graphs*, *deep learning for graphs*, *Graph Neural Networks* (GNN), *Graph Convolutional Networks* (GCN), and the framework of **Deep Graph Networks** (DGN; Bacciu, Errica, Micheli, Podda, 2020).

21.2 Deep Graph Networks

21.2.1 The graphs considered

We consider **labelled graphs**: each **node** (vertex) v has a **vector label** $\mathbf{l}(v)$ (e.g. $[1, 0, 1, 0.7]$), each **edge** (link) can have a label (e.g. a position) and can be directed; there can be **cycles**. A is the **adjacency matrix** of the graph.

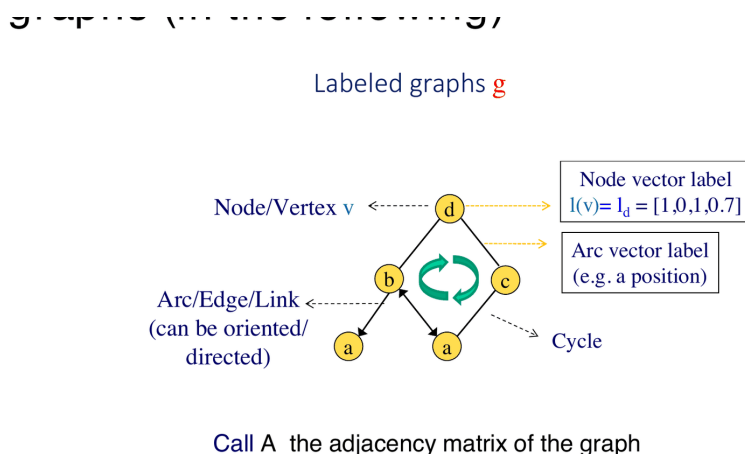


Figure 21.3: A labelled graph: vector labels on nodes and edges, possible cycles.

21.2.2 The representation problem

There was no **systematic** way (valid for any task) to extract features or relationship metrics between structured examples. It is an instance of **representation learning**, extended to structured domains.

- **Feature**-based representations are incomplete or strongly task-dependent (for example topological indices in chemistry).
- Representations with **adjacency matrices** or other fixed-size representations have problems: over-sizing or incompleteness (waste with padding or loss of information), **alignment** between different graphs (which node of one graph corresponds to which node of the other?), topological order (which makes generalisation difficult).

Tip — Moving the models towards the data

“The ability to treat the inherent nature of the input data is the key to a successful application of ML methodologies.” Instead of **bringing the data to the models** (turning graphs into vectors or trees into sequences, with alignment problems and loss of information), one **brings the models to the data**.

Example — Molecules and transductions

In **QSPR/QSAR** one wants to correlate the chemical structure of molecules with their properties: Property/Activity = $T(\text{Structure})$, a property value (regression) or “toxic yes/no” (classification). Molecules **are not vectors**: they are more naturally represented by variable-size structures. Can we predict directly from the structures?

21.2.3 Transductions on graphs

The goal is to learn a **transduction** T from a structured domain to a discrete or continuous space, starting from examples (graph _{i} , target _{i}) with variable-size graphs.

- **Structure** → **structure** (input-output isomorphic): **node-level** tasks (e.g. node classification).
- **Structure** → **scalar/element**: **graph-level** tasks (graph regression or classification).
- In general also non-isomorphic; and **edge-level** tasks (e.g. *link prediction*).

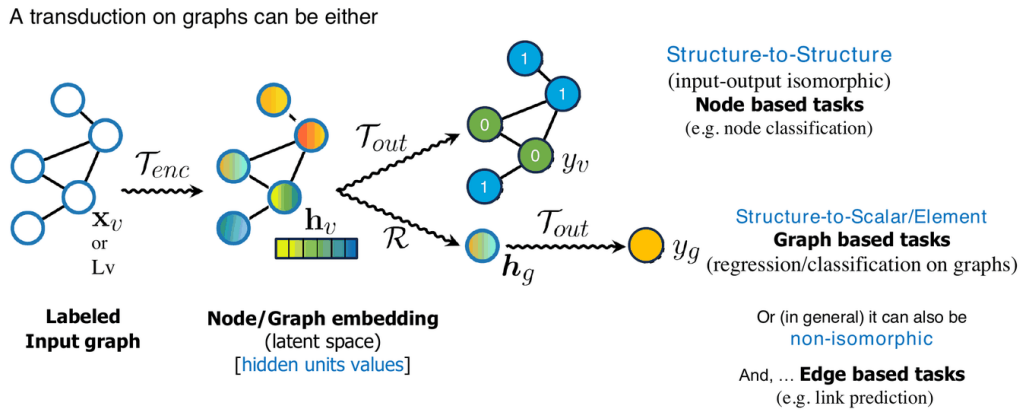
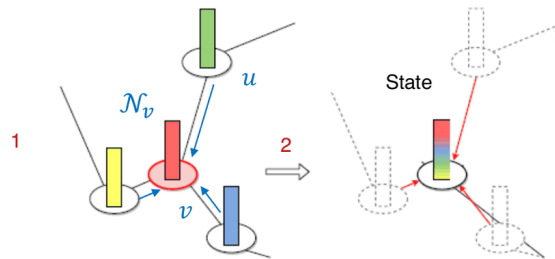


Figure 21.4: Transductions on graphs: the graph is encoded into a node embedding, from which per-node or whole-graph outputs are obtained.

21.2.4 Message passing

How does a DGN work? The abstract concept is **message passing**: nodes “talk” to each other, exchanging messages to inform one another and become aware of the surrounding structure.

1. Each node computes a **message**, a function of its own label, of its neighbours $\mathcal{N}_v$ and of the edges between them, and sends it to its neighbours.
2. Nodes **aggregate** the received messages with a **permutation-invariant** function (the order in which they arrive does not matter).
3. Each node **updates** its own attributes (the **state**) by combining the messages with the free parameters W .



Nodes “talk” each other to inform about themselves and to be aware of the surrounding structure. h_v

Figure 21.5: Message passing: messages from neighbours, aggregation, state update.

The state does not depend only on the local neighbourhood: **iterating** message passing, information **spreads** from each node to all the others. Finally, an output can be emitted **for each node**, or **for the whole graph** through a permutation-invariant aggregation of the node representations, called **global pooling** (for example sum or mean).

21.2.5 The link with CNNs

Both CNNs and convolutional networks for graphs visit each node of the data, but:

- in the **CNN** the convolutional kernel is applied on a **regular 2D grid**: a weighted average of the pixels in the window is computed, and the neighbours of a pixel are **ordered** and **fixed in number**;
- on **graphs** the CNN cannot be applied directly: the neighbours of a vertex are **unordered** and **variable in number**. The receptive field is limited to the **neighbours**, **weight sharing**

is used in a permutation-invariant way (abstracting from any node ordering), and the local neighbourhood is extended through **layering**.

CNN and GCN both visit each node of the input data, but ...

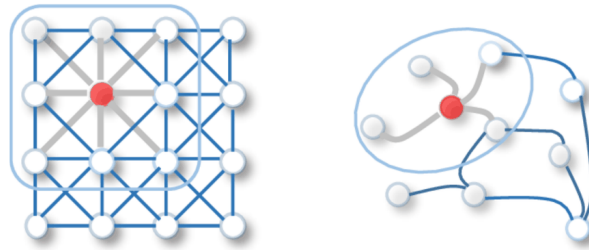


Figure 21.6: Convolution on a grid (CNN) and on a graph.

21.2.6 The general formula

Message passing is iterated over layers (or iterations) l :

$$\mathbf{h}_v^{(l)} = \text{AGG}_{W^{(l)}}(\mathbf{L}_v, \mathbf{h}_v^{(l-1)}, \{\mathbf{h}_u^{(l-1)} : u \in \mathcal{N}(v)\}), \quad l = 1, \dots, L,$$

where AGG aggregates (propagates) the messages from the neighbours u with a **permutation-invariant** operator over the set of neighbours (e.g. a sum) and combinations of the unit activations, W are the free parameters, $\mathbf{L}_v$ is the node label, and $\mathcal{N}$ is given by the adjacency matrix A . Applying this computation to each node corresponds to a **parallel, unordered visit** of the graph at each iteration.

There are many variants:

- **GCN** (Kipf, Welling, 2017): $\mathbf{h}^{(l)} = \text{ReLU}(\hat{A} \mathbf{h}^{(l-1)} W^{(l)})$, with $\hat{A}$ the degree-normalised adjacency matrix;
- **GraphConv** (Morris et al., 2019): $\mathbf{h}^{(l)} = f(W_1^{(l)} \mathbf{h}^{(l-1)} + W_2^{(l)} A \mathbf{h}^{(l-1)})$;
- **GAT** (Veličković et al., 2018): **attention**-based aggregation.

(Many are available in *PyTorch Geometric*.)

Tip — Why it is relevant for deep learning

DGNs are **intrinsically deep**: the process is not local. Iterating over the layers, the latent representations of the nodes progressively include the **contextual** information of increasingly distant nodes (diffusion).

21.2.7 NN4G and GCN: iterations as layers

In convolutional networks for graphs (GCN, NN4G), **each message passing iteration is a new layer**, and for each node the context is **composed** across the layers:

- the **receptive field** extends incrementally as the layers increase;
- **depth serves the growth of the context**;
- by training the model one learns **how to represent the graph**: end-to-end for general GCNs, **layer by layer** (incrementally) for NN4G, with an automatic number of layers and without vanishing gradient problems.

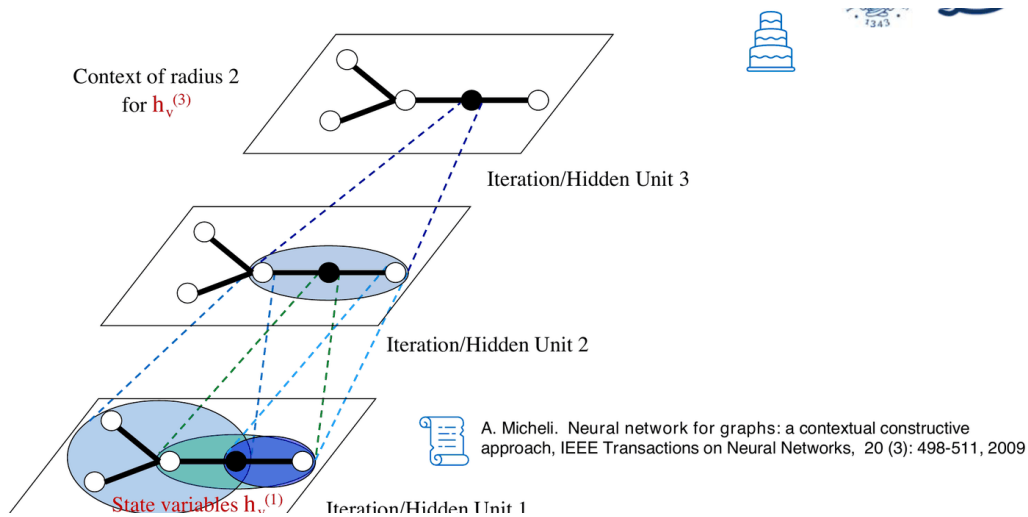


Figure 21.7: Evolution of the (compositional) context through the iterations: at the third layer the state of the node has a context of radius 2.

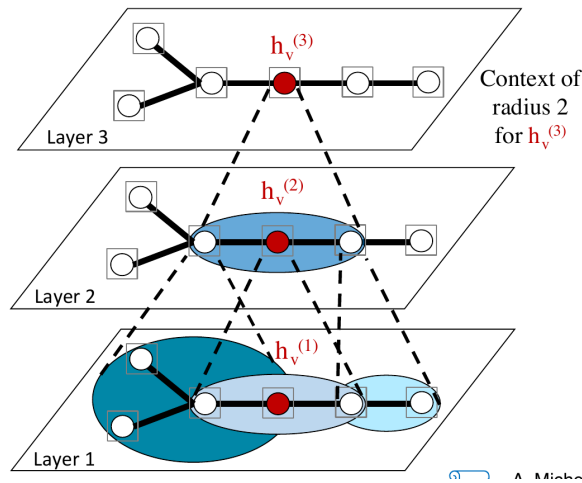


Figure 21.8: GCN and NN4G: each message passing iteration is a new layer.

NN4G (Micheli, 2009) computes a state variable for each layer and each vertex:

$$h_v^{(1)} = f\left(\sum_{j=0}^{|L_v|} \bar{w}_{1j} L_j(v)\right), \quad h_v^{(l)} = f\left(\underbrace{\sum_{j=0}^{|L_v|} \bar{w}_{lj} L_j(v)}_{\text{label of } v} + \underbrace{\sum_{j=1}^{l-1} \hat{w}_{lj} \sum_{u \in \mathcal{N}(v)} h_u^{(j)}}_{\text{context from neighbours, from all previous layers}}\right), \quad l = 2, \dots, N.$$

It is not recursive: each layer uses the states of the **previous** layers. Units are added one at a time (in Cascade Correlation style), and the outputs of the various layers are combined to produce the result on the graph.

21.2.8 GNN and GraphESN: recursive approach

In **recursive** approaches (GNN, GraphESN) the diffusion process is similar, but l is the number of **iterations on the same layer** (or layers with shared weights). It is a **dynamical system**: typically **convergence to a fixed point** is required, used as the graph embedding, and for this reason the message passing dynamics is constrained to be **contractive**.

- **GNN** (Scarselli et al., 2009): with training of the embedding.

- **GraphESN** (Gallicchio, Micheli, 2010): it extends Echo State Networks: the message passing function **is not trained** (it is a random reservoir), but the parameters of the dynamical system are controlled directly.

$$\mathbf{h}_v^{(l)} = \tanh \left(\sum_{j=0}^{|L_v|} \tilde{w}_{lj} L_j(v) + \sum_{u \in \mathcal{N}(v)} \hat{W} \mathbf{h}_u^{(l-1)} \right),$$

iterated until convergence; the output for a graph is obtained with a global pooling X and a **linear readout** trained with the pseudoinverse or ridge regression: $y(g) = W_{out} X(\mathbf{h}(g))$.

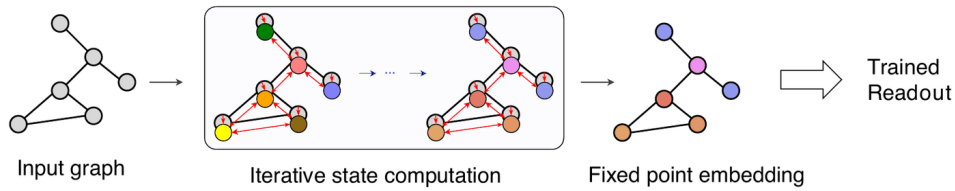


Figure 21.9: GraphESN: iterative computation of the states up to a fixed-point embedding, then trained readout.

21.3 Open problems and some proposals

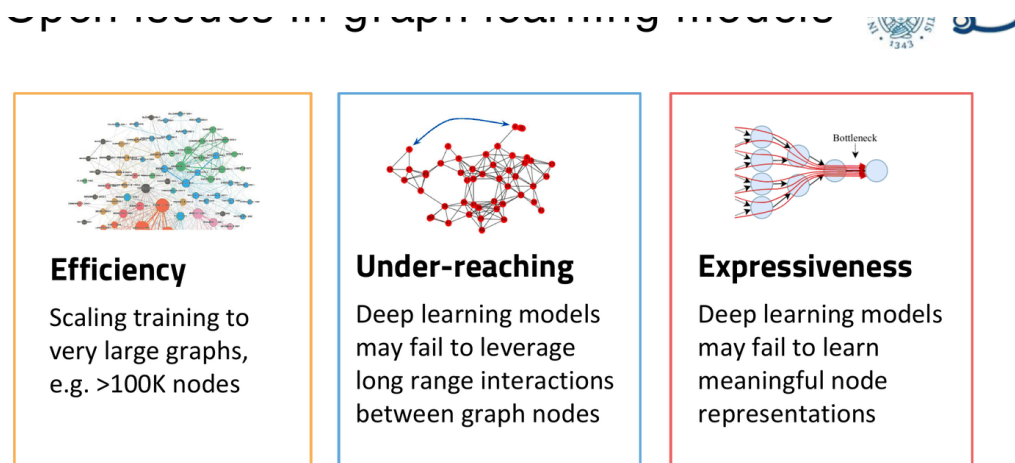


Figure 21.10: Open problems in graph learning models: efficiency, under-reaching, expressiveness.

21.3.1 Efficiency: beyond end-to-end backpropagation

- **GraphESN for node classification:** on very large graphs (e.g. Twitch-gamers, 168,000 nodes) it achieves accuracy equal to or higher than most end-to-end trained models, with a speed-up of up to 10 times, and training scales easily beyond 100,000 nodes.
- **NN4G+** (automatic architecture design): the network is built **incrementally**, with a synergistic double expansion. It is trained **one unit at a time** (no end-to-end backpropagation across layers); a layer is expanded as long as the validation accuracy improves, and new layers are added as long as it improves. The resulting architectures are adapted to the task, consistently deeper and leaner compared to an explicit architecture search, and they are built in minutes instead of hours (more than 10 times faster, without loss of accuracy).

21.3.2 The problems of depth: the “over-”

Depth is needed, but message passing has biases and problems:

- **Over-smoothing** (Li et al., 2018): accuracy typically drops beyond 4–5 layers; node representations **collapse** in the deep layers. Message passing acts as a **diffusion**, and the high-level representations of the nodes all become similar.
- **Over-squashing** (Alon, Yahav, 2021): it is a **bottleneck**. The receptive field grows **exponentially** with depth, but all this information must be compressed into a fixed-size embedding per node. The **sensitivity** of a node’s representation to the input features of distant nodes, measured by the Jacobian $\partial \mathbf{h}_v^{(L)} / \partial \mathbf{x}_u$, decreases exponentially with the number of layers.
- For end-to-end training, the **difficulty of backpropagating the gradient** through many message passing layers.

The interaction among these phenomena is an open research problem. The effects:

1. **Under-reaching**: the over-* prevent having the receptive field needed to learn interactions between distant nodes.
2. **Heterophily** in node classification. In **homophilous** graphs nodes in the same neighbourhood mostly share the same class; in **heterophilous** graphs nodes of the same class are generally far apart, and predictions based mainly on immediate neighbours can be misleading. These are potentially hard tasks for standard message passing, because of the over-* and the bias towards “homogeneous” graphs.

- Heterophilic graphs: nodes belonging to the same class are generally further apart (predictions that rely chiefly on a node’s immediate neighbours can be misleading)

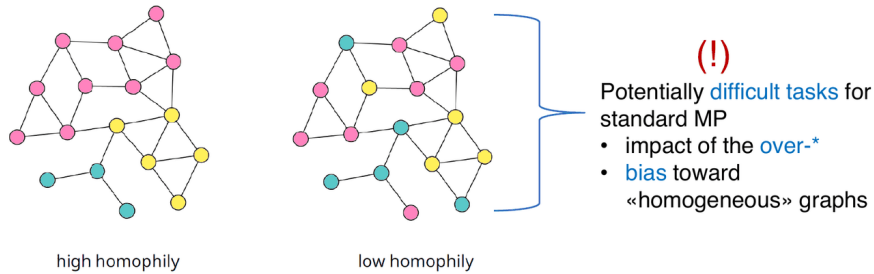


Figure 21.11: High- and low-homophily graphs: the latter are hard for standard message passing.

Analysis tools

- **Models that separate the bias of message passing from the problems of end-to-end training**: NN4G (incremental, without propagating the gradient across layers) and GraphESN (no training of the embedding, direct control of the spectral norm $\|W\|$, i.e. of the Lipschitz constant of the recursive message passing function). Since

$$\left\| \frac{\partial \mathbf{h}_v^{(L)}}{\partial \mathbf{x}_u} \right\| \leq \prod_{l=1}^L \|W^{(l)}\| \cdot (A^L)_{u,v},$$

imposing $\|W\| > 1$ prevents the sensitivity with respect to the input from vanishing exponentially (over-squashing).

- **Spectral analysis** of the layers. “Frequency” on a graph is the variability of the signal (labels, embeddings) between neighbouring nodes. The **Dirichlet energy** $E(\mathbf{h}) = \sum_v \sum_{u \in \mathcal{N}_v} \|\mathbf{h}_v - \mathbf{h}_u\|^2$ measures how much the signal varies over the graph (spatial view) and, through the graph Fourier transform (obtained from the decomposition of the Laplacian $L = D - A$), how the “mass” of the signal is distributed over the frequency spectrum: a low-frequency (smooth) signal has low energy. Heterophilous graphs have **high-frequency** signals, and heterophilous tasks require **preserving high frequencies**.

On the **Chameleon** dataset (a network of Wikipedia pages, 2277 nodes, 31,400 edges, homophily 0.23), a “stress test” for heterophily, NN4G and GraphESN **better preserve the energy** across layers, and while the accuracy of the other models drops as message passing increases, theirs does not. In an experiment with 12 layers: GraphESN 73.7%, NN4G 68.3%, GCN 57.8%, GAT 55.0%, GraphConv 33.8%.

21.3.3 Ongoing extensions

- **Dynamic/temporal graphs**: relationships or node values that evolve over time (spread of infections, content on social networks, signals on nodes). **Dynamic Graph Echo State Networks** implement a spatio-temporal convolution with a multi-layer reservoir, as effective as fully trained temporal graph networks, but with a 100-fold speed-up in training and 10-fold in inference.
- **Graph generation**: learning the distribution $p(G)$ of a set of graphs in order to sample new ones (e.g. fragment-based molecule generation), useful for fairness, robustness, privacy.
- **Explainability** (XAI) for graph networks: for example identifying the benzene rings that determine a classification, or the known structural fragments (such as the Michael acceptor) that affect the activation of the P53 pathway in the Tox21 data.
- **Network quantification**: predicting the **prevalence** of the classes among the unlabelled nodes instead of classifying them individually (voting, market research, epidemiology).
- **Bioinformatics**: predicting dynamic properties (robustness, sensitivity) of biochemical pathways and protein-protein interaction networks represented as graphs.

21.4 Other approaches: kernels for structures

Kernel methods extend easily to graphs of different types (sequences, trees, directed or undirected graphs, cyclic or acyclic), because it is enough to define a dot product $k(x, x') = \langle \phi(x), \phi(x') \rangle$, i.e. a **similarity** between data of any type, with an implicit embedding in a Euclidean space.

- **Marginalised kernels**: the similarity between two graphs is based on the common label paths obtained with random walks; the kernel is the dot product of the count vectors, averaged over all possible paths. The cost typically scales quadratically with the size of the graph.
- **Convolution kernels**: objects are decomposed into **sub-structures** and the kernel is defined by combining kernels between the sub-structures.

Convolution Kernels

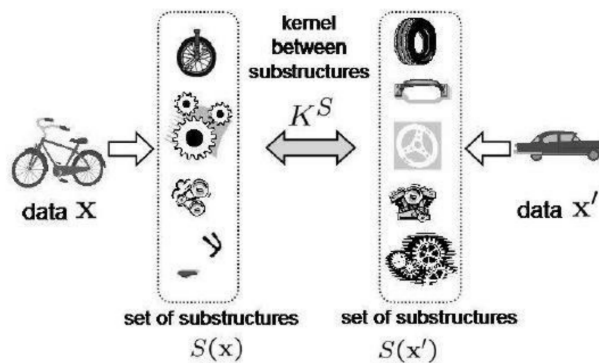


Figure 21.12: Convolution kernel: the similarity between objects is computed from the similarity between their sub-structures.

Warning — Limits of kernels for structures

- **Efficiency:** the cost is critical, because of the decomposition into sub-structures.
- **Adaptivity:** they are knowledge-driven methods, with a similarity measure **fixed before learning**: the encoding in the feature space is not learned. The advantage is being able to exploit prior knowledge for a specific problem.
- For structured domains **universality is not known**: the choice of the kernel is critical, and computing a **complete** kernel for graphs is at least as hard as the **graph isomorphism** problem. (Adaptive kernels are being studied.)

Summary — Conclusions

- Deep learning on complex data can be done **efficiently**, without fully end-to-end training: models such as GraphESN and NN4G build accurate DGNs efficiently and make it possible to study the bias of message passing.
- **Depth** in DGNs is necessary (it makes the context grow), but it can cause problems (over-smoothing, over-squashing, training difficulties); going beyond end-to-end can help both for efficiency and for the over-^{*}.
- ML for structured data and graphs is a concrete reality, still under development, that offers new solutions where **relationships** matter. If the data have relationships (serial, hierarchical, networks), **it is not advisable to use a flat representation**. It is also a great opportunity to treat different data types uniformly.

Questions — Possible exam questions

- Why treat structured data directly, instead of turning them into vectors?
- Which types of transduction can be defined on graphs?
- Describe message passing and the general formula of DGNs.
- Difference between layered (GCN, NN4G) and recursive (GNN, GraphESN) approaches.
- What are over-smoothing and over-squashing? What is heterophily?
- Pros and cons of kernels for structures.